

SOLUTIONS MANUAL

PART 2: CHAPTERS 6-10

Rev 06/04/2017

Rev 20180306

DIGITAL DESIGN

**WITH AN INTRODUCTION to the VERILOG HDL,
VHDL, and SystemVerilog
Sixth Edition**

M. MORRIS MANO

Professor Emeritus

California State University, Los Angeles

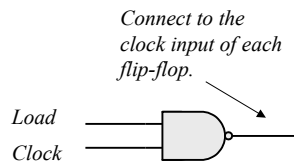
MICHAEL D. CILETTI

Professor Emeritus

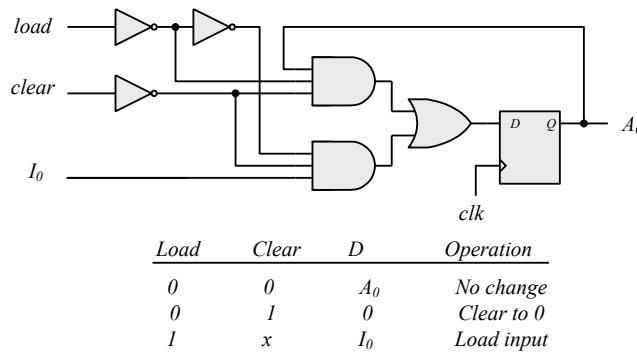
University of Colorado, Colorado Springs

CHAPTER 6

- 6.1** The structure shown below gates the clock through a nand gate. In practice, the circuit can exhibit two problems if the load signal is asynchronous: (1) the gated clock arrives in the setup interval of the clock of the flip-flop, causing metastability, and (2) the load signal truncates the width of the clock pulse. Additionally, the propagation delay through the nand gate might compromise the synchronicity of the overall circuit.



- 6.2** Modify Fig. 6.2, with each stage replicating the first stage shown below:



Note: In this design, *clear* has priority over *load*.

- 6.3** Serial data is transferred one bit at a time, while parallel data is transferred n bits at a time ($n > 1$).

A shift register can convert serial data into parallel data by first shifting one bit a time into the register and then taking the parallel data from the register outputs.

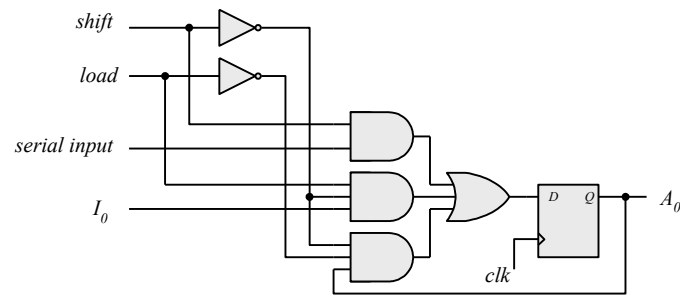
A shift register with parallel load can convert parallel data to a serial format by first loading the data in parallel and then shifting the bits one at a time.

- 6.4** 1101; 0110; 1011; 1101; 0110; 1011

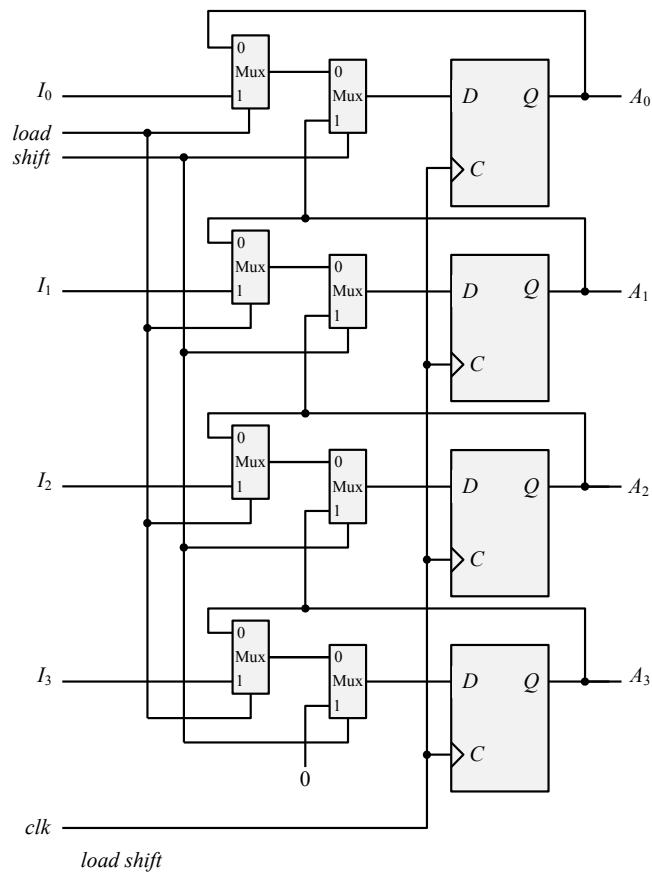
- 6.5** (a) See Fig. 10.12: IC 74194

(b) See Fig. 10.12. Connect two 74194 ICs to form an 8-bit register.

6.6 First stage of register:

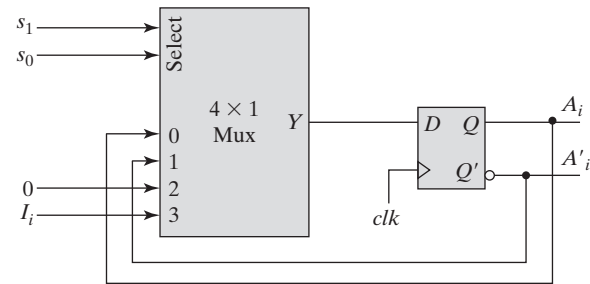


Implementation with muxes:



0	0	No change
0	1	Shift right (Towards LSB)
1	0	Parallel load
1	1	Shift right

6.7 First stage of register:



6.8 A = 1101, 0110, 1011, 1101.
Carry = 0, 1, 0, 0.

6.9 (a) In Fig. 6.5, complement the serial output of shift register B (with an inverter), and set the initial value of the carry to 1.

(b)

Present state	Inputs		Next state	Output	FF inputs	
Q	x	y	Q	D	J_Q	K_Q
0	0	0	0	0	0	x
0	0	0	1	1	1	x
0	0	1	0	1	0	x
0	0	1	0	0	0	x
1	1	0	1	1	x	0
1	1	0	1	0	x	0
1	1	1	0	0	x	1
1	1	1	1	1	x	0

$Q \backslash xy$		x			
		00	01	11	10
Q	0	m_0	m_1 1	m_3	m_2
	1	m_4 x	m_5 x	m_7 x	m_6 x

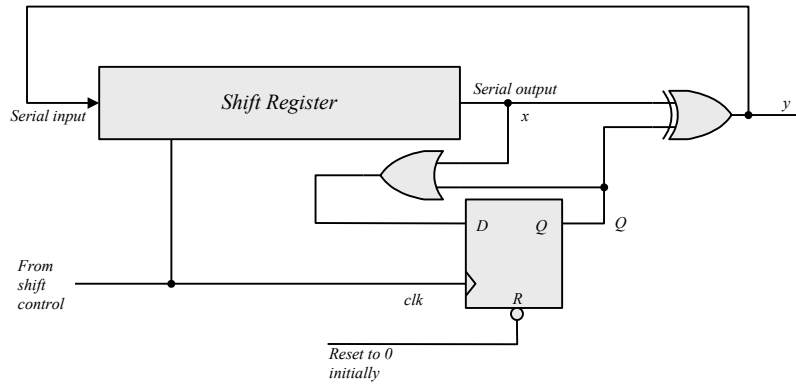
$J_Q = x'y$

$Q \backslash xy$		x			
		00	01	11	10
Q	0	m_0 x	m_1 x	m_3 x	m_2 x
	1	m_4	m_5	m_7	m_6 1

$K_Q = xy' \oplus \oplus$
 $D = Q \oplus x \oplus y$

6.10

See solution to Problem 5.7.

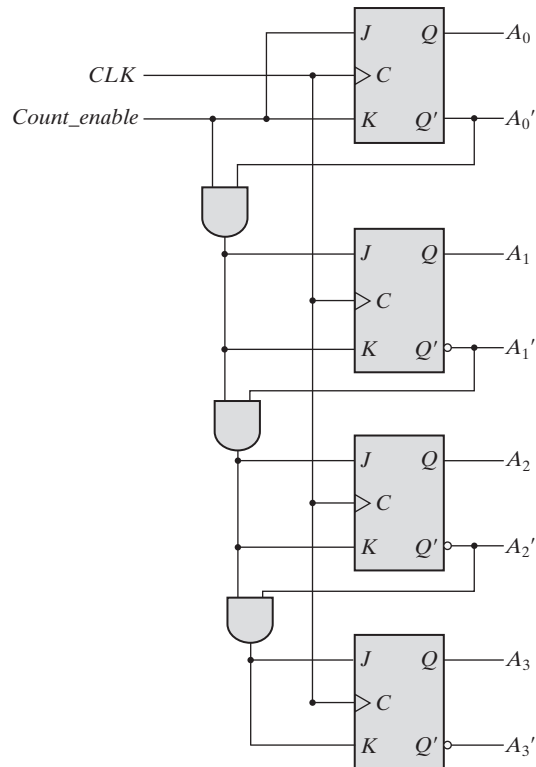
Note that $y = x$ if $Q = 0$, and $y = x'$ if $Q = 1$. Q is set on the first 1 from x .Note that $x \oplus 0 = x$, and $x \oplus 1 = x'$.

6.11

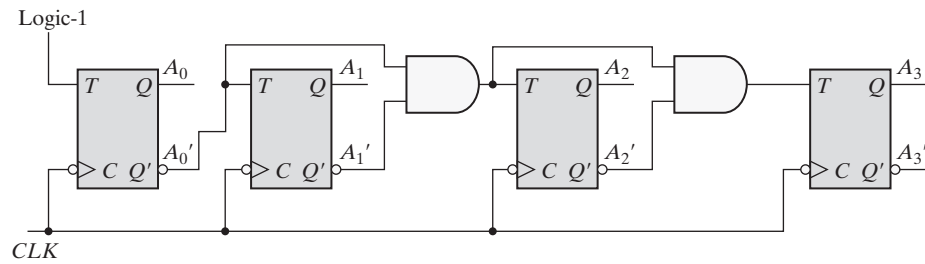
(a) The counter will be a down counter and count backwards.

(b) The counter will be an up counter and count forwards.

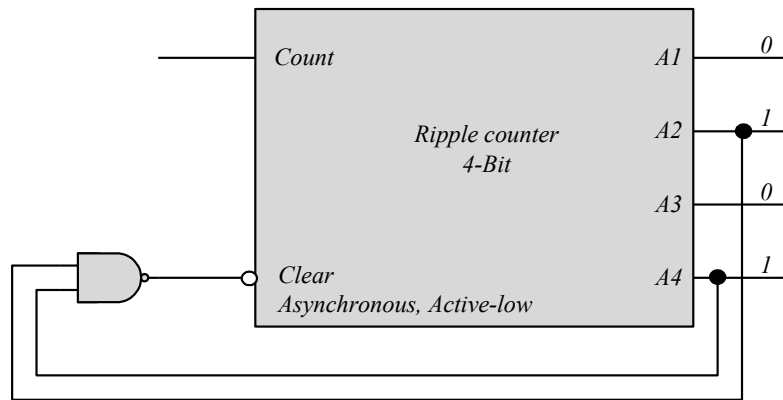
6.12

(a) With the bubbles in C removed (positive-edge).

(b) With complemented flip-flops connected to C .



6.13



An active-low asynchronous reset should also be added.

- 6.14 (a) On adding 1 to 1001100111, we get 1001101000. As 4-bits have changed in the result, 4 flip-flops will be complemented.
- (b) On adding 1 to 0000111111, we get 0001000000. As 7-bits have changed in the result, 7 flip-flops will be complemented.
- (c) On adding 1 to 0011111111, we get 0100000000. As 9-bits have changed in the result, 9 flip-flops will be complemented.

- 6.15 The worst case is when all 10 flip-flops are complemented.
 Maximum delay = $10 \times 5 \text{ ns} = 50 \text{ ns}$
 Maximum frequency = $1/50 \text{ ns} = 10^9/100 = 20 \text{ MHz}$

- 6.16
- | | | | | |
|---------------|------|------|------|-----------------|
| Q8 Q4 Q2 Q1 : | 1010 | 1100 | 1110 | Self correcting |
| Next state: | 1011 | 1101 | 1111 | |
| Next state: | 0100 | 0100 | 0000 | |
- 1010 $\rightarrow$ 1011 $\rightarrow$ 0100
 1100 $\rightarrow$ 1101 $\rightarrow$ 0100
 1110 $\rightarrow$ 1111 $\rightarrow$ 0000

- 6.17** With E denoting the count enable, the inputs to the four D flip-flops of the binary synchronous counter are determined by:

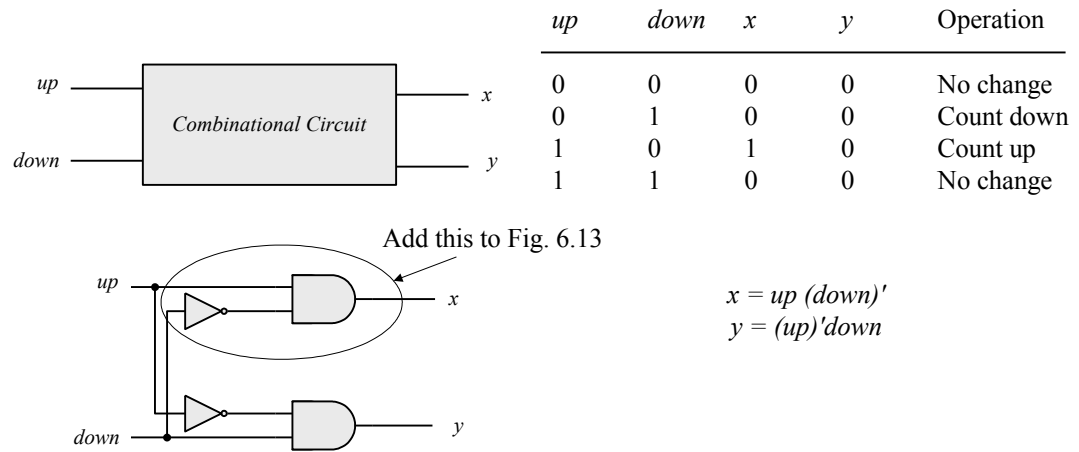
$$D_{A0} = A_0 \oplus E$$

$$D_{A1} = A_1 \oplus (A_0 E)$$

$$D_{A2} = A_2 \oplus (A_1 A_0 E)$$

$$D_{A3} = A_3 \oplus (A_2 A_1 A_0 E)$$

- 6.18** When $up = down = 1$ the circuit counts up.



- 6.19 (b)** From the state table in Table 6.5:

$$D_{Q1} = Q_1'$$

$$D_{Q2} = \sum (1, 2, 5, 6)$$

$$D_{Q4} = \sum (3, 4, 5, 6)$$

$$D_{Q8} = \sum (7, 8)$$

$$\text{Don't care: } d = \sum (10, 11, 12, 13, 14, 15)$$

Simplifying with maps:

$$D_{Q2} = Q_2 Q_1' + Q_8' Q_2' Q_1$$

$$D_{Q4} = Q_4 Q_1' + Q_4 Q_2' + Q_4' Q_2 Q_1$$

$$D_{Q8} = Q_8 Q_1' + Q_4 Q_2 Q_1$$

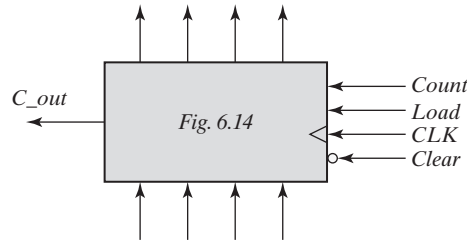
(a)

Present state	Next state	Flip-flop inputs			
$A_8 A_4 A_2 A_1$	$A_8 A_4 A_2 A_1$	$J_{A8} K_{A8}$	$J_{A4} K_{A4}$	$J_{A2} K_{A2}$	$J_{A1} K_{A1}$
0000	0001	0 x	0 x	0 x	1 x
0001	0010	0 x	0 x	1 x	x 1
0010	0011	0 x	0 x	x 0	1 x
0011	0100	0 x	1 x	x 1	x 1
0100	0101	0 x	x 0	0 x	1 x
0101	0110	0 x	x 0	1 x	x 1
0110	0111	0 x	x 0	x 0	1 x
0111	1000	1 x	x 1	x 1	x 1
1000	1001	x 0	0 x	0 x	1 x
1001	0000	x 1	0 x	0 x	x 1

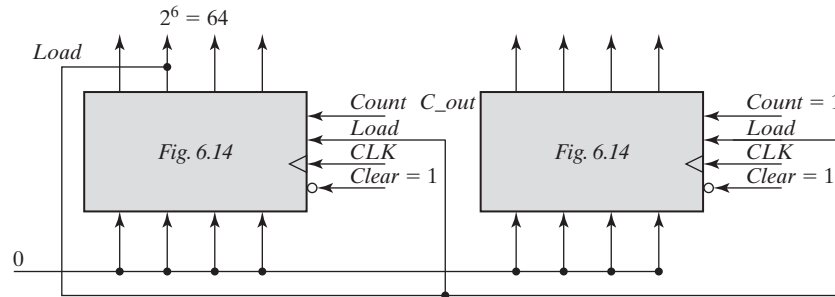
$$\begin{aligned}
 J_{A1} &= 1 \\
 K_{A1} &= 1 \\
 J_{A2} &= A_1 A_8' \\
 K_{A2} &= A_1 \\
 J_{A4} &= A_1 A_2 \\
 K_{A4} &= A_1 A_2' \\
 J_{A8} &= A_1 A_2' A_4 \\
 K_{A8} &= A_1
 \end{aligned}$$

$$d(A_8, A_4, A_2, A_1) = \sum (10, 11, 12, 13, 14, 15)$$

6.20 Block diagram of 4-bit circuit:



(a) Block diagram of binary counter that counts from 0 through 64:



(b) If all 8-outputs are used, then the maximum value the counter can count will be $2^8 - 1 = 255$.

6.21 (a)

$$J_{A0} = LI_0 + L'C \quad KA_0 = LI'_0 + L'C$$

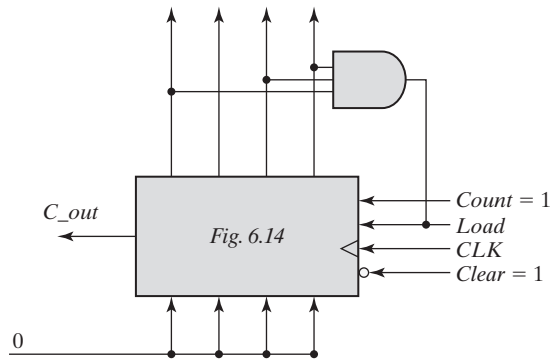
(b)

$$J = [L(LI)']'(L + C) = (L' + LI)(L + C)$$

$$LI + L'C + LIC = LI + L'C \text{ (use a map)}$$

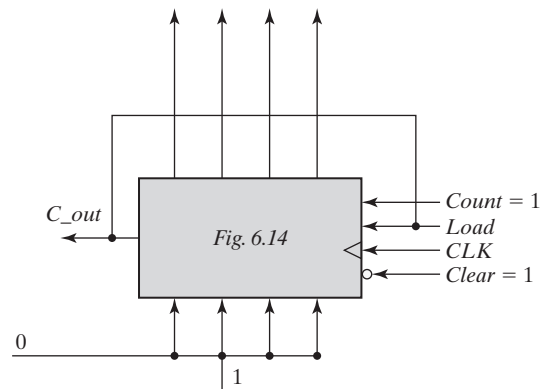
$$K = (LI)'(L + C) = (L' + I')(L + C) = LI' + L'C$$

6.22 (a)



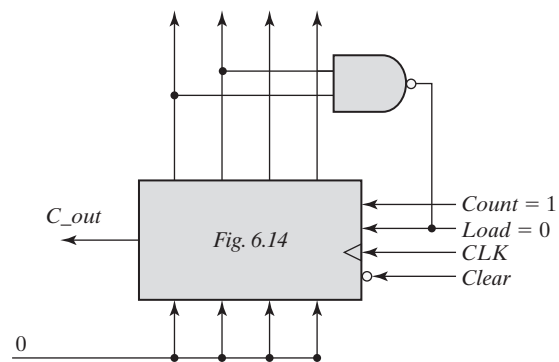
Count sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

(b)



Count sequence: 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15

(c)



Count sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

- 6.23** Use a 3-bit counter and a flip-flop (initially at 0). A start signal sets the flip-flop, which in turn enables the counter. On the count of 7 (binary 111) reset the flip-flop to 0 to disable the count (with the value of 000).

6.24

Present state			Next state			Flip-flop inputs		
A	B	C	A	B	C	T _A	T _B	T _C
0	0	0	0	1	1	0	1	1
0	0	1	1	1	1	1	1	0
0	1	0	×	×	×	×	×	×
0	1	1	0	0	1	0	1	0
1	0	0	0	0	0	1	0	0
1	0	1	×	×	×	×	×	×
1	1	0	1	0	0	0	1	0
1	1	1	1	1	0	0	0	1

On simplifying, we get:

$$T_A = AB' + B'C$$

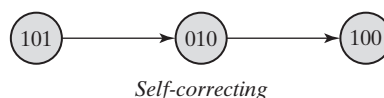
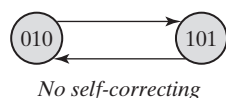
$$T_B = A' + BC'$$

$$T_C = AC + A'C'$$

$$= AC + A'B'C'$$

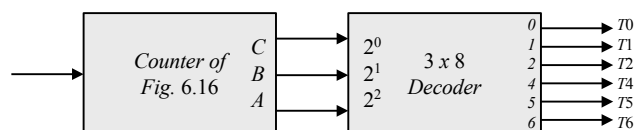
(not self-starting)

(self-starting)



6.25 (a) Use a 6-bit ring counter.

(b)



6.26 Frequency of clock generator = 80 MHz
Period of clock generator = $1/80 \text{ MHz} = 12.5 \text{ ns}$

We need to use a 2-bit counter (four states) to count four pulses, as $80/4 = 20 \text{ MHz}$;
cycle time = $1000 \times 10^{-9} / 20 = 50 \text{ ns}$.

6.27 (a)

Present state			Next state			Flip-flop inputs					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	1	0	0	1	×	0	×	0	×
0	0	1	1	1	0	1	×	1	×	×	1
0	1	0	0	0	1	0	×	×	1	1	×
×	×	×	×	×	×	×	×	×	×	×	×
1	0	0	0	1	0	×	1	1	×	0	×
×	×	×	×	×	×	×	×	×	×	×	×
1	1	0	0	0	0	×	1	×	1	0	×
×	×	×	×	×	×	×	×	×	×	×	×

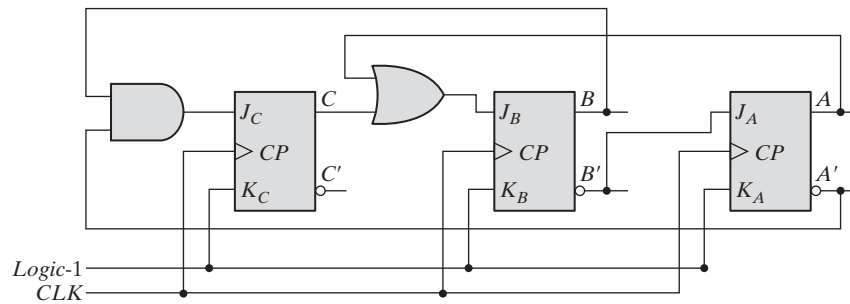
On simplifying, we get:

$$J_A = B' \quad K_A = 1$$

$$J_B = A + C \quad K_B = 1$$

$$J_C = A'B \quad K_C = 1$$

(b)



6.28_a, b

Present state <i>ABC</i>	Next state <i>ABC</i>
000	001
001	010
010	100
011	xxx
100	110
101	xxx
110	000
111	xxx

		<i>BC</i>			
		00	01	11	10
<i>A</i>	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

$D_A = A \oplus B$

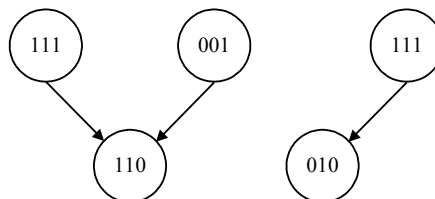
		<i>BC</i>			
		00	01	11	10
<i>A</i>	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

$D_B = AB' + C$

		<i>BC</i>			
		00	01	11	10
<i>A</i>	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

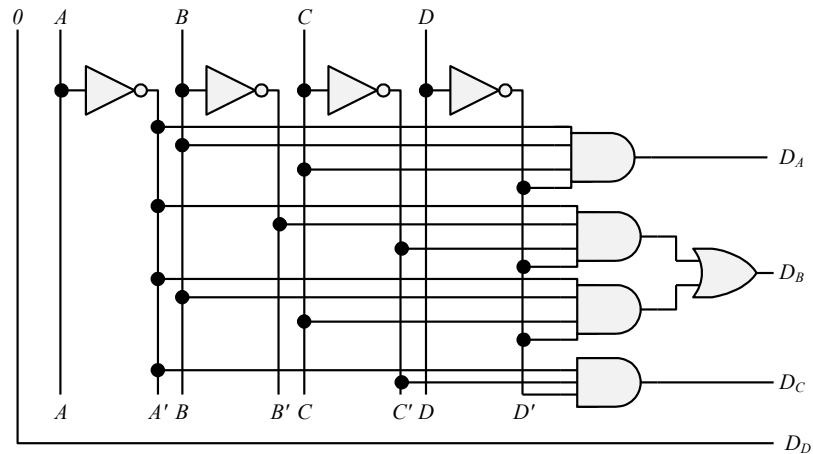
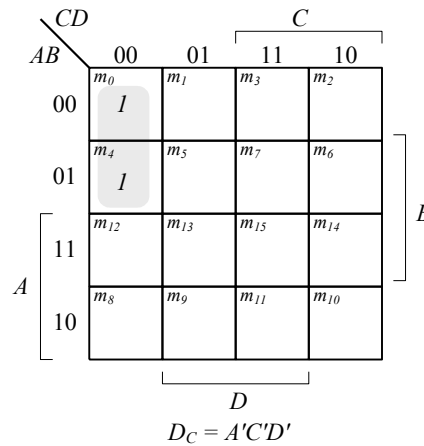
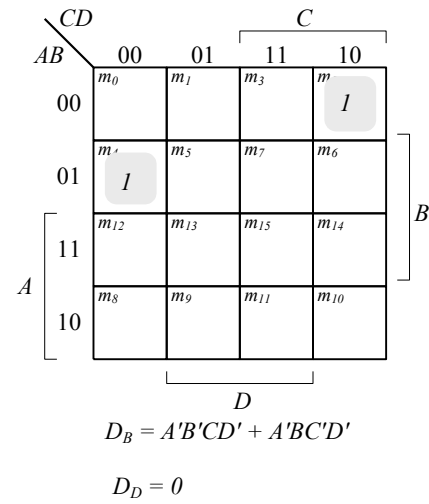
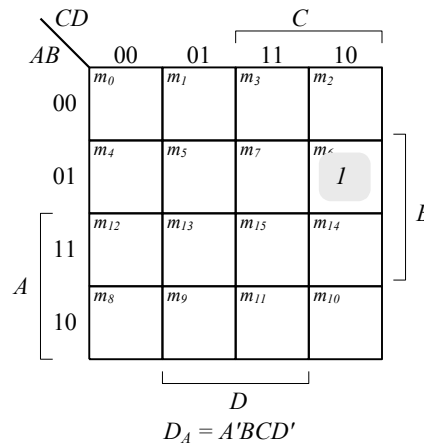
$D_C = A'B'C'$

Self-correcting



6.28_a, b

Present State	Next State
ABCD	ABCD
0000	0010
0001	0000
0010	0100
0011	0000
0100	0110
0101	0000
0110	1000
0111	0000
1000	0000



(c)

Present state			Next state			Flip-flop inputs		
A	B	C	A	B	C	D_A	D_B	D_C
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	0
0	1	0	1	1	0	1	1	0
0	1	1	0	0	1	0	0	1
×	×	×	×	×	×	×	×	×
1	0	1	0	1	1	0	1	1
1	1	0	1	0	1	1	0	1
×	×	×	×	×	×	×	×	×

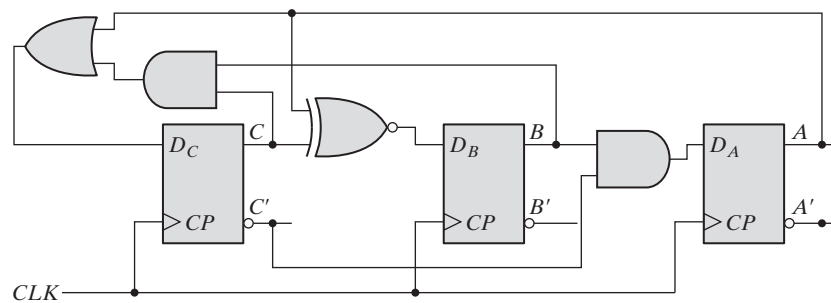
On simplifying, we get:

$$D_A = BC'$$

$$D_B = A'C' + AC = (A \oplus C)'$$

$$D_C = A + BC$$

(d)



Sequence number	A	Flip-flop outputs				E	AND gate required for output
1	0	0	0	0	0	0	$A'E'$
2	1	0	0	0	0	0	AB'
3	1	1	0	0	0	0	BC'
4	1	1	1	0	0	0	CD'
5	1	1	1	1	0	0	DE'
6	1	1	1	1	1	1	AE
7	0	1	1	1	1	1	$A'B$
8	0	0	1	1	1	1	$B'C$
9	0	0	0	1	1	1	$C'D$
10	0	0	0	0	1	1	$D'E$

Number of unused states = $32 - 10 = 22$

6.31 Note: The register has active-low asynchronous reset.

Verilog – Behavioral

```

module Reg_4_bit_beh (output reg A3, A2, A1, A0, input I3, I2, I1, I0, Clock, Clear_b);
  always @ (posedge Clock, negedge Clear_b)
    if (Clear_b == 0) {A3, A2, A1, A0} <= 4'b0;
    else {A3, A2, A1, A0} <= {I3, I2, I1, I0};
endmodule

```

- VHDL – Behavioral

```

entity Prob_6_31_vhdl is
  port (A3, A2, A1, A0: out bit; I3, I2, I1, I0, Clock, Clear_b: in bit);
end Prob_6_31_vhdl;

```

architecture Behavioral **of** Prob_6_31_vhdl **is**

signal A_word: bit_vector (3 **downto** 0);

begin

A3 <= A_word(3);

A2 <= A_word(2);

A1 <= A_word(1);

A0 <= A_word(0);

process (Clock, Clear_b)

begin

if Clear_b = '0' **then** A_word <= "0000";

elsif (Clock'event **and** Clock = '1') **then** A_word <= (I3 & I2 & I1 & I0); **end if**;

end process;

end Behavioral;

Verilog - Structural

entity Reg_4_bit_Structural_vhdl **is**

port (A3, A2, A1, A0: **out bit**; I3, I2, I1, I0, Clock, Clear_b: **in bit**);

DFF M3DFF (A3, I3, Clock, Clear_b);

DFF M2DFF (A2, I2, Clock, Clear_b);

DFF M1DFF (A1, I1, Clock, Clear_b);

```
DFF M0DFF (A0, I0, Clock, Clear_b);
endmodule
```

```
module DFF(output reg Q, input D, clk, clear_b);
  always @ (posedge clk, negedge clear_b)
    if (clear_b == 0) Q <= 0; else Q <= D;
endmodule
```

VHDL - Structural

```
architecture Structural of Prob_6_31_vhdl is
  component D_FF port (Q: out bit; D: in bit; clk, clear_b: in bit); end component;
```

```
begin
  M3: D_FF port map (Q => A3, D => I3, clk => Clock, clear_b => Clear_b);
  M2: D_FF port map (Q => A2, D => I2, clk => Clock, clear_b => Clear_b);
  M1: D_FF port map (Q => A1, D => I1, clk => Clock, clear_b => Clear_b);
  M0: D_FF port map (Q => A0, D => I0, clk => Clock, clear_b => Clear_b);
end Structural;
```

```
entity D_FF is
  port (Q: out bit; D, clk, clear_b: in bit);
end D_FF;
```

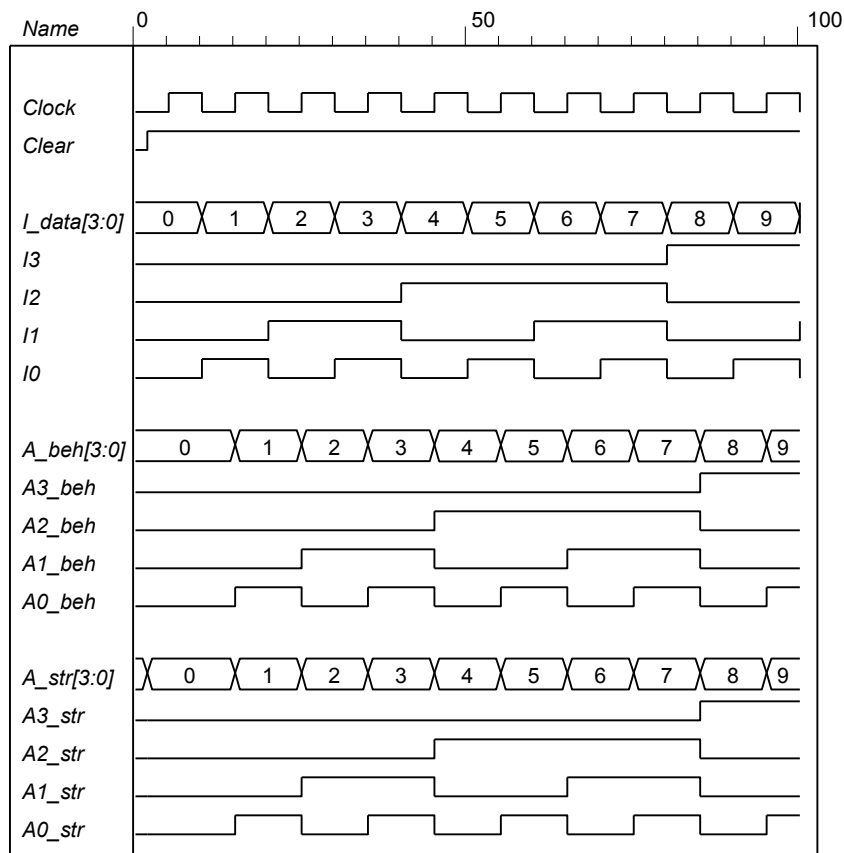
```
architecture Behavioral of D_FF is
begin
  process (clk, clear_b) begin
    if (clear_b = '0') then Q <= '0';
    elsif clk'event and clk = '1' then Q <= D;
    end if;
  end process;
end Behavioral;
```

Testbench - Verilog

```
module t_Reg_4_bit ();
  wire A3_beh, A2_beh, A1_beh, A0_beh;
  wire A3_str, A2_str, A1_str, A0_str;
  reg I3, I2, I1, I0, Clock, Clear_b;
  wire [3: 0] I_data = {I3, I2, I1, I0};
  wire [3: 0] A_beh = {A3_beh, A2_beh, A1_beh, A0_beh};
  wire [3: 0] A_str = {A3_str, A2_str, A1_str, A0_str};

  Reg_4_bit_beh M_beh (A3_beh, A2_beh, A1_beh, A0_beh, I3, I2, I1, I0, Clock, Clear_b);
  Reg_4_bit_Str M_str (A3_str, A2_str, A1_str, A0_str, I3, I2, I1, I0, Clock, Clear_b);

  initial #100 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial begin Clear_b = 0; #2 Clear_b = 1; end
  integer K;
  initial begin
    for (K = 0; K < 16; K = K + 1) begin {I3, I2, I1, I0} = K; #10 ; end
  end
endmodule
```



VHDL

```

entity Reg_4_bit_vhdl is
  port(A3, A2, A1, A0 : out bit; I3, I2, I1, I0, Clock, Clear: in bit); is
end
  always @ (posedge Clock, negedge Clear)
    if (Clear == 0) {A3, A2, A1, A0} <= 4'b0;
    else {A3, A2, A1, A0} <= {I3, I2, I1, I0};
endmodule

module Reg_4_bit_Str (output A3, A2, A1, A0, input I3, I2, I1, I0, Clock, Clear);
  DFF M3DFF (A3, I3, Clock, Clear);
  DFF M2DFF (A2, I2, Clock, Clear);
  DFF M1DFF (A1, I1, Clock, Clear);
  DFF M0DFF (A0, I0, Clock, Clear);
endmodule

module DFF(output reg Q, input D, clk, clear);
  always @ (posedge clk, posedge clear)
    if (clear == 0) Q <= 0; else Q <= D;
endmodule

```

6.32 (a) Verilog

```

module Reg_4_bit_Load (output reg A3, A2, A1, A0, input I3, I2, I1, I0, Load, Clock, Clear);
  always @ (posedge Clock, negedge Clear)
    if (Clear == 0) {A3, A2, A1, A0} <= 4'b0;
    else if (Load) {A3, A2, A1, A0} <= {I3, I2, I1, I0};
endmodule

```

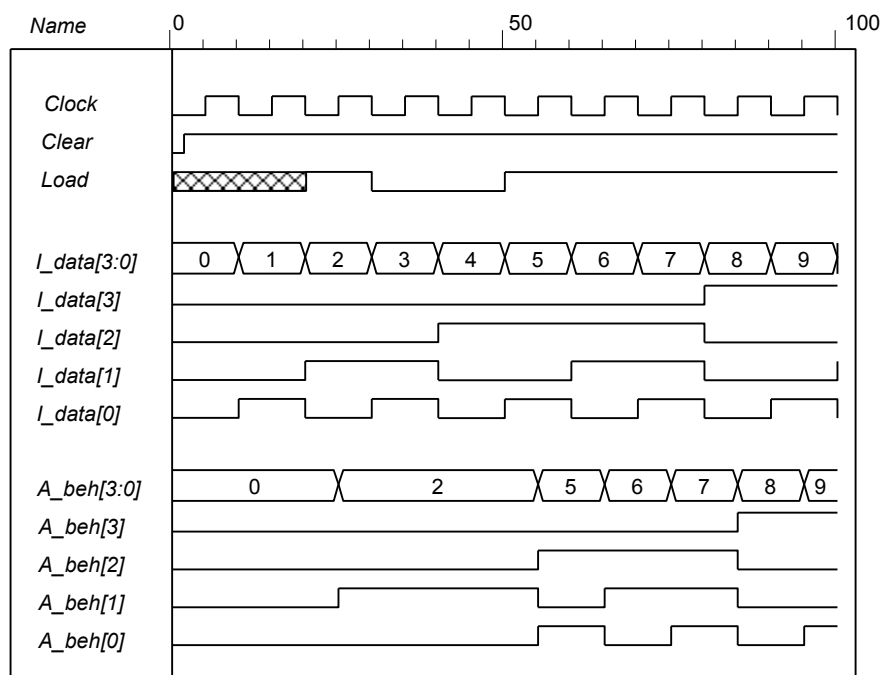
```

module t_Reg_4_Load ();
  wire A3_beh, A2_beh, A1_beh, A0_beh;
  reg I3, I2, I1, I0, Load, Clock, Clear;
  wire [3: 0] I_data = {I3, I2, I1, I0};
  wire [3: 0] A_beh = {A3_beh, A2_beh, A1_beh, A0_beh};

  Reg_4_bit_Load M0 (A3_beh, A2_beh, A1_beh, A0_beh, I3, I2, I1, I0, Load, Clock, Clear);

  initial #100 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial begin Clear = 0; #2 Clear = 1; end
  integer K;
  initial fork
    #20 Load = 1;
    #30 Load = 0;
    #50 Load = 1;
  join
  initial begin
    for (K = 0; K < 16; K = K + 1) begin {I3, I2, I1, I0} = K; #10 ; end
  end
endmodule

```



(a) VHDL

```

entity Prob_6_32a_vhdl is
  port (A3, A2, A1, A0: out bit; I3, I2, I1, I0, Load, Clock, Clear: in bit);
end Prob_6_32a_vhdl;

architecture Behavioral of Prob_6_32a_vhdl is
  signal A_word: bit_vector (3 downto 0);
  begin
    A3 <= A_word(3);
    A2 <= A_word(2);
    A1 <= A_word(1);

```

```

A0 <= A_word(0);

process (Clock, Clear) begin
    if (Clear = '0') then A_word <= "0000";
    elsif (Load = '1') then A_word <= I3 & I2 & I1 & I0;
    end if;
end process;
end Behavioral;

```

(b) Verilog

```

module Reg_4_bit_Load_str (output A3, A2, A1, A0, input I3, I2, I1, I0, Load, Clock, Clear);
    wire y3, y2, y1, y0;
    mux_2 M3 (y3, A3, I3, Load);
    mux_2 M2 (y2, A2, I2, Load);
    mux_2 M1 (y1, A1, I1, Load);
    mux_2 M0 (y0, A0, I0, Load);

```

```

    DFF M3DFF (A3, y3, Clock, Clear);
    DFF M2DFF (A2, y2, Clock, Clear);
    DFF M1DFF (A1, y1, Clock, Clear);
    DFF M0DFF (A0, y0, Clock, Clear);
endmodule

```

```

module DFF(output reg Q, input D, clk, clear);
    always @ (posedge clk, posedge clear)
        if (clear == 0) Q <= 0; else Q <= D;
endmodule

```

```

module mux_2 (output y, input a, b, sel);
    assign y = sel ? a : b;
endmodule

```

```

module t_Reg_4_Load_str ();
    wire A3, A2, A1, A0;
    reg I3, I2, I1, I0, Load, Clock, Clear;
    wire [3: 0] I_data = {I3, I2, I1, I0};
    wire [3: 0] A = {A3, A2, A1, A0};

    Reg_4_bit_Load_str M0 (A3, A2, A1, A0, I3, I2, I1, I0, Load, Clock, Clear);
initial #100 $finish;
initial begin Clock = 0; forever #5 Clock = ~Clock; end
initial begin Clear = 0; #2 Clear = 1; end
    integer K;
    initial fork
        #20 Load = 1;
        #30 Load = 0
        #50 Load = 1;
        #80 Load = 0;
    join
    initial begin
        for (K = 0; K < 16; K = K + 1) begin {I3, I2, I1, I0} = K; #10 ; end
    end
endmodule

```


VHDL

```
entity Prob_6_32b_vhdl is
  port (A3, A2, A1, A0: buffer bit; I3, I2, I1, I0, Load, Clock, Clear: in bit);
end Prob_6_32b_vhdl;
```

```
architecture Structural of Prob_6_32b_vhdl is
  signal y3, y2, y1, y0: bit;
  component mux_2 port (y: out bit; a, b, sel: in bit); end component;
  component D_FF port (Q: out bit; D, clk, clear: in bit); end component;
begin
```

```
  M3: mux_2 port map (y3, A3, I3, Load);
  M2: mux_2 port map (y2, A2, I2, Load);
  M1: mux_2 port map (y1, A1, I1, Load);
  M0: mux_2 port map (y0, A0, I0, Load);
```

```
  M3DFF: D_FF port map (A3, y3, Clock, Clear);
  M2DFF: D_FF port map (A2, y2, Clock, Clear);
  M1DFF: D_FF port map (A1, y1, Clock, Clear);
  M0DFF: D_FF port map (A0, y0, Clock, Clear);
end Structural;
```

```
entity D_FF is
  port (Q: out bit; D, clk, clear: in bit);
end D_FF;
```

```
architecture Behavioral of D_FF is
begin
  process (clk, clear) begin
    if (clear = '1') then Q <= '0';
    elsif clk'event and clk = '1' then Q <= D;
    end if;
  end process;
end Behavioral;
```

```
entity mux_2 is
  port (y: out bit; a, b, sel: in bit);
end mux_2;
```

```
architecture Behavioral of mux_2 is
begin
  y <= a when sel = '1' else b;
end Behavioral;
```

(c)

```
module Reg_4_bit_Load_beh (output reg A3, A2, A1, A0, input I3, I2, I1, I0, Load, Clock, Clear);
  always @ (posedge Clock, negedge Clear)
    if (Clear == 0) {A3, A2, A1, A0} <= 4'b0;
    else if (Load) {A3, A2, A1, A0} <= {I3, I2, I1, I0};
endmodule
```

```
module Reg_4_bit_Load_str (output A3, A2, A1, A0, input I3, I2, I1, I0, Load, Clock, Clear);
  wire y3, y2, y1, y0;
  mux_2 M3 (y3, A3, I3, Load);
  mux_2 M2 (y2, A2, I2, Load);
  mux_2 M1 (y1, A1, I1, Load);
  mux_2 M0 (y0, A0, I0, Load);
```

```

DFF M3DFF (A3, y3, Clock, Clear);
DFF M2DFF (A2, y2, Clock, Clear);
DFF M1DFF (A1, y1, Clock, Clear);
DFF M0DFF (A0, y0, Clock, Clear);
endmodule

module DFF(output reg Q, input D, clk, clear);
  always @ (posedge clk, posedge clear)
    if (clear == 0) Q <= 0; else Q <= D;
endmodule

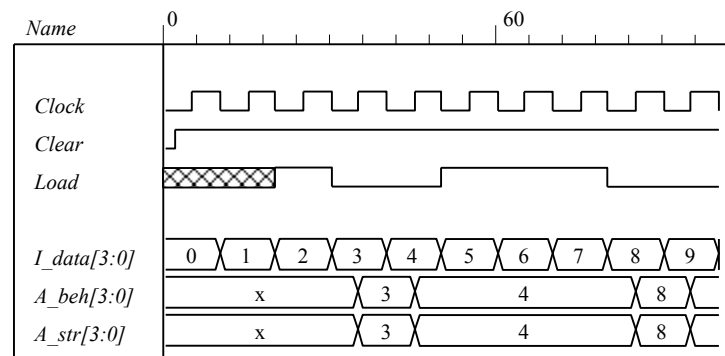
module mux_2 (output y, input a, b, sel);
  assign y = sel ? a: b;
endmodule

module t_Reg_4_Load_str ();
  wire A3_beh, A2_beh, A1_beh, A0_beh;
  wire A3_str, A2_str, A1_str, A0_str;
  reg I3, I2, I1, I0, Load, Clock, Clear;
  wire [3: 0] I_data, A_beh, A_str;
  assign I_data = {I3, I2, I1, I0};
  assign A_beh = {A3_beh, A2_beh, A1_beh, A0_beh};
  assign A_str = {A3_str, A2_str, A1_str, A0_str};

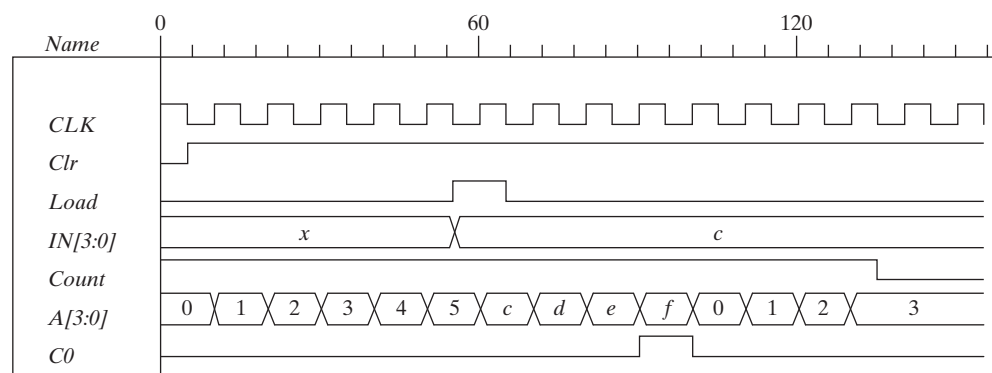
  Reg_4_bit_Load_str M0 (A3_beh, A2_beh, A1_beh, A0_beh, I3, I2, I1, I0, Load, Clock, Clear);
  Reg_4_bit_Load_str M1 (A3_str, A2_str, A1_str, A0_str, I3, I2, I1, I0, Load, Clock, Clear);

  initial #100 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial begin Clear = 0; #2 Clear = 1; end
  integer K;
  initial fork
    #20 Load = 1;
    #30 Load = 0;
    #50 Load = 1;
    #80 Load = 0;
  join
  initial begin
    for (K = 0; K < 16; K = K + 1) begin {I3, I2, I1, I0} = K; #10 ; end
  end
endmodule

```



6.33



6.34 Verilog

```

module Shiftreg (SI, SO, CLK);
  input    S  I, CLK;
  output    SO;
  reg [3: 0] Q;
  assign    SO = Q[0];    // Output from LSB
  always @ (posedge CLK)
    Q = {SI, Q[3: 1]};    // Loads from MSB
endmodule

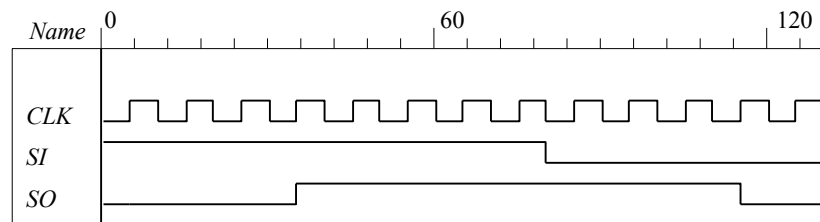
// Test plan
//
// Verify that data shift through the register
// Set SI =1 at startup
// Hold SI =1 for 8 clock cycles then set to 0
// Verify that data shifts out of the register correctly

module t_Shiftreg;
  reg    SI, CLK;
  wire   SO;

  Shiftreg M0 (SI, SO, CLK);

  initial #130 $finish;
  initial begin CLK = 0; forever #5 CLK = ~CLK; end
  initial fork
    SI = 1'b1;
    #80 SI = 0;
  join
endmodule

```



VHDL

```

entity Shiftreg is
  port (SI, CLK: in bit; SO: out bit);
end Shiftreg;

architecture Behavioral of Shiftreg is
  signal Q: Std_Bit_Vector (3 down to 0);
  begin
    SO <= Q(0);
    process (CLK) begin
      if CLK'event and CLK = '1' then Q <= SI & Q(3: 1);
    end process;
  end Behavioral;

entity t_Shiftreg is
end t_Shiftreg;

architecture Testbench of t_Shiftreg is
  component Shiftreg port (SI, CLK: in Std_Logic; SO: out Std_Logic);
  signal t_CLK, t_SI, t_SO: Std_Logic;
  begin

```

```
UUT Shiftreg port map (SI => t_SI, CLK => t_CLK: SO => t_SO);  
t_SI <= '1';  
t_SI <= 0 after 80 ns  
process begin  
    t_CLK <= '0';  
    wait for 5 ns;  
    t_CLK <= '1';  
    wait for 5 ns;  
end process;  
end Testbench;
```

6.35 (a) Structural model for Prob. 6.2

Note that *Load* has priority over *Clear*.

Verilog

```

module Prob_6_35a (output [3: 0] A, input [3:0] I, input Load, Clock, Clear);
    Register_Cell R0 (A[0], I[0], Load, Clock, Clear);
    Register_Cell R1 (A[1], I[1], Load, Clock, Clear);
    Register_Cell R2 (A[2], I[2], Load, Clock, Clear);
    Register_Cell R3 (A[3], I[3], Load, Clock, Clear);
endmodule

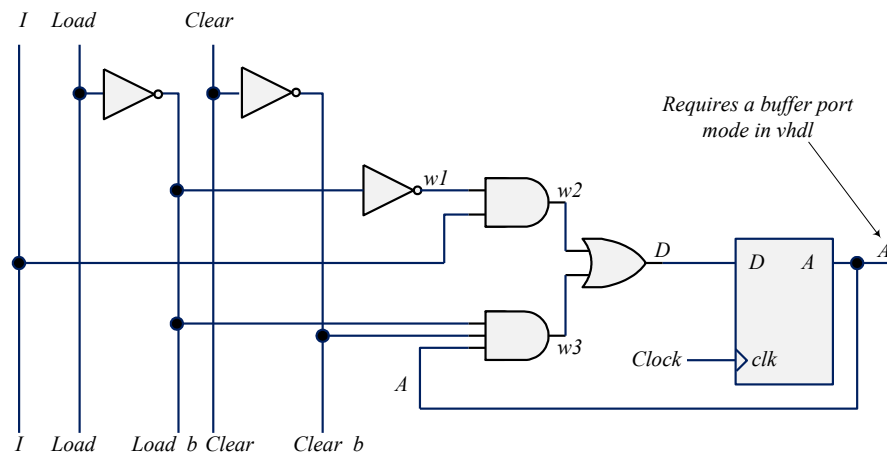
```

```

module Register_Cell (output A, input I, Load, Clock, Clear);
    D_FF M0 (A, D, Clock);
    not (Load_b, Load);
    not (w1, Load_b);
    not (Clear_b, Clear);
    and (w2, I, w1);
    and (w3, A, Load_b, Clear_b);
    or (D, w2, w3);
endmodule

```

Cell detail:



```

module D_FF (output reg Q, input D, clk);
always @ (posedge clk) Q <= D;
endmodule

```

```

module t_Prob_6_35a ( );

```

```

    wire [3: 0] A;
    reg [3: 0] I;
    reg Clock, Clear, Load;

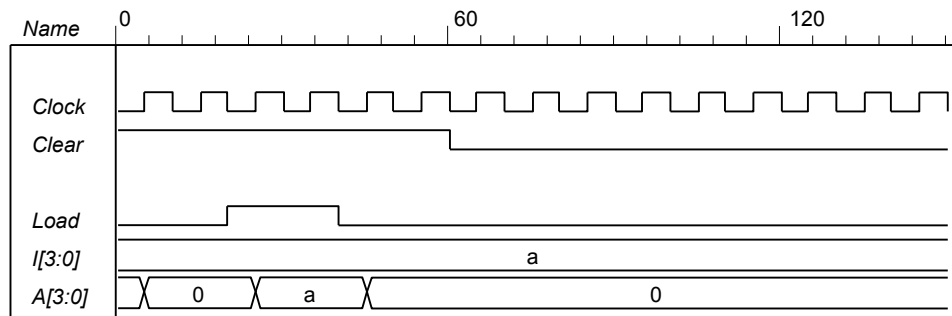
```

```

    Prob_6_35a M0 ( A, I, Load, Clock, Clear);
    initial #150 $finish;
    initial begin Clock = 0; forever #5 Clock = ~Clock; end
    initial fork
    I = 4'b1010; Clear = 1;
    #40 Clear = 0;
    Load = 0;
    #20 Load = 1;
    #40 Load = 0;

```

```
join
endmodule
```



VHDL

```
entity Prob_6_35a_vhdl is
```

```
    port (A: out bit_vector (3 downto 0); I: in bit_vector (3 downto 0); Load, Clock, Clear: in bit);
end Prob_6_35a_vhdl;
```

```
architecture Structural of Prob_6_35a_vhdl is
```

```
    component Register_Cell port (A: buffer bit; I: in bit; Load, Clock, Clear: in bit);
    end component;
```

```
begin
```

```
    R0: Register_Cell port map (A(0), I(0), Load, Clock, Clear);
```

```
    R1: Register_Cell port map (A(1), I(1), Load, Clock, Clear);
```

```
    R2: Register_Cell port map (A(2), I(2), Load, Clock, Clear);
```

```
    R3: Register_Cell port map (A(3), I(3), Load, Clock, Clear);
```

```
end Structural;
```

```
entity Register_Cell is
```

```
    port (A: buffer bit; I, Load, Clock, Clear: in bit);
end Register_Cell;
```

```
architecture Structural of Register_Cell is
```

```
    signal D, Load_b, Clear_b, w1, w2, w3: bit;
```

```
    component D_FF port (A: buffer bit; D, Clock: in bit); end component;
```

```
begin
```

```
    Load_b <= not Load;
```

```
    w1 <= not Load_b;
```

```
    Clear_b <= not Clear;
```

```
    w2 <= I and w1;
```

```
    w3 <= A and Load_b and Clear_b;
```

```
    D <= w2 or w3;
```

```
    M0: D_FF port map (A, D, Clock);
```

```
end Structural;
```

```
entity D_FF is
```

```
    port (Q: out bit; D, clk: in bit);
end D_FF;
```

```
architecture Behavioral of D_FF is
```

```
begin
```

```
    process (clk) begin
```

```
        if clk'event and clk = '1' then Q <= D; end if;
```

```
    end process;
```

```
end Behavioral;
```

(b) Verilog – Behavioral model for Prob 6.2.

```
module Prob_6_35b (output reg [3: 0] A, input [3:0] I, input Load, Clock, Clear);
```

```

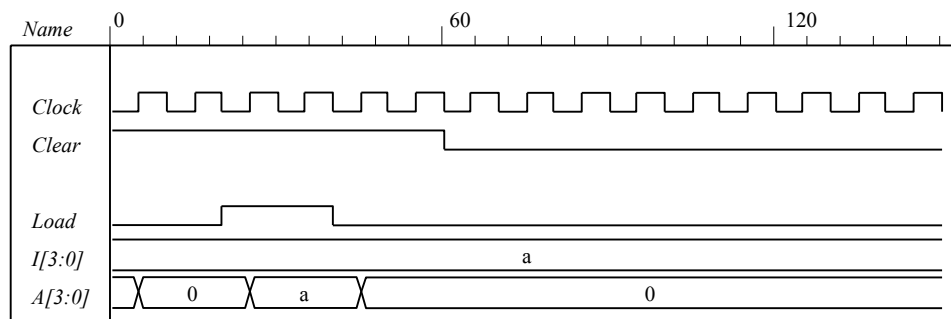
always @ (posedge Clock)
  if (Load) A <= I;
  else if (Clear) A <= 4'b0;
  //else A <= A;          // redundant statement
endmodule

module t_Prob_6_35b ( );

  wire [3: 0] A;
  reg [3: 0] I;
  reg Clock, Clear, Load;

  Prob_6_35b M0 ( A, I, Load, Clock, Clear);
  initial #150 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial fork
    I = 4'b1010; Clear = 1;
    #60 Clear = 0;
    Load = 0;
    #20 Load = 1;
    #40 Load = 0;
  join
endmodule

```



VHDL

```

entity Prob_6_35b is
  port (A: out bit_vector (3 downto 0); I: in bit_vector (3 downto 0),
        Load, Clock, Clear: in std_logic);
end Prob_6_35b;

architecture Behavioral of Prob_6_35b is
begin
  process (Clock) begin
    if Clock'event and Clock = '1' then
      if Load = '1' then A <= I;
      elsif Clear = '1' then A <= '0000'; end if;
    end Behavioral;
  end process;
end Behavioral;

```

(c) Verilog – Structural model for Prob. 6.6.

```

module Prob_6_35c (output [3: 0] A, input [3:0] I, input Shift, Load, Clock);
  Register_Cell R0 (A[0], I[0], A[1], Shift, Load, Clock);
  Register_Cell R1 (A[1], I[1], A[2], Shift, Load, Clock);
  Register_Cell R2 (A[2], I[2], A[3], Shift, Load, Clock);
  Register_Cell R3 (A[3], I[3], A[0], Shift, Load, Clock);
endmodule

```



```

module Register_Cell (output A, input I, Serial_in, Shift, Load, Clock);
  D_FF M0 (A, D, Clock);
  not (Shift_b, Shift);
  not (Load_b, Load);
  and (w1, Shift, Serial_in);
  and (w2, Shift_b, Load, I);

```

```

  and (w3, A, Shift_b, Load_b);
  or (D, w1, w2, w3);
endmodule

```

```

module D_FF (output reg Q, input D, clk);
always @ (posedge clk) Q <= D;
endmodule

```

```

module t_Prob_6_ ( );

```

```

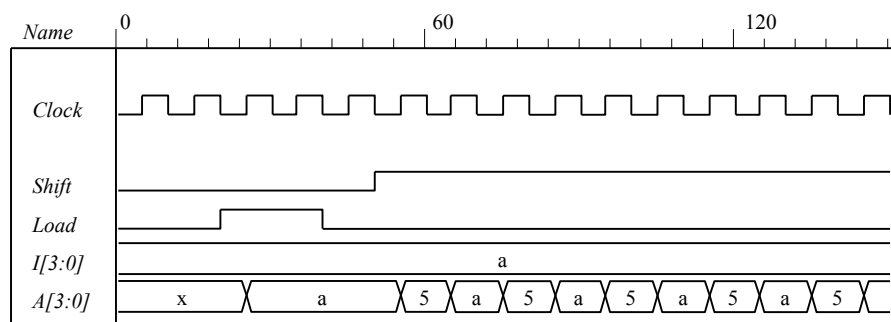
  wire [3: 0] A;
  reg [3: 0] I;
  reg Clock, Shift, Load;

```

```

  Prob_6_35c M0 (A, I, Shift, Load, Clock);
  initial #150 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial fork
    I = 4'b1010;
    Load = 0; Shift = 0;
    #20 Load = 1;
    #40 Load = 0;
    #50 Shift = 1;
  join
endmodule

```



VHDL

```

entity Prob_6_35c is
  port (A: out bit_vector (3 downto 0); I: in bit_vector (3 downto 0);
        Shift, Load, Clock: in bit);
end Prob_6_35c;

```

architecture Structural of Prob_6_35c is

```

  component Register_Cell port (A: buffer bit; I: in bit; D: in bit;
                                Shift, Load, Clock: in bit);

```

```

  end component;

```

```

begin

```

```

  R0: Register_Cell port map (A[0], I[0], A[1], Shift, Load, Clock);
  R1: Register_Cell port map (A[1], I[1], A[2], Shift, Load, Clock);
  R2: Register_Cell port map (A[2], I[2], A[3], Shift, Load, Clock);
  R3: Register_Cell port map (A[3], I[3], A[0], Shift, Load, Clock);

```

```

end Structural;

entity Register_Cell is
    port ( A, : buffer bit; I, Serial_in, Shift, Load, Clock: in bit);
end Register_Cell;

architecture Structural of Register_Cell is
    component D_FF (Q: buffer bit; D, clk: in bit); end component;
begin
    M0: DFF port map (A, D, Clock);
    Shift_b <= not Shift;
    Load_b <= not Load;
    w1 <= Shift and Serial_in;
    w2 <= Shift_b and Load and I;
    w3 <= A and Shift_b and Load_b;
    D <= w1 or w2 or w3;
end Structural;

entity D_FF is
    port (Q: out bit; D, clk: in bit);
end D_FF;

architecture Behavioral of D_FF is
begin
    process (clk) begin
        if clk'event and clk = '1' then Q <= D; end if;
    end Behavioral;
end Behavioral;

```

(d) Behavioral model for Prob. 6.6

Verilog

```

module Prob_6_35d (output reg [3: 0] A, input [3:0] I, input Shift, Load, Clock, Clear);
    always @ (posedge Clock)
        if (Shift) A <= {A[0], A[3:1]};
        else if (Load) A <= I;
        else if (Clear) A <= 4'b0;
        //else A <= A;           // redundant statement
endmodule

```

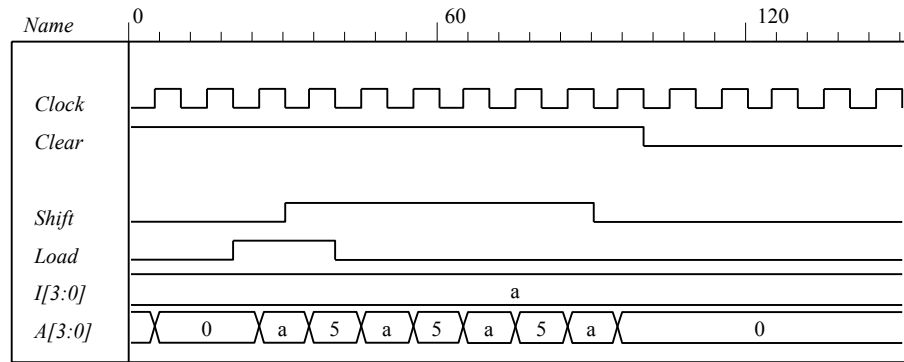
```

module t_Prob_6_35d ( );

    wire [3: 0] A;
    reg [3: 0] I;
    reg Clock, Clear, Shift, Load;

    Prob_6_35d M0 ( A, I, Shift, Load, Clock, Clear);
    initial #150 $finish;
    initial begin Clock = 0; forever #5 Clock = ~Clock; end
    initial fork
        I = 4'b1010; Clear = 1;
        #100 Clear = 0;
        Load = 0;
        #20 Load = 1;
        #40 Load = 0;
        #30 Shift = 1;
        #90 Shift = 0;
    join
endmodule

```



VHDL

```
entity Prob_6_35d_vhdl is
    port (A: buffer bit_vector (3 downto 0); I: in bit_vector (3 downto 0);
          Shift, Load, Clock, Clear: in bit);
end Prob_6_35d_vhdl;
```

```
architecture Behavioral of Prob_6_35d_vhdl is
begin
```

```
    process (Clock) begin
        if Clock'event and Clock = '1' then
            if (Shift = '1') then A <= (A(0) & A(3 downto 1));
            elsif (Load = '1') then A <= I;
            elsif (Clear = '1') then A <= "0000";
            end if; end if;
        end process;
    end Behavioral;
```

```
(c) module Prob_6.35e
    (output [3: 0] A, input [3: 0] I_par, input s1, s0, CLK, Clear);
    wire y3, y2, y1, y0;
    DFF D3 (A[3], y3, CLK, Clear);
    DFF D2 (A[2], y2, CLK, Clear);
    DFF D1 (A[1], y1, CLK, Clear);
    DFF D0 (A[0], y0, CLK, Clear);
    MUX_4x1 M3 (y3, I_par[3], 0, ~A[3], A[3], s1, s0);
    MUX_4x1 M2 (y2, I_par[2], 0, ~A[2], A[2], s1, s0);
    MUX_4x1 M1 (y1, I_par[1], 0, ~A[1], A[1], s1, s0);
    MUX_4x1 M0 (y0, I_par[0], 0, ~A[0], A[0], s1, s0);
endmodule
```

```
module MUX_4x1 (output reg y, input I3, I2, I1, I0, s1, s0);
    always @ (I3, I2, I1, I0, s1, s0)
        case ({s1, s0})
            2'b11: y = I3;
            2'b10: y = I2;
            2'b01: y = I1;
            2'b00: y = I0;
        endcase
endmodule
```

```
module DFF (output reg Q, input D, clk, reset_b);
    always @ (posedge clk, negedge reset_b) if (reset_b == 0) Q <= 0; else Q <= D;
endmodule
```



```

(f) module Prob_6.35f_BEH
  (output [3: 0] A, input [3: 0] I_par, input s1, s0, CLK, Clear);
  always @ (posedge CLK, negedge Clear) if (Clear == 0) A <= 4'b0;
  else case ({s1, s0})
    2'b11: A <= I_par;
    2'b01: A <= ~A;
    2'b10: A <= 0;
    2'b00: A <= A;
  endcase
endmodule

```

(g) Verilog - Behavioral model for Problem 6.8

```

module Ripple_Counter_4bit_a (output [3: 0] A, input Count, reset_b);
  reg A0, A1, A2, A3;
  assign A = {A3, A2, A1, A0};
  always @ (negedge Count, negedge reset_b)
    if (reset_b == 0) A0 <= 0; else if (T) A0 <= !A0;

  always @ (negedge A0, negedge reset_b)
    if (reset_b == 0) A1 <= 0; else if (T) A1 <= !A1;

  always @ (negedge A1, negedge reset_b)
    if (reset_b == 0) A2 <= 0; else if (T) A2 <= !A2;

  always @ (negedge A2, negedge reset_b)
    if (reset_b == 0) A3 <= 0; else if (T) A3 <= !A3;
endmodule

module t_Ripple_Counter_4bit ();
  wire [3: 0] A;
  reg Count, reset_b;

  Ripple_Counter_4bit_a M0 (A, Count, reset_b);

  initial #300 $finish;
endmodule

```

```

initial fork
    reset_b = 0;          // Active-low reset
    #60 reset_b = 1;

    Count = 1;
    #15 Count = 0;
    #30 Count = 1;
    #85 begin Count = 0; forever #10 Count = ~Count; end
join
endmodule

module Ripple_Counter_4bit_b (output [3: 0] A, input Count, reset_b);
    reg A0, A1, A2, A3;
    assign A = {A3, A2, A1, A0};
    always @ (negedge Count, negedge reset_b)
        if (reset_b == 0) A0 <= 0; else A0 <= !A0;

    always @ (negedge A0, negedge reset_b)
        if (reset_b == 0) A1 <= 0; else A1 <= !A1;

    always @ (negedge A1, negedge reset_b)
        if (reset_b == 0) A2 <= 0; else A2 <= !A2;

    always @ (negedge A2, negedge reset_b)
        if (reset_b == 0) A3 <= 0; else A3 <= !A3;
endmodule

```

```

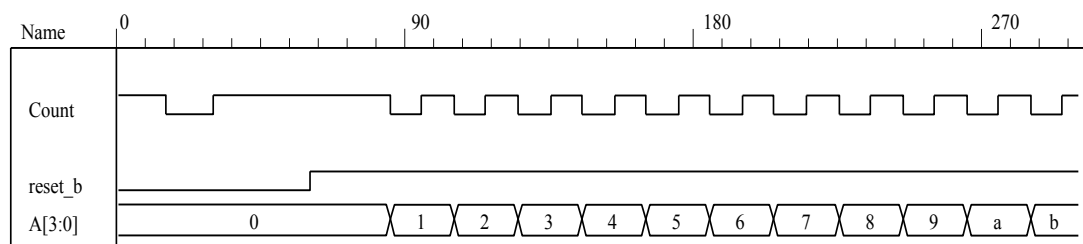
Testbench
module t_Ripple_Counter_4bit ();
    wire [3: 0] A;
    reg Count, reset_b;

    Ripple_Counter_4bit_b M0 (A, Count, reset_b);

    initial #300 $finish;
    initial fork
        reset_b = 0;          // Active-low reset
        #60 reset_b = 1;

        Count = 1;
        #15 Count = 0;
        #30 Count = 1;
        #85 begin Count = 0; forever #10 Count = !Count; end
    join
endmodule

```



```

VHDL
entity Prob_6_35g_vhdl is    -- Ripple_Counter_4bit_
    port (A: out bit_vector (3 downto 0); Count, reset_b: in bit);
end Prob_6_35g_vhdl;

```

```

architecture Behavioral of Prob_6_35g_vhdl is
  signal A0, A1, A2, A3: bit;
begin
  A <= A3 & A2 & A1 & A0;
  process (Count, reset_b) begin
    if reset_b = '0' then A0 <= '0';
    elsif Count'event and Count = '1' then A0 <= not A0;
    end if;
  end process;

  process (Count, reset_b) begin
    if reset_b = '0' then A1 <= '0';
    elsif A0'event and A0 = '0' and Count = '1' then A1 <= not A1;
    end if;
  end process;

  process (Count, reset_b) begin
    if reset_b = '0' then A2 <= '0';
    elsif A1'event and A1 = '0' and Count = '1' then A2 <= not A2;
    end if;
  end process;

  process (Count, reset_b) begin
    if reset_b = '0' then A3 <= '0';
    elsif A2'event and A2 = '0' and Count = '1' then A3 <= not A3;
    end if;
  end process;
end Behavioral;

```

- (h) Note: This version of the solution situates the data shift registers in the test bench.

Verilog

```

module Serial_Subtractor (output SO, input SI_A, SI_B, shift_control, clock, reset_b);
  // See Fig. 6.5 and Problem 6.9a (2s complement serial subtractor)
  reg [1: 0] sum;           // bitwise result
  wire mem = sum[1];
  assign SO = sum[0];

```

```

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) sum <= 2'b10;
    else if (shift_control) begin
      sum <= SI_A + (~SI_B) + sum[1]; // 2s comp bitwise subtraction
    end
endmodule

```

```

module t_Serial_Subtractor ();
  wire SI_A, SI_B;
  reg shift_control, clock, reset_b;

```

```

  Serial_Subtractor M0 (SO, SI_A, SI_B, shift_control, clock, reset_b);

```

```

  initial #250 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork
    shift_control = 0;
    #10 reset_b = 0;
    #20 reset_b = 1;
    #22 shift_control = 1;
    #105 shift_control = 0;
    #112 reset_b = 0;
    #114 reset_b = 1;
    #122 shift_control = 1;
  join

```

```

#205 shift_control = 0;
join
reg [7: 0] A, B, SO_reg;
wire s7;
assign s7 = SO_reg[7];
assign SI_A = A[0];           // LSB of data register
assign SI_B = B[0];
wire SI_B_bar = ~SI_B;
initial fork
  A = 8'h5A;
  B = 8'h0A;
  #122 A = 8'h0A;
  #122 B = 8'h5A;
join

always @ (negedge clock, negedge reset_b)
  if (reset_b == 0) SO_reg <= 0;
  else if (shift_control == 1) begin
    SO_reg <= {SO, SO_reg[7: 1]};
    A <= A >> 1;
    B <= B >> 1;
  end
wire negative = !M0.sum[1]; // M0.sum[1] is the hierarchical path to sum[1]
wire [7: 0] magnitude = (!negative)? SO_reg: 1'b1 + ~SO_reg;
endmodule

```

Simulation results are shown for 5Ah – 0Ah = 50h = 80 and 0Ah – 5Ah = -80.
The magnitude of the result is also shown.

VHDL

See Fig. 6.5 and Problem 6.9a (2s complement serial subtractor).

```

entity Prob_6_35h_vhdl is
  port (SO: out bit; SI_A, SI_B: in bit, shift_control, clock, reset_b: in bit);
end Prob_6_35h_vhdl;

architecture Behavioral of Prob_6_35h_vhdl is
  signal sum: bit_vector (1 downto 0);
  signal mem: bit;
begin
  mem <= sum(1);
  SO <= sum(0);

  process (clock, reset_b) begin
    if (reset_b = '0') then sum <= "10";
    elsif clock'event and clock = '1' then
      if (shift_control = '1') then
        sum(0) <= SI_A xor SI_B xor sum(0);
        sum(1) <= ((SI_A xor SI_B) and sum(0)) ;
      end if;
    end if;
  end process;
end Behavioral;

```

(i) See Prob. 6.35h.

(j) VERILOG

```

module Serial_Twos_Comp (output y, input [7: 0] data, input load, shift_control, Clock, reset_b);
  reg [7: 0] SReg;
  reg Q;
  wire SO = SReg [0];
  assign y = SO ^ Q;
  always @ (posedge Clock, negedge reset_b)
    if (reset_b == 0) begin
      SReg <= 0;
      Q <= 0;
    end
  else begin
    if (load) SReg = data;
    else if (shift_control) begin
      Q <= Q | SO;
      SReg <= {y, SReg[7: 1]};
    end
  end
endmodule

module t_Serial_Twos_Comp ();
  wire y;
  reg [7: 0] data;
  reg load, shift_control, Clock, reset_b;

  Serial_Twos_Comp M0 (y, data, load, shift_control, Clock, reset_b);

  reg [7: 0] twos_comp;

  always @ (posedge Clock, negedge reset_b)

```

```

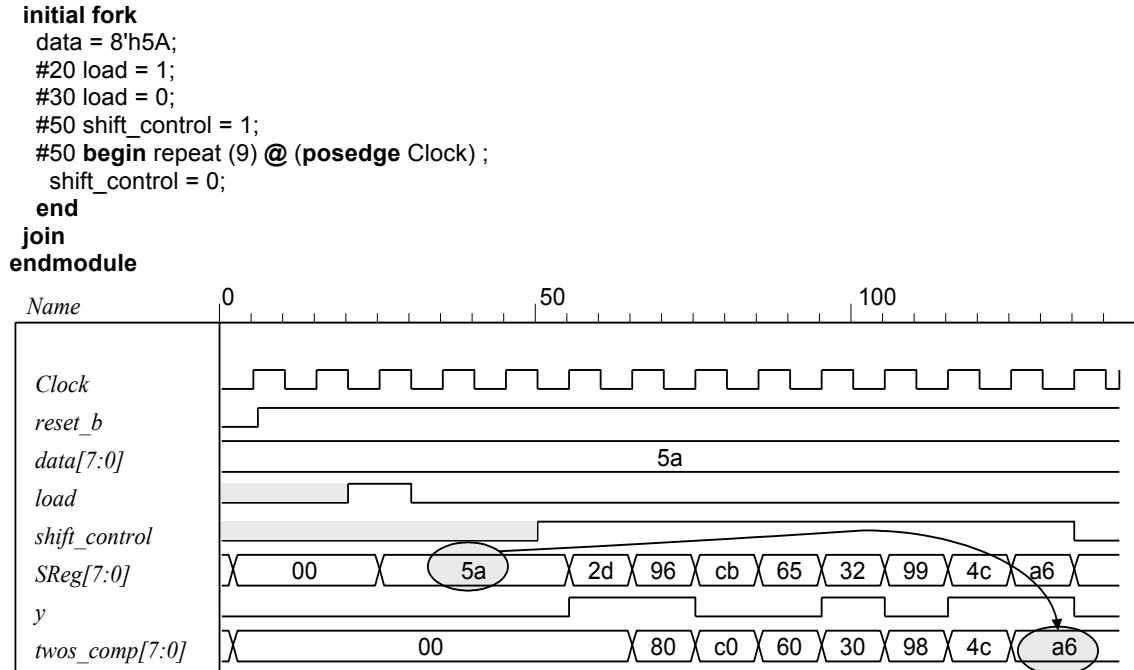
if (reset_b == 0) twos_comp <= 0;
else if (shift_control && !load) twos_comp <= {y, twos_comp[7: 1]};

```

```

initial #200 $finish;
initial begin Clock = 0; forever #5 Clock = ~Clock; end
initial begin #2 reset_b = 0; #4 reset_b = 1; end

```



Note: Example forms $a6_h$ as the two's complement of $5a_h$.

VHDL

```

entity Prob_6_35j_vhdl is          -- Serial_Twos_Comp
  port (y: buffer bit; data: in bit_vector (7 downto 0); load, shift_control, Clock, reset_b: in bit);
end Prob_6_35j_vhdl;

```

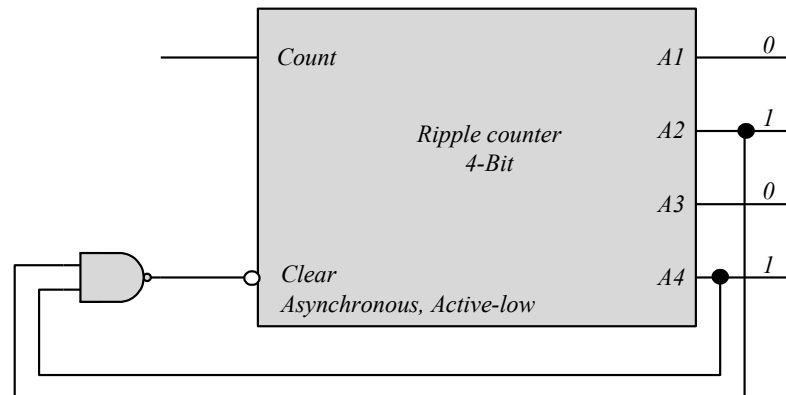
architecture Behavioral of Prob_6_35j_vhdl is

```

  signal Sreg: bit_vector (7 downto 0);
  signal Q, SO: bit;
begin
  SO <= Sreg (0);
  y <= SO xor Q;
  process (Clock, reset_b) begin
    if (reset_b = '0') then
      Sreg <= "00000000";
      Q <= '0';
    elsif Clock'event and Clock = '1' then
      if (load = '1') then Sreg <= data;
      elsif (shift_control = '1') then
        Q <= Q xor SO;
        Sreg <= y & Sreg (7 downto 1);
      end if;
    end if;
  end process;
end Behavioral;

```

(k) From the solution to Problem 6.13:



Verilog

```
module Prob_6_35k_BCD_Counter (output A4, A3, A2, A1, input Count, clk);
  wire A = {A1, A2, A3, A4};
  nand (Clear, A2, A4);
  Ripple_Counter_4bit M0 (A, Clear, reset_b);
endmodule
```

VHDL (Note: active-low, asynchronous reset is included).

entity Prob_6_35k_BCD_Counter_vhdl **is**

port (Q1, Q2, Q4, Q8: **buffer bit**; Count, clk, reset_b: **in bit**);

end Prob_6_35k_BCD_Counter_vhdl;

architecture Behavioral **of** Prob_6_35k_BCD_Counter_vhdl **is**

signal Clear: bit;

signal A: bit_vector (3 **downto** 0);

component Ripple_Counter_4bit **port** (A: **out** bit_vector (3 **downto** 0); Count, Clear, reset_b: **in bit**); **end**

component;

component nand2_gate **port** (y: **out bit**; xin1, xin2: **in bit**); **end component**;

begin

 Q1 <= A(0);

 Q2 <= A(1);

 Q4 <= A(2);

 Q8 <= A(3);

 G1: nand2_gate **port map** (Clear, Q2, Q4);

 G2: Ripple_Counter_4bit **port map** (A, Count, Clear, reset_b);

end Behavioral;

entity Ripple_Counter_4bit **is**

port (A: **buffer** bit_vector (3 **downto** 0); Count, Clear, reset_b: **in bit**);

end Ripple_Counter_4bit;

architecture Behavioral **of** Ripple_counter_4bit **is**

signal Q1, Q2, Q4, Q8: bit;

begin

process (Count, reset_b) **begin**

if reset_b = '0' **then** A(0) <= '0';

elsif Count 'event **and** Count = '0' **then** A(0) <= not A(0);

end if;

end process;

process (A(0), reset_b) **begin**

```

if (reset_b = '0') then A(1) <= '0';
elseif A(0)'event and A(0) = '0' then
if A(3) = '1' then A(1) <= '0';
elseif A(3) = '0' then A(1) <= not A(1);
end if;
end if;
end process;

process (A(1), reset_b) begin
if (reset_b = '0') then A(2) <= '0';
elseif A(1)'event and A(1) = '0' then A(2) <= not A(2);
end if;
end process;

process (A(0), reset_b) begin
if (reset_b = '0') then A(3) <= '0';
elseif A(0)'event and A(0) = '0' then
if Clear = '1' then A(3) <= '0';
else A(3) <= not A(3);
end if;
end if;
end process;
end Behavioral;

entity nand2_gate is
port (y: out bit; xin1, xin2: in bit);
end nand2_gate;

architecture Behavioral of nand2_gate is
begin
y <= not (xin1 and xin2);
end Behavioral;

```

(I) Verilog

```

module Prob_6_35I_Up_Dwn_Beh (output reg [3: 0] A, input CLK, Up, Down, reset_b);

```

```

always @ (posedge CLK, negedge reset_b)
if (reset_b == 0) A <= 4'b0000;
else case ({Up, Down})
2'b10: A <= A + 4'b0001; // Up
2'b01: A <= A - 4'b0001; // Down
default: A <= A; // Suspend (Redundant statement)
endcase
endmodule

```

```

module t_Prob_6_35I_Up_Dwn_Beh ();
wire [3: 0] A;
reg CLK, Up, Down, reset_b;

```

```

Prob_6_35I_Up_Dwn_Beh M0 (A, CLK, Up, Down, reset_b);

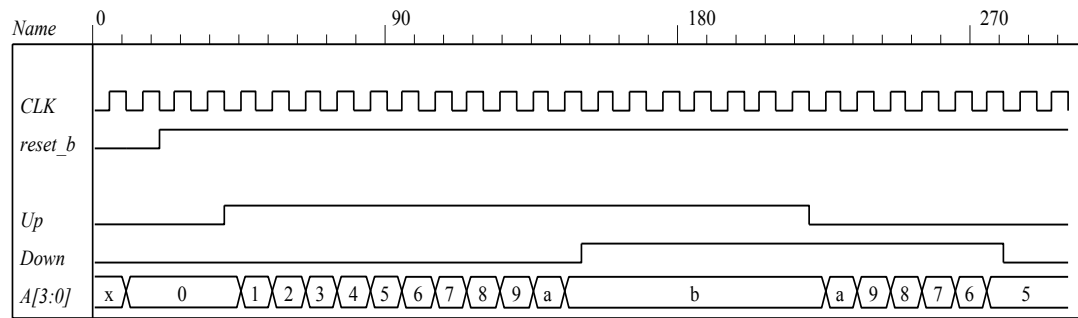
```

```

initial #300 $finish;
initial begin CLK = 0; forever #5 CLK = ~CLK; end
initial fork
Down = 0; Up = 0;
#10 reset_b = 0;
#20 reset_b = 1;
#40 Up = 1;
#150 Down = 1;
#220 Up = 0;
#280 Down = 0;
join

```

endmodule



VHDL

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_6_35l_vhdl is      -- Up_Dwn_Beh
    port (A: buffer integer; CLK, Up, Down, reset_b: bit);
end Prob_6_35l_vhdl;

architecture Behavioral of Prob_6_35l_vhdl is
begin
    process (CLK, reset_b) begin
        if (reset_b = '0') then A <= 0;
        elsif CLK'event and CLK = '1' then
            case (Up & Down) is
                when "10" => A <= A + 1;      -- Up
                when "01" => A <= A - 1;      -- Down
                when others => A <= A;      -- Suspend (Redundant statement)
            end case;
        end if;
    end process;
end Behavioral;
```

6.36 (a) Verilog (Behavioral)

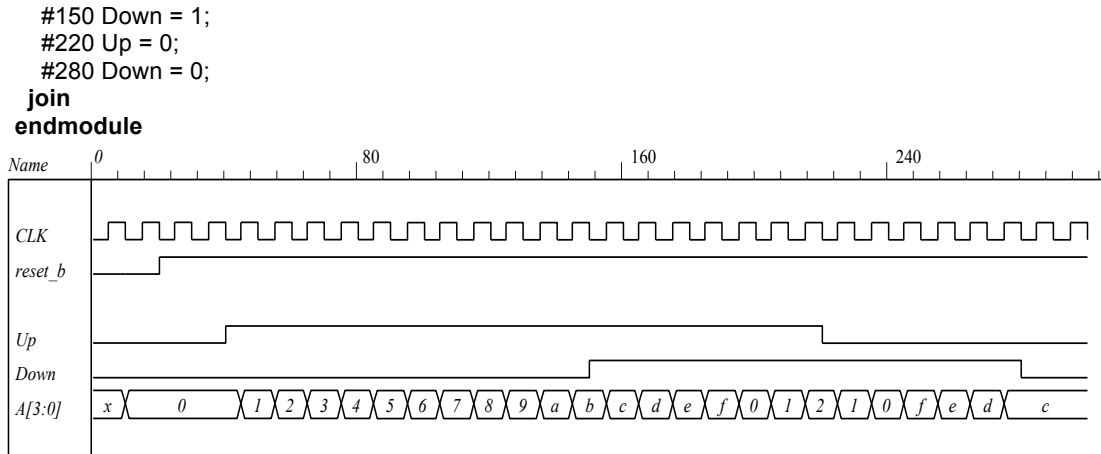
```
// See Fig. 6.13, 4-bit Up-Down Binary Counter
module Prob_6_36_Up_Dwn_Beh (output reg [3: 0] A, input CLK, Up, Down, reset_b);

    always @ (posedge CLK, negedge reset_b)
        if (reset_b == 0) A <= 4'b0000;
        else if (Up) A <= A + 4'b0001;
        else if (Down) A <= A - 4'b0001;
endmodule

module t_Prob_6_36_Up_Dwn_Beh ();
    wire [3: 0] A;
    reg CLK, Up, Down, reset_b;

    Prob_6_36_Up_Dwn_Beh M0 (A, CLK, Up, Down, reset_b);

    initial #300 $finish;
    initial begin CLK = 0; forever #5 CLK = ~CLK; end
    initial fork
        Down = 0; Up = 0;
        #10 reset_b = 0;
        #20 reset_b = 1;
        #40 Up = 1;
    end
```



(b) Verilog (Structural)

```

module Prob_6_36_Up_Dwn_Str (output [3: 0] A, input CLK, Up, Down, reset_b);
  wire Down_3, Up_3, Down_2, Up_2, Down_1, Up_1;
  wire A_0b, A_1b, A_2b, A_3b;

  stage_register SR3 (A[3], A_3b, Down_3, Up_3, Down_2, Up_2, A[2], A_2b, CLK, reset_b);
  stage_register SR2 (A[2], A_2b, Down_2, Up_2, Down_1, Up_1, A[1], A_1b, CLK, reset_b);
  stage_register SR1 (A[1], A_1b, Down_1, Up_1, Down_not_Up, Up, A[0], A_0b, CLK, reset_b);
  not (Up_b, Up);
  and (Down_not_Up, Down, Up_b);
  or (T, Up, Down_not_Up);
  Toggle_flop TF0 (A[0], A_0b, T, CLK, reset_b);
endmodule

module stage_register (output A, A_b, Down_not_Up_out, Up_out, input Down_not_Up, Up, A_in,
  A_in_b, CLK, reset_b);

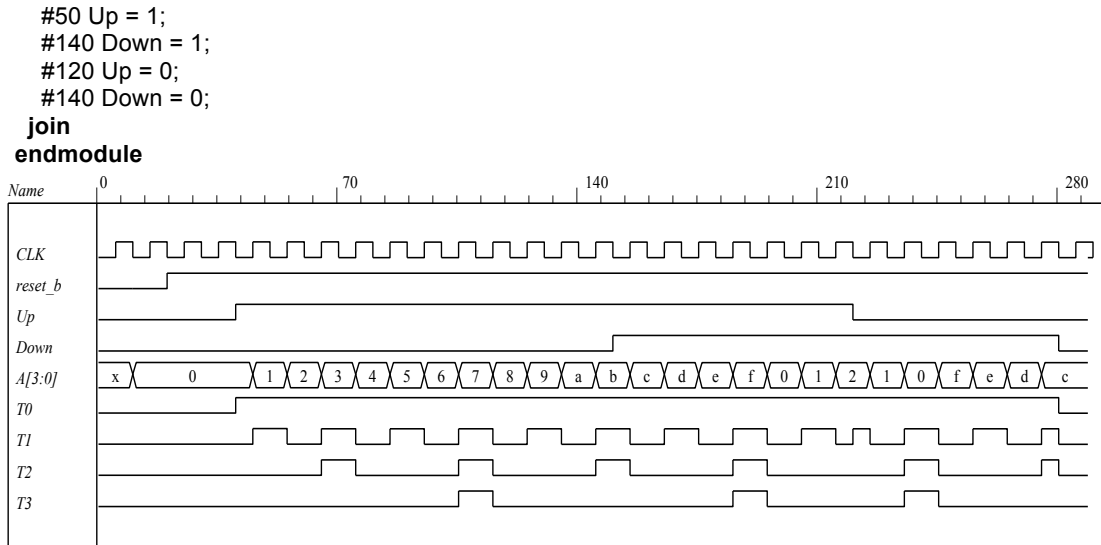
  Toggle_flop T0 (A, A_b, T, CLK, reset_b);
  or (T, Down_not_Up_out, Up_out);
  and (Down_not_Up_out, Down_not_Up, A_in_b);
  and (Up_out, Up, A_in);
endmodule

module Toggle_flop (output reg Q, output Q_b, input T, CLK, reset_b);
  always @ (posedge CLK, negedge reset_b) if (reset_b == 0) Q <= 0; else Q <= Q ^ T;
  assign Q_b = ~Q;
endmodule

module t_Prob_6_36_Up_Dwn_Str ();
  wire [3: 0] A;
  reg CLK, Up, Down, reset_b;
  wire T3 = M0.SR3.T;
  wire T2 = M0.SR2.T;
  wire T1 = M0.SR1.T;
  wire T0 = M0.T;
  Prob_6_36_Up_Dwn_Str M0 (A, CLK, Up, Down, reset_b);

  initial #150 $finish;
  initial begin CLK = 0; forever #5 CLK = ~CLK; end
  initial fork
    Down = 0; Up = 0;
    #10 reset_b = 0;
    #20 reset_b = 1;
  join
endmodule

```



VHDL (Structural)

entity Prob_6_36_Up_Dwn_Str_vhdl **is**

port (A: **buffer bit_vector** (3 **downto** 0); CLK, Up, Down, reset_b: **in bit**);

end Prob_6_36_Up_Dwn_Str_vhdl;

architecture Structural **of** Prob_6_36_Up_Dwn_Str_vhdl **is**

signal Down_3, Up_3, Down_2, Up_2, Down_1, Up_1: **bit**;

signal A_0b, A_1b, A_2b, A_3b: **bit**;

signal Down_not_Up: **bit**;

signal T, Up_b: **bit**;

component stage_register **port** (A: **out bit**; A_b, Down_not_Up_out, Up_out: **out bit**;

Down_not_Up, Up, A_in, A_in_b, CLK, reset_b: **in bit**); **end component**;

component Toggle_flop **port** (Q, Q_b: **out bit**; T, CLK, reset_b: **in bit**); **end component**;

begin

M3: stage_register **port map** (A => A(3), A_b => A_3b, Down_not_Up_out => Down_3, Up_out
=> Up_3, Down_not_up => Down_2, Up => Up_2, A_in => A(2), A_in_b => A_2b, CLK => CLK,
reset_b => reset_b);

M2: stage_register **port map** (A(2), A_2b, Down_2, Up_2, Down_1, Up_1, A(1), A_1b, CLK,
reset_b);

M1: stage_register **port map** (A(1), A_1b, Down_1, Up_1, Down_not_Up, Up, A(0), A_0b, CLK,
reset_b);

M0: Toggle_flop **port map** (A(0), A_0b, T, CLK, reset_b);

Up_b <= not Up;

Down_not_Up <= Down and Up_b;

T <= Up or Down_not_Up;

end Structural;

entity stage_register **is**

port (A, A_b: **out bit**; Down_not_Up_out: **buffer bit**; Up_out: **buffer bit**; Down_not_Up, Up, A_in,
A_in_b, CLK, reset_b: **in bit**);

end stage_register;

architecture Structural **of** stage_register **is**

signal T: **bit**;

component Toggle_flop **port** (Q: **buffer bit**; Q_b: **out bit**; T, CLK, reset_b: **in bit**); **end**
component;

begin

T0: Toggle_flop **port map** (A, A_b, T, CLK, reset_b);

T <= Down_not_Up_out or Up_out;


```

    Down_not_Up_out <= Down_not_Up and A_in_b;
    Up_out <= Up and A_in;
end Structural;

entity Toggle_flop is
    port (Q: buffer bit; Q_b: out bit; T, CLK, reset_b: in bit);
end Toggle_flop;

architecture Behavioral of Toggle_flop is
begin
    process (CLK, reset_b) begin
        if (reset_b = '0') then Q <= '0'; else Q <= Q xor T; end if;
        Q_b <= not Q;
    end process;
end Behavioral;

```

6.37 (a) module Counter_if (output reg [3: 0] Count, input clock, reset);
 always @ (posedge clock , posedge reset)
 if (reset)Count <= 0;
 else if (Count == 0) Count <= 3;
 else if (Count == 1) Count <= 7; // Default interpretation is decimal
 else if (Count == 3) Count <= 1;
 else if (Count == 4) Count <= 0;
 else if (Count == 6) Count <= 4;
 else if (Count == 7) Count <= 6;
 else Count <= 0;
endmodule

(b) module Counter_case (output reg [3: 0] Count, input clock, reset);
 always @ (posedge clock , posedge reset)
 if (reset)Count <= 0;
 else begin
 Count <= 0;
 case (Count)
 0: Count <= 3;
 1: Count <= 7;
 3: Count <= 1;
 4: Count <= 0;
 6: Count <= 4;
 7: Count <= 6;
 default: Count <= 0;
 endcase
 end
endmodule

```

(c) module Counter_FSM (output reg [3: 0] Count, input clock, reset);
    reg [2: 0] state, next_state;
    parameter s0 = 0, s1 = 1, s2 = 2, s3 = 3, s4 = 4, s5 = 5, s6 = 6, s7 = 7;
    always @ (posedge clock , posedge reset)
        if (reset) state <= s0; else state <= next_state;
    always @ (state) begin
        Count = 0;
        case (state)
            s0: begin next_state = s1; Count = 0; end
            s1: begin next_state = s2; Count = 3; end
            s2: begin next_state = s3; Count = 1; end
            s3: begin next_state = s4; Count = 7; end
            s4: begin next_state = s5; Count = 6; end
            s5: begin next_state = s6; Count = 4; end
            default: begin next_state = s0; Count = 0; end
        endcase
    end
endmodule

```

6.38 (a)

Verilog

```

module Prob_6_38a_Updown (OUT, Up, Down, Load, IN, CLK); // Verilog 1995
    output [3: 0] OUT;
    input [3: 0] IN;
    input Up, Down, Load, CLK;
    reg [3:0] OUT;

    always @ (posedge CLK)
        if (Load) OUT <= IN;
        else if (Up) OUT <= OUT + 4'b0001;
        else if (Down) OUT <= OUT - 4'b0001;
        else OUT <= OUT;
    endmodule

```

VHDL

```

entity Prob_6_38a_vhdl is
    port (OUT_sig: out std_logic_vector (3 downto 0); IN_sig: in std_logic_vector (3 downto 0);
        Up, down, Load, CLK: in std_logic);
end Prob_6_38a;

architecture Behavioral of Prob_6_38a is
    begin
        process (CLK) begin
            if CLK'event and CLK = 1 then
                if Load = 1 then OUT_sign <= IN_sig;
                elsif UP = 1 then OUT_sign <= OUT_sig + "0001";
                elsif DOWN = 1 then OUT_sign <= OUT_sign - "0001";
                else OUT_sign <= OUT_sign;
            end if
        end process
    end

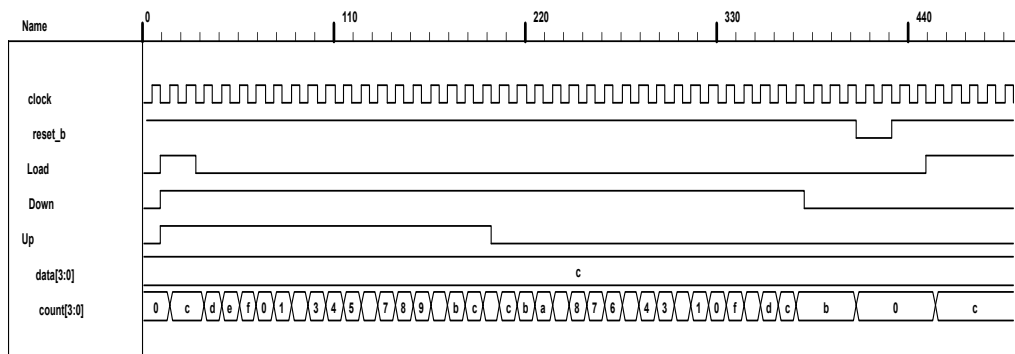
```



```

    end if;
  end process;
end Behavioral;

```



(b)

Verilog

```

module Prob_6_38b_Updown (output reg [3: 0] OUT, input [3: 0] IN, input s1, s0, CLK);

```

```

    always @ (posedge CLK)

```

```

    case ({s1, s0})

```

```

        2'b00: OUT <= OUT + 4'b0001;

```

```

        2'b01: OUT <= OUT - 4'b0001;

```

```

        2'b10: OUT <= IN;

```

```

        2'b11: OUT <= OUT;

```

```

    endcase

```

```

endmodule

```

```

module t_Prob_6_38b_Updown ();

```

```

    wire [3: 0] OUT;

```

```

    reg [3: 0] IN;

```

```

    reg s1, s0, CLK;

```

```

    Prob_6_38b_Updown M0 (OUT, IN, s1, s0, CLK);

```

```

    initial #150 $finish;

```

```

    initial begin CLK = 0; forever #5 CLK = ~CLK; end

```

```

    initial fork

```

```

        IN = 4'b1010;

```

```

        #10 begin s1 = 1; s0 = 0; end // Load IN

```

```

        #20 begin s1 = 1; s0 = 1; end // no change

```

```

        #40 begin s1 = 0; s0 = 0; end // UP;

```

```

        #80 begin s1 = 0; s0 = 1; end // DOWN

```

```

        #120 begin s1 = 1; s0 = 1; end

```

```

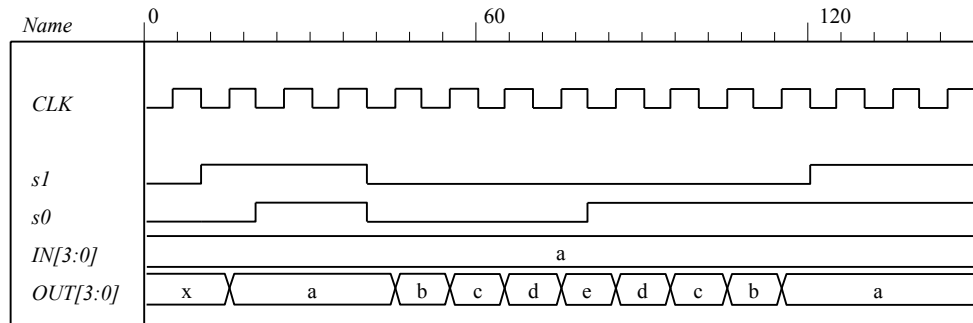
    join

```

```

endmodule

```



VHDL

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_6_38b_Updown_vhdl is
    port (count: buffer unsigned (3 downto 0);
          data_IN: in unsigned (3 downto 0); s1, s0, CLK: in bit);
end Prob_6_38b_Updown_vhdl;
```

```
architecture Behavioral of Prob_6_38b_Updown_vhdl is
begin
    process (CLK) begin
        if CLK'event and CLK = '1' then
            case (s1 & s0) is
                when "00" => count <= count + "0001";
                when "01" => count <= count - "0001";
                when "10" => count <= data_IN;
                when "11" => count <= count;
            end case;
        end if;
    end process;
end Behavioral;
```

6.39 Verilog (Behavioral)

```
module Prob_6_39_Counter_BEH (output reg [2: 0] Count, input Clock, reset_b);
    always @ (posedge Clock, negedge reset_b)
        if (reset_b == 0) Count <= 0;
        else case (Count)
            0: Count <= 1;
            1: Count <= 2;
            2: Count <= 4;
            3: Count <= 4;
            4: Count <= 5;
            5: Count <= 6;
            6: Count <= 0;
            7: Count <= 0;
            default: Count <= 0;
        endcase
endmodule
```

Verilog (Structural)

```

module Prob_6_39_Counter_STR (output [2: 0] Count, input Clock, reset_b);
  supply1 PWR;
  wire Count_1_b = ~Count[1];

  JK_FF M2 (Count[2], Count[1], Count[1], Clock, reset_b);
  JK_FF M1 (Count[1], Count[0], PWR, Clock, reset_b);
  JK_FF M0 (Count[0], Count_1_b, PWR, Clock, reset_b);
endmodule

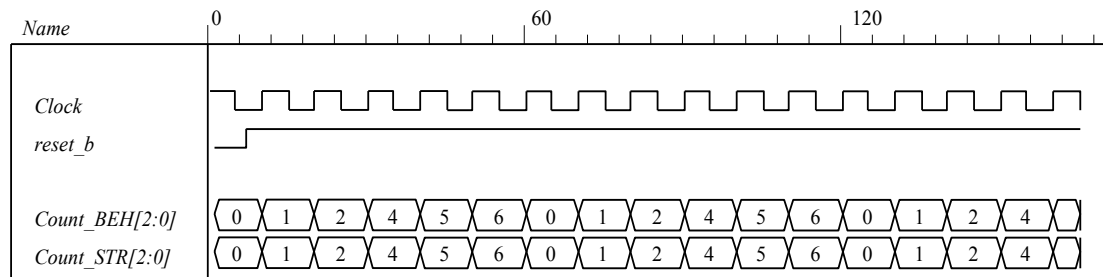
module JK_FF (output reg Q, input J, K, clk, reset_b);
  always @ (posedge clk, negedge reset_b) if (reset_b == 0) Q <= 0; else
    case ({J,K})
      2'b00: Q <= Q;
      2'b01: Q <= 0;
      2'b10: Q <= 1;
      2'b11: Q <= ~Q;
    endcase
endmodule

module t_Prob_6_39_Counter ();
  wire [2: 0] Count_BEH, Count_STR;
  reg Clock, reset_b;

  Prob_6_39_Counter_BEH M0_BEH (Count_STR, Clock, reset_b);
  Prob_6_39_Counter_STR M0_STR (Count_BEH, Clock, reset_b);

  initial #250 $finish;
  initial fork #1 reset_b = 0; #7 reset_b = 1; join
  initial begin Clock = 1; forever #5 Clock = ~Clock; end
endmodule

```



VHDL (Behavioral)

```

entity Prob_6_39_Counter_BEH_vhdl is
  port (Count: buffer bit_vector (2 downto 0); Clock, reset_b: in bit);
end Prob_6_39_Counter_BEH_vhdl;

```

```

architecture Behavioral of Prob_6_39_Counter_BEH_vhdl is
begin

```

```

  process (Clock, reset_b) begin
    if (reset_b = '0') then Count <= "000";
    elsif Clock'event and Clock = '1' then
      case (Count) is
        when "000" => Count <= "001";
        when "001" => Count <= "010";
        when "010" => Count <= "100";
        when "011" => Count <= "100";
        when "100" => Count <= "101";
        when "101" => Count <= "110";
        when "110" => Count <= "000";
        when "111" => Count <= "000";
      endcase
    end if
  end process

```

```

        when others => Count <= "000";
    end case;
end if;
end process;
end Behavioral;

```

VHDL (Structural)

```

entity Prob_6_39_Counter_STR_vhdl is
    port (Count: out bit_vector (2 downto 0); Clock, reset_b: in bit);

```

```

end Prob_6_39_Counter_STR_vhdl;

```

```

architecture Structural of Prob_6_39_Counter_STR_vhdl is
    constant PWR: bit := '1';
    signal Count_1_b: bit;
    component JK_FF port (Q: buffer bit; J, K, clk, reset_b: in bit); end component;
begin
    Count_1_b <= not Count (1);
    M2: JK_FF port map (Count(2), Count(1), Count(1), Clock, reset_b);
    M1: JK_FF port map (Count(1), Count(0), PWR, Clock, reset_b);
    M0: JK_FF port map (Count(0), Count_1_b, PWR, Clock, reset_b);
end Structural;

```

```

entity JK_FF is
    port (Q: buffer bit; J, K, clk, reset_b: in bit);
end JK_FF;

```

```

architecture Behavioral of JK_FF is
begin
process (clk, reset_b) begin
    if (reset_b = '0') then Q <= 0; else
        if clk'event and clk = '1' then
            case (J & K) is
                when "00" => Q <= Q;
                when "01" => Q <= '0';
                when "10" => Q <= '1';
                when "11" => Q <= not Q;
            end case;
        end if;
    end if;
end process;
end Behavioral;

```

```

entity t_Prob_6_39_Counter_vhdl () is
end t_Prob_6_39_Counter_vhdl;

```

```

architecture Structural of t_Prob_6_39_Counter_vhdl is
    signal Count_BEH, Count_STR: bit_vector (2 downto 0);
    signal Clock, reset_b: bit;
    signal t_clk: integer range 0 to 5000;

    component Prob_6_39_Counter_BEH_vhdl
        port (Count: out bit_vector (2 downto 0); Clock, reset_b: in bit); end component;
    component Prob_6_39_Counter_STR_vhdl
        port (Count: out bit_vector (2 downto 0); Clock, reset_b: in bit); end component;
begin
    M0_BEH: Prob_6_39_Counter_BEH_vhdl port map (Count_BEH, Clock, reset_b);
    M0_STR: Prob_6_39_Counter_STR_vhdl port map (Count_STR, Clock, reset_b);

    reset_b <= 0 after 1 ns;

```

```

reset_b <= 1 after 7 ns;

process () -- clock for testbench
begin
    t_clk <= '0';
    wait for 5 ns;
    t_clk <= '1';
    wait for 5 ns;
end process;
end Behavioral;

```

6.40 Verilog

```

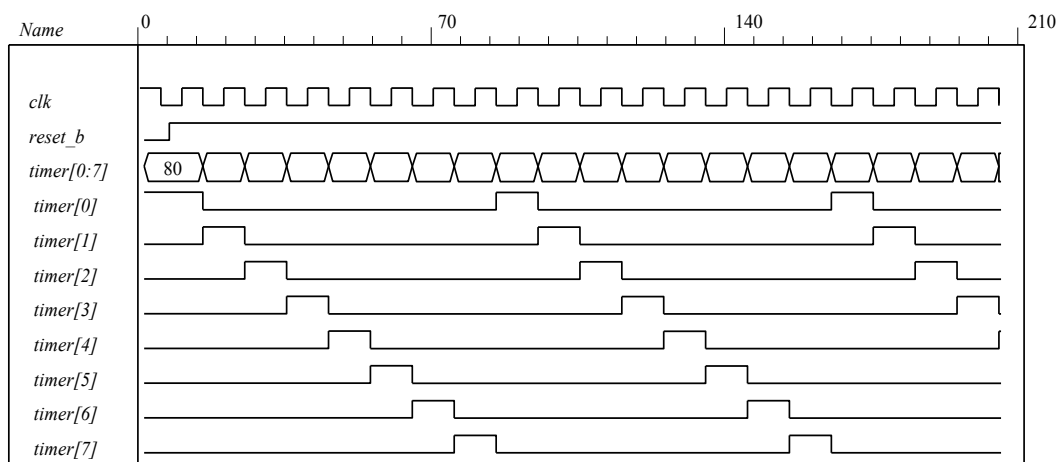
module Prob_6_40_vhdl (output reg [0:7] timer, input clk, reset_b);
    always @ (negedge clk, negedge reset_b)
        if (reset_b == 1'b0) timer <= 8'b1000_0000; else
            case (timer)
                8'b1000_0000: timer <= 8'b0100_0000;
                8'b0100_0000: timer <= 8'b0010_0000;
                8'b0010_0000: timer <= 8'b0001_0000;
                8'b0001_0000: timer <= 8'b0000_1000;
                8'b0000_1000: timer <= 8'b0000_0100;
                8'b0000_0100: timer <= 8'b0000_0010;
                8'b0000_0010: timer <= 8'b0000_0001;
                8'b0000_0001: timer <= 8'b1000_0000;
                default:      timer <= 8'b1000_0000;
            endcase;
end Behavioral;

module t_Prob_6_40 ();
    wire [0: 7] timer;
    reg clk, reset_b;

    Prob_6_40 M0 (timer, clk, reset_b);

    initial #250 $finish;
    initial fork #1 reset_b = 0; #7 reset_b = 1; join
    initial begin clk = 1; forever #5 clk = ~clk; end
endmodule

```



VHDL

```

entity Prob_6_40_vhdl is
  port (timer: buffer bit_vector (0 to 7); clk, reset_b: in bit);
end Prob_6_40_vhdl;

architecture Behavioral of Prob_6_40_vhdl is
begin
  process (clk, reset_b) begin
    if (reset_b = '0') then timer <= "10000000";
    elsif (clk'event and clk = '1') then
      case (timer) is
        when "10000000" => timer <= "01000000";
        when "01000000" => timer <= "00100000";
        when "00100000" => timer <= "00010000";
        when "00010000" => timer <= "00001000";
        when "00001000" => timer <= "00000100";
        when "00000100" => timer <= "00000010";
        when "00000010" => timer <= "00000001";
        when "00000001" => timer <= "10000000";
        when others => timer <= "10000000";
      end case;
    end if;
  end process;
end Behavioral;

```

6.41**Verilog**

```

module Prob_6_41_Switched_Tail_Johnson_Counter (output [0: 3] Count, input CLK, reset_b);
  wire Q_0b, Q_1b, Q_2b, Q_3b;

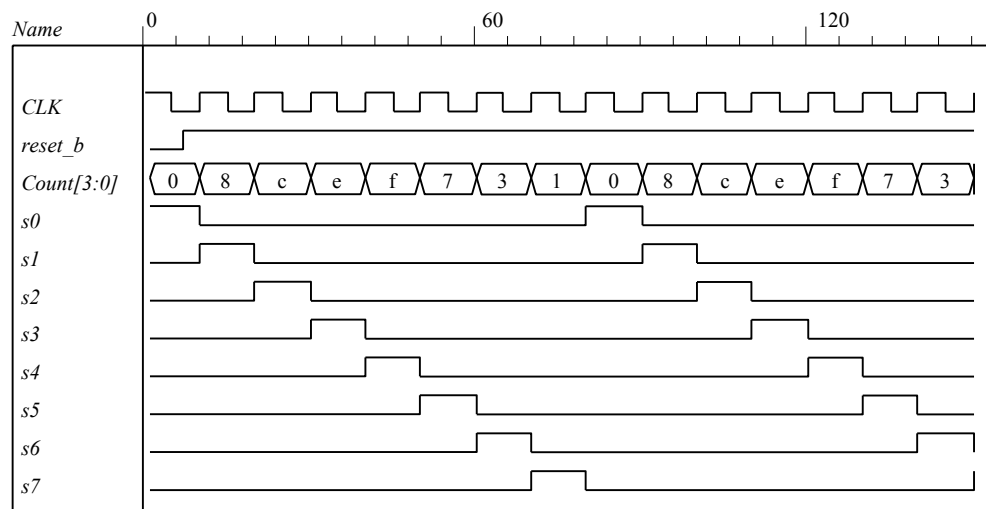
  D_FF M3 (Count[3], Q_3b, Count[2], CLK, reset_b);
  D_FF M2 (Count[2], Q_2b, Count[1], CLK, reset_b);
  D_FF M1 (Count[1], Q_1b, Count[0], CLK, reset_b);
  D_FF M0 (Count[0], Q_0b, Q_3b, CLK, reset_b);
endmodule

module DFF (output reg Q, output Q_b, input D, clk, reset_b);
  assign Q_b = ~Q;
  always @ (posedge clk, negedge reset_b) if (reset_b == 0) Q <= 0; else Q <= D;
endmodule

module t_Prob_6_41_Switched_Tail_Johnson_Counter ();
  wire [3: 0] Count;
  reg CLK, reset_b;
  wire s0 = ~ M0.Count[0] && ~M0.Count[3];
  wire s1 = M0.Count[0] && ~M0.Count[1];
  wire s2 = M0.Count[1] && ~M0.Count[2];
  wire s3 = M0.Count[2] && ~M0.Count[3];
  wire s4 = M0.Count[0] && M0.Count[3];
  wire s5 = ~ M0.Count[0] && M0.Count[1];
  wire s6 = ~ M0.Count[1] && M0.Count[2];
  wire s7 = ~ M0.Count[2] && M0.Count[3];

  Prob_6_41_Switched_Tail_Johnson_Counter M0 (Count, CLK, reset_b);
  initial #150 $finish;
  initial fork #1 reset_b = 0; #7 reset_b = 1; join
  initial begin CLK = 1; forever #5 CLK = ~CLK; end
endmodule

```



VHDL

entity Prob_6_41_vhdl **is**

port (Count: **buffer** bit_vector (0 to 3); CLK, reset_b: **in** bit);

end Prob_6_41_vhdl;

architecture Structural of Prob_6_41_vhdl **is**

signal Q_0b, Q_1b, Q_2b, Q_3b: **bit**;

component D_FF **port** (Q, Q_b: **buffer** bit; D, clk, reset_b: **in** bit); **end component**;

begin

 M3: D_FF **port** map (Count (3), Q_3b, Count (2), CLK, reset_b);

 M2: D_FF **port** map (Count (2), Q_2b, Count (1), CLK, reset_b);

 M1: D_FF **port** map (Count (1), Q_1b, Count (0), CLK, reset_b);

 M0: D_FF **port** map (Count (0), Q_0b, Q_3b, CLK, reset_b);

end Structural;

entity D_FF **is**

port (Q: **buffer** bit; Q_b: **out** bit; D, clk, reset_b: **in** bit);

end D_FF;

architecture Behavioral of D_FF **is**

begin

 Q_b <= **not** Q;

process (clk, reset_b) **begin**

if (reset_b = '0') **then** Q <= '0';

else if clk'event and clk = '1' **then** Q <= D;

end if;

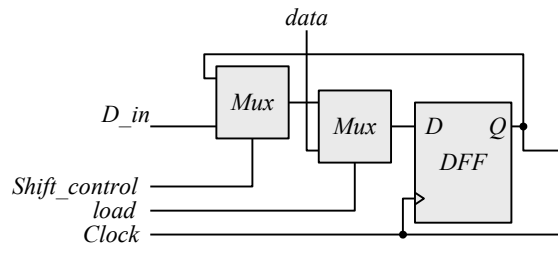
end if;

end process;

end Behavioral;

6.42 Intra-assignment delay is a form of delay that has the effect of postponing the assignment of a value (a constant or value of a variable/expression) to a variable.

6.43 Shift register cell:



Note: When Shift_Control selects the recirculation path, Q from the flip-flop is clocked back into the flip-flop, creating the action of a suspended clock.

Verilog

```
module Prob_6_43_Str (output SO, input [7:0] data, input load, Shift_control, Clock, reset_b);
    supply0 gnd;
    wire SO_A;
```

```
begin
```

```
    ShiftReg_with_Load M_A (SO_A, D_in, data_A, load, Shift_control, Clock, reset_b);
```

```
    ShiftReg_with_Load M_B (SO_B, SO_A, data_B, gnd, Shift_control, Clock, reset_b);
```

```
endmodule;
```

```
module ShiftReg_with_Load port (output SO_A, in D_in, input [7:0] data, input
    load, Shift_control, Clock, reset_b);
```

```
    wire [7: 0] Q;
```

```
    assign SO = Q[0];
```

```
    SR_cell M7 (Q[7], D_in, data[7], load, select, Clock, reset_b);
```

```
    SR_cell M6 (Q[6], Q[7], data[6], load, select, Clock, reset_b);
```

```
    SR_cell M5 (Q[5], Q[6], data[5], load, select, Clock, reset_b);
```

```
    SR_cell M4 (Q[4], Q[5], data[4], load, select, Clock, reset_b);
```

```
    SR_cell M3 (Q[3], Q[4], data[3], load, select, Clock, reset_b);
```

```
    SR_cell M2 (Q[2], Q[3], data[2], load, select, Clock, reset_b);
```

```
    SR_cell M1 (Q[1], Q[2], data[1], load, select, Clock, reset_b);
```

```
    SR_cell M0 (Q[0], Q[1], data[0], load, select, Clock, reset_b);
```

```
endmodule
```

```
module SR_cell (output Q, input D, data, load, select, Clock, reset_b);
```

```
    wire y;
```

```
    DFF_with_load M0 (Q, y, data, load, Clock, reset_b);
```

```
    Mux_2 M1 (y, Q, D, select);
```

```
endmodule
```

```
module DFF_with_load (output reg Q, input D, data, load, Clock, reset_b);
```

```
    always @ (posedge Clock, negedge reset_b)
```

```
        if (reset_b == 0) Q <= 0; else if (load) Q <= data; else Q <= D;
```

```
endmodule
```

```
module Mux_2 (output reg y, input a, b, sel);
```

```
    always @ (a, b, sel) if (sel ==1) y = b; else y = a;
```

```
endmodule
```

```
module t_Fig_6_4_Str ();
```

```
    wire SO;
```

```
    reg load, Shift_control, Clock, reset_b;
```

```
    reg [7: 0] data, Serial_Data;
```

```
    Prob_6_43_Str M0 (SO, data, load, Shift_control, Clock, reset_b);
```

```
    always @ (posedge Clock, negedge reset_b)
```

```
        if (reset_b == 0) Serial_Data <= 0;
```

```
else if (Shift_control ) Serial_Data <= {M0.SO_A, Serial_Data [7: 1]};
```

```
initial #200 $finish;
initial begin Clock = 0; forever #5 Clock = ~Clock; end
initial begin #2 reset_b = 0; #4 reset_b = 1; end
```

```
initial fork
  data = 8'h5A;
  #20 load = 1;
  #30 load = 0;
  #50 Shift_control = 1;
  #50 begin repeat (9) @ (posedge Clock) ;
    Shift_control = 0;
  end
join
endmodule
```

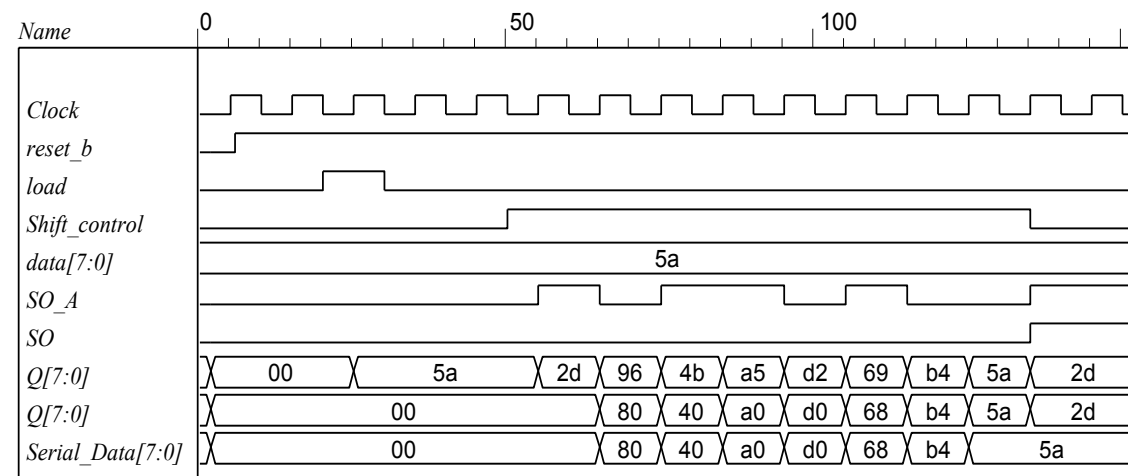
```
module t_Fig_6_4_Str ();
  wire SO;
  reg load, Shift_control, Clock, reset_b;
  reg [7: 0] data, Serial_Data;

  Prob_6_43_Str M0 (SO, data, load, Shift_control, Clock, reset_b);

  always @ (posedge Clock, negedge reset_b)
  if (reset_b == 0) Serial_Data <= 0;
  else if (Shift_control ) Serial_Data <= {M0.SO_A, Serial_Data [7: 1]};
```

```
initial #200 $finish;
initial begin Clock = 0; forever #5 Clock = ~Clock; end
initial begin #2 reset_b = 0; #4 reset_b = 1; end
```

```
initial fork
  data = 8'h5A;
  #20 load = 1;
  #30 load = 0;
  #50 Shift_control = 1;
  #50 begin repeat (9) @ (posedge Clock) ;
    Shift_control = 0;
  end
join
endmodule
```



Alternative: a behavioral model for synthesis is given below. The behavioral description implies the need for a mux at the input to a D-type flip-flop.

```

module Fig_6_4_Beh (output SO, input [7: 0] data, input load, Shift_control, Clock, reset_b);
  reg [7: 0] Shift_Reg_A, Shift_Reg_B;
  assign SO = Shift_Reg_B[0];
  always @ (posedge Clock, negedge reset_b)
    if (reset_b == 0) begin
      Shift_Reg_A <= 0;
      Shift_Reg_B <= 0;
    end
    else if (load) Shift_Reg_A <= data;
    else if (Shift_control) begin
      Shift_Reg_A <= { Shift_Reg_A[0], Shift_Reg_A[7: 1]};
      Shift_Reg_B <= {Shift_Reg_A[0], Shift_Reg_B[7: 1]};
    end

```

endmodule

```

module t_Fig_6_4_Beh ();
  wire SO;
  reg load, Shift_control, Clock, reset_b;
  reg [7: 0] data, Serial_Data;

```

Fig_6_4_Beh M0 (SO, data, load, Shift_control, Clock, reset_b);

```

always @ (posedge Clock, negedge reset_b)
  if (reset_b == 0) Serial_Data <= 0;
  else if (Shift_control ) Serial_Data <= {M0.Shift_Reg_A[0], Serial_Data [7: 1]};

```

```

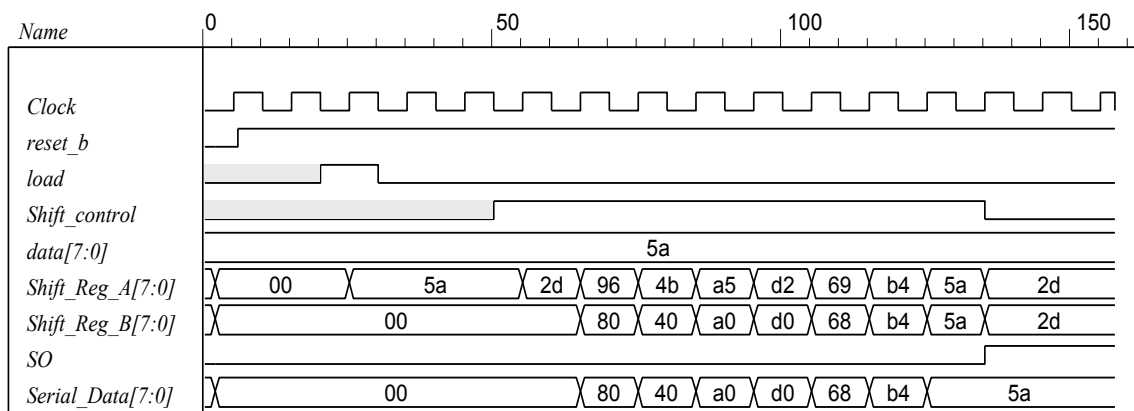
initial #200 $finish;
initial begin Clock = 0; forever #5 Clock = ~Clock; end
initial begin #2 reset_b = 0; #4 reset_b = 1; end

```

```

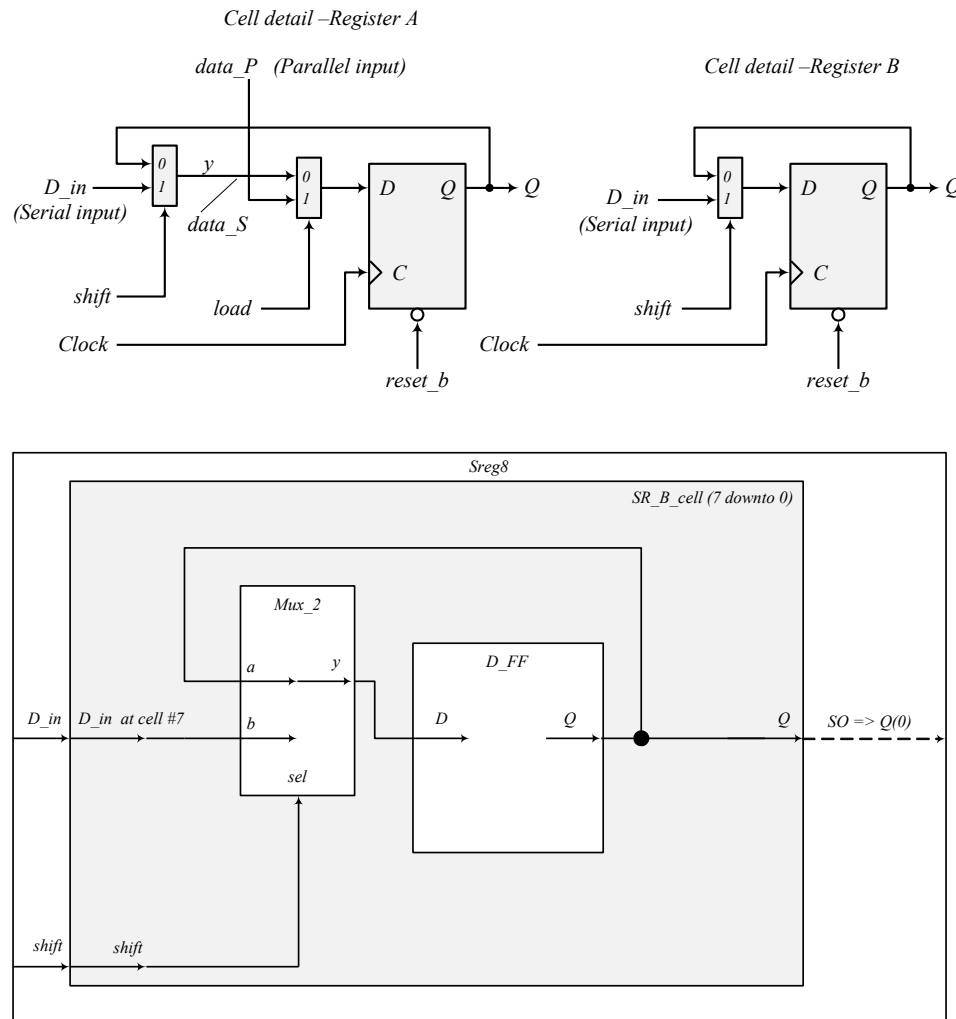
initial fork
  data = 8'h5A;
  #20 load = 1;
  #30 load = 0;
  #50 Shift_control = 1;
  #50 begin repeat (9) @ (posedge Clock) ;
    Shift_control = 0;
  end
join
endmodule

```



VHDL

The datapath into each flip-flop will be modified to include a 2-input mux. One input of the mux is connected to the output of the flip-flop. The other input is connected to the data path that was previously connected to the flip-flop. A control signal is supplied to the mux so that the flip-flop appears to have a gated clock signal. When the machine is to be idle (i.e., not shifting and not loading), the output of the flip-flop recirculates through the mux and back to the D-input of the flip-flop. The cells of register A have an additional mux to accommodate parallel load as well as ordinary operation. The modified registers cells are shown below. Additionally, the output of the least significant bit of Register A connects to the input mux of the left-most cell of the register.



entity Prob_6_43_Str_vhdl is

port (SO: **out bit**; data_A: **in bit_vector** (7 **downto** 0); D_in, load, shift, Clock, reset_b: **in bit**);
end Prob_6_43_Str_vhdl;

architecture Structural of Prob_6_43_Str_vhdl is

signal SO_A: bit;

-- Register A

component SReg8_with_Load **port** (SO: **out bit**; D_in: **in bit**; data: **in bit_vector** (7 **downto** 0); load, shift, Clock, reset_b: **in bit**); **end component**;

-- Register B

component SReg8 **port** (SO: **out bit**; D_in: **in bit**; shift, Clock, reset_b: **in bit**); **end component**;

```

begin
    -- Register A
    M_A: Sreg8_with_Load port map (SO => SO_A, D_in => D_in, data => data_A, load => load,
    shift => shift, Clock => clock, reset_b => reset_b);

    -- Register B
    M_B: Sreg8 port map (SO => SO, D_in => SO_A, shift => shift, Clock => Clock, reset_b =>
    reset_b);
end structural;

entity Sreg8_with_Load is -- Register A
port (SO: out bit; D_in: in bit; data: in bit_vector (7 downto 0); load, shift, Clock, reset_b: in bit);
end Sreg8_with_Load;

architecture Structural of Sreg8_with_Load is
    signal Q: bit_vector (7 downto 0);
    component SR_A_cell port (Q: out bit; D, data, load, shift, Clock, reset_b: in bit); end component;
begin
    SO <= Q(0);
    M7: SR_A_cell port map (Q(7), D_in, data(7), load, shift, Clock, reset_b);
    M6: SR_A_cell port map (Q(6), Q(7), data(6), load, shift, Clock, reset_b);
    M5: SR_A_cell port map (Q(5), Q(6), data(5), load, shift, Clock, reset_b);
    M4: SR_A_cell port map (Q(4), Q(5), data(4), load, shift, Clock, reset_b);
    M3: SR_A_cell port map (Q(3), Q(4), data(3), load, shift, Clock, reset_b);
    M2: SR_A_cell port map (Q(2), Q(3), data(2), load, shift, Clock, reset_b);
    M1: SR_A_cell port map (Q(1), Q(2), data(1), load, shift, Clock, reset_b);
    M0: SR_A_cell port map (Q(0), Q(1), data(0), load, shift, Clock, reset_b);
end Structural;

entity Sreg8 is -- Register B
port (SO: out bit; D_in: in bit; shift, Clock, reset_b: in bit);
end Sreg8;

architecture Structural of Sreg8 is
    signal Q: bit_vector (7 downto 0);
    component SR_B_cell port (Q: out bit; D_in, shift, Clock, reset_b: in bit); end component;
begin
    SO <= Q(0);
    M7: SR_B_cell port map (Q => Q(7), D_in => D_in, shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M6: SR_B_cell port map (Q => Q(6), D_in => Q(7), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M5: SR_B_cell port map (Q => Q(5), D_in => Q(6), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M4: SR_B_cell port map (Q => Q(4), D_in => Q(5), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M3: SR_B_cell port map (Q => Q(3), D_in => Q(4), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M2: SR_B_cell port map (Q => Q(2), D_in => Q(3), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M1: SR_B_cell port map (Q => Q(1), D_in => Q(2), shift => shift, Clock => Clock, reset_b =>
    reset_b);
    M0: SR_B_cell port map (Q => Q(0), D_in => Q(1), shift => shift, Clock => Clock, reset_b =>
    reset_b);
end Structural;

entity SR_A_cell is
port (Q: buffer bit; D_in, data, load, shift, Clock, reset_b: in bit);
end SR_A_cell;

architecture Structural of SR_A_cell is
    signal y: bit;

```

```

component D_FF_with_load port (Q: buffer bit; data_S, data_P, load, shift, Clock, reset_b: in bit);
end component;
component Mux_2 port (y : out bit; a, b, sel: in bit); end component;
begin
  M0: D_FF_with_load port map (Q => Q, data_S => y, data_P => data, load => load, shift => shift,
    Clock => Clock, reset_b => reset_b);
  M1: Mux_2 port map (y, Q, D_in, shift);
end Structural;

```

```

entity D_FF_with_load is
  port (Q: out bit ; data_S, data_P, load, shift, Clock, reset_b: in bit);
end D_FF_with_load;

```

```

architecture Behavioral of D_FF_with_load is
begin
  process (Clock, reset_b) begin
    if (reset_b = '0') then Q <= '0';
    elsif Clock'event and Clock = '1' then
      if (load = '1') then Q <= data_P; else Q <= data_S; end if;
    end if;
  end process;
end Behavioral;

```

```

entity Mux_2 is
  port (y: out bit; a, b, sel: in bit);
end Mux_2;

```

```

architecture behavioral of Mux_2 is
begin
  process (a, b, sel) begin
    if (sel = '1') then y <= b; else y <= a; end if;
  end process;
end Behavioral;

```

```

entity SR_B_cell is
  port (Q: buffer bit; D_in, shift, Clock, reset_b: in bit);
end SR_B_cell;

```

```

architecture Structural of SR_B_cell is
  signal y: bit;
  component D_FF port (Q: buffer bit; D, Clock, reset_b: in bit); end component;
  component Mux_2 port (y : out bit; a, b, sel: in bit); end component;
begin
  M0: D_FF port map (Q => Q, D => y, Clock => Clock, reset_b => reset_b);
  M1: Mux_2 port map (y, Q, D_in, shift);
end Structural;

```

```

entity D_FF is
  port (Q: out bit ; D, Clock, reset_b: in bit);
end D_FF;

```

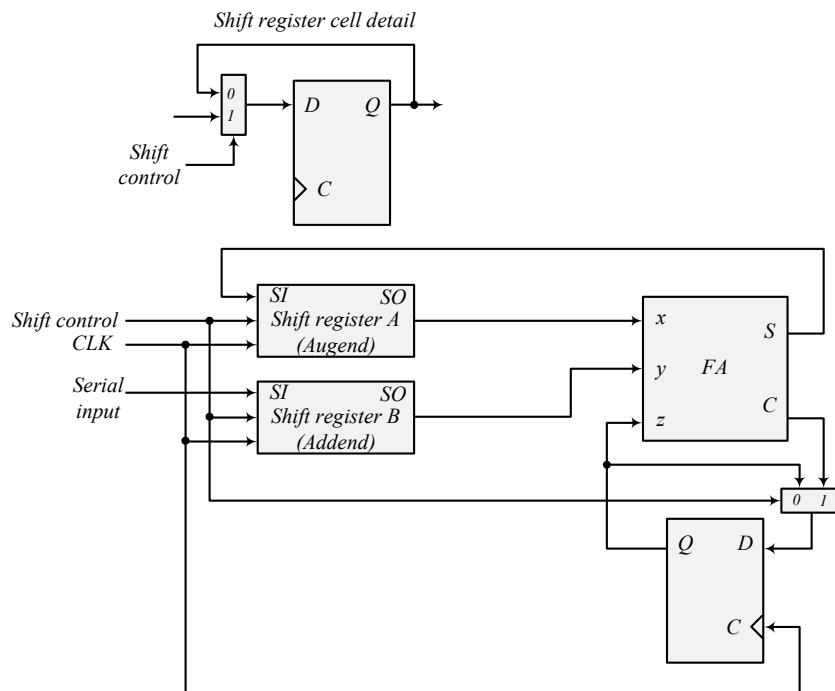
```

architecture Behavioral of D_FF is
begin
  process (Clock, reset_b) begin
    if (reset_b = '0') then Q <= '0';
    elsif Clock'event and Clock = '1' then
      Q <= D;
    end if;
  end process;
end Behavioral;

```


6.44

// See Figure 6.5
 // Note: Sum is stored in shift register A; carry is stored in Q
 // Note: Clear is active-low.

**Verilog**

```
module Prob_6_44_Str (output SO, input [7: 0] data_A, data_B, input S_in, load, Shift_control, CLK,
    Clear);
    supply0 gnd;
    wire sum, carry;
    assign SO = sum;
    wire SO_A, SO_B;
```

```
    Shift_Reg_gated_clock M_A (SO_A, sum, data_A, load, Shift_control, CLK, Clear);
    Shift_Reg_gated_clock M_B (SO_B, S_in, data_B, load, Shift_control, CLK, Clear);
    FA M_FA (carry, sum, SO_A, SO_B, Q);
    DFF_gated M_FF (Q, carry, Shift_control, CLK, Clear);
```

```
endmodule
```

```
module Shift_Reg_gated_clock (output SO, input S_in, input [7: 0] data, input load, Shift_control,
    Clock, reset_b);
    reg [7: 0] SReg;
    assign SO = SReg[0];
```

```
    always @ (posedge Clock, negedge reset_b)
        if (reset_b == 0) SReg <= 0;
        else if (load) SReg <= data;
        else if (Shift_control) SReg <= {S_in, SReg[7: 1]};
```

```
endmodule
```

```
module DFF_gated (output Q, input D, Shift_control, Clock, reset_b);
    DFF M_DFF (Q, D_internal, Clock, reset_b);
```

```

Mux_2 M_Mux (D_internal, Q, D, Shift_control);
endmodule

module DFF (output reg Q, input D, Clock, reset_b);
  always @ (posedge Clock, negedge reset_b)
    if (reset_b == 0) Q <= 0; else Q <= D;
endmodule

module Mux_2 (output reg y, input a, b, sel);
  always @ (a, b, sel) if (sel == 1) y = b; else y = a;
endmodule

module FA (output reg carry, sum, input a, b, C_in);
  always @ (a, b, C_in) {carry, sum} = a + b + C_in;
endmodule

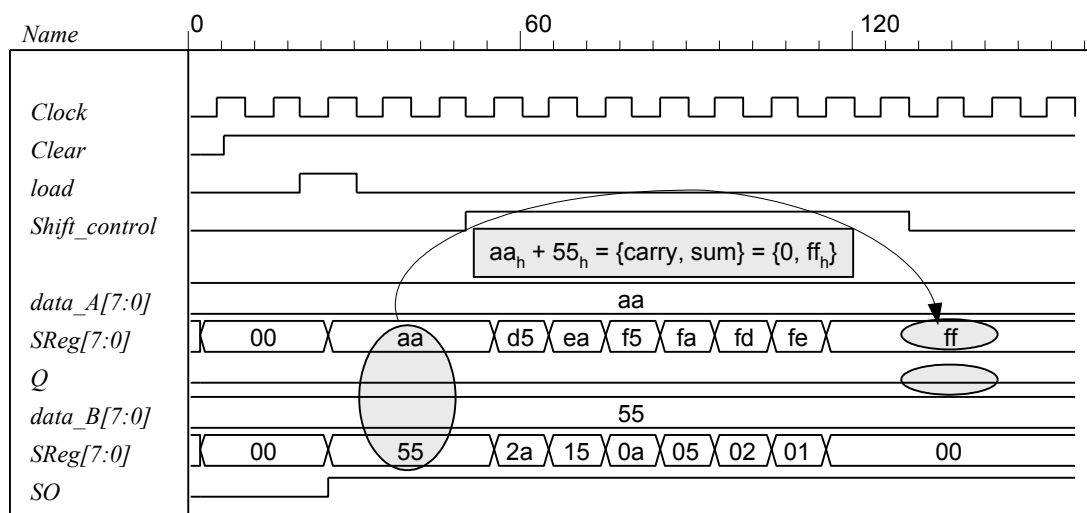
module t_Prob_6_44_Str ();
  wire SO;
  reg SI, load, Shift_control, Clock, Clear;
  reg [7: 0] data_A, data_B;

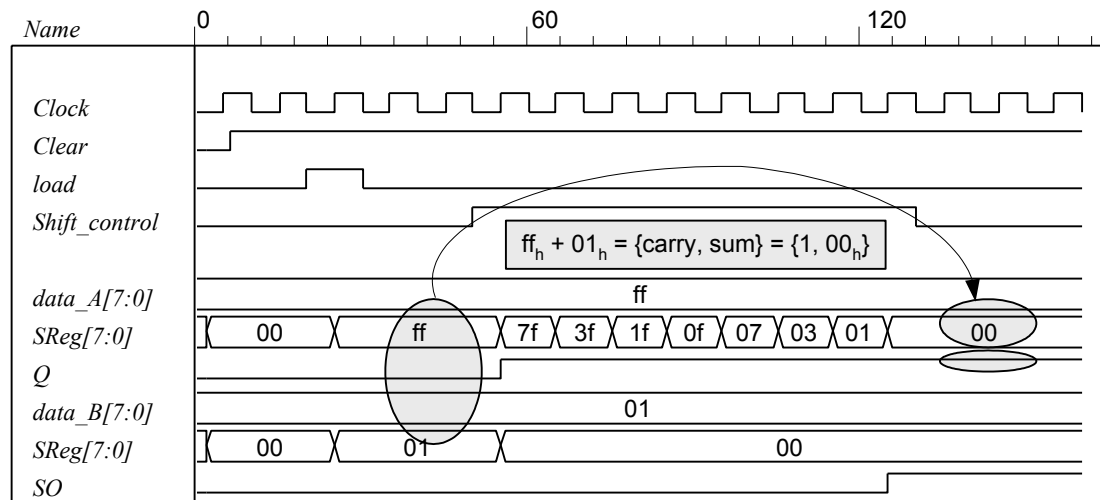
  Prob_6_44_Str M0 (SO, data_A, data_B, SI, load, Shift_control, Clock, Clear);

  initial #200 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial begin #2 Clear = 0; #4 Clear = 1; end

  initial fork
    data_A = 8'hAA; //8'h ff;
    data_B = 8'h55; //8'h01;
    SI = 0;
    #20 load = 1;
    #30 load = 0;
    #50 Shift_control = 1;
    #50 begin repeat (8) @ (posedge Clock) ;
      #5 Shift_control = 0;
    end
  join
endmodule

```





VHDL

```
entity Prob_6_44_Str_VHDL is
    port (SO: out bit; data_A, data_B: in bit_vector (7 downto 0); S_in, load, Shift_control, CLK,
        Clear: in bit);
end Prob_6_44_Str_VHDL;
```

```
architecture Structural of Prob_6_44_Str_VHDL is
    signal gnd: bit;
    signal sum, carry: bit;
    signal SO_A, SO_B, Q: bit;
    component Shift_Reg_gated_clock port (SO: out bit; S_in: in bit; data: in bit_vector (7 downto 0);
        load, Shift_control, Clock, reset_b: in bit); end component;
    component D_FF_gated port (Q: out bit; D, Shift_control, Clock, reset_b: in bit); end component;
    component FA port (carry, sum: out bit; a, b, C_in: in bit); end component;
    component Mux_2 port (y: out bit; a, b, sel: in bit); end component;
begin
    SO <= sum;

    M_A: Shift_Reg_gated_clock port map (SO_A, sum, data_A, load, Shift_control, CLK, Clear);
    M_B: Shift_Reg_gated_clock port map (SO_B, S_in, data_B, load, Shift_control, CLK, Clear);
    M_FA: FA port map (carry, sum, SO_A, SO_B, Q);
    M_DFF: D_FF_gated port map (Q, carry, Shift_control, CLK, Clear);
end Structural;
```

```
entity Shift_Reg_gated_clock is
    port (SO: out bit; S_in: in bit; data: in bit_vector (7 downto 0); load, Shift_control, Clock,
        reset_b: in bit);
end Shift_Reg_gated_clock;
```

```
architecture Behavioral of Shift_Reg_gated_clock is
    signal Sreg: bit_vector (7 downto 0);
begin
    SO <= Sreg(0);
    process (Clock, reset_b) begin
        if (reset_b = '0') then SReg <= "00000000";
        elsif Clock'event and Clock = '1' then
            if (load = '1') then SReg <= data;
            elsif (Shift_control = '1') then SReg <= S_in & Sreg(7 downto 1);
            end if;
        end if;
    end process;
end Behavioral;
```

```
entity D_FF_gated is
    port (Q: buffer bit ; D, Shift_control, Clock, reset_b: in bit);
end D_FF_gated;
```

```
architecture Structural of D_FF_gated is
    signal D_internal: bit;
    component D_FF port (Q: buffer bit; D: in bit; Clock, reset_b: in bit); end component;
    component Mux_2 port (y: out bit; a, b, sel: in bit); end component;
```

```
begin
    M_D_FF: D_FF port map (Q, D_internal, Clock, reset_b);
    M_Mux: Mux_2 port map (D_internal, Q, D, Shift_control);
end Structural;
```

```
entity D_FF is
    port (Q: out bit; D: in bit; Clock, reset_b: in bit);
end D_FF;
architecture Behavioral of D_FF is
begin
```

```

        process (Clock, reset_b) begin
            if (reset_b = '0') then Q <= '0';
            elsif Clock'event and Clock = '1' then Q <= D; end if;
        end process;
    end Behavioral;

    entity Mux_2 is
        port (y: out bit; a, b, sel: in bit);
    end Mux_2;

    architecture Behavioral of Mux_2 is
    begin
        process (a, b, sel) begin
            if (sel = '1') then y <= b;
            else y <= a;
            end if;
        end process;
    end Behavioral;

    entity FA is
        port (carry, sum: out bit; a, b, C_in: in bit);
    end FA;

    architecture Behavioral of FA is
    begin
        process (a, b, C_in) begin
            sum <= a xor b xor C_in;
            carry <= ((a xor b) and C_in) or (a and b);
        end process;
    end Behavioral;

```

6.45 **module** Prob_6_45 (**output reg** y_out, **input** start, clock, reset_bar);
parameter s0 = 4'b0000,
s1 = 4'b0001,
s2 = 4'b0010,
s3 = 4'b0011,
s4 = 4'b0100,
s5 = 4'b0101,
s6 = 4'b0110,
s7 = 4'b0111,
s8 = 4'b1000;
reg [3: 0] state, next_state;
always @ (posedge clock, negedge reset_bar)
if (!reset_bar) state <= s0; **else** state <= next_state;
always @ (state, start) begin
y_out = 1'b0;
case (state)
s0: **if** (start) next_state = s1; **else** next_state = s0;
s1: **begin** next_state = s2; y_out = 1; **end**
s2: **begin** next_state = s3; y_out = 1; **end**
s3: **begin** next_state = s4; y_out = 1; **end**
s4: **begin** next_state = s5; y_out = 1; **end**
s5: **begin** next_state = s6; y_out = 1; **end**
s6: **begin** next_state = s7; y_out = 1; **end**
s7: **begin** next_state = s8; y_out = 1; **end**
s8: **begin** next_state = s0; y_out = 1; **end**
default: next_state = s0;
endcase
end
endmodule

6.46 Verilog

```
module Prob_6_46 (output reg [0: 3] timer, input clk, reset_b);
```

```
  always @ (negedge clk, negedge reset_b)
```

```
  if (reset_b == 0) timer <= 4'b1000; else
```

```
  case (timer)
```

```
    4'b1000: timer <= 4'b0100;
```

```
    4'b0100: timer <= 4'b0010;
```

```
    4'b0010: timer <= 4'b0001;
```

```
    4'b0001: timer <= 4'b1000;
```

```
    default: timer <= 4'b1000;
```

```
  endcase;
```

```
endmodule
```

```
module t_Prob_6_46 ();
```

```
  wire [0: 3] timer;
```

```
  reg clk, reset_b;
```

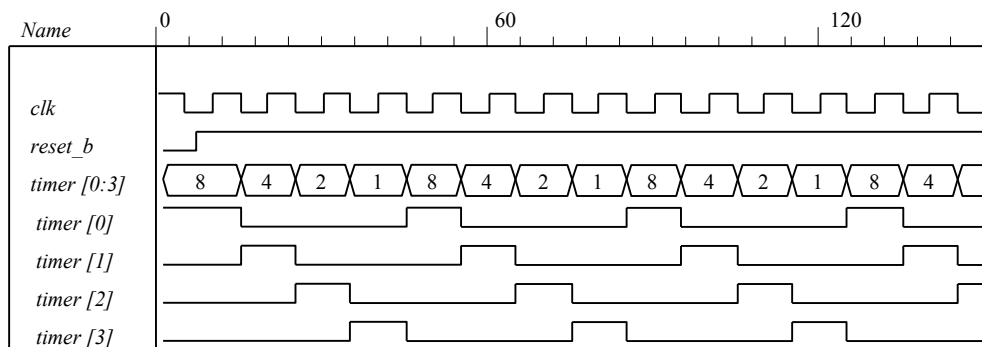
```
  Prob_6_46 M0 (timer, clk, reset_b);
```

```
  initial #150 $finish;
```

```
  initial fork #1 reset_b = 0; #7 reset_b = 1; join
```

```
  initial begin clk = 1; forever #5 clk = ~clk; end
```

```
endmodule
```



VHDL

State machine version:

```
entity Prob_6_46_sm_vhdl is
```

```
  port (timer: buffer bit_vector (0 to 3); count_enable, clock, reset_b: in bit);
```

```
end Prob_6_46_vhdl;
```

```
architecture Behavioral of Prob_6_46_vhdl is
```

```
  type State_type is (s0, s1, s2, s3, s4, s5);
```

```
  signal state, next_state: State_type;
```

```
begin
```

```
  process (clock, reset_b) begin
```

```
    if (reset_b = '0') then state <= s0;
```

```
    elsif clock'event and clock = '0' then state <= next_state;
```

```
    end if;
```

```
end process;
```

```

process (state, count_enable) begin
    next_state <= s0;
    case (state) is
        when s0 =>      if count_enable = '1' then next_state <= s1;
                       else next_state <= s0; end if;
        when s1 =>      if count_enable = '1' then next_state <= s2;
                       else next_state <= s1; end if;
        when s2 =>      if count_enable = '1' then next_state <= s3;
                       else next_state <= s2; end if;
        when s3 =>      if count_enable = '1' then next_state <= s4;
                       else next_state <= s3; end if;
        when s4 =>      if count_enable = '1' then next_state <= s5;
                       else next_state <= s4; end if;
        when s5 =>      if count_enable = '1' then next_state <= s0;
                       else next_state <= s5; end if;
        when others =>  next_state <= s0;
    end case;
end process;
process (state) begin
    case (state) is
        when s0 =>      timer (0 to 3) <= "0001";
        when s1 =>      timer (0 to 3) <= "1000";
        when s2 =>      timer (0 to 3) <= "0100";
        when s3 =>      timer (0 to 3) <= "0010";
        when s4 =>      timer (0 to 3) <= "0001";
        when s5 =>      timer (0 to 3) <= "0001";
        when others =>  timer (0 to 3) <= "0000";
    end case;
end process;
end Behavioral;

```

6.47 Verilog

```

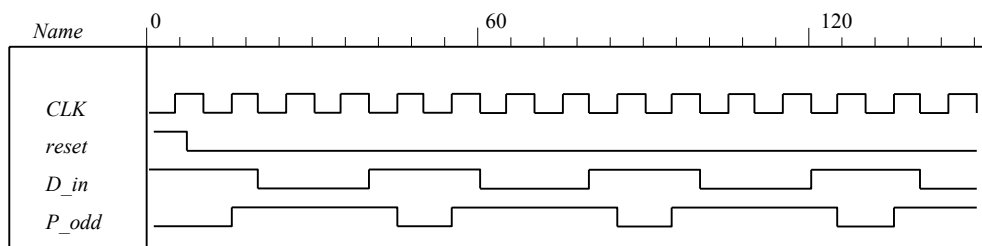
module Prob_6_47 (
    output reg P_odd,
    input D_in, CLK, reset
);
    wire D;
    assign D = D_in ^ P_odd;
    always @ (posedge CLK, posedge reset)
        if (reset) P_odd <= 0;
        else P_odd <= D;
endmodule

module t_Prob_6_47 ();
    wire P_odd;
    reg D_in, CLK, reset;

    Prob_6_47 M0 (P_odd, D_in, CLK, reset);

    initial #150 $finish;
    initial fork #1 reset = 1; #7 reset = 0; join
    initial begin CLK = 0; forever #5 CLK = ~CLK; end
    initial begin D_in = 1; forever #20 D_in = ~D_in; end
endmodule

```

VHDL

```
entity Prob_6_47_vhdl is
    port (P_odd: buffer bit; D_in, CLK, reset: in bit);
end Prob_6_47_vhdl;
```

```
architecture Behavioral of Prob_6_47_vhdl is
    signal D: bit;
begin
    D <= D_in xor P_odd;
    process (CLK, reset)
        if (reset = '1') then P_odd <= '0';
        elsif CLK'event and CLK = '1' then P_odd <= D;
        end if;
    end process;
end Behavioral;
```

6.48 (a) module Prob_6_48a (output reg [7: 0] count, input clk, reset_b);

```
reg [3: 0] state;
always @ (posedge clk, negedge reset_b)
    if (reset_b == 0) state <= 0; else state <= state + 1;
always @ (state)
    case (state)
        1, 3, 5, 7, 9, 11, 13: count = 8'b0000_0001;
        0: count = 8'b0000_0010;
        2: count = 8'b0000_0100;
        4: count = 8'b0000_1000;
        6: count = 8'b0001_0000;
        8: count = 8'b0010_0000;
        10: count = 8'b0100_0000;
        12: count = 8'b1000_0000;
        default: count = 8'b0000_0000;
    endcase
endmodule
```

(b) module Prob_6_48a (output reg [7: 0] count, input clk, reset_b);

```
reg [3: 0] state;
always @ (posedge clk, negedge reset_b)
    if (reset_b == 0) state <= 0; else state <= state + 1;
always @ (state)
    case (state)
        1, 3, 5, 7, 9, 11, 13: count = 8'b1000_0000;
        0: count = 8'b0100_0000;
        2: count = 8'b0010_0000;
        4: count = 8'b0001_0000;
        6: count = 8'b0000_1000;
        8: count = 8'b0000_0100;
        10: count = 8'b0000_0010;
        12: count = 8'b0000_0001;
        default: count = 8'b0000_0000;
    endcase
endmodule
```


6.49

Verilog

// Behavioral description of a 4-bit universal shift register

// Fig. 6.7 and Table 6.3

module Prob_6_49_Shift_Register_4_beh (// V2001, 2005

output reg [3: 0] A_par, // Register output

input [3: 0] I_par, // Parallel input

input s1, s0, // Select inputs

MSB_in, LSB_in, // Serial inputs

CLK, Clear // Clock and Clear

);

always @ (posedge CLK, negedge Clear) // V2001, 2005

if (~Clear) A_par <= 4'b0000;

else

case ({s1, s0})

2'b00: A_par <= A_par; // No change

2'b01: A_par <= {MSB_in, A_par[3: 1]}; // Shift right

2'b10: A_par <= {A_par[2: 0], LSB_in}; // Shift left

2'b11: A_par <= I_par; // Parallel load of input

endcase

endmodule

// Test plan:

// test reset action load

// test parallel load

// test shift right

// test shift left

// test circulation of data

// test reset on-the-fly

module t_Shift_Register_4_beh ();

reg s1, s0, // Select inputs

MSB_in, LSB_in, // Serial inputs

clk, reset_b; // Clock and Clear

reg [3: 0] I_par; // Parallel input

wire [3: 0] A_par; // Register output

Prob_6_49_Shift_Register_4_beh M0 (A_par, I_par, s1, s0, MSB_in, LSB_in, clk, reset_b);

initial #200 \$finish;

initial begin clk = 0; **forever** #5 clk = ~clk; **end**

initial fork

// test reset action load

#3 reset_b = 1;

#4 reset_b = 0;

#9 reset_b = 1;

// test parallel load

#10 I_par = 4'hA;

#10 {s1, s0} = 2'b11;

// test shift right

#30 MSB_in = 1'b0;

#30 {s1, s0} = 2'b01;

// test shift left

#80 LSB_in = 1'b1;

#80 {s1, s0} = 2'b10;

// test circulation of data

#130 {s1, s0} = 2'b11;

#140 {s1, s0} = 2'b00;

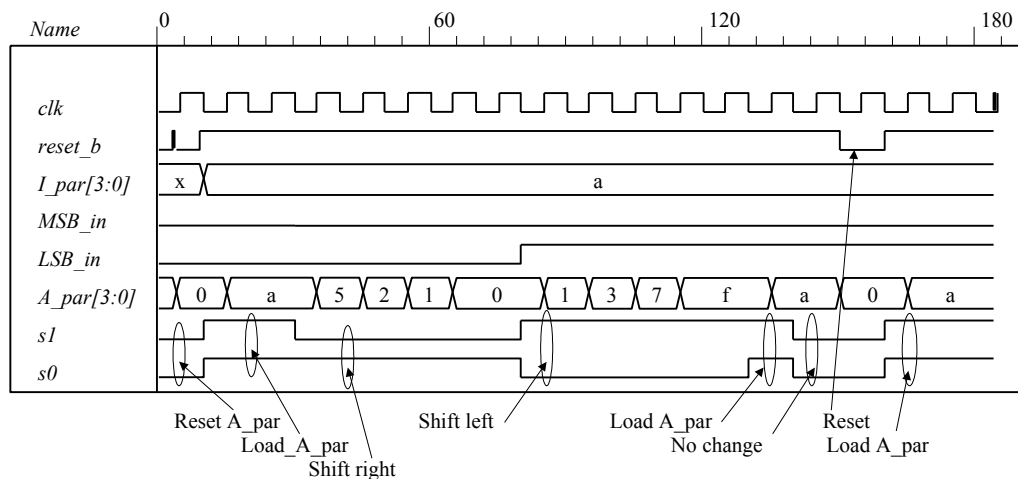
```

// test reset on the fly

#150 reset_b = 1'b0;
#160 reset_b = 1'b1;
#160 {s1, s0} = 2'b11;

join
endmodule

```



VHDL

-- Behavioral description of a 4-bit universal shift register

-- Fig. 6.7 and Table 6.3

entity Prob_6_49_vhdl is -- Shift_Register_4_beh_vhdl

port (A_par: buffer bit_vector (3 downto 0); I_par: in bit_vector (3 downto 0);
s1, s0, MSB_in, LSB_in, CLK, Clear: in bit);

end Shift_Register_4_beh_vhdl;

architecture Behavioral **of** Prob_6_49_Shift_Register_4_beh_vhdl **is**

begin

process (CLK, Clear) **begin**

if (Clear = '0') then A_par <= "0000";

elsif CLK'event and CLK = '1' then

case (s1 & s0) is

when "00" => A_par <= A_par;

-- No change

when "01" => A_par <= MSB_in & A_par(3 downto 1);

-- Shift right

when "10" => A_par <= A_par(2 downto 0) & LSB_in;

-- Shift left

when "11" => A_par <= I_par;

-- Parallel load

end case;

end if;

end process;

end Behavioral;

6.50 (a) `module Prob_6_50a (output reg [2: 0] count, input clk, reset_b);`
`always @ (posedge clk, negedge reset_b)`
`if (!reset_b) count <= 0;`
`else case (count)`
`3'd0: count <= 3'd4;`
`3'd1: count <= 3'd6;`
`3'd2: count <= 3'd1;`
`3'd4: count <= 3'd2;`
`3'd6: count <= 3'd0;`
`default: count <= 3'd0;`
`endcase`
`endmodule`

(b) `module Prob_6_50b (output reg [2: 0] count, input clk, reset_b);`
`always @ (posedge clk, negedge reset_b)`
`if (!reset_b) count <= 0;`
`else case (count)`
`3'd0: count <= 3'd2;`
`3'd1: count <= 3'd0;`
`3'd2: count <= 3'd6;`
`3'd3: count <= 3'd1;`
`3'd5: count <= 3'd3;`
`3'd6: count <= 3'd5;`
`default: count <= 3'd0;`
`endcase`
`endmodule`
`module t_Prob_6_50b;`
`wire [2: 0] count;`
`reg clock, reset_b ;`

6.51 Verilog

`module Prob_6_51_Seq_Detector (output detect, input bit_in, clk, reset_b);`
`reg [2: 0] sample_reg;`
`assign detect = (sample_reg == 3'b111);`
`always @ (posedge clk, negedge reset_b) if (reset_b == 0) sample_reg <= 0;`
`else sample_reg <= {bit_in, sample_reg [2: 1]};`
`endmodule`

`module Prob_5_45_Seq_Detector (output detect, input bit_in, clk, reset_b);`
`parameter S0 = 0, S1 = 1, S2 = 2, S3 = 3;`
`reg [1: 0] state, next_state;`

`assign detect = (state == S3);`
`always @ (posedge clk, negedge reset_b)`
`if (reset_b == 0) state <= S0; else state <= next_state;`

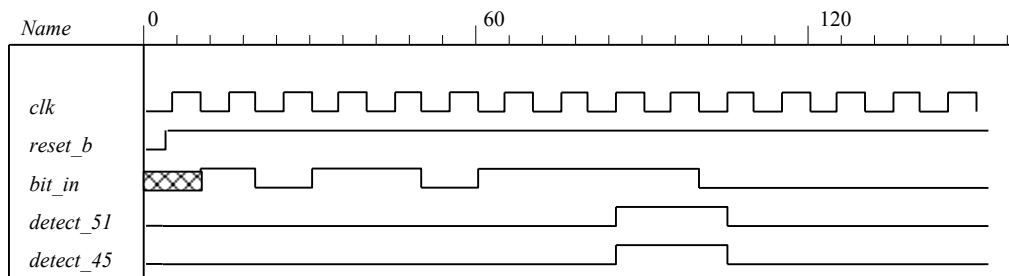
`always @ (state, bit_in) begin`
`next_state = S0;`
`case (state)`
`0: if (bit_in) next_state = S1; else state = S0;`
`1: if (bit_in) next_state = S2; else next_state = S0;`
`2: if (bit_in) next_state = S3; else state = S0;`
`3: if (bit_in) next_state = S3; else next_state = S0;`
`default: next_state = S0;`
`endcase`
`end`
`endmodule`


```
// Comparison
module t_Prob_6_51_Seq_Detector_ ();
  wire detect_45, detect_51;
  reg bit_in, clk, reset_b;

  Prob_5_51_Seq_Detector M0 (detect_51, bit_in, clk, reset_b);
  Prob_5_45_Seq_Detector M1 (detect_45, bit_in, clk, reset_b);

  initial #350$finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork

    reset_b = 0;
    #4 reset_b = 1;
    #10 bit_in = 1;
    #20 bit_in = 0;
    #30 bit_in = 1;
    #50 bit_in = 0;
    #60 bit_in = 1;
    #100 bit_in = 0;
  join
endmodule
```



The circuit using a shift register uses less hardware.

VHDL

```
entity Prob_6_51_Seq_Detector_vhdl is
  port (detect: out bit; bit_in, clk, reset_b: in bit);
end Prob_6_51_Seq_Detector_vhdl;

architecture Behavioral of Prob_6_51_Seq_Detector_vhdl is
  signal sample_reg: bit_vector (2 downto 0);
begin
  process (sample_reg) begin
    if sample_reg = '111' then detect <= '1'; else detect <= '0'; end if;
  end process;

  process (clk, reset_b) begin
    if (reset_b = '0') then sample_reg <= "000";
    elsif clk'event and clk = '1' then sample_reg <= bit_in & sample_reg (2 downto 1);
    end if;
  end process;
end Behavioral;
```

6.52 Universal Shift Register

Verilog

```
module Prob_6_52 (
  output [3:0] A_par,
```

```

input [3: 0] In_par,
input MSB_in, LSB_in,
input [1: 0] s1, s0,
input CLK, Clear_b
);
wire y0, y1, y2, y3;
Mux_4x1 M0 (y0, In_par[0], LSB_in, A_par[1], A_par[0], s1, s0);
Mux_4x1 M1 (y1, In_par[1], A_par[0], A_par[2], A_par[1], s1, s0);
Mux_4x1 M2 (y2, In_par[2], A_par[1], A_par[3], A_par[2], s1, s0);
Mux_4x1 M3 (y3, In_par[3], A_par[2], MSB_in, A_par[3], s1, s0);
DFF D0 (A_par[0], y0, CLK, Clear_b);
DFF D1 (A_par[1], y1, CLK, Clear_b);
DFF D2 (A_par[2], y2, CLK, Clear_b);
DFF D3 (A_par[3], y3, CLK, Clear_b);
endmodule

```

```

module Mux_4x1 (output reg y, input in3, in2, in1, in0, s1, s0);
  always @ (in3, in2, in1, in0, s1, s0)
    case ({s1, s0})
      2'b00: y = in0;
      2'b01: y = in1;
      2'b10: y = in2;
      2'b11: y = in3;
    endcase
endmodule

```

```

module DFF (output reg q, input d, clk, clr_b);
  always @ (posedge clk, negedge clr_b)
    if (clr_b == 1'b0) q <= 0; else q <= d;
endmodule

```

Features to be tested:

Action of Clear_b

Power-up initialization

On-the-fly

Action of mode controls

s1 s0

0 0 No change

0 1 Shift right

1 0 Shift left

1 1 Parallel load

```

module t_Problem_6_52 ();
  wire [3:0] A_par;
  reg [3: 0] In_par;
  reg MSB_in, LSB_in;
  reg s1, s0;
  reg CLK, Clear_b;
  reg [3:0] In_par;

  Prob_6_52 M0 (A_par, In_par, MSB_in, LSB_in, s1, s0, CLK, Clear_b);
  initial #300 $finish;
  initial begin CLK = 0, forever #5 CLK = ~CLK; end
  initial fork
    Clear_b = 0; // Power-up initialization
    #20 Clear_b = 1; // Running

    In_par = 4'b1010; // Word for parallel load
    MSB_in = 1'b1; // Bit for serial load
    LSB_in = 1'b0; // Bit for serial load
  endfork

```

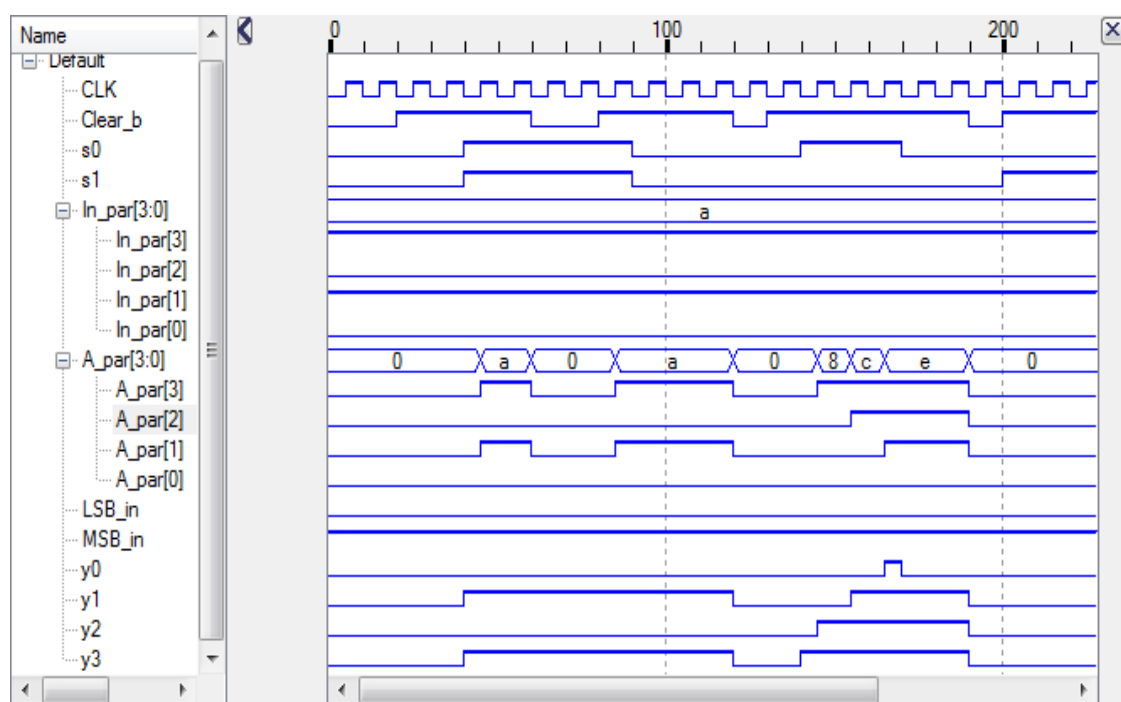
```

s1 = 0; s0 = 0;           // Initial action to no change
#40 begin s1 = 1; s0 = 1; end // parallel load
#60 Clear_b = 1'b0;       // Reset on-the-fly
#80 Clear_b = 1'b1;       // Resume action with parallel load at next clock edge
#90 begin s1 = 0; s0 = 0; end // No action – register holds 4'b1010

#120 Clear_b = 1'b0;      // Clear register
#130 Clear_b = 1'b1;
#140 begin s1 = 1'b0; s0 = 1'b1; end // Shifting to right (from MSB)
#170 begin s1 = 1'b0; s0 = 1'b0; end // Register should hold 4'b1111

#190 begin Clear_b = 1'b0; s1 = 1'b0; s0 = 1'b0; end
#200 begin Clear_b = 1'b1; s1 = 1'b1; s0 = 1'b0; end // Resume action – shift left
#230 begin s1 = 1'b0; s0 = 1'b0; end
join
endmodule

```



VHDL

entity Prob_6_52_vhdl is

port (A_par: **buffer** bit_vector (3 **downto** 0); In_par: **in** bit_vector (3 **downto** 0);
MSB_in, LSB_in: **in bit**; s1, s0 **downto** 0); CLK, Clear_b: **in bit**);

end Prob_6_52_vhdl;

architecture Structural **of** Prob_6_52_vhdl is

signal y0, y1, y2, y3: **bit**;

component Mux_4x1 **port** (y: **out bit**; in3, in2, in1, in0, s1, s0: **in bit**); **end component**;

component D_FF **port** (q: **out bit**; d, clk, clr_b: **in bit**); **end component**;

begin

M0: Mux_4x1 port map (y0, In_par(0), LSB_in, A_par(1), A_par(0), s1, s0);

M1: Mux_4x1 port map (y1, In_par(1), A_par(0), A_par(2), A_par(1), s1, s0);

M2: Mux_4x1 port map (y2, In_par(2), A_par(1), A_par(3), A_par(2), s1, s0);

M3: Mux_4x1 port map (y3, In_par(3), A_par(2), MSB_in, A_par(3), s1, s0);

D0: D_FF port map (A_par(0), y0, CLK, Clear_b);

D1: D_FF port map (A_par(1), y1, CLK, Clear_b);

D2: D_FF port map (A_par(2), y2, CLK, Clear_b);

```
D3: D_FF port map (A_par(3), y3, CLK, Clear_b);
end Structural;
```

```
entity Mux_4x1 is
    port (y: out bit; in3, in2, in1, in0, s1, s0: in bit);
end Mux_4x1;
```

```
architecture Behavioral of Mux_4x1 is
begin
    process (in3, in2, in1, in0, s1, s0) begin
        case (s1 & s0) is
            when "00" => y <= in0;
            when "01" => y <= in1;
            when "10" => y <= in2;
            when "11" => y <= in3;
        end case;
    end process;
end Behavioral;
```

```
entity D_FF is
    port (q: out bit; d, clk, clr_b: in bit);
end D_FF;
```

```
architecture Behavioral of D_FF is
begin
    process (clk, clr_b) begin
        if (clr_b = '0') then q <= '0'; else q <= d; end if;
    end process;
endmodule
```

6.53

Verilog

```
module Prob_6_53 (output reg [3:0] SR_A, input Shift_control, SI, CLK, Clear_b);
    reg [3: 0] SR_B;
    wire Sum, Carry;
    wire SO_A = SR_A[3];
    wire SO_B = SR_B[3];
    wire SI_A = Sum;
    wire SI_B = SI;
    wire Q;

    always @ (posedge CLK)
        if (Clear_b == 1'b0) SR_A <= 4'b0; else if (Shift_control) SR_A <= {Sum, SR_A[3:1]};

    always @ (posedge CLK)
        if (Clear_b == 1'b0) SR_B <= 4'b0; else if (Shift_control) SR_B <= {SI, SR_B[3:1]};

    FA M0 (Sum, Carry, SO_A, SO_B, Q);
    and (clk_to_DFF, CLK, Shift_control); // Caution: gated clock

    DFF M1 (Q, Carry, clk_to_DFF, Clear_b);
endmodule

module FA (output S, C, input x, y, z);
    assign {C, S} = x + y + z;
endmodule

module DFF (output reg Q, input D, C, Clear_b);
    always @ (posedge C) if (Clear_b == 1'b0) Q <= 1'b0; else Q <= D;
endmodule
```

```

module t_Prob_6_53 ();
  wire [3:0] SR_A;
  reg Shift_control, SI, CLK, Clear_b;

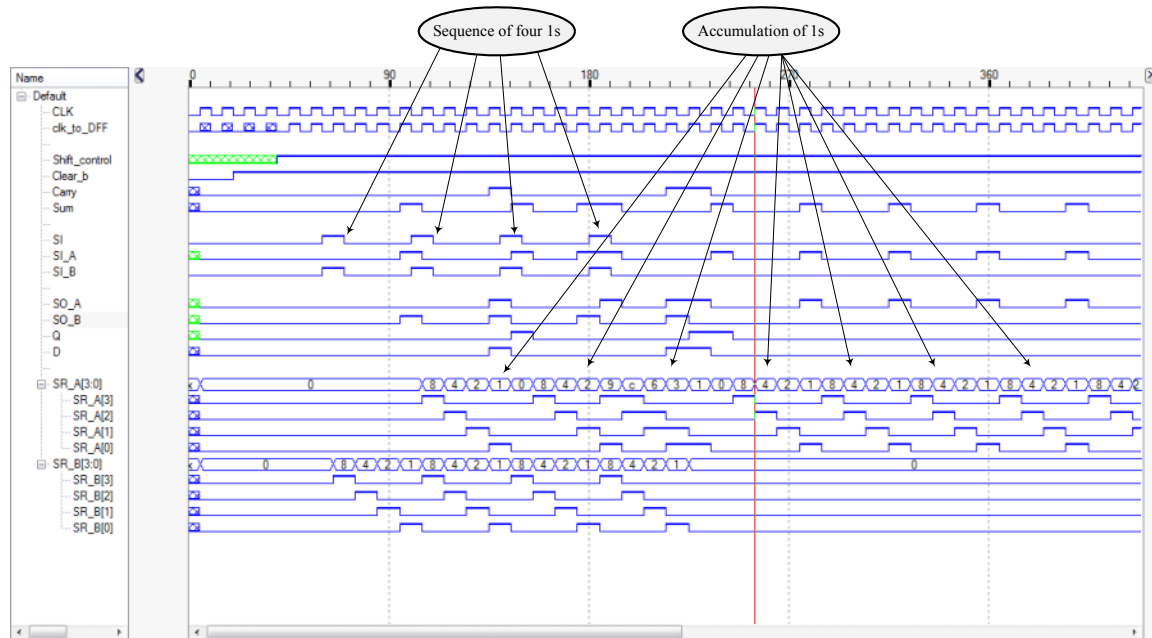
  Prob_6_53 M0 (SR_A, Shift_control, SI, CLK, Clear_b);
  initial #300 $finish;
  initial begin CLK = 0; forever #5 CLK = ~CLK; end
  initial fork
    Clear_b = 0;
    #20 Clear_b = 1;

    #40 Shift_control = 1;
    SI = 0;
  // Sequence of 1s
  /*
    #60 SI = 1;
    #70 SI = 0;
    #100 SI = 1;
    #110 SI = 0;
    #140 SI = 1;
    #150 SI = 0;
    #180 SI = 1;
    #190 SI = 0;
  */

  // Sequence of threes
    #60 SI = 1;
    #80 SI = 0;
    #100 SI = 1;
    #120 SI = 0;
    #140 SI = 1;
    #160 SI = 0;
    #180 SI = 1;
    #200 SI = 0;
  join
endmodule

```

Simulation results for accumulating a sequence of four 1s.



Simulation results for accumulating a sequence of four 1s.


```

    component FA port (carry, sum: out bit; a, b, C_in: in bit); end component;
    component D_FF port (Q: out bit; D, C, Clear_b: in bit); end component;
    component and2_gate port (y: out bit; xin1, xin2: in bit);
    end component;

begin

    SO_A <= SR_A(3);
    SO_B <= SR_B(3);
    SI_A <= Sum;
    SI_B <= SI;

    process (CLK) begin
        if CLK'event and CLK = '1' then
            if (Clear_b = '0') then SR_A <= "0000";
            elsif (Shift_control = '1') then SR_A <= Sum & SR_A(3 downto 1);
            end if;
        end if;
    end process;

    process (CLK) begin
        if CLK'event and CLK = '1' then
            if (Clear_b = '0') then SR_B <= "0000";
            else if (Shift_control = '1') then SR_B <= SI & SR_B(3 downto 1);
            end if;
        end if;
    end process;

    M0: FA port map (Carry, Sum, SO_A, SO_B, Q);
    M1: and2_gate port map (clk_to_DFF, CLK, Shift_control); -- Caution: gated clock
    M2: D_FF port map (Q, Carry, clk_to_DFF, Clear_b);
end Behavioral;

entity FA is
    port (carry, sum: out bit; a, b, C_in: in bit);
end FA;

architecture Behavioral of FA is
begin
    process (a, b, C_in) begin
        sum <= a xor b xor C_in;
        carry <= ((a xor b) and C_in) or (a and b);
    end process;
end Behavioral;

entity D_FF is
    port (Q: out bit; D, C, Clear_b: in bit);
end D_FF;

architecture Behavioral of D_FF is
begin
    process (C) begin
        if C'event and C = '1' then
            if (Clear_b = '0') then Q <= '0';
            else Q <= D;
            end if;
        end if;
    end process;
end Behavioral;

entity and2_gate is

```



```
port (y: out bit; xin1, xin2: in bit);  
end and2_gate;  
  
architecture Behavioral of and2_gate is  
begin  
    y <= xin1 and xin2;  
end Behavioral;
```

6.54--

Verilog

```

module Prob_6_54 (output reg [3:0] SR_A, input Shift_control, SI, CLK, Clear_b);
  reg [3:0] SR_B;
  wire S;
  wire Q;
  wire SI_A = S;
  wire SO_A = SR_A[0];
  wire SO_B = SR_B[0];
  wire SI_B = SI;
  wire J_in, K_in;
  and (J_in, SO_A, SO_B);
  nor (K_in, SO_A, SO_B);

  xor (S, SO_A, SO_B, Q);
  and (clk_to_JKFF, Shift_control, CLK);

  always @ (posedge CLK)
    if (Clear_b == 1'b0) SR_A <= 4'b0; else if (Shift_control) SR_A <= {SI_A, SR_A[3:1]};

  always @ (posedge CLK)
    if (Clear_b == 1'b0) SR_B <= 4'b0; else if (Shift_control) SR_B <= {SI_B, SR_B[3:1]};

  and (clk_to_JKFF, CLK, Shift_control);      // Caution: gated clock

  JKFF M1 (Q, J_in, K_in, clk_to_JKFF, Clear_b);
endmodule

module FA (output S, C, input x, y, z);
  assign {C, S} = x + y + z;
endmodule

module JKFF (buffer reg Q, input J_in, K_in, C, Clear_b);
  always @ (posedge C) if (Clear_b == 1'b0) Q <= 1'b0; else
    case ({J_in, K_in})
      2'b00:   Q <= Q;
      2'b01:   Q <= 1'b0;
      2'b10:   Q <= 1'b1;
      2'b11:   Q <= ~Q;
    endcase
endmodule

module t_Prob_6_54 ();
  wire [3:0] SR_A;
  reg Shift_control, SI, CLK, Clear_b;

  Prob_6_54 M0 (SR_A, Shift_control, SI, CLK, Clear_b);
  //initial #200 $finish;      // sequence of 1s
  initial #400 $finish;      // sequence of 3s
  initial begin CLK = 0; forever #5 CLK = ~CLK; end
  initial fork
    Clear_b = 0;
    #20 Clear_b = 1;

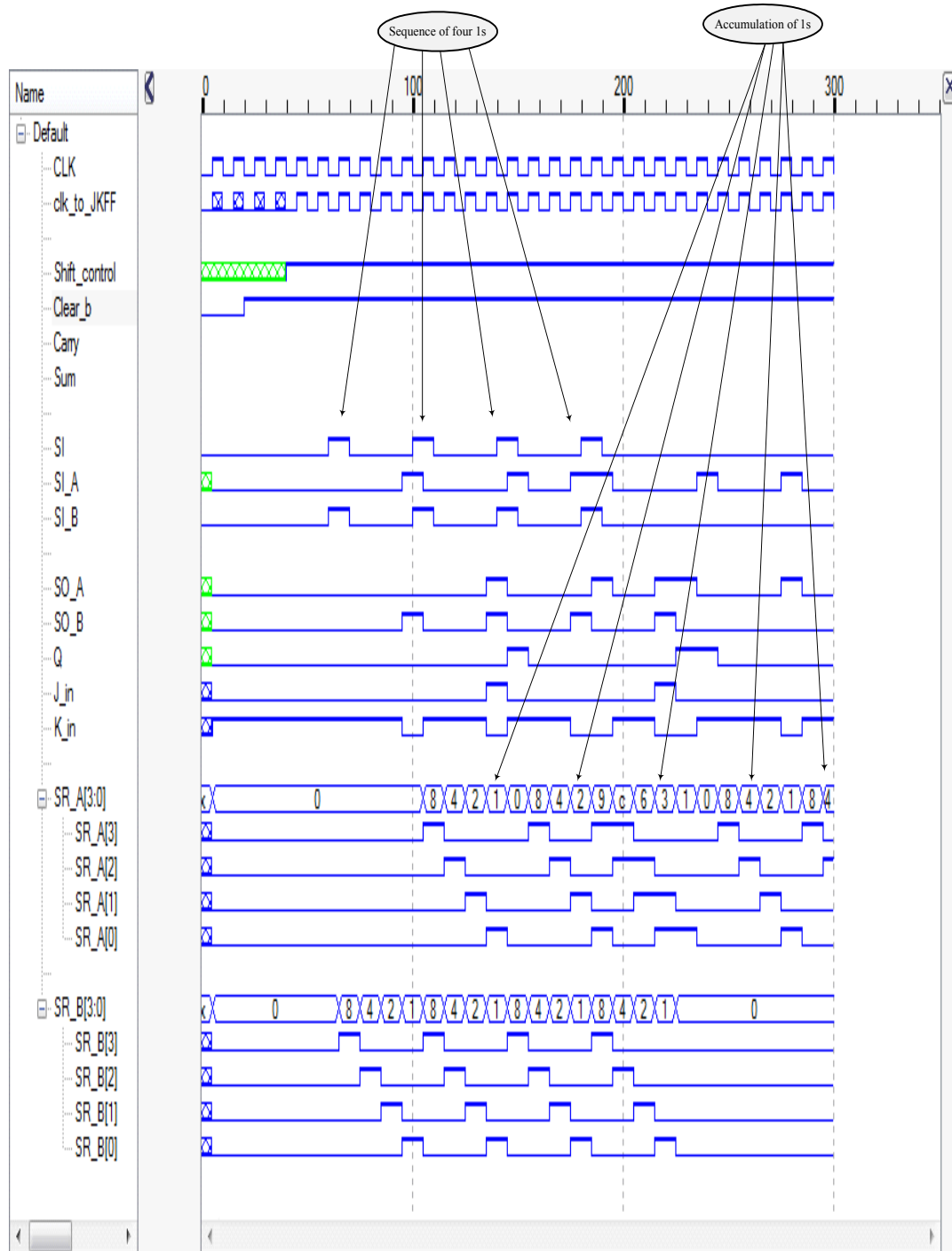
    #40 Shift_control = 1;
    SI = 0;
  // Sequence of 1s
  /*
    #60 SI = 1;
    #70 SI = 0;
    #100 SI = 1;
    #110 SI = 0;
    #140 SI = 1;
  */

```

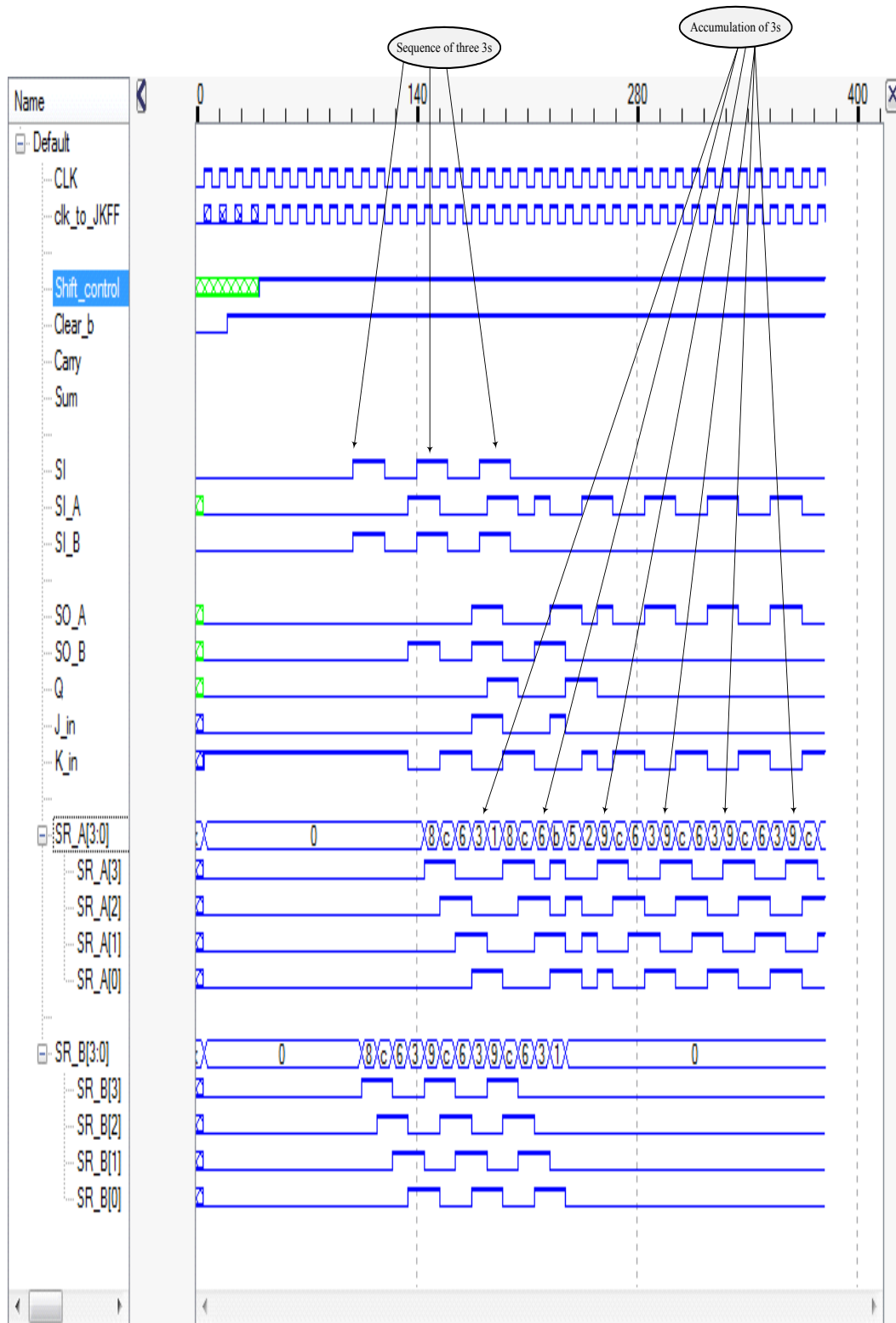
```
#150 SI = 0;
#180 SI = 1;
#190 SI = 0;
*/

// Sequence of threes
#60 SI = 1;
#80 SI = 0;
#100 SI = 1;
#120 SI = 0;
#140 SI = 1;
#160 SI = 0;
#180 SI = 1;
#200 SI = 0;
join
endmodule
```

Simulation results: Accumulation of a sequence of four 1s.



Accumulation of a sequence of 3s:



```

VHDL
--library IEEE;
--use IEEE.Std_Logic_1164.all;
entity Prob_6_54_vhdl is
    port (SR_A: buffer Bit_vector (3 downto 0); Shift_control, SI, CLK, Clear_b: in Bit);
end Prob_6_54_vhdl;

architecture Behavioral of Prob_6_54_vhdl is
    signal SR_B: Bit_vector (3 downto 0);
    signal S: Bit;
    signal Q: Bit;
    signal SI_A: Bit;
    signal SO_A: Bit;
    signal SO_B: Bit;
    signal SI_B: Bit;
    signal J_in, K_in: Bit;
    signal clk_to_JK_FF: bit;
    component and2_gate port (y: out Bit; xin1, xin2: in Bit); end component;
    component nor2_gate port (y: out Bit; xin1, xin2: in Bit); end component;
    component xor3_gate port (y: out Bit; xin1, xin2, xin3: in Bit); end component;
    component JK_FF port (Q: buffer Bit; J_in, K_in, C, Clear_b: in Bit); end component;

begin
    SI_A <= S;
    SO_A <= SR_A(0);
    SO_B <= SR_B(0);
    SI_B <= SI;
    G1: and2_gate port map (J_in, SO_A, SO_B);
    G2: nor2_gate port map (K_in, SO_A, SO_B);
    G3: xor3_gate port map (S, SO_A, SO_B, Q);
    G4: and2_gate port map (clk_to_JK_FF, Shift_control, CLK);
    G5: JK_FF port map (Q, J_in, K_in, clk_to_JK_FF, Clear_b);

    process (CLK) begin
        if CLK'event and CLK = '1' then
            if Clear_b = '0' then SR_A <= "0000";
            elsif (Shift_control = '1') then SR_A(3 downto 0) <= SI_A & SR_A(3 downto 1); end if;
            end if;
        end process;

    process (CLK) begin
        if (CLK'event and CLK = '1') then if (Clear_b = '0') then SR_B <= "0000";
        elsif (Shift_control = '1') then SR_B <= SI_B & SR_B(3 downto 1); end if;
        end if;
    end process;
end Behavioral;

entity FA is
    port (S, C: out Bit; x, y, z: in Bit);
end FA;

architecture Boolean_Eq of FA is
begin
    C <= ((x xor y) and z) or (x and y);
    S <= x xor y xor z;
end Boolean_Eq;

entity JK_FF is
    port (Q: buffer bit; J_in, K_in, C, Clear_b: in Bit);
end JK_FF;

architecture Behavioral of JK_FF is

```

```

begin
  process (C) begin
    if C'event and C = '1' then
      if (Clear_b = '0') then Q <= '0';
      else
        case (J_in & K_in) is
          when "00" => Q <= Q;
          when "01" => Q <= '0';
          when "10" => Q <= '1';
          when "11" => Q <= not Q;
        end case;
      end if;
    end if;_vhdl
  end process;
end Behavioral;

```

```

entity and2_gate is
  port (y: out Bit; xin1, xin2: in Bit);
end and2_gate;

```

```

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

```

entity nor3_gate is
  port (y: out Bit; xin1, xin2, xin3: in Bit);
end nor3_gate;

```

```

architecture Behavioral of nor3_gate is
begin
  y <= not( xin1 or xin2 or Xin3);
end Behavioral;

```

```

entity nor2_gate is
  port (y: out Bit; xin1, xin2: in Bit);
end nor2_gate;

```

```

architecture Behavioral of nor2_gate is
begin
  y <= xin1 nor xin2;
end Behavioral;

```

```

entity xor3_gate is
  port (y: out Bit; xin1, xin2, xin3: in Bit);
end xor3_gate;

```

```

architecture Behavioral of xor3_gate is
begin
  y <= ( xin1 xor xin2 xor Xin3);
end Behavioral;

```

6.55 Verilog

```

module Prob_6_55 (output Q8, Q4, Q2, Q1, input Count, Clear_b);
  supply1 Pwr;
  not (Q8_bar, Q8);
  and (J_in_M8, Q2, Q4);
  JKFF M1 (Q1, Pwr, Pwr, Count, Clear_b);
  JKFF M2 (Q2, Q8_bar, Pwr, Q1, Clear_b);
  JKFF M4 (Q4, Pwr, Pwr, Q2_Clear_b);
  JKFF M8 (Q8, J_in_M8, Pwr, Q1_Clear_b);

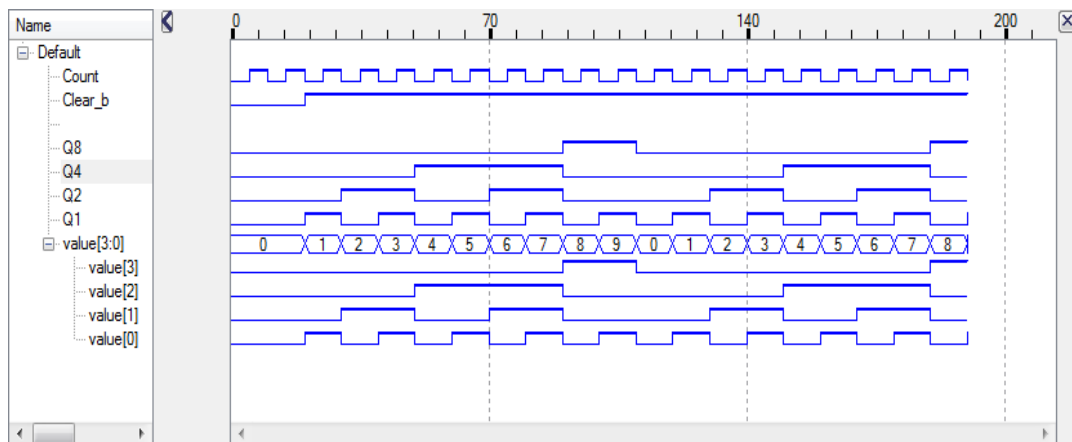
```

endmodule

```
module JKFF (output reg Q, input J_in, K_in, C, Clear_b);
always @ (negedge C) if (Clear_b == 1'b0) Q <= 1'b0; else
  case ({J_in, K_in})
    2'b00: Q <= Q;
    2'b01: Q <= 1'b0;
    2'b10: Q <= 1'b1;
    2'b11: Q <= ~Q;
  endcase
endmodule
```

```
module t_Prob_6_55 ();
wire Q8, Q4, Q2, Q1;
reg Count, Clear_b;
wire [3:0] value = {Q8, Q4, Q2, Q1}; // Display counter
Prob_6_55 M0 (Q8, Q4, Q2, Q1, Count, Clear_b);

initial #200 $finish;
initial begin Count = 0; forever #5 Count = ~Count; end
initial fork
  Clear_b = 0;
  #20 Clear_b = 1;
join
endmodule
```



Note: This problem illustrates the importance of defining a buffer port mode when a signal is both an output and is read internally. Failure to observe this requirement will result in wasted time debugging the code.

VHDL

```

entity Prob_6_55_vhdl is
  port (Q8, Q4, Q2, Q1: buffer Bit; Count, Clear_b: in Bit);
end Prob_6_55_vhdl;

architecture Behavioral of Prob_6_55_vhdl is
  signal Pwr: Bit := '1';
  signal Q8_bar: bit;
  signal J_in_M8: bit;
  component not_gate port (y: out Bit; xin: in Bit); end component;
  component and2_gate port (y: out Bit; xin1, xin2: in Bit); end component;
  component JK_FlipFlop port (Q: buffer Bit; J_in, K_in, C, Clear_b: in Bit); end component;
begin
  Pwr <= '1';
  G1: not_gate port map (Q8_bar, Q8);
  G2: and2_gate port map (J_in_M8, Q2, Q4);
  M1: JK_FlipFlop port map (Q1, Pwr, Pwr, Count, Clear_b);
  M2: JK_FlipFlop port map (Q2, Q8_bar, Pwr, Q1, Clear_b);
  M4: JK_FlipFlop port map (Q4, Pwr, Pwr, Q2, Clear_b);
  M8: JK_FlipFlop port map (Q8, J_in_M8, Pwr, Q1, Clear_b);
end Behavioral;

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

entity not_gate is
  port (y: out bit; xin: in bit);
end not_gate;

architecture Behavioral of not_gate is
begin
  y <= not xin;
end Behavioral;

entity JK_FlipFlop is
  port (Q: buffer Bit; J_in, K_in: in Bit; C: in Bit; Clear_b: in Bit);
end JK_FlipFlop;

architecture Behavioral of JK_FlipFlop is
begin
  process (C) begin
    if C'event and C = '0' then
      if (Clear_b = '0') then Q <= '0';
    else
      case (J_in & K_in) is
        when "00" => Q <= Q;
        when "01" => Q <= '0';
        when "10" => Q <= '1';
        when "11" => Q <= not Q;
      end case;
    end case;
  end process;

```

```

        end if;
        end if;
    end process;
end Behavioral;

entity Prob_6_55_vhdl_tb is -- Append tb for compatibility with compiler
    -- port ();
end Prob_6_55_vhdl_tb;

architecture Behavioral of Prob_6_55_vhdl_tb is
    signal t_Q8, t_Q4, t_Q2, t_Q1: bit;
    signal t_Count, t_Clear_b: bit;
    signal t_value: bit_vector (3 downto 0); -- Display counter
    component Prob_6_55_vhdl port (Q8, Q4, Q2, Q1: out bit; Count, Clear_b: in bit); end component;

begin
    t_value <= t_Q8 & t_Q4 & t_Q2 & t_Q1;
    M0: Prob_6_55_vhdl port map (t_Q8, t_Q4, t_Q2, t_Q1, t_Count, t_Clear_b);

    process
    begin
        t_count <= '0';
        wait for 5 ns;
        t_count <= '1';
        wait for 5 ns;
    end process;
    t_Clear_b <= '0';
    t_Clear_b <= '1' after 20 ns;

end Behavioral;

```

6.56 Verilog

```

module Prob_6_56 (A3, A2, A1, A0: buffer bit; Next_stage: out bit; Count_enable, CLK, Clear_b: in bit);
    assign Next_stage = A3 && A2 && A1 && A0;
    wire JK_in_M1, JK_in_M2, JK_in_M3,
    and (JK_in_M1, Count_enable, A0);
    and (JK_in_M2, JK_in_M1, A1);
    and (JK_in_M3, JK_in_M2, A2);
    // and (Next_stage, JK_in_M3, A3);
    JKFF M0 (A0, Count_enable, Count_enable, CLK, Clear_b);
    JKFF M1 (A1, JK_in_M1, J_in_M1, CLK, Clear_b);
    JKFF M2 (A2, JK_in_M2, JK_in_M2, CLK, Clear_b);
    JKFF M3 (A3, JK_in_M3, JK_in_M3, A3, CLK, Clear_b);
endmodule

module JKFF (output reg Q, input J_in, K_in, C, Clear_b);
    always @ (posedge C) if (Clear_b == 1'b0) Q <= 0; else
        case ({J_in, K_in})
            2'b00: Q <= Q;
            2'b01: Q <= 1'b0;
            2'b10: Q <= 1'b1;
            2'b11: Q <= ~Q;
        endcase
    endmodule

module t_Prob_6_56 ();

```

```

wire A3, A2, A1, A0;
wire Next_stage;
reg Count_enable;
reg CLK, Clear_b;

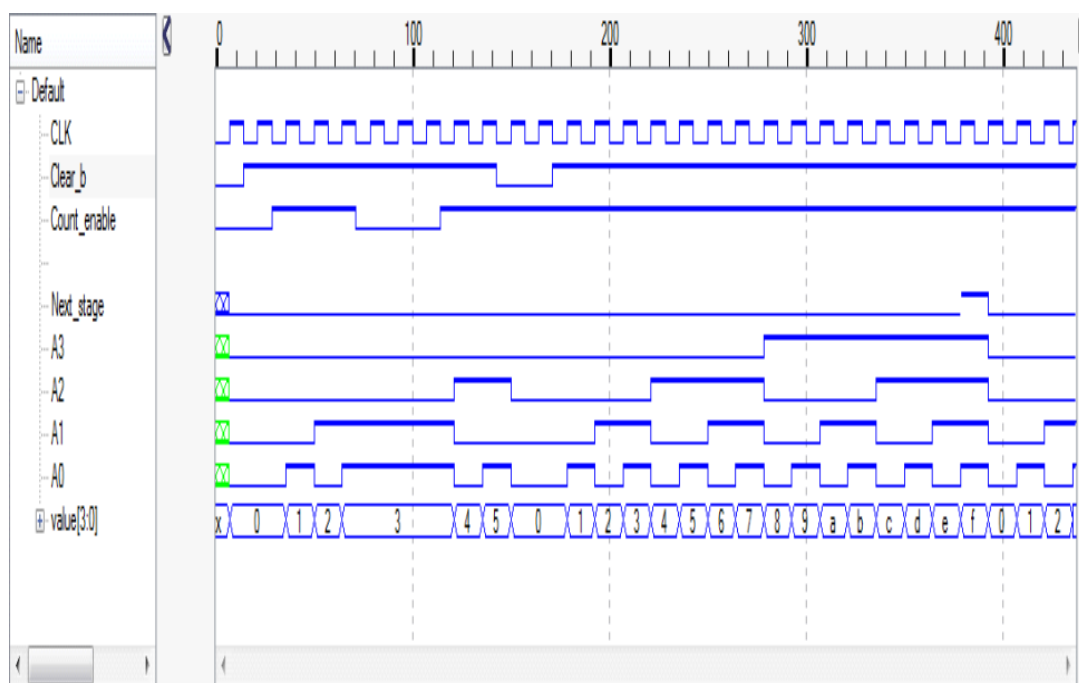
wire [3:0] value = {A3, A2, A1, A0};

Prob_6_56 M0 (A3, A2, A1, A0, Next_stage, Count_enable, CLK, Clear_b);

initial #400 $finish;
initial begin CLK = 0; forever #5 CLK = ~CLK; end
initial fork
    Clear_b = 0;
    #10 Clear_b = 1;
    #100 Clear_b = 0;    // Reset on the fly
    #120 Clear_b = 1;

    Count_enable = 0;
    #20 Count_enable = 1;
    #50 Count_enable = 0;
    #80 Count_enable = 1;
join
endmodule

```



VHDL

```

entity Prob_6_56_vhdl is
    port (A3, A2, A1, A0: buffer bit; next_stage: out bit; Count_enable, CLK, Clear_b: in bit);
end Prob_6_56_vhdl;

```

```

architecture Structural of Prob_6_56_vhdl is
    signal JK_in_M1, JK_in_M2, JK_in_M3: bit;
    component and2_gate port (y: out bit; xin1, xin2: in bit); end component;
    component JK_FF port (Q: buffer bit; J_in, K_in, C, Clear_b: in bit); end component;
begin

```

```

G0: and2_gate port map (JK_in_M1, Count_enable, A0);
G1: and2_gate port map (JK_in_M2, JK_in_M1, A1);
G2: and2_gate port map (JK_in_M3, JK_in_M2, A2);
G3: and2_gate port map (Next_stage, JK_in_M3, A3);
M0: JK_FF port map (A0, Count_enable, Count_enable, CLK, Clear_b);
M1: JK_FF port map (A1, JK_in_M1, JK_in_M1, CLK, Clear_b);
M2: JK_FF port map (A2, JK_in_M2, JK_in_M2, CLK, Clear_b);
M3: JK_FF port map (A3, JK_in_M3, JK_in_M3, CLK, Clear_b);
end Structural;

```

```

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;
architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

```

entity JK_FF is
  port (Q: buffer bit; J_in, K_in, C, Clear_b: in bit);
end JK_FF;
architecture Behavioral of JK_FF is
begin
  process (C) begin
    if C'event and C = '1' then
      if (Clear_b = '0') then Q <= '0';
    else
      case (J_in & K_in) is
        when "00" => Q <= Q;
        when "01" => Q <= '0';
        when "10" => Q <= '1';
        when "11" => Q <= not Q;
      end case;
    end if;
  end if;
end process;
end Behavioral;

```

```

entity Prob_6_56_vhdl_tb is
  -- port ();
end Prob_6_56_vhdl_tb;

```

```

architecture Behavioral of Prob_6_56_vhdl_tb is
  signal t_Q8, t_Q4, t_Q2, t_Q1: bit;
  signal t_next_stage, t_Count, t_CLK, t_Clear_b: bit;
  signal t_value: bit_vector (3 downto 0); -- Display counter
  component Prob_6_56_vhdl port (A3, A2, A1, A0: buffer bit; next_stage: out bit; Count_enable, CLK,
Clear_b: in bit); end component;
begin
  t_value <= t_Q8 & t_Q4 & t_Q2 & t_Q1;
  M0: Prob_6_56_vhdl port map (t_Q8, t_Q4, t_Q2, t_Q1, t_next_stage, t_Count, t_CLK, t_Clear_b);
  t_count <= '1' after 2 ns;
  t_count <= '0' after 7 ns;
  t_count <= '1' after 11 ns;
  process -- () "clock" for testbench
  begin
    t_CLK <= '0';
    wait for 5 ns;
    t_CLK <= '1';
    wait for 5 ns;
  end process;
end Behavioral;

```

6.57

Verilog

```

module Prob_6_57 (output A3, A2, A1, A0, input Up, Down, CLK, Clear_b);
    not (Up_bar, Up);
    not (A0_bar, A0);
    not (A1_bar, A1);
    not (A2_bar, A2);
    and (w1, Up_bar, Down);
    and (w2, w1, A0_bar);
    and (w3, Up, A0);
    and (w4, w2, A1_bar);
    and (w5, w3, A1);
    and (w6, w4, A2_bar);
    and (w7, w5, A2);
    or (T0, w1, Up);
    or (T1, w2, w3);
    or (T2, w4, w5);
    or (T3, w6, w7);
    TFF M0 (A0, A0_bar, T0, CLK, Clear_b);
    TFF M1 (A1, A1_bar, T1, CLK, Clear_b);
    TFF M2 (A2, A2_bar, T2, CLK, Clear_b);
    TFF M3 (A3, A3_bar, T3, CLK, Clear_b);
endmodule

module TFF (output reg Q, output Q_bar, input T, Clear_b, C, Clear_b); // Active low reset
    assign Q_bar = ~Q;
    always @ (posedge C) if (Clear_b == 1'b0) Q <= 0; else if (T) Q <= ~Q;
endmodule

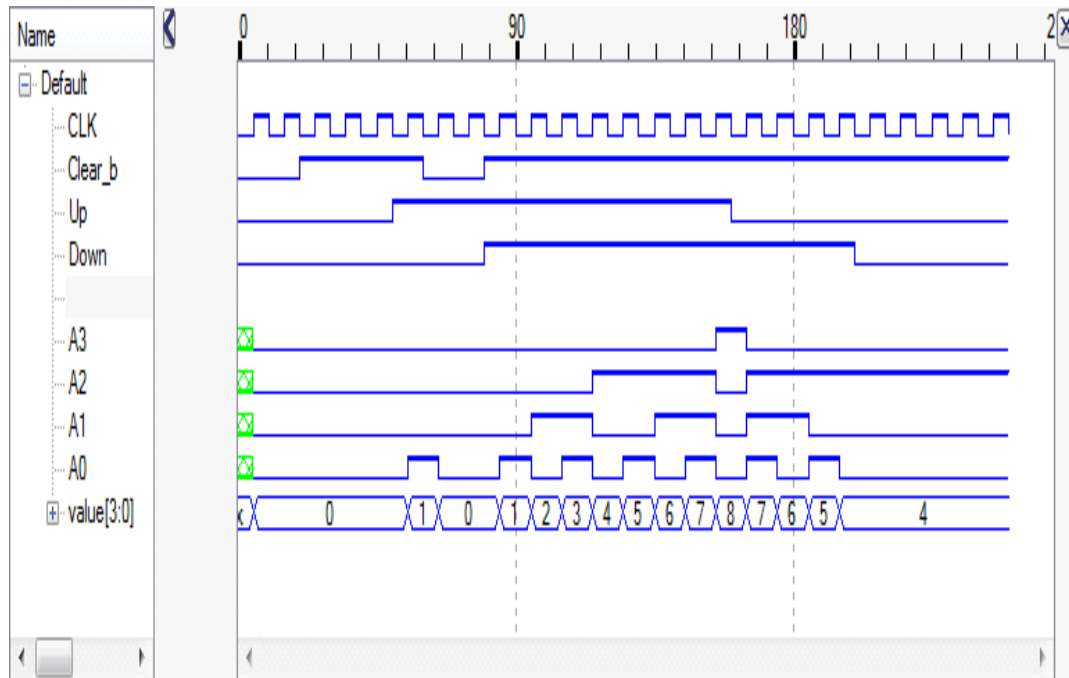
module t_Prob_6_57 ();
    wire A3, A2, A1, A0;
    reg Up, Down, CLK, Clear_b;
    wire [3:0] value = {A3, A2, A1, A0}; // Display count

    Prob_6_57 M0(A3, A2, A1, A0, Up, Down, CLK, Clear_b);

    initial #250 $finish;
    initial begin CLK = 0; forever #5 CLK = ~CLK; end
    initial fork
        Clear_b = 1'b0;
        #20 Clear_b = 1'b1;
        #60 Clear_b = 0; // Reset on the fly
        #80 Clear_b = 1;
        Up = 1'b0;
        Down = 1'b0;

        #50 Up = 1'b1;
        #80 Down = 1'b1; // Up has priority
        #160 Up = 1'b0;
        #200 Down = 1'b0;
    join
endmodule

```



VHDL

entity Prob_6_57_vhdl **is**

port (A3: out bit; A2, A1, A0: **buffer bit**; Up, Down, CLK, Clear_b: **in bit**);
end Prob_6_57_vhdl;

architecture Structural **of** Prob_6_57_vhdl **is**

component not_gate **port** (y: **out bit**; xin1: **in bit**); **end component**;
component and2_gate **port** (y: **out bit**; xin1, xin2: **in bit**); **end component**;
component or2_gate **port** (y: **out bit**; xin1, xin2: **in bit**); **end component**;
component T_FFLOP **port** (Q: **buffer bit**; Q_bar: **out bit**; T, C, Clear_b: **in bit**); **end component**;
signal Up_bar, A0_bar, A1_bar, A2_bar, A3_bar, w1, w2, w3, w4, w5, w6, w7: **bit**;
signal T0, T1, T2, T3: **bit**;
begin
 M0: not_gate **port map** (Up_bar, Up);
 M1: and2_gate **port map** (w1, Up_bar, Down);
 M2: and2_gate **port map** (w2, w1, A0_bar);
 M3: and2_gate **port map** (w3, Up, A0);
 M4: and2_gate **port map** (w4, w2, A1_bar);
 M5: and2_gate **port map** (w5, w3, A1);
 M6: and2_gate **port map** (w6, w4, A2_bar);
 M7: and2_gate **port map** (w7, w5, A2);
 M8: or2_gate **port map** (T0, w1, Up);
 M9: or2_gate **port map** (T1, w2, w3);
 M10: or2_gate **port map** (T2, w4, w5);
 M11: or2_gate **port map** (T3, w6, w7);
 M12: T_FFlop **port map** (A0, A0_bar, T0, CLK, Clear_b);
 M13: T_FFlop **port map** (A1, A1_bar, T1, CLK, Clear_b);
 M14: T_FFlop **port map** (A2, A2_bar, T2, CLK, Clear_b);
 M15: T_FFlop **port map** (A3, A3_bar, T3, CLK, Clear_b);
end Structural;

entity T_Flop **is**

port (Q, Q_bar: **buffer bit**; T, C, Clear_b: **in bit**); -- Active-low, synchronous reset
end T_Flop;

architecture Behavioral of T_FFLOP is
begin

```

    Q_bar <= not Q;
    process (C) begin
        if C'event and C = '1' then
            if (Clear_b = '0') then Q <= '0';
            elsif (T = '1') then Q <= Q_bar;
            end if;
        end if;
    end process;

```

end Behavioral;

entity and2_gate is

```

    port (y: out bit; xin1, xin2: in bit);

```

end and2_gate;

architecture Behavioral of and2_gate is
begin

```

    y <= xin1 and xin2;

```

end Behavioral;

entity or2_gate is

```

    port (y: out bit; xin1, xin2: in bit);

```

end or2_gate;

architecture Behavioral of or2_gate is

begin y <= xin1 or xin2;

end Behavioral;

entity not_gate is

```

    port (y: out bit; xin1: in bit);

```

end not_gate;

architecture Behavioral of not_gate is

begin

```

    y <= not xin1;

```

end Behavioral;

6.58 (a) Structural Model

Verilog

```

module Problem_6_58 (output A3, A2, A1, A0, C_out, input I3, I2, I1, I0, Count, Load, CLK, Clear_b);
    not (Load_bar, Load);
    not (I0_bar, I0);
    not (I1_bar, I1);
    not (I2_bar, I2);
    not (I3_bar, I3);
    and (w0, Count, Load_bar);
    and (w1, Load, I0);
    and (w2, Load, I0_bar);
    and (w3, Load, I1);
    and (w4, Load, I1_bar);
    and (w5, Load, I2);
    and (w6, Load, I2_bar);
    and (w7, Load, I3);
    and (w8, Load, I3_bar);
    or (w9, w1, w0);
    or (w10, w2, w0);
    or (w11, w3, w17);
    or (w12, w4, w17);
    or (w13, w5, w18);
    or (w14, w6, w18);
    or (w15, w7, w19);
    or (w16, w8, w19);
    and (w17, w0, A0);
    and (w18, w0, A0, A1);
    and (w19, w0, A0, A1, A2);
    and (C_out, w0, A0, A1, A2, A3);

    JKFF M0 (A0, w9, w10, CLK, Clear_b);
    JKFF M1 (A1, w11, w12, CLK, Clear_b);
    JKFF M2 (A2, w13, w14, CLK, Clear_b);
    JKFF M3 (A3, w15, w16, CLK, Clear_b);
endmodule

```

```

module JKFF (output reg Q, input J_in, K_in, C, Clear_b);
    always @ (posedge C) if (Clear_b == 1'b0) Q <= 0; else
        case ({J_in, K_in})
            2'b00:    Q <= Q;
            2'b01:    Q <= 1'b0;
            2'b10:    Q <= 1'b1;
            2'b11:    Q <= ~Q;
        endcase
endmodule

```

```

module t_Problem_6_58 ();
    wire A3, A2, A1, A0, C_out;
    reg I3, I2, I1, I0, Count, Load, CLK, Clear_b;
    wire [3:0] value = {A3, A2, A1, A0};
    wire [3:0] Par_word = {I3, I2, I1, I0};

    Problem_6_58 M0 (A3, A2, A1, A0, C_out, I3, I2, I1, I0, Count, Load, CLK, Clear_b);

```

```

    initial #400 $finish;
    initial begin CLK = 0; forever #5 CLK = ~CLK; end
    initial fork
        {I3, I2, I1, I0} = 4'b0101; // Data for parallel load
        Clear_b = 0;

```



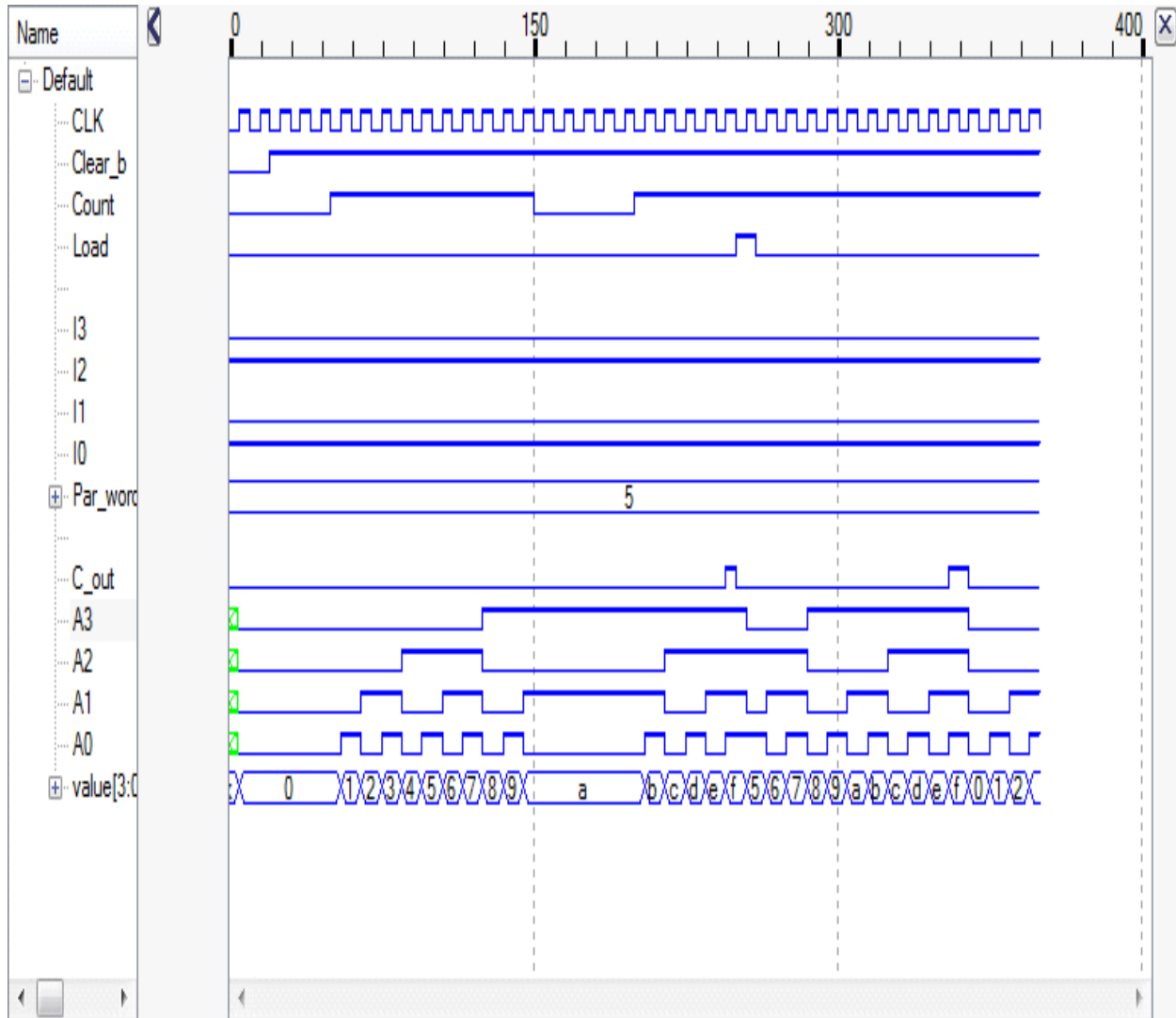
```

#20 Clear_b = 1;

Count = 0;
#50 Count = 1;    // Counting
#150 Count = 0;   // Pause
#200 Count = 1;   // Resume counting
Load = 0;
#250 Load = 1; // Parallel load
#260 Load = 0;

join
endmodule

```



VHDL – Structural Model

entity Prob_6_58_vhdl **is**

port (A3, A2, A1, A0: **buffer bit**; C_out: **out bit**; I3, I2, I1, I0, Count, Load, CLK, Clear_b: **in bit**);
end Prob_6_58_vhdl;

architecture Structural of Prob_6_58_vhdl **is**

signal Load_bar, I0_bar, I1_bar, I2_bar, I3_bar, Clear_bar: **bit**;
signal w0, w1, w2, w3, w4, w5, w6, w7, w8, w9, w10: **bit**;

```

signal w11, w12, w13, w14, w15, w16, w17, w18, w19: bit;
component not_gate port (y: out bit; xin1: in bit); end component;
component and2_gate port (y: out bit; xin1, xin2: in bit); end component;
component and3_gate port (y: out bit; xin1, xin2, xin3: in bit); end component;
component and4_gate port (y: out bit; xin1, xin2, xin3, xin4: in bit); end component;
component and5_gate port (y: out bit; xin1, xin2, xin3, xin4, xin5: in bit); end component;
component or2_gate port (y: out bit; xin1, xin2: in bit); end component;
component JK_FF port (Q: buffer bit; J_in, K_in, C, Clear_b: in bit); end component;
begin
  G0: not_gate port map(Load_bar, Load);
  G1: not_gate port map(I0_bar, I0);
  G2: not_gate port map(I1_bar, I1);
  G3: not_gate port map(I2_bar, I2);
  G4: not_gate port map(I3_bar, I3);
  G5: and2_gate port map(w0, Count, Load_bar);
  G6: and2_gate port map(w1, Load, I0);
  G7: and2_gate port map(w2, Load, I0_bar);
  G8: and2_gate port map(w3, Load, I1);
  G9: and2_gate port map(w4, Load, I1_bar);
  G10: and2_gate port map(w5, Load, I2);
  G11: and2_gate port map(w6, Load, I2_bar);
  G12: and2_gate port map(w7, Load, I3);
  G13: and2_gate port map(w8, Load, I3_bar);
  G14: or2_gate port map( w9, w1, w0);
  G15: or2_gate port map( w10, w2, w0);
  G16: or2_gate port map( w11, w3, w17);
  G17: or2_gate port map( w12, w4, w17);
  G18: or2_gate port map( w13, w5, w18);
  G19: or2_gate port map( w14, w6, w18);
  G20: or2_gate port map( w15, w7, w19);
  G21: or2_gate port map( w16, w8, w19);
  G22: and2_gate port map(w17, w0, A0);
  G23: and3_gate port map(w18, w0, A0, A1);
  G24: and4_gate port map(w19, w0, A0, A1, A2);
  G25: and5_gate port map(C_out, w0, A0, A1, A2, A3);
  M0: JK_FF port map(A0, w9, w10, CLK, Clear_b);
  M1: JK_FF port map(A1, w11, w12, CLK, Clear_b);
  M2: JK_FF port map(A2, w13, w14, CLK, Clear_b);
  M3: JK_FF port map(A3, w15, w16, CLK, Clear_b);
end Structural;

entity JK_FF is
  port (Q: buffer bit; J_in, K_in, C, Clear_b: in bit);
end JK_FF;

architecture Behavioral of JK_FF is
begin
  process (C) begin
    if C'event and C = '1' then
      if (Clear_b = '0' ) then Q <= '0';
    else
      case (J_in & K_in) is
        when "00" =>    Q <= Q;
        when "01" =>    Q <= '0';
        when "10" =>    Q <= '1';
        when "11" =>    Q <= not Q;
      end case;
    end if;
  end if;
  end process;
end Behavioral;

```

```

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;

```

```

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

```

entity and3_gate is
  port (y: out bit; xin1, xin2, xin3: in bit);
end and3_gate;

```

```

architecture Behavioral of and3_gate is
begin
  y <= xin1 and xin2 and xin3;
end Behavioral;

```

```

entity and4_gate is
  port (y: out bit; xin1, xin2, xin3, xin4: in bit);
end and4_gate;

```

```

architecture Behavioral of and4_gate is
begin
  y <= xin1 and xin2 and xin3 and xin4;
end Behavioral;

```

```

entity and5_gate is
  port (y: out bit; xin1, xin2, xin3, xin4, xin5: in bit);
end and5_gate;

```

```

architecture Behavioral of and5_gate is
begin
  y <= xin1 and xin2 and xin3 and xin4 and xin5;
end Behavioral;

```

```

entity or2_gate is
  port (y: out bit; xin1, xin2: in bit);
end or2_gate;

```

```

architecture Behavioral of or2_gate is
begin
  y <= xin1 or xin2;
end Behavioral;

```

```

entity not_gate is
  port (y: out bit; xin1: in bit);
end not_gate;

```

```

architecture Behavioral of not_gate is
begin
  y <= not xin1;
end Behavioral;

```

6.58 (B) Verilog - Behavioral Model

```

module Prob_6_58b (output reg[3:0] A_count, output C_out,
  input [3:0] Data_in, input Count, Load, CLK, Clear_b);
  wire A3 = A_count [3];
  wire A2 = A_count [2];
  wire A1 = A_count [1];

```

```

wire A0 = A_count [0];
wire Load_bar = !Load;
assign C_out = A3 && A2 && A1 && A0 && (Count && Load_bar);

always @ (posedge CLK, negedge Clear_b)
if (Clear_b == 1'b0) A_count <= 4'b0000;
else if (Load == 1'b1) A_count <= Data_in;
else if (Count == 1'b1) A_count <= A_count + 1'b1;
endmodule // Also see HDL Example 6.3)

```

6.58 (B) VHDL - Behavioral Model

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_6_58b_vhdl is
  port (A_count: buffer unsigned (3 downto 0); C_out: out Std_Logic;
        Data_in: in unsigned (3 downto 0); Count, Load, CLK, Clear_b: in bit);
end Prob_6_58b_vhdl;

architecture Behavioral of Prob_6_58b_vhdl is
  signal Load_bar: bit;
begin
    Load_bar <= not Load;

    C_out <= '1' when A_count = 15 and Count = '1' and Load_bar = '1';

    process (CLK, Clear_b)
      if (Clear_b = "0") then A_count <= "0000";
      elsif CLK'event and CLK = '1' then
        if (Load = '1') then A_count <= Data_in;
        elsif (Count = '1') then A_count <= A_count + 1;
        else A_count <= A_count;
      end if;
    end process;
end Behavioral;

```

Also see HDL Example 6.3.

6.59 (a) Verilog

Circuit is modified for gated clock controlled by *count*.

```

module Prob_6_59a (out [3:0] STRC, in [3:0] I_par, in Count, Load, CLK, Clear_b);
  wire D_A, D_B, D_C, D_E;
  wire E_bar = !STRC[0];
  mux_2x1 mux_A (D_A, STRC[3], E_bar, count);
  D_FF D_FF_A (STRC[3], D_A, clk, clr_bar);
  mux_2x1 mux_B (D_B, STRC[2], STRC[3], count);
  D_FF D_FF_B (STRC[2], D_B, clk, clr_bar);
  mux_2x1 mux_C (D_C, STRC[1], STRC[2], count);
  D_FF D_FF_B (STRC[1], D_C, clk, clr_bar);
  mux_2x1 mux_C (D_E, STRC[0], STRC[1], count);
  D_FF D_FF_B (STRC[0], D_E, clk, clr_bar);
endmodule

module D_FF (output Q, input D, clk, clr_bar);
  always @ (posedge clk, negedge clr_bar)
    if clr_bar == 1'b0) Q <= 1'b0;
    else Q <= D;
endmodule;

module mux_2x1 (out y, in xin1, xin2, sel);
  always @ (xin1, xin2, sel)
    if (sel == 1'b0) y = xin1;
    else if (sel == 1'b1) y = xin2;
endmodule;

```

(a) VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;

entity Prob_6_59a_vhdl is
  port (STRC: buffer Std_Logic_vector (3 downto 0); I_par: in Std_Logic_vector (3 downto 0);
    Count, Load, CLK, clr_b: in bit);
end Prob_6_59a_vhdl;

architecture Structural of Prob_6_59a_vhdl is
  signal E_bar: Std_Logic;
  component D_FF port (Q: out Std_Logic; D: in Std_Logic; clk, clr_bar: in bit); end component;
begin
  E_bar <= not STRC(0);
  M_A: D_FF port map (Q => STRC(3), D => E_bar, clk => clk, clr_bar => clr_bar);
  M_B: D_FF port map (Q => STRC(2), D => STRC(3), clk => clk, clr_bar => clr_bar);
  M_C: D_FF port map (Q => STRC(1), D => STRC(2), clk => clk, clr_bar => clr_bar);
  M_E: D_FF port map (Q => STRC(0), D => STRC(1), clk => clk, clr_bar => clr_bar);
end Structural;

entity D_FF is
  port (Q: out Std_Logic; D: in Std_Logic; clk, clr_bar: in bit);
end D_FF;
architecture Behavioral of D_FF is
begin
  process (clk, clr_bar) begin
    if clr_bar = '0' then Q <= '0';
    elsif clk'event and clk = '1' then Q <= D;
    end if;
  end process;
end Behavioral;

```

(b) Verilog

```

module Prob_6_59b_ver (output reg [3:0] STRC, input [3:0] I_par, input Count, Load, CLK, Clear_b);
  wire E_bar = !STRC[0];
  always @ (posedge CLK) if (Clear_b == 1'b0) STRC <= 4'b0; else
    if (Load) STRC <= I_par
    else if (Count) STRC <= {E_bar, STRC[3:1]};
endmodule

```

```

module t_Prob_6_59b_ver ();
wire [3:0] t_STRC;
reg t_Load, t_Count, t_CLK, t_Clear_b;
reg [3:0] t_I_par;

```

```

  Prob_6_59b_ver M0 (t_STRC, t_I_par, t_Count, t_Load, t_CLK, t_Clear_b);

```

```

initial #400 $finish;
initial begin t_CLK = 0; forever #5 t_CLK = ~t_CLK; end
initial fork
  t_I_par = 4'b0101; // Data for parallel load
  t_Clear_b = 0;
  #20 t_Clear_b = 1;

  t_Count = 0;
  #50 t_Count = 1;      // Counting
  #150 t_Count = 0;    // Pause
  #200 t_Count = 1;    // Resume counting
  t_Load = 0;
  #250 t_Load = 1;     // Parallel load
  #260 t_Load = 0;
join
endmodule

```

(b) VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

```

```

entity Prob_6_59b_vhdl is
  port (STRC: buffer Std_Logic_vector (3 downto 0); I_par: in Std_Logic_vector (3 downto 0);
    Count, Load, CLK, Clear_b: in bit);
end Prob_6_59b_vhdl;

```

```

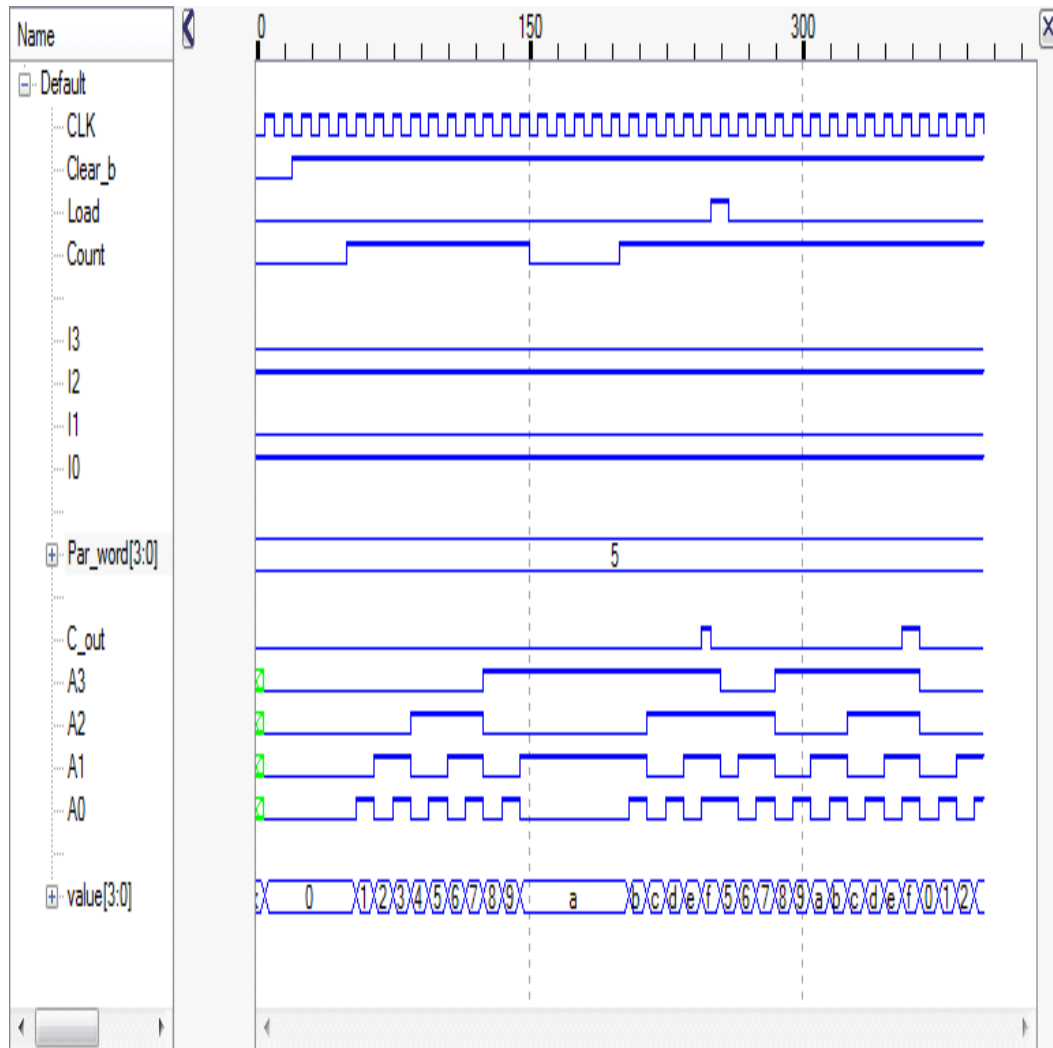
architecture Behavioral of Prob_6_59b_vhdl is
  signal E_bar <= not STRC(0);

```

```

begin
  process (CLK) begin
    if CLK'event and CLK = '1' then
      if (Clear_b = '0') then STRC <= "0000";
      elsif (Load = '1') then STRC <= I_par;
      elsif (Count = '1') STRC <= E_bar & STRC (3 downto 1);
      end if;
    end if;
  end process;
end Behavioral;

```



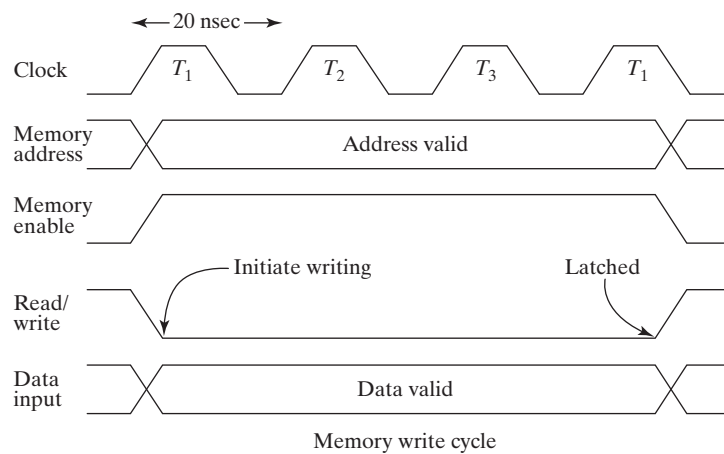
Chapter 7

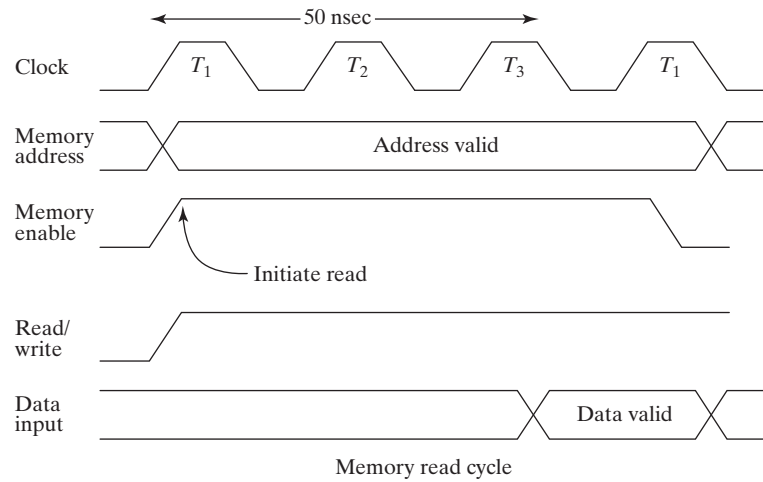
- 7.1**
- | | | | |
|-----|--|--------------------|-----------------|
| (a) | $4 \text{ K} \times 16 = 2^{12} \times 16$ | Address lines = 12 | Data lines = 16 |
| (b) | $2 \text{ G} \times 8 = 2^{31} \times 8$ | Address lines = 31 | Data lines = 8 |
| (c) | $16 \text{ M} \times 32 = 2^{24} \times 32$ | Address lines = 24 | Data lines = 32 |
| (d) | $256 \text{ K} \times 64 = 2^{18} \times 64$ | Address lines = 18 | Data lines = 64 |

- 7.2** (a) 2^{13} bytes (b) 2^{31} bytes (c) 2^{26} bytes (d) 2^{21} bytes

- 7.3**
- $723 = 512 + 128 + 64 + 16 + 2 + 1$
 $3451 = 2048 + 1024 + 256 + 64 + 32 + 16 + 8 + 2 + 1$
- 10-bit address: 10 1101 0011 = $2D3_{16}$
 16-bit data: 0000 1101 0111 1011 = $0D7B_{16}$

- 7.4** The clock frequency of CPU is 50 MHz. Hence, the clock cycle time with which CPU is operating will be $1/50 \text{ MHz} = 20 \text{ ns}$. Since memory access and cycle time is 50 ns, one memory read/memory write operation will be completed in two and a half (or possibly three) CPU clock cycles.

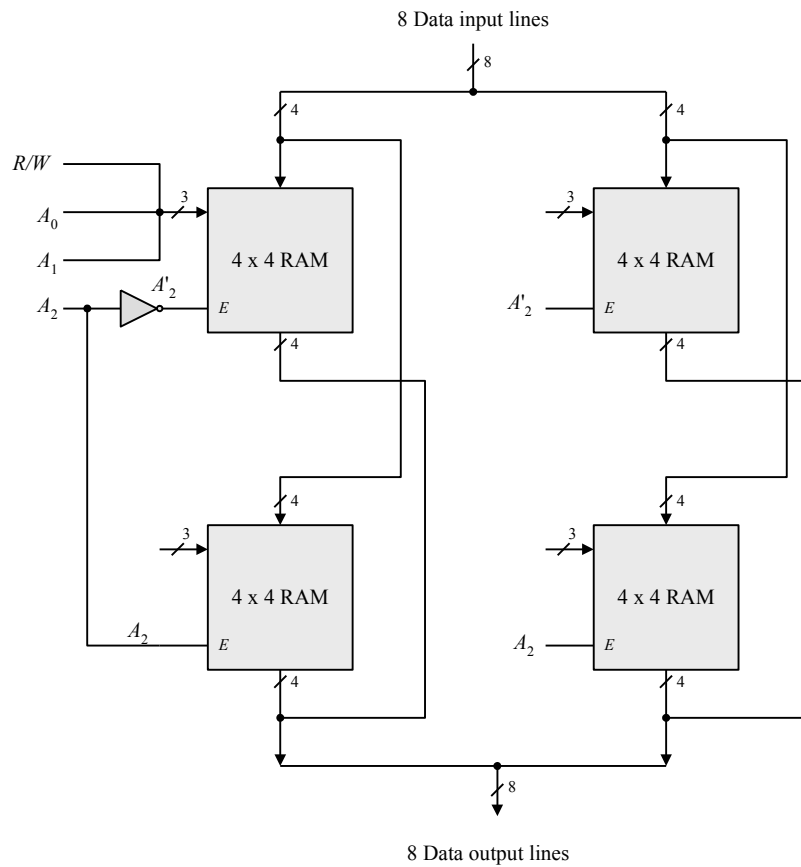




7.5

Pending

7.6



- 7.7** (a) $16\text{ K} = 2^{14} = 2^7 \times 2^7 = 128 \times 128$
Each decoder is 7×128 .
Decoders require 256 AND gates, each with 7 inputs.
- (b) $6000 = 0101110\ 1110000$
 $x = 46\ y = 112$
- 7.8** (a) Number of chips = $512\text{ K}/32\text{ K} = 16$
As $512\text{ K} = 2^{19}$, total 19 address lines are required to access 512 K bytes.
- (b) Among these 19 lines, 15 lines (as $32\text{ K} = 2^{15}$) will be connected to the address inputs of all chips.
- (c) Remaining 4 lines (as $19 - 15 = 4$) must be decoded for the chip select inputs using a 4×16 decoder.
- 7.9** $13 + 12 = 25$ address lines. Memory capacity = 2^{25} words.
- 7.10** $01011011 = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10\ 11\ 12\ 13$
 $P_1\ P_2\ 0\ P_4\ 1\ 0\ 1\ P_8\ 1\ 0\ 1\ 1\ P_{13}$
- $P_1 = \text{XOR of bits } (3, 5, 7, 9, 11) = 0, 1, 1, 1, 1 = 0$ (even # of 0s)
 $P_2 = \text{XOR of bits } (3, 6, 7, 10, 11) = 0, 0, 1, 0, 1 = 0$
 $P_4 = \text{XOR of bits } (5, 6, 7, 12) = 1, 0, 1, 1 = 1$ (odd # of 0s)
 $P_8 = \text{XOR of bits } (9, 10, 11, 12) = 1, 0, 1, 1 = 1$
 $P_{13} = \text{XOR of all 12 bits} = 1$
- Thus, composite 13-bit code word: $0001\ 1011\ 1011\ 1$
- 7.11** $11001001010 = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10\ 11\ 12\ 13\ 14\ 15$
 $P_1\ P_2\ 1\ P_4\ 1\ 0\ 0\ P_8\ 1\ 0\ 0\ 1\ 0\ 1\ 0$
- $P_1 = \text{XOR of bits } (3, 5, 7, 9, 11, 13, 15) = 1, 1, 0, 1, 0, 0, 0 = 1$ (odd # of 0s)
 $P_2 = \text{XOR of bits } (3, 6, 7, 10, 11, 14, 15) = 1, 0, 0, 0, 0, 1, 0 = 0$ (even # of 0s)
 $P_4 = \text{XOR of bits } (5, 6, 7, 12, 13, 14, 15) = 1, 0, 0, 1, 0, 1, 0 = 1$
 $P_8 = \text{XOR of bits } (9, 10, 11, 12, 13, 14, 15) = 1, 0, 0, 1, 0, 1, 0 = 1$
- Thus, the 15-bit Hamming code word is $101\ 110\ 011\ 001\ 010$.
- 7.12** (a) $1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10\ 11\ 12$
 $0\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 0\ 1\ 0$
- $C_1 (1, 3, 5, 7, 9, 11) = 0, 0, 1, 1, 1, 1 = 0$
 $C_2 (2, 3, 6, 7, 10, 11) = 0, 0, 1, 1, 0, 1 = 1$
 $C_4 (4, 5, 6, 7, 12) = 0, 1, 1, 1, 0 = 1$
 $C_8 (8, 9, 10, 11, 12) = 0, 1, 0, 1, 0 = 0$
- $C = 0110$
Error in bit 6.
Correct data: $0101\ 1010$

(b) 1 2 3 4 5 6 7 8 9 10 11 12
 1 0 1 1 1 0 0 0 0 1 1 0

$$C_1 (1, 3, 5, 7, 9, 11) = 1, 1, 1, 0, 0, 1 = 0$$

$$C_2 (2, 3, 6, 7, 10, 11) = 0, 1, 0, 0, 1, 1 = 1$$

$$C_4 (4, 5, 6, 7, 12) = 1, 1, 0, 0, 0 = 0$$

$$C_8 (8, 9, 10, 11, 12) = 0, 0, 1, 1, 0 = 0$$

$$C = 0010$$

Error in bit 2 = Parity bit P_2 .

Correct 8-bit data: 3 5 6 7 9 10 11 12
 1 1 0 0 0 1 1 0

(c) 1 2 3 4 5 6 7 8 9 10 11 12
 1 0 1 1 1 1 1 1 0 1 0 0

$$C = 0000 \text{ (No errors)}$$

$$C_1 (1, 3, 5, 7, 9, 11) = 1, 1, 1, 0, 0, 1 = 0$$

$$C_2 (2, 3, 6, 7, 10, 11) = 0, 1, 0, 0, 1, 1 = 1$$

$$C_4 (4, 5, 6, 7, 12) = 1, 1, 0, 0, 0 = 0$$

$$C_8 (8, 9, 10, 11, 12) = 0, 0, 1, 1, 0 = 0$$

Correct 8-bit data: 3 5 6 7 9 10 11 12
 1 1 1 1 0 1 0 0

7.13 (a) 5 check bits + 1 = 6 parity bits

(b) 6 check bits + 1 = 7 parity bits

(c) 7 check bits + 1 = 8 parity bits

7.14 (a) 1 2 3 4 5 6 7 $P_1 = \text{Xor}(3, 5, 7) = 0, 0, 0 = 1$
 P_1 P_2 0 P_4 0 1 0 $P_2 = \text{Xor}(3, 6, 7) = 0, 1, 0 = 0$
 $P_4 = \text{Xor}(5, 6, 7) = 0, 1, 0 = 1$

7-bit word: 0101010

(b) No error:

$$C_1 = \text{Xor}(1, 3, 5, 7) = 0, 0, 0, 0 = 0$$

$$C_2 = \text{Xor}(2, 3, 6, 7) = 1, 0, 1, 0 = 0$$

$$C_4 = \text{Xor}(4, 5, 6, 7) = 1, 0, 1, 0 = 0$$

(c) Error in bit 5:

1	2	3	4	5	6	7
0	1	0	1	1	1	0

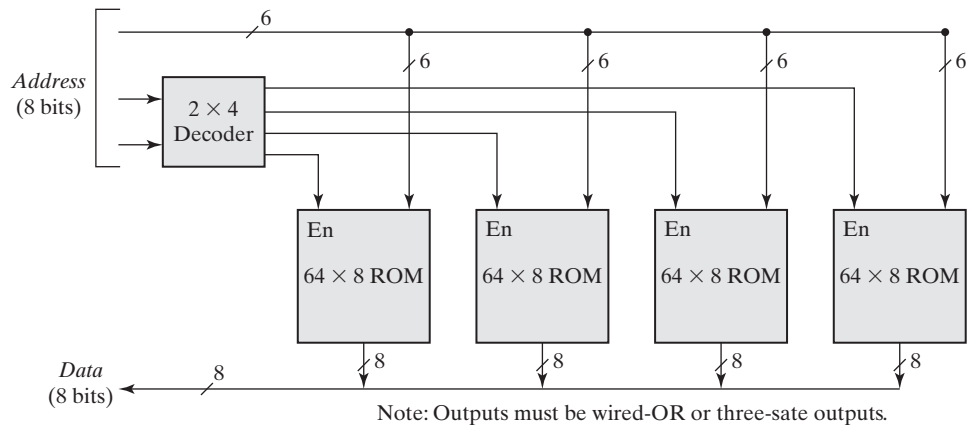
$$C_1 = \text{Xor}(0, 0, 1, 0) = 1$$

$$C_2 = \text{Xor}(1, 0, 1, 0) = 0$$

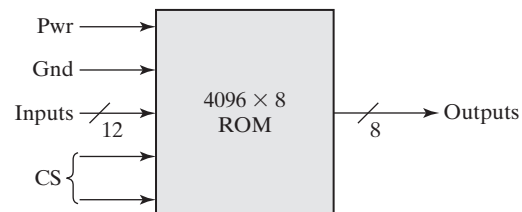
$$C_4 = \text{Xor}(1, 1, 1, 0) = 1$$

Error in bit 5: C = 101

(d) 8-bit word 1 2 3 4 5 6 7 8
 0 1 0 1 0 1 0 1
 Error in bits 2 and 5: 0 0 1 1 1 0 1
 $C_1 = \text{Xor}(0, 0, 1, 0) = 1$
 $C_2 = \text{Xor}(0, 0, 1, 0) = 1$
 $C_4 = \text{Xor}(1, 1, 1, 0) = 1$
 $P = 0$
 $C = (1, 1, 1) \neq 0$ and $P = 0$ indicates double error.

7.15

- 7.16** Number of input lines = 12 address lines (as $4096 = 2^{12}$) + 4 (two for chip select + one for power + one for ground) = 16
 Number of output lines = 8
 Thus, a 24 pins IC is needed.



7.17

Input Address				Output of ROM	
$I_5 I_4 I_3 I_2 I_1$	$D_6 D_5 D_4$	$D_3 D_2 D_1$	$D_0 (2^0)$	Decimal	
0 0 0 0 0	0 0 0	0 0 0	0, 1	0, 1	
0 0 0 0 1	0 0 0	0 0 1	0, 1	2, 3	
...	...	...	...	...	
...	...	...	...	...	
0 1 0 0 0	0 0 1	0 1 1	0, 1	16, 17	
0 1 0 0 1	0 0 1	1 0 0	0, 1	18, 19	
...	...	...	...	...	
...	...	...	...	...	
1 1 1 1 0	1 1 0	0 0 0	0, 1	60, 61	
1 1 1 1 1	1 1 0	0 0 1	0, 1	62, 63	

- 7.18**
- (a) 5 inputs and 7 outputs: $2^5 \times 7 = 32 \times 7$
 - (b) 10 inputs and 4 outputs: $2^{10} \times 4 = 1024 \times 4$
 - (c) 8 inputs and 8 outputs: $2^8 \times 8 = 256 \times 8$
 - (d) 9 inputs and 5 outputs: $2^9 \times 5 = 512 \times 5$

7.19

$x \begin{matrix} yz \\ 0 \\ 1 \end{matrix}$		$y \begin{matrix} 00 & 01 & 11 & 10 \end{matrix}$																																																																																	
		m_0	m_1	m_3	m_2																																																																														
0		0	1	1	0																																																																														
1		m_4	m_5	m_7	m_6																																																																														
		0	1	0	1																																																																														
		z																																																																																	
		$A = y'z + x'z + xyz'$ $A' = y'z' + x'z' + xyz$																																																																																	
$x \begin{matrix} yz \\ 0 \\ 1 \end{matrix}$		$y \begin{matrix} 00 & 01 & 11 & 10 \end{matrix}$																																																																																	
		m_0	m_1	m_3	m_2																																																																														
0		1	1	0	0																																																																														
1		m_4	m_5	m_7	m_6																																																																														
		0	0	1	1																																																																														
		z																																																																																	
		$B = xy + x'y'$ $B' = xy' + x'y$																																																																																	
$x \begin{matrix} yz \\ 0 \\ 1 \end{matrix}$		$y \begin{matrix} 00 & 01 & 11 & 10 \end{matrix}$																																																																																	
		m_0	m_1	m_3	m_2																																																																														
0		0	0	1	0																																																																														
1		m_4	m_5	m_7	m_6																																																																														
		0	1	0	0																																																																														
		z																																																																																	
		$C = x'yz + xy'z$ $C' = z' + x'y' + xyz$																																																																																	
$x \begin{matrix} yz \\ 0 \\ 1 \end{matrix}$		$y \begin{matrix} 00 & 01 & 11 & 10 \end{matrix}$																																																																																	
		m_0	m_1	m_3	m_2																																																																														
0		0	1	0	1																																																																														
1		m_4	m_5	m_7	m_6																																																																														
		1	1	1	0																																																																														
		z																																																																																	
		$D = xy' + xz + y'z + x'yz'$ $D' = x'y'z' + x'yz + xyz'$																																																																																	
		<table> <tr> <th colspan="2">Product Inputs</th><th colspan="4">Outputs</th></tr> <tr> <th>term</th><th>$x y z$</th><th>A</th><th>B</th><th>C</th><th>D</th></tr> <tr><td>$y'z$</td><td>1 - 0 1</td><td>1</td><td>-</td><td>-</td><td>1</td></tr> <tr><td>$x'z$</td><td>2 0 - 1</td><td>1</td><td>-</td><td>-</td><td>-</td></tr> <tr><td>xyz'</td><td>3 1 1 0</td><td>1</td><td>-</td><td>-</td><td>-</td></tr> <tr><td>xy</td><td>4 1 1 -</td><td>-</td><td>1</td><td>-</td><td>-</td></tr> <tr><td>$x'y'$</td><td>5 0 0 -</td><td>-</td><td>1</td><td>-</td><td>-</td></tr> <tr><td>$x'yz$</td><td>6 0 1 1</td><td>-</td><td>-</td><td>1</td><td>-</td></tr> <tr><td>$xy'z$</td><td>7 1 0 1</td><td>-</td><td>-</td><td>1</td><td>-</td></tr> <tr><td>xy'</td><td>8 1 0 -</td><td>-</td><td>-</td><td>-</td><td>1</td></tr> <tr><td>Xz</td><td>9 1 - 1</td><td>-</td><td>-</td><td>-</td><td>1</td></tr> <tr><td>$y'z$</td><td>10 - 0 1</td><td>-</td><td>-</td><td>-</td><td>1</td></tr> <tr><td>$x'yz'$</td><td>11 0 1 0</td><td>-</td><td>-</td><td>-</td><td>1</td></tr> </table>				Product Inputs		Outputs				term	$x y z$	A	B	C	D	$y'z$	1 - 0 1	1	-	-	1	$x'z$	2 0 - 1	1	-	-	-	xyz'	3 1 1 0	1	-	-	-	xy	4 1 1 -	-	1	-	-	$x'y'$	5 0 0 -	-	1	-	-	$x'yz$	6 0 1 1	-	-	1	-	$xy'z$	7 1 0 1	-	-	1	-	xy'	8 1 0 -	-	-	-	1	Xz	9 1 - 1	-	-	-	1	$y'z$	10 - 0 1	-	-	-	1	$x'yz'$	11 0 1 0	-	-	-	1
Product Inputs		Outputs																																																																																	
term	$x y z$	A	B	C	D																																																																														
$y'z$	1 - 0 1	1	-	-	1																																																																														
$x'z$	2 0 - 1	1	-	-	-																																																																														
xyz'	3 1 1 0	1	-	-	-																																																																														
xy	4 1 1 -	-	1	-	-																																																																														
$x'y'$	5 0 0 -	-	1	-	-																																																																														
$x'yz$	6 0 1 1	-	-	1	-																																																																														
$xy'z$	7 1 0 1	-	-	1	-																																																																														
xy'	8 1 0 -	-	-	-	1																																																																														
Xz	9 1 - 1	-	-	-	1																																																																														
$y'z$	10 - 0 1	-	-	-	1																																																																														
$x'yz'$	11 0 1 0	-	-	-	1																																																																														

7.20

Inputs	Outputs
$x y z$	A, B, C, D
0 0 0	0 1 0 0
0 0 1	1 1 0 1
0 1 0	1 0 1 1
0 1 1	0 0 0 1
1 0 0	1 0 0 0
1 0 1	0 0 0 1
1 1 0	1 1 1 0
1 1 1	0 1 0 1

Memory content at address 2 = $M[010] = 1011$

Memory content at address 6 = $M[110] = 1110$

7.21 Note: See truth table in Fig. 7.12(b).

$A_2 \backslash A_1 A_0$		A_1			
		00	01	11	10
A_2	0	m_0 0	m_1 0	m_3 0	m_2 0
	1	m_4 0	m_5 0	m_7 1	m_6 1

A_0

$$F_1 = A_2 A_1$$

$$F_1' = A_2' + A_1'$$

$A_2 \backslash A_1 A_0$		A_1			
		00	01	11	10
A_2	0	m_0 0	m_1 0	m_3 0	m_2 0
	1	m_4 1	m_5 1	m_7 1	m_6 0

A_0

$$F_2 = A_2 A_1' + A_2 A_0$$

$$F_2' = A_2' + A_1 A_0'$$

$A_2 \backslash A_1 A_0$		A_1			
		00	01	11	10
A_2	0	m_0 0	m_1 0	m_3 1	m_2 0
	1	m_4 0	m_5 1	m_7 0	m_6 0

A_0

$$F_3 = A_2' A_1 A_0 + A_2 A_1' A_0$$

$$F_3' = A_0' + A_2' A_1' + A_2 A_1$$

$A_2 \backslash A_1 A_0$		A_1			
		00	01	11	10
A_2	0	m_0 0	m_1 0	m_3 0	m_2 1
	1	m_4 0	m_5 0	m_7 0	m_6 1

A_0

$$F_4 = A_1 A_0'$$

$$F_4' = A_1' + A_0$$

Product term	Inputs			Outputs			
	A_2	A_1	A_0	F_1	F_2	F_3	F_4
$A_2 A_1$	1	1	1	-	-	-	-
A_2'	2	0	-	-	1	-	-
$A_1 A_0'$	3	-	1	0	-	1	-
$A_2' A_1 A_0$	4	-	1	1	-	-	1
$A_2 A_1'$	5	1	0	1	-	-	1

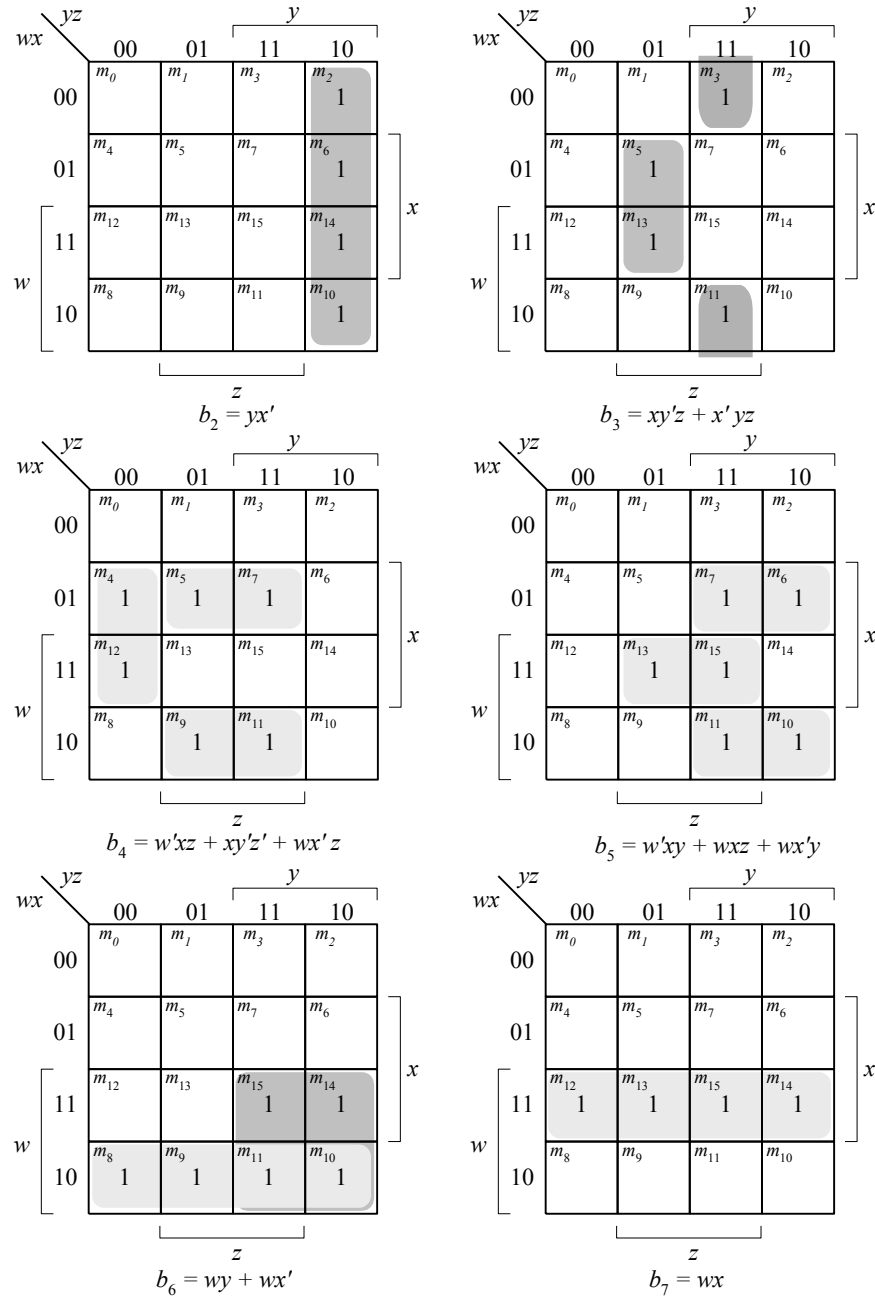
$T \quad C \quad T \quad T$

Alternative: F_1', F_2', F_3, F_4
(5 terms)

7.22

Decimal	w	x	y	z	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	1
2	4	0	0	1	0	0	0	0	0	1	0	0
3	9	0	0	1	0	0	0	0	1	0	0	1
4	16	0	1	0	0	0	0	1	0	0	0	0
5	25	0	1	0	0	0	0	1	1	0	0	1
6	36	0	1	1	0	0	1	0	0	1	0	0
7	49	0	1	1	0	0	1	1	0	0	0	1
8	64	1	0	0	0	1	0	0	0	0	0	0
9	81	1	0	0	0	1	0	1	0	0	0	1
10	100	1	0	1	0	1	1	0	0	1	0	0
11	121	1	0	1	0	1	1	1	1	0	0	1
12	144	1	1	0	0	1	0	0	1	0	0	0
13	169	1	1	0	1	0	1	0	1	0	0	1
14	196	1	1	1	0	1	1	0	0	0	1	0
15	225	1	1	1	1	1	1	0	0	0	0	1

Note: $b_0 = z$, and $b_1 = 0$.
ROM would have 4 inputs
and 6 outputs. A 4 x 8
ROM would waste two
outputs.



7.23

From Fig. 4-3:

$$w = A + BC + BD$$

$$w' = A'B' + A'C'D'$$

$$x = B'C + B'D + BC'D'$$

$$x' = B'C'D' + BC \quad BD$$

$$y = CD + C'D'$$

$$y' = C'D + CD'$$

$$z = D'$$

$$z' = D$$

Use w, x', y, z (7 terms)

Product term	Inputs				Outputs			
	A	B	C	D	F_1	F_2	F_3	F_4
A	1	1	-	-	-	1	-	-
BC	2	-	1	1	-	1	1	-
BD	3	-	1	-	1	1	-	-
$B'C'D'$	4	-	0	0	0	-	1	-
CD	5	-	-	1	1	-	-	1
$C'D'$	6	-	-	0	0	-	-	1
D'	7	-	-	-	0	-	-	1
					T	C	T	T

7.24

AND				
Product term	Inputs			
	A	B	C	D
1	1	-	-	-
2	-	1	1	-
3	-	1	-	1
4	-	0	1	-
5	-	0	-	1
6	-	1	0	0
7	-	-	1	1
8	-	-	0	0
9	-	-	-	-
10	-	-	-	0
11	-	-	-	-
12	-	-	-	-

$$w = A + BC + BD$$

$$x = B'C + B'D + BC'D'$$

$$y = CD + C'D'$$

$$z = D'$$

7.25 On simplifying, we get:

$$A = yz' + xz' + x'y'z$$

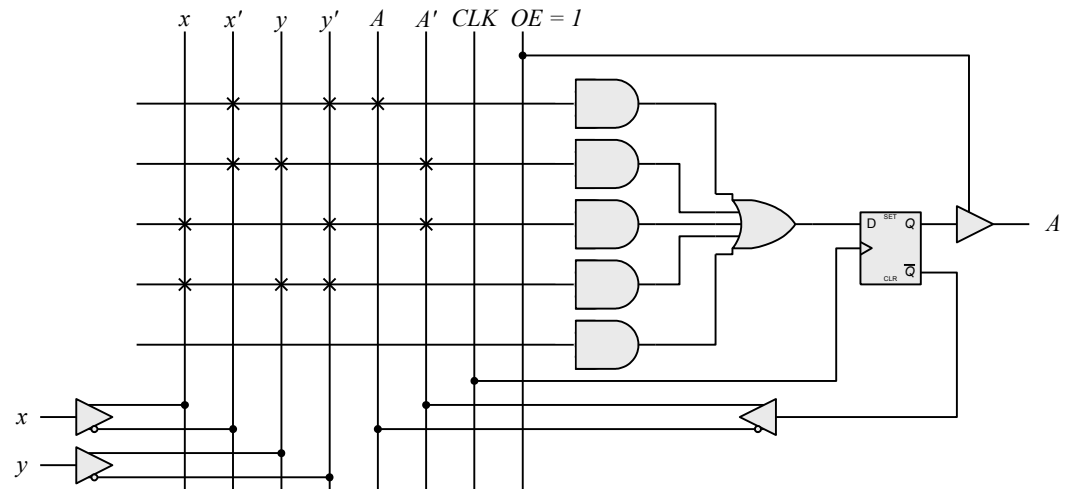
$$B = x'y' + xy + yz$$

$$C = A + xyz$$

$$D = z + x'y$$

Product term	AND Inputs				Outputs
	x	y	z	A	
1	–	1	0	–	$A = yz' + xz' + x'y'z$
2	1	–	0	–	
3	0	0	1	–	
4	0	0	–	–	$B = x'y' + xy + yz$
5	1	1	–	–	
6	–	1	1	–	
7	–	–	–	1	$C = A + xyz$
8	1	1	1	–	
9	–	–	–	–	
10	–	–	1	–	$D = z + x'y$
11	0	1	–	–	
12	–	–	–	–	

7.26



7.27

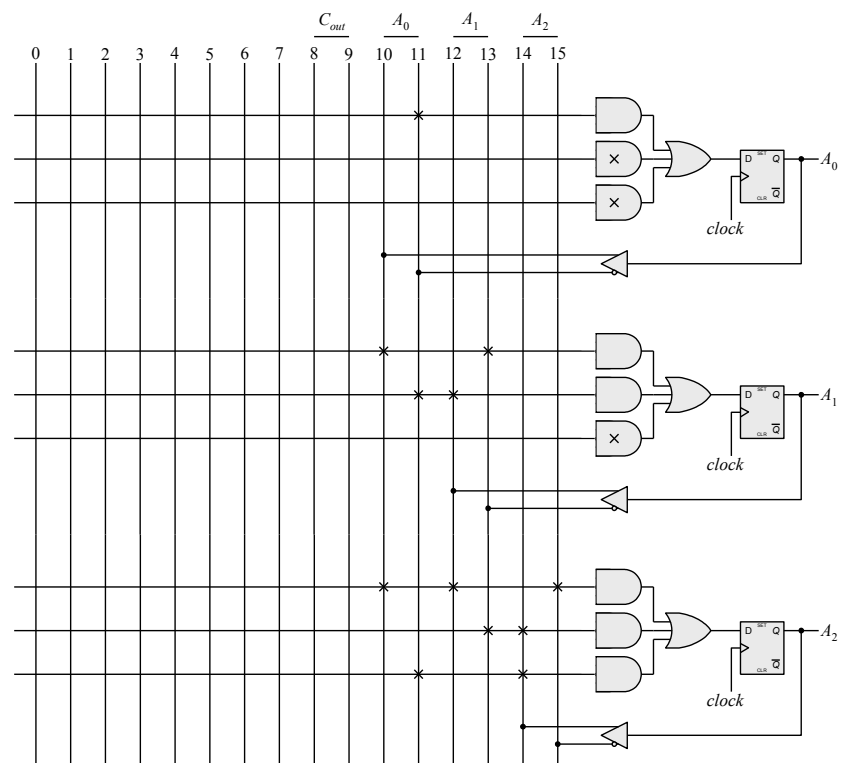
The results of Prob. 6.17 can be used to develop the equations for a three-bit binary counter with D-type flip-flops.

$$DA_0 = A'_0$$

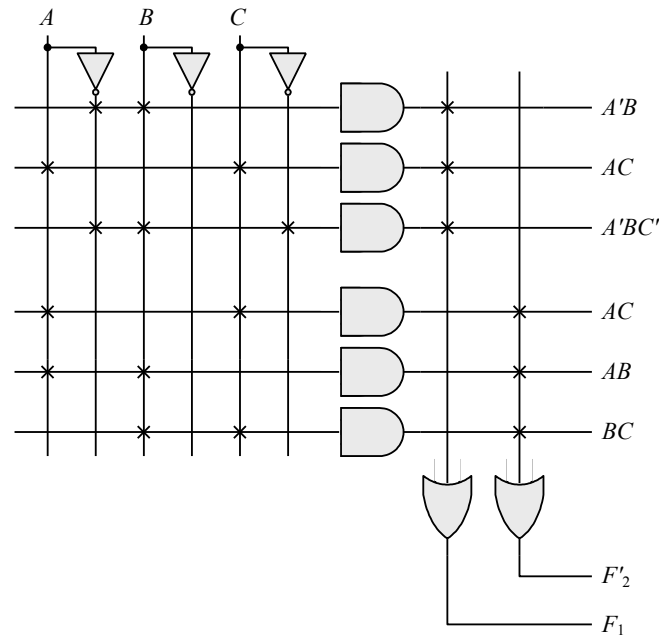
$$DA_1 = A'_1 A_0 + A_1 A'_0$$

$$DA_2 = A'_2 A_1 A_0 + A_2 A'_1 + A_2 A'_0$$

$$C_{out} = A_2 A_1 A_0$$



7.28



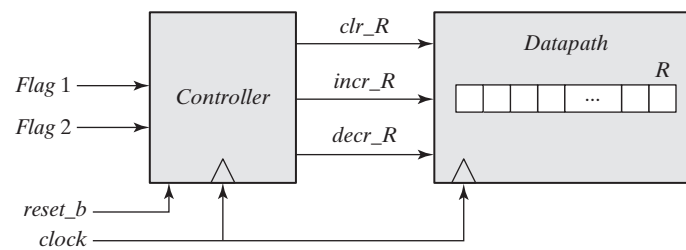
7.29

Product term	Inputs	Output
	$x y A$	D_A
$x'y'A$	1 0 0 1	1
$x'yA'$	2 0 1 0	1
$xy'A'$	3 1 0 0	1
xyA	4 1 1 1	1

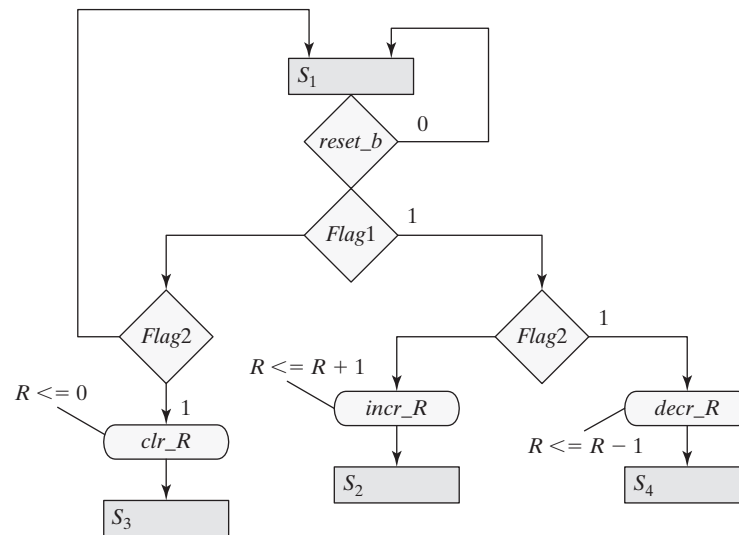
CHAPTER 8

- 8.1 (a) $R2 \leftarrow R2 + 1, R1 \leftarrow R2$
 (b) $R3 \leftarrow R3 - 1$
 (c) If $(T1 = 1)$ then $(R0 \leftarrow R1)$ else $(R0 \leftarrow R2)$

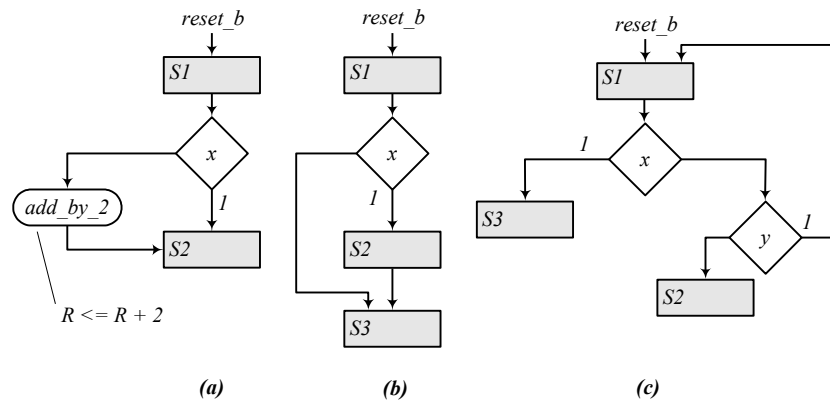
- 8.2 Logic diagram:



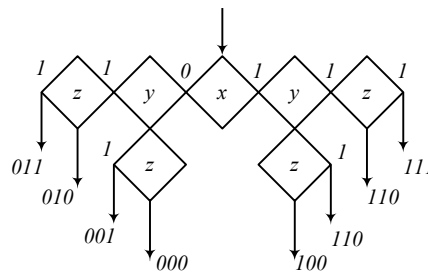
ASMD chart:



8.3



8.4



8.5 An algorithmic state machine (ASM) chart is a special-purpose flowchart that has been developed to specifically define algorithms for execution on digital hardware.

An ASM chart is composed of three basic elements, such as the state box, the decision box and the conditional box. The boxes themselves are connected by directed edges indicating the sequential precedence and evolution of the states as the machine operates.

The ASM chart is similar to a state transition diagram. Each state block is equivalent to a state in a sequential circuit. The decision box is equivalent to the binary information written along the directed lines that connect two states in a state diagram. As a consequence, it is sometimes convenient to convert the chart into a state diagram and then use sequential circuit procedures to design the control logic.

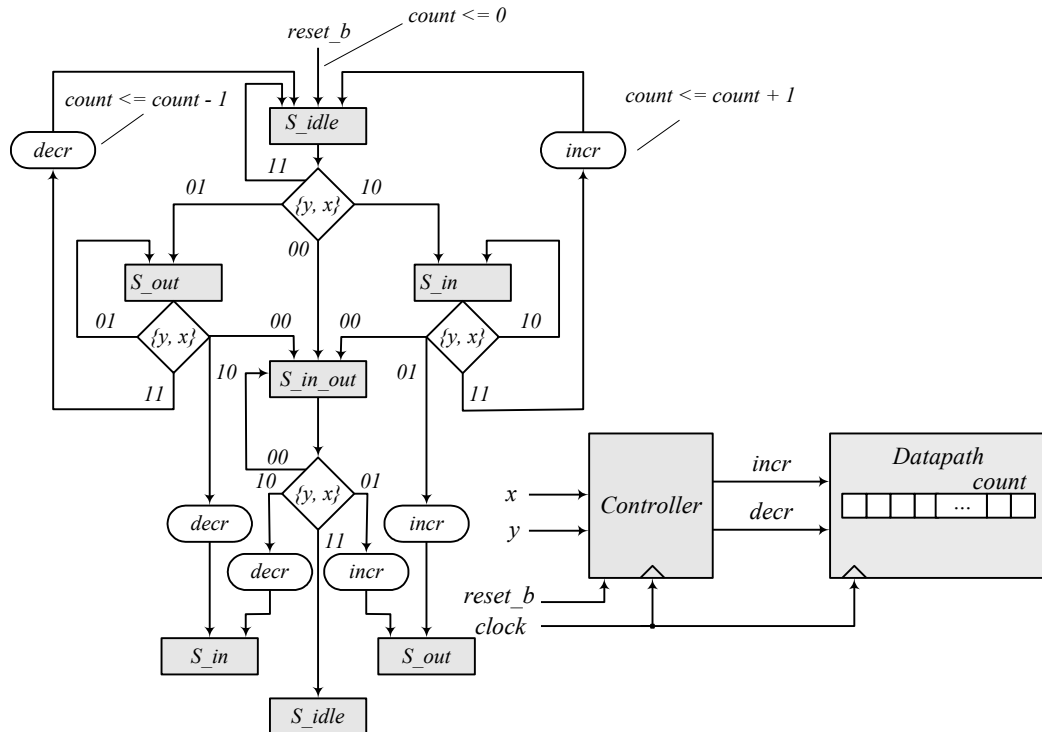
Algorithmic state machine and datapath (ASMD) charts clarify the information displayed by ASM charts and provide a systematic, effective and efficient tool for designing a control unit for a given datapath unit. The three steps that form an ASMD chart are as follows.

1. Form an ASM chart showing only the states of the controller, decision boxes and the names of input signals that cause state transitions,

- Convert the ASM chart into an ASMD chart by annotating the edges of the ASM chart to indicate the concurrent register operations of the datapath unit (i.e., register operations that are concurrent with a state transition).
- Modify the ASMD chart to identify the control signals that are generated by the controller and that cause the indicated operations in the datapath unit.

8.6

Note: In practice, the asynchronous inputs x and y should be synchronized to the clock to avoid metastable conditions in the flip-flops..



Note: To avoid counting a person more than once, the machine waits until x or y is de-asserted before incrementing or decrementing the counter. The machine also accounts for persons entering and leaving simultaneously.

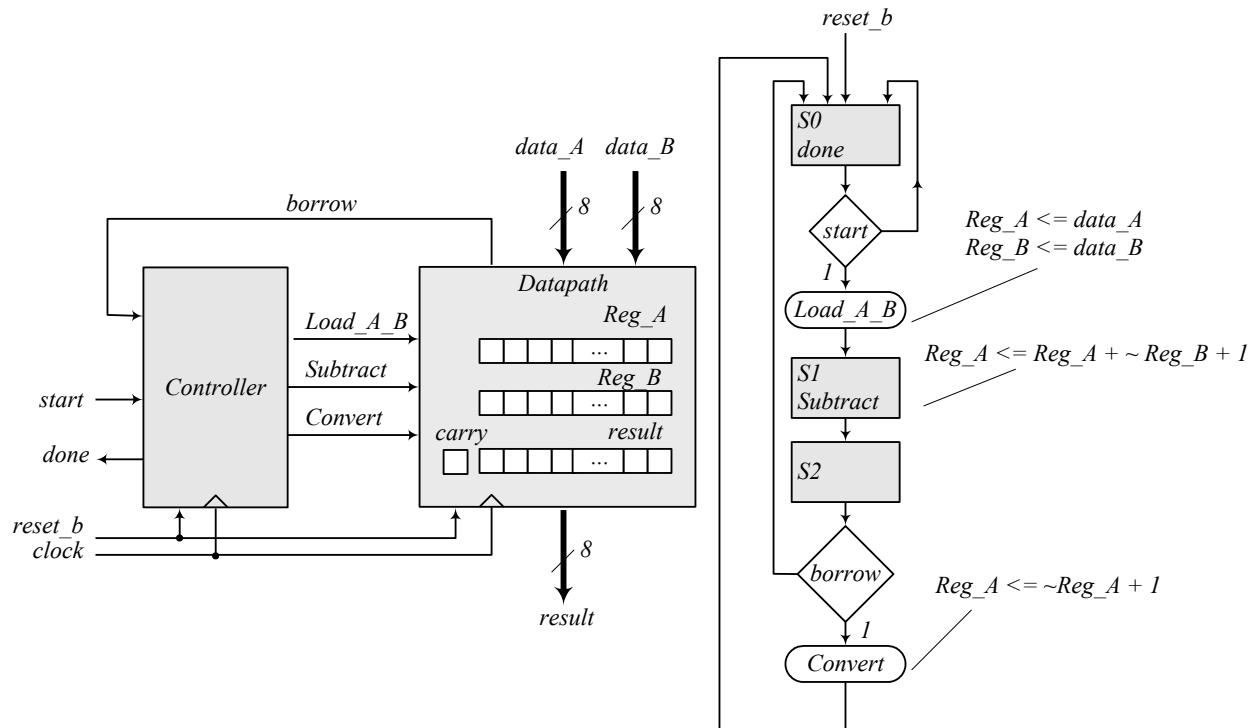
8.7 RTL notation:

S0: Initial state: if (start = 1) then ($RA \leftarrow data_A$, $RB \leftarrow data_B$, go to *S1*).

S1: $\{Carry, RA\} \leftarrow RA + (2\text{'s complement of } RB)$, go to *S2*.

S2: If (borrow = 0) go to *S0*. If (borrow = 1) then $RA \leftarrow (2\text{'s complement of } RA)$, go to *S0*.

Block diagram and ASMD chart:



module Subtractor_P8_7

(**output** done, **output** [7:0] result, **input** [7:0] data_A, data_B, **input** start, clock, reset_b);

Controller_P8_7 M0 (Load_A_B, Subtract, Convert, done, start, borrow, clock, reset_b);

Datapath_P8_7 M1 (result, borrow, data_A, data_B, Load_A_B, Subtract, Convert, clock, reset_b);

endmodule

module Controller_P8_7 (**output reg** Load_A_B, Subtract, **output reg** Convert, **output** done, **input** start, borrow, clock, reset_b);

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;

reg [1:0] state, next_state;

assign done = (state == S0);

always @ (posedge clock, negedge reset_b)
if (!reset_b) state <= S0; **else** state <= next_state;

always @ (state, start, borrow) **begin**

Load_A_B = 0;

Subtract = 0;

Convert = 0;

case (state)

S0: **if** (start) **begin** Load_A_B = 1; next_state = S1; **end**

S1: **begin** Subtract = 1; next_state = S2; **end**

S2: **begin** next_state = S0; **if** (borrow) Convert = 1; **end**

default: next_state = S0;

```

    endcase
end
endmodule

module Datapath_P8_7 (output [7: 0] result, output borrow, input [7: 0] data_A, data_B,
    input Load_A_B, Subtract, Convert, clock, reset_b);
    reg    carry;
    reg [8:0] diff;
    reg [7: 0] RA, RB;
    assign    borrow = carry;
    assign result = RA;

    always @ (posedge clock, negedge reset_b)
        if (!reset_b) begin carry <= 1'b0; RA <= 8'b0000_0000; RB <= 8'b0000_0000; end
        else begin
            if (Load_A_B) begin RA <= data_A; RB <= data_B; end
            else if (Subtract) {carry, RA} <= RA + ~RB + 1;

                // In the statement above, the math of the LHS is done to the wordlength of the LHS
                // The statement below is more explicit about how the math for subtraction is done:
                // else if (Subtract) {carry, RA} <= {1'b0, RA} + {1'b1, ~RB } + 9'b0000_0001;
                // If the 9-th bit is not considered, the 2s complement operation will generate a carry bit,
                // and borrow must be formed as borrow = ~carry.

            else if (Convert) RA <= ~RA + 8'b0000_0001;
        end
    endmodule

// Test plan – Verify;
// Power-up reset
// Subtraction with data_A > data_B
// Subtraction with data_A < data_B
// Subtraction with data_A = data_B
// Reset on-the-fly: left as an exercise

module t_Subtractor_P8_7;
    wire    done;
    wire [7:0] result;
    reg [7: 0] data_A, data_B;
    reg    start, clock, reset_b;

    Subtractor_P8_7 M0 (done, result, data_A, data_B, start, clock, reset_b);

    initial #200 $finish;
    initial begin clock = 0; forever #5 clock = ~clock; end
    initial fork
        reset_b = 0;
        #2 reset_b = 1;
        #90 reset_b = 1;
        #92 reset_b = 1;
    join

    initial fork
        #20 start = 1;
        #30 start = 0;
        #70 start = 1;
        #110 start = 1;
    join

    initial fork
        data_A = 8'd50;
        data_B = 8'd20;

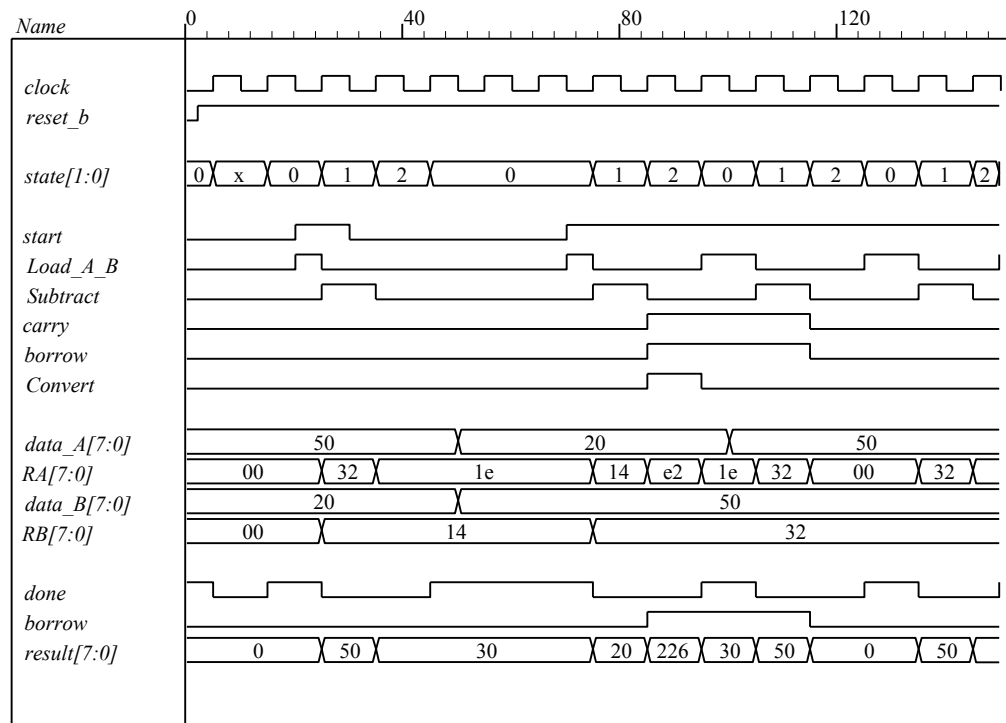
```

```

#50 data_A = 8'd20;
#50 data_B = 8'd50;

#100 data_A = 8'd50;
#100 data_B = 8'd50;
join
endmodule

```



VHDL

entity Datapath_P8_7 **is**

port (result: **out** std_logic_vector (7 **downto** 0);

data_a, data_b: in std_logic_vector (7 **downto** 0); Load_A_B, Subtract, Convert, clock: in std_logic;

borrow: out std_logic);

end Datapath_P8_7;

architecture Behavioral **of** Datpath_P8_7 **is**

begin

process (clock, reset_b) **begin**

if reset_b'event **and** reset_b = '0' **then** carry <= 0; Reg_A <= "00000000";
Reg_B <= "00000000";

elsif clock'event **and** clock = 1 **then**

if Load_A_B = 1 **then** Reg_A <= data_A; Reg_B <= data_B;

elsif Subtract = 1 **then** carry & Reg_A <= Reg_A + (not Reg_b) + 1;

elsif Convert **then** Reg_A <= (not Reg_A) + "00000001"; **end if**;

end Behavioral;

begin

entity Controller_P8_7 **is**

port (Load_A_B, Subtract, Convert, done: **out** Std_Logic; start, borrow, clock,
reset_b: **in** Std_Logic);

end Controller_P8_7;

```

architecture Behavioral of Controller_P8_7 is
    constant S0: Std_Logic_Vector = "00";
    constant S1: Std_Logic_Vector := "01";
    constant S2: Std_Logic_Vector := "10";
    signal state, next_state: std_logic_vector (1 downto 0);
begin
    process (clock, reset_b)
    begin
        if reset_b'event and reset_b = 0 then state <= S0;
        elsif clock'event and clock = 1 then state <= next_state; end if;
    end process;
    process (state, start, borrow) begin
        Load_A_B <= '0';
        Subtract <= '0';
        Convert <= '0';
        case state is
            when S0 => if start = '1' then Load_A_B <= '1'; next_state <= S1; end if;
            when S1 => Subtract <= '1'; next_state <= S2;
            when S2 => next_state <= S0; if borrow = '1' then convert = '1'; end if;
            when others next_state <= S0;
        end case;
    end process;
end Behavioral;

entity Subtractor_P8_7 is
    port (done: out std_logic; result: out std_logic_vector (7 downto 0);
        data_a, data_b: in std_logic_vector (7 downto 0); start, clock, reset_b: in std_logic);
end Subtractor_P8_7;

architecture ASMD is
    component Controller_P8_7 is
        port (Load_A_B, Subtract, Convert, done: out Std_Logic; start, borrow, clock,
            reset_b: in Std_Logic);
    end component;

    component Datapath_P8_7 is
        port (result: out std_logic_vector (7 downto 0);
            data_a, data_b: in std_logic_vector (7 downto 0); Load_A_B, Subtract, Convert, clock: in std_logic;
            borrow: out std_logic);
    end component;
begin
    M0: Controller_P8_7
        port map (Load_A_B, Subtract, Convert, done, start, borrow, clock, reset_b);
    M1: Datapath_P8_7
        port map(result, data_a, data_b, Load_A_B, Subtract, Convert, clock, borrow);
end ASMD;

```

8.8 RTL notation:

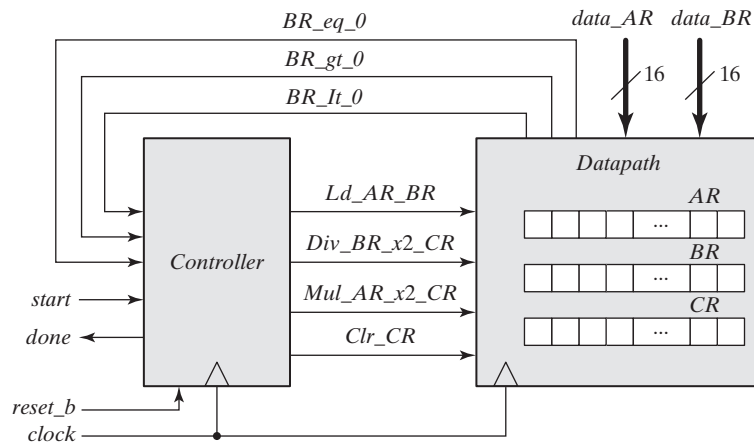
S0: if (start = 1) $AR \leftarrow \text{input data}$, $BR \leftarrow \text{input data}$, go to S1.

S1: if ($BR[15] = 1$) (sign bit negative) then $CR \leftarrow BR$ (shifted right, sign extension).

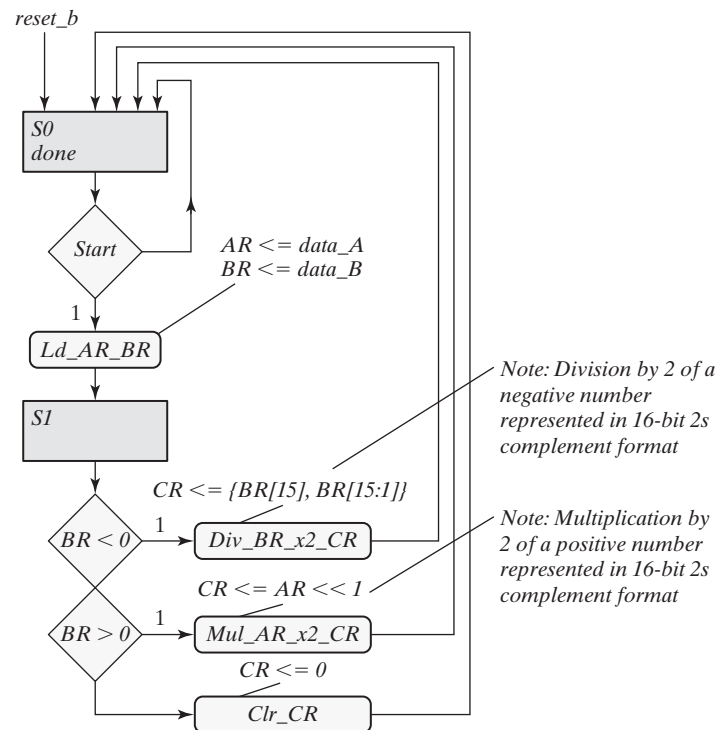
else if (positive non-zero) then ($\text{Overflow} \leftarrow AR[15] \oplus [14]$), $CR \leftarrow AR$ (shifted left)

else if ($BR = 0$) then ($CR \leftarrow 0$).

Block diagram:



ASMD chart:



HDL description:

```
module Prob_8_8 (output done, input [15: 0] data_AR, data_BR, input start, clock, reset_b);
  Controller_P8_8 M0 (Ld_AR_BR, Div_BR_x2_CR, Mul_AR_x2_CR, Clr_CR, done, start,
    BR_lt_0, BR_gt_0, BR_eq_0, clock, reset_b);
  Datapath_P8_8 M1 (Overflow, BR_lt_0, BR_gt_0, BR_eq_0, data_AR, data_BR, Ld_AR_BR,
    Div_BR_x2_CR, Mul_AR_x2_CR, Clr_CR, clock, reset_b);
endmodule
```

```
module Controller_P8_8 (
  output reg Ld_AR_BR, Div_BR_x2_CR, Mul_AR_x2_CR, Clr_CR,
  output done, input start, BR_lt_0, BR_gt_0, BR_eq_0, clock, reset_b);
  parameter S0 = 1'b0, S1 = 1'b1;
  reg state, next_state;
  assign done = (state == S0);
  always @ (posedge clock, negedge reset_b)
  if (!reset_b) state <= S0; else state <= next_state;
  always @ (state, start, BR_lt_0, BR_gt_0, BR_eq_0) begin
    Ld_AR_BR = 0;
    Div_BR_x2_CR = 0;
    Mul_AR_x2_CR = 0;
    Clr_CR = 0;
    case (state)
      S0: if (start) begin Ld_AR_BR = 1; next_state = S1; end
      S1: begin
        next_state = S0;
        if (BR_lt_0) Div_BR_x2_CR = 1;
        else if (BR_gt_0) Mul_AR_x2_CR = 1;
        else if (BR_eq_0) Clr_CR = 1;
        end
      default: next_state = S0;
    endcase
  end
endmodule
```

```
module Datapath_P8_8 (output reg Overflow, output BR_lt_0, BR_gt_0, BR_eq_0, input [15: 0]
data_AR, data_BR, input Ld_AR_BR, Div_BR_x2_CR, Mul_AR_x2_CR, Clr_CR, clock, reset_b);
  reg [15: 0] AR, BR, CR;
  assign BR_lt_0 = BR[15];
  assign BR_gt_0 = (!BR[15]) && (! BR[14:0]); // Reduction-OR
  assign BR_eq_0 = (BR == 16'b0);
  always @ (posedge clock, negedge reset_b)
  if (!reset_b) begin AR <= 16'b0; BR <= 16'b0; CR <= 16'b0; end
  else begin
    if (Ld_AR_BR) begin AR <= data_AR; BR <= data_BR; end
    else if (Div_BR_x2_CR) CR <= {BR[15], BR[15:1]};
    else if (Mul_AR_x2_CR) {Overflow, CR} <= (AR << 1);
    else if (Clr_CR) CR <= 16'b0;
  end
endmodule
```


8.9

Design equations:

$$D_{S_idle} = S_2 + S_idle \text{ Start'}$$

$$D_{S_1} = S_idle \text{ Start} + S_1 (A2 \ A3)'$$

$$D_{S_2} = A2 \ A3 \ S_1$$

HDL description:

Verilog

```
module Prob_8_9 (output E, F, output [3: 0] A, output A2, A3, input Start, clock, reset_b);
wire set_E, clr_E, set_F, clr_A_F, incr_A;
```

```
    Prob_8_9_Controller M0 (set_E, clr_E, set_F, clr_A_F, incr_A, Start, A2, A3, clock, reset_b);
    Prob_8_9_Datapath M1 (E, F, A, A2, A3, set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b);
endmodule
```

```
// Structural version of the controller (one-hot)
// Note that the flip-flop for S_idle must have a set input and reset_b is wire to the set
// Simulation results match Fig. 8-13
```

```
module Prob_8_9_Controller (
    output set_E, clr_E, set_F, clr_A_F, incr_A,
    input Start, A2, A3, clock, reset_b
);

    wire D_S_idle, D_S_1, D_S_2;
    wire q_S_idle, q_S_1, q_S_2;
    wire w0, w1, w2, w3;
    wire [2:0] state = {q_S_2, q_S_1, q_S_idle};

    // Next-State Logic
    or (D_S_idle, q_S_2, w0); // input to D-type flip-flop for q_S_idle
    and (w0, q_S_idle, Start_b);
    not (Start_b, Start);

    or (D_S_1, w1, w2, w3); // input to D-type flip-flop for q_S_1
    and (w1, q_S_idle, Start);
    and (w2, q_S_1, A2_b);
    not (A2_b, A2);
    and (w3, q_S_1, A2, A3_b);
    not (A3_b, A3);

    and (D_S_2, A2, A3, q_S_1); // input to D-type flip-flop for q_S_2

    D_flop_S M0 (q_S_idle, D_S_idle, clock, reset_b);
    D_flop M1 (q_S_1, D_S_1, clock, reset_b);
    D_flop M2 (q_S_2, D_S_2, clock, reset_b);

    // Output Logic
    and (set_E, q_S_1, A2);
    and (clr_E, q_S_1, A2_b);
    buf (set_F, q_S_2);
    and (clr_A_F, q_S_idle, Start);
    buf (incr_A, q_S_1);
endmodule

module D_flop (output reg q, input data, clock, reset_b);
    always @ (posedge clock, negedge reset_b)
        if (!reset_b) q <= 1'b0; else q <= data;
endmodule
```

```

module D_flop_S (output reg q, input data, clock, set_b);
    always @ (posedge clock, negedge set_b)
        if (!set_b) q <= 1'b1; else q <= data;
endmodule

/*
// RTL Version of the controller
// Simulation results match Fig. 8-13

module Prob_8_9_Controller (
    output reg set_E, clr_E, set_F, clr_A_F, incr_A,
    input Start, A2, A3, clock, reset_b
);
    parameter S_idle = 3'b001, S_1 = 3'b010, S_2 = 3'b100; // One-hot
    reg [2: 0] state, next_state;

    always @ (posedge clock, negedge reset_b)
        if (!reset_b) state <= S_idle; else state <= next_state;

    always @ (state, Start, A2, A3) begin
        set_E = 1'b0;
        clr_E = 1'b0;
        set_F = 1'b0;
        clr_A_F = 1'b0;
        incr_A = 1'b0;
        case (state)
            S_idle: if (Start) begin next_state = S_1; clr_A_F = 1; end
                     else next_state = S_idle;

            S_1: begin
                    incr_A = 1;
                    if (!A2) begin next_state = S_1; clr_E = 1; end
                    else begin
                        set_E = 1;
                        if (A3) next_state = S_2; else next_state = S_1;
                    end
                end

            S_2: begin next_state = S_idle; set_F = 1; end
            default: next_state = S_idle;
        endcase
    end
endmodule
*/

module Prob_8_9_Datapath
    output reg E, F, output reg [3: 0] A, output A2, A3,
    input set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b
);
    assign A2 = A[2];
    assign A3 = A[3];

    always @ (posedge clock, negedge reset_b) begin
        if (!reset_b) begin E <= 0; F <= 0; A <= 0; end
        else begin
            if (set_E) E <= 1;
            if (clr_E) E <= 0;
            if (set_F) F <= 1;
            if (clr_A_F) begin A <= 0; F <= 0; end
            if (incr_A) A <= A + 1;
        end
    end
endmodule

```

```
// Test Plan - Verify: (1) Power-up reset, (2) match ASMD chart in Fig. 8-9 (d),
// (3) recover from reset on-the-fly
```

```
module t_Prob_8_9;
  wire E, F;
  wire [3: 0] A;
  wire A2, A3;
  reg Start, clock, reset_b;

  Prob_8_9 M0 (E, F, A, A2, A3, Start, clock, reset_b);

  initial #500 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin reset_b = 0; #2 reset_b = 1; end
  initial fork
    #20 Start = 1;
    #40 reset_b = 0;
    #62 reset_b = 1;
  join
endmodule
```

VHDL

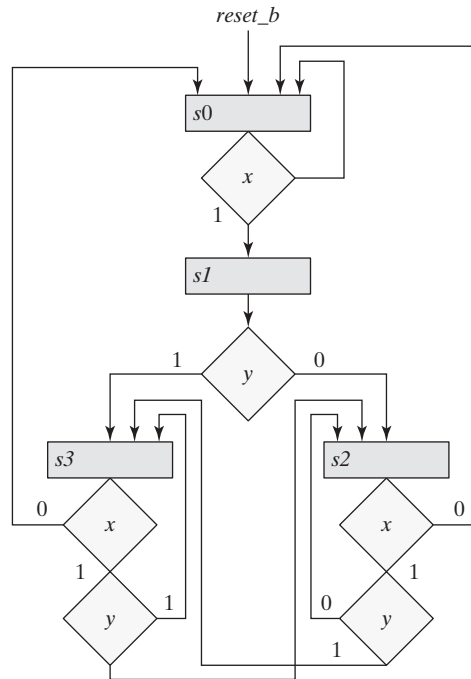
See Answers to Selected Problems in the text.

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
```

```
entity Prob_8_9_Datapath_vhdl is
  port (A2, A3: out Std_Logic; set_E, clr_E, set_F, clr_A_F, incr_A, clock, reset_b: in Std_Logic);
end Prob_8_9_Datapath_vhdl;
```

```
architecture Behavioral of Prob_8_9_Datapath_vhdl is
  signal E, F: out Std_Logic;
  signal A: out Std_Logic_Vector (3 downto 0);
begin
  A2 <= A(2);
  A3 <= A(3);
  process (clock, reset_b) begin
    if reset_b = '0' then E <= '0', F <= '0', A <= "0000";
    else
      if set_E = '1' then E <= '1'; end if;
      if clr_E = '1' then E <= '0'; end if;
      if set_F = '1' then F <= '1'; end if;
      if clr_a_F = '1' then A = "0000"; F <= '0'; end if;
      if incr_A = '1' then A <= A + "0001"; end if;
    end if;
  end process;
end Behavioral;
```

8.10



```

module Prob_8_10 (input x, y, clock, reset_b);
  reg [ 1: 0] state, next_state;
  parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= s0; else state <= next_state;
  always @ (state, x, y) begin
    next_state = s0;
    case (state)
      s0: if (x == 0) next_state = s0; else next_state = s1;
      s1: if (y == 0) next_state = s2; else next_state = s3;
      s2: if (x == 0) next_state = s0; else if (y == 0) next_state = s2; else next_state = s3;
      s3: if (x == 0) next_state = s0; else if (y == 0) next_state = s2; else next_state = s3;
    endcase
  end
endmodule

```


8.11

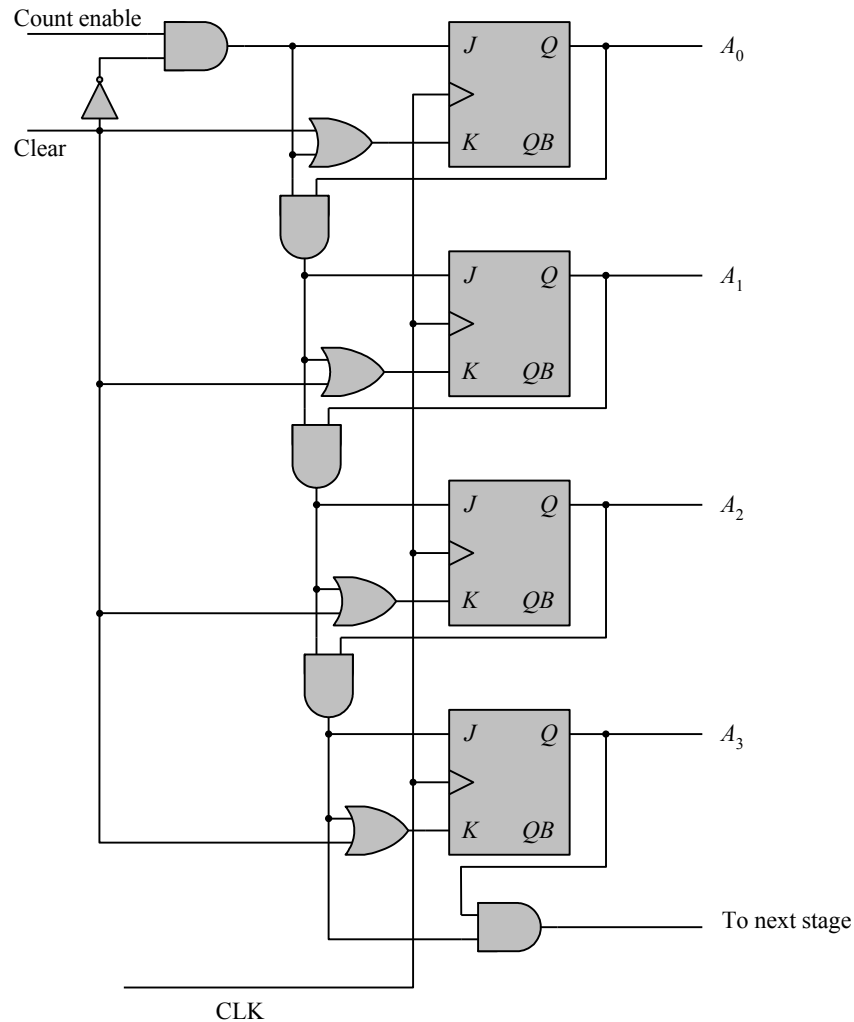
Present state		Inputs		Next state	
A	B	x	y	A	B
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	1	1
0	1	1	0	1	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	1	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	1	0
1	1	1	1	1	1

On simplifying, we get the D flip-flop equations as:

$$D_A = A'B + Ax$$

$$D_B = A'B'x + A'By + xy$$

- 8.12** For the 4-bit synchronous counter, modify the counter in Fig. 6.12 to add a signal, Clear, to clear the counter synchronously, as shown in the circuit diagram below.



(a) VERILOG

```

module Counter_4bit_Synch_Clr (output [3: 0] A, output next_stage, input Count_enable, Clear, CLK);
  wire A0, A1, A2, A3;
  assign A[3: 0] = {A3, A2, A1, A0};
  JK_FF M0 (A0, J0, K0, CLK);
  JK_FF M1 (A1, J1, K1, CLK);
  JK_FF M2 (A2, J2, K2, CLK);
  JK_FF M3 (A3, J3, K3, CLK);

  not (Clear_b, Clear);
  and (J0, Count_enable, Clear_b);
  and (J1, J0, A0);
  and (J2, J1, A1);
  and (J3, J2, A2);

  or (K0, Clear, J0);
  or (K1, Clear, J1);
  or (K2, Clear, J2);
  or (K3, Clear, J3);

  and (next_stage, A3, J3);
endmodule

```

```

module JK_FF (output reg Q, input J, K, clock);
  always @ (posedge clock)
    case ({J,K})
      2'b00: Q <= Q;
      2'b01: Q <= 0;
      2'b10: Q <= 1;
      2'b11: Q <= ~Q;
    endcase
endmodule

```

```

module t_Counter_4bit_Synch_Clr ();
  wire [3: 0] A;
  wire next_stage;
  reg Count_enable, Clear, clock;

```

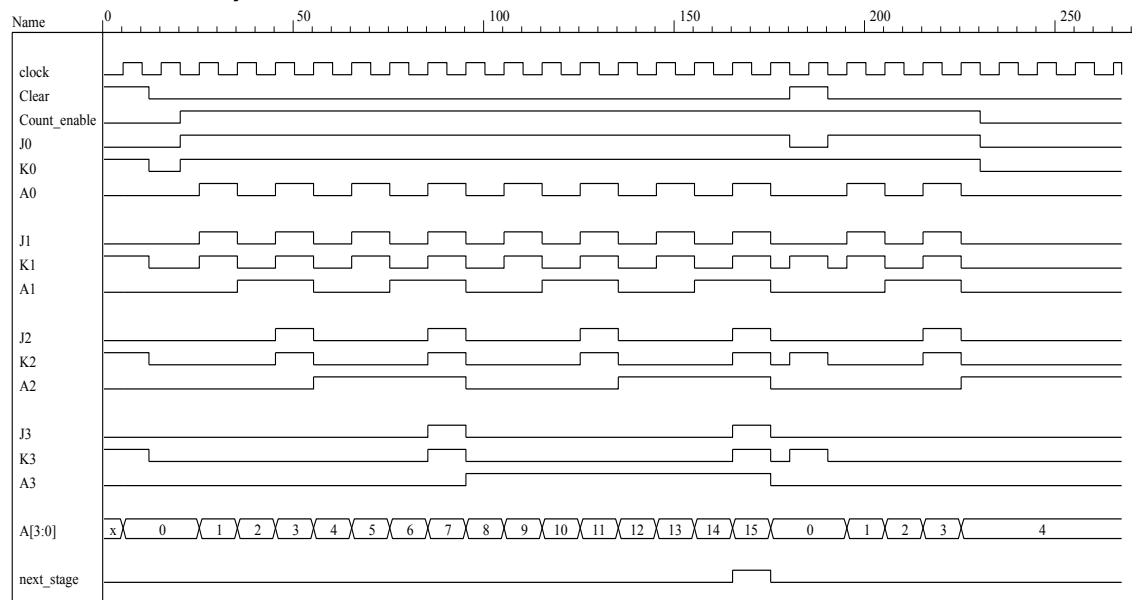
Counter_4bit_Synch_Clr M0 (A, next_stage, Count_enable, Clear, clock);

```

initial #300 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
  Clear = 1;
  Count_enable = 0;
  #12 Clear = 0;
  #20 Count_enable = 1;
  #180 Clear = 1;
  #190 Clear = 0;
  #230 Count_enable = 0;
join
endmodule

```

Simulation results: synchronous clear.



(a) VHDL

```

entity Prob_8_12a_Counter_4bit_Synch_Clr_vhdl is
  port (A: out Std_Logic_vector (3 downto 0); next_stage: out Std_Logic; Count_enable, Clear, CLK: in
bit);
end Prob_8_12_Counter_4bit_Synch_Clr_vhdl;

```

```

architecture Structural of Prob_8_12a_Counter_4bit_Synch_Clr_vhdl is
    signal A0, A1, A2, A3: Std_Logic;
    component JK_FF port (Q: buffer Std_Logic; J, K, clock: in Std_Logic); end component;
    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic); end component;
    component or2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic); end component;
    component not_gate port (y: out Std_Logic; xin: in Std_Logic); end component;

begin
    A(3 downto 0) <= A3 & A2 & A1 & A0;

    M0: JK_FF port map (A0, J0, K0, CLK);
    M1: JK_FF port map (A1, J1, K1, CLK);
    M2: JK_FF port map (A2, J2, K2, CLK);
    M3: JK_FF port map (A3, J3, K3, CLK);
    M4: not_gate port map (Clear_b, Clear);
    M6: and port map (J0, Count_enable, Clear_b);
    M7: and port map (J1, J0, A0);
    M8: and port map (J2, J1, A1);
    M9: and port map (J3, J2, A2);
    M10: or port map (K0, Clear, J0);
    M11: or port map (K1, Clear, J1);
    M12: or port map (K2, Clear, J2);
    M13: or port map (K3, Clear, J3);
    M14: and port map (next_stage, A3, J3);
end Structural;

entity JK_FF is
    port (Q: buffer bit; J, K, clock: in bit);
end JK_FF;

architecture Behavioral of JK_FF is
begin
    process (clock) begin
        if clock'event and clock = '1' then
            case (J & K) is
                when "00": Q <= Q;
                when "01": Q <= '0';
                when "10": Q <= '1';
                when "11": Q <= not Q;
            end case;
        end if;
    end Behavioral;

entity and2_gate is
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavioral of and2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;

entity or2_gate is
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end or2_gate;

architecture Behavioral of or2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;

entity not_gate is

```

```

    port (y: out Std_Logic; xin: in Std_Logic);
end not_gate;

```

```

architecture Behavioral of not_gate is
begin
    y <= not xin;
end Behavioral;

```

(b) Verilog

```

module Counter_4bit_Asynch_Clr_b (
    output [3: 0] A, output next_stage, input Count_enable, Clk, Clear_b
);
    wire A0, A1, A2, A3;
    assign A[3: 0] = {A3, A2, A1, A0};
    wire J0, K0, J1, K1, J2, K2, J3, K3;
    assign K0 = J0;
    assign K1 = J1;
    assign K2 = J2;
    assign K3 = J3;

    JK_FF M0 (A0, J0, K0, Clk, Clear_b);
    JK_FF M1 (A1, J1, K1, Clk, Clear_b);
    JK_FF M2 (A2, J2, K2, Clk, Clear_b);
    JK_FF M3 (A3, J3, K3, Clk, Clear_b);

    and (J0, Count_enable, Clear_b);
    and (J1, J0, A0);
    and (J2, J1, A1);
    and (J3, J2, A2);

    and (next_stage, A3, J3);
endmodule

```

```

entity JK_FF is
    port (Q: buffer bit; J, K, clock, Clear_b: in bit);
end JK_FF;

```

```

architecture Behavioral of JK_FF is
    process (clock, Clear_b) begin
        if (Clear_b = '0') then Q <= '0';
        elsif clock'event and clock = '1' then
            case (J & K) is
                when "00" => Q <= Q;
                when "01" => Q <= '0';
                when "10" => Q <= '1';
                when "11" => Q <= not Q;
            end case;
        end if;
    end Behavioral;

```

```

module t_Counter_4bit_Asynch_Clr_b ();
    wire [3: 0] A;
    wire next_stage;
    reg Count_enable, Clk, Clear_b;

    Counter_4bit_Asynch_Clr_b M0 (A, next_stage, Count_enable, Clk, Clear_b);

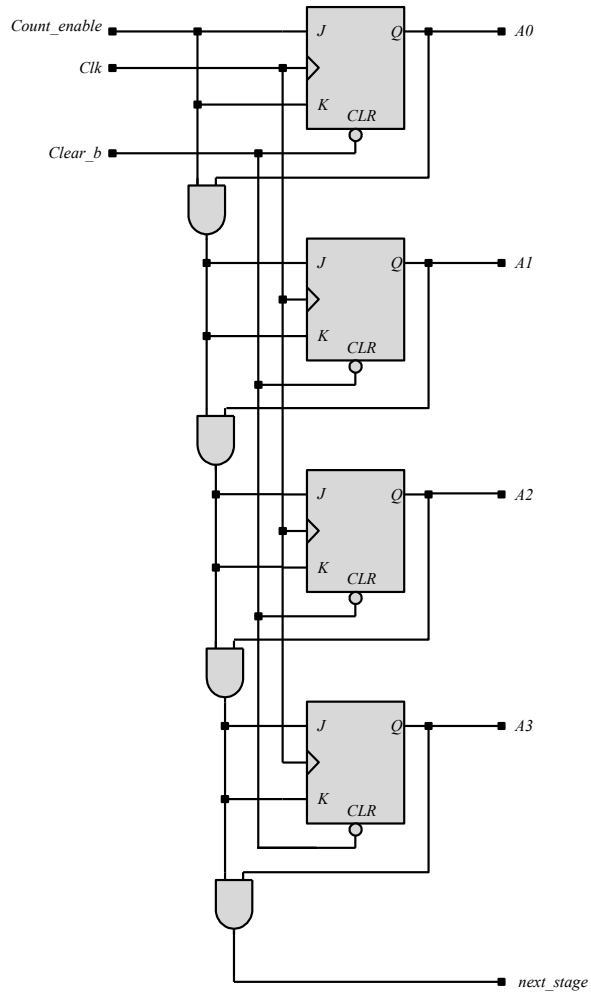
    initial #200 $finish;
    initial begin Clk = 0; forever #5 Clk = ~Clk; end
    initial fork
        Count_enable = 0;

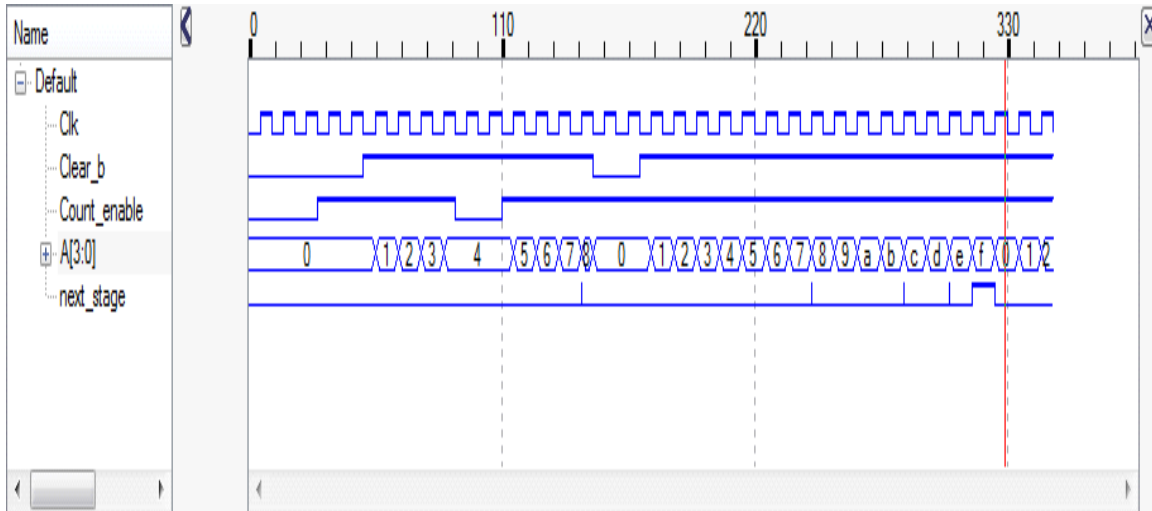
```

```

Clear_b = 0;
#30 Count_enable = 1;
#50 Clear_b = 1;
#90 Count_enable = 0;
#110 Count_enable = 1;
#150 Clear_b = 0;
#170 Clear_b = 1;
join
endmodule

```





(b) VHDL

entity Prob_8_12b_Counter_4bit_Asynch_Clr_b_vhdl **is**

port (A: **out** bit_vector (3 **downto** 0); next_stage: **out** bit; Count_enable, Clk, Clear_b: **in** bit);
end Prob_8_12b_Counter_4bit_Asynch_Clr_b_vhdl;

architecture Structural **of** Prob_8_12b_Counter_4bit_Asynch_Clr_b_vhdl **is**

signal A0, A1, A2, A3: **bit**;

signal J0, K0, J1, K1, J2, K2, J3, K3: **bit**;

component JK_FF **port** (Q: **buffer** bit; J, K, clock, Clear_b: **in** bit); **end component**;

component and2_gate **port** (y: **out** bit; xin1, xin2: **in** bit); **end component**;

begin

A(3: 0) <= A3 & A2 & A1 & A0;

K0 <= J0;

K1 <= J1;

K2 <= J2;

K3 <= J3;

M0: JK_FF **port map** (A0, J0, K0, Clk, Clear_b);

M1: JK_FF **port map** (A1, J1, K1, Clk, Clear_b);

M2: JK_FF **port map** (A2, J2, K2, Clk, Clear_b);

M3: JK_FF **port map** (A3, J3, K3, Clk, Clear_b);

M4: and2_gate **port map** (J0, Count_enable);

M5: and2_gate **port map** (J1, J0, A0);

M6: and2_gate **port map** (J2, J1, A1);

M7: and2_gate **port map** (J3, J2, A2);

M8: and2_gate **port map** (next_stage, A3, J3);

end Structural;

entity JK_FF **is**

port (Q: **buffer** bit; J, K, clock, Clear_b: **in** bit);

end JK_FF;

architecture Behavioral **of** JK_FF **is**

begin

process (clock, Clear_b) **begin**

if (Clear_b = '0') **then** Q <= '0';

elsif clock'event **and** clock = '1' **then**

case (J & K) **is**

when "00" => Q <= Q;

```
        when "01" => Q <= '0';
        when "10" => Q <= '1';
        when "11" => Q <= not Q;
    end case;
end Behavioral;

entity and2_gate
    port (y: out bit; xin1, xin2: in bit);
end and2_gate;

architecture Behavioral of and2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;
```

8.13

```
// Structural description of design example (Fig. 8-10, 8-12)
module Design_Example_STR

  ( output [3:0]  A,
    output       E, F,
    input       Start, clock, reset_b
  );

  Controller_STR M0 (clr_A_F, set_E, clr_E, set_F, incr_A, Start, A[2], A[3], clock, reset_b );
  Datapath_STR M1 (A, E, F, clr_A_F, set_E, clr_E, set_F, incr_A, clock);
endmodule

module Controller_STR
  ( output clr_A_F, set_E, clr_E, set_F, incr_A,
    input Start, A2, A3, clock, reset_b
  );

  wire      G0, G1;
  parameter S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11;
  wire      w1, w2, w3;

  not (G0_b, G0);
  not (G1_b, G1);
  buf (incr_A, w2);
  buf (set_F, G1);
  not (A2_b, A2);
  or (D_G0, w1, w2);
  and (w1, Start, G0_b);
  and (clr_A_F, G0_b, Start);
  and (w2, G0, G1_b);
  and (set_E, w2, A2);
  and (clr_E, w2, A2_b);
  and (D_G1, w3, w2);
  and (w3, A2, A3);
  D_flip_flop_AR M0 (G0, D_G0, clock, reset_b);
  D_flip_flop_AR M1 (G1, D_G1, clock, reset_b);
endmodule

// datapath unit

module Datapath_STR
  ( output [3: 0] A,
    output       E, F,
    input       clr_A_F, set_E, clr_E, set_F, incr_A, clock
  );

  JK_flip_flop_2 M0 (E, E_b, set_E, clr_E, clock);
  JK_flip_flop_2 M1 (F, F_b, set_F, clr_A_F, clock);
  Counter_4 M2 (A, incr_A, clr_A_F, clock);

endmodule

module Counter_4 (output reg [3: 0] A, input incr, clear, clock);
  always @ (posedge clock)
    if (clear)  A <= 0; else if (incr) A <= A + 1;
endmodule

module D_flip_flop_AR (Q, D, CLK, RST);
  output  Q;
  input D, CLK, RST;
  reg    Q;
```



```

always @ (posedge CLK, negedge RST)
  if (RST == 0) Q <= 1'b0;
  else Q <= D;
endmodule

module JK_flip_flop_2 (Q, Q_not, J, K, CLK);
  output Q, Q_not;
  input J, K, CLK;
  reg Q;

  assign Q_not = ~Q;
  always @ (posedge CLK)
    case ({J, K})
      2'b00: Q <= Q;
      2'b01: Q <= 1'b0;
      2'b10: Q <= 1'b1;
      2'b11: Q <= ~Q;
    endcase
endmodule

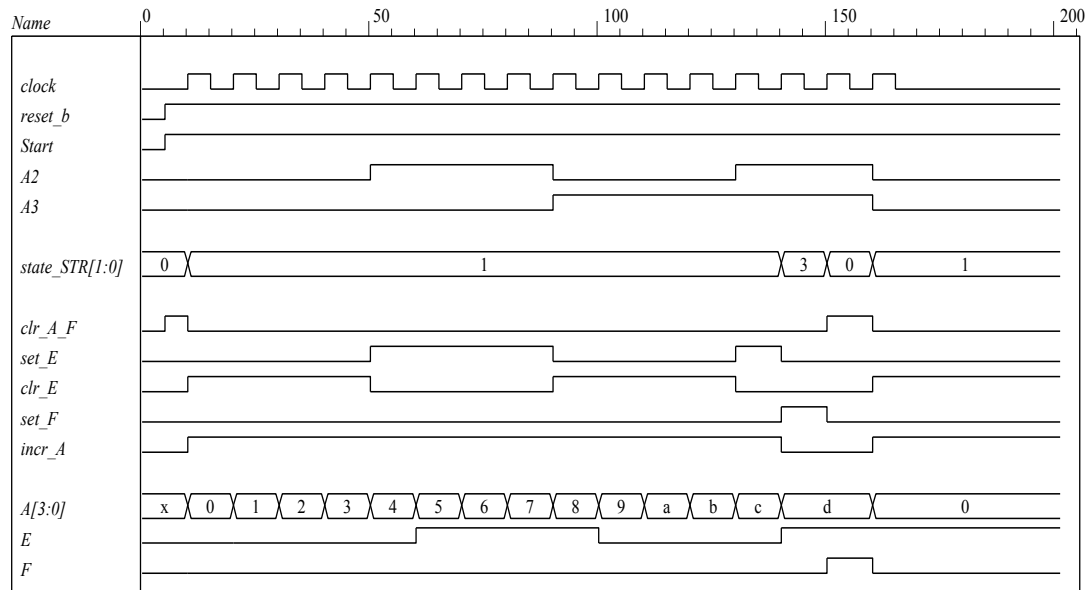
module t_Design_Example_STR;
  reg Start, clock, reset_b;
  wire [3:0] A;
  wire E, F;
  wire [1:0] state_STR = {M0.M0.G1, M0.M0.G0};

  Design_Example_STR M0 (A, E, F, Start, clock, reset_b);

  initial #500 $finish;
  initial
    begin
      reset_b = 0;
      Start = 0;
      clock = 0;
      #5 reset_b = 1; Start = 1;
      repeat (32)
        begin
          #5 clock = ~ clock;
        end
      end
    initial
      $monitor ("A = %b E = %b F = %b time = %0d", A, E, F, $time);
endmodule

```

The simulation results shown below match Fig. 8.13.



8.14 The state code 2'b10 is unused. If the machine enters an unused state, with the controller written with default assignment to *next_state*, the default assignment forces the state to *S_idle*, so the machine recovers from the condition.

8.15 Modify the test bench to insert a reset event and extend the clock.

// RTL description of design example (see Fig.8-11)

module Design_Example_RTL (A, E, F, Start, clock, reset_b);

// Specify ports of the top-level module of the design
// See block diagram Fig. 8-10

output [3: 0] A;
output E, F;
input Start, clock, reset_b;

// Instantiate controller and datapath units

Controller_RTL M0 (set_E, clr_E, set_F, clr_A_F, incr_A, A[2], A[3], Start, clock, reset_b);
Datapath_RTL M1 (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);

endmodule

module Controller_RTL (set_E, clr_E, set_F, clr_A_F, incr_A, A2, A3, Start, clock, reset_b);

output reg set_E, clr_E, set_F, clr_A_F, incr_A;
input Start, A2, A3, clock, reset_b;
reg [1:0] state, next_state;
parameter S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11; // State codes

always @ (posedge clock or negedge reset_b) // State transitions (edge-sensitive)
if (reset_b == 0) state <= S_idle;
else state <= next_state;

// Code next state logic directly from ASMD chart (Fig. 8-9d)

always @ (state, Start, A2, A3) begin // Next state logic (level-sensitive)

```

    next_state = S_idle;
case (state)
    S_idle: if (Start) next_state = S_1; else next_state = S_idle;
    S_1:    if (A2 & A3) next_state = S_2; else next_state = S_1;
    S_2:    next_state = S_idle;
    default: next_state = S_idle;
endcase
end

// Code output logic directly from ASMD chart (Fig. 8-9d)

always @ (state, Start, A2) begin
    set_E = 0;      // default assignments; assign by exception
    clr_E = 0;
    set_F = 0;
    clr_A_F = 0;
    incr_A = 0;
    case (state)
    S_idle:         if (Start) clr_A_F = 1;
    S_1:            begin incr_A = 1; if (A2) set_E = 1; else clr_E = 1; end
    S_2:            set_F = 1;
    endcase
end
endmodule

module Datapath_RTL (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
    output reg [3: 0] A;      // register for counter
    output reg      E, F;    // flags
    input          set_E, clr_E, set_F, clr_A_F, incr_A, clock;

    // Code register transfer operations directly from ASMD chart (Fig. 8-9d)

    always @ (posedge clock) begin
        if (set_E)          E <= 1;
        if (clr_E)          E <= 0;
        if (set_F)          F <= 1;
        if (clr_A_F)        begin A <= 0; F <= 0; end
        if (incr_A)         A <= A + 1;
    end
endmodule

module t_Design_Example_RTL;
    reg      Start, clock, reset_b;
    wire [3: 0] A;
    wire      E, F;

    // Instantiate design example

    Design_Example_RTL M0 (A, E, F, Start, clock, reset_b);

    // Describe stimulus waveforms

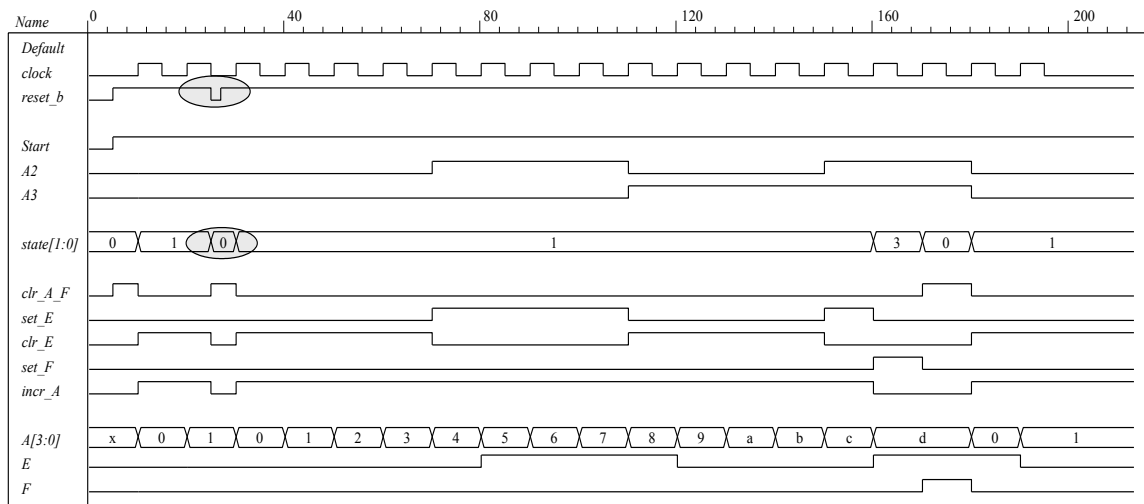
    initial #500 $finish;    // Stopwatch
    initial fork
        #25 reset_b = 0;    // Test for recovery from reset on-the-fly.
        #27 reset_b = 1;
    join
    initial
    begin
        reset_b = 0;
        Start = 0;
        clock = 0;

```

```

#5 reset_b = 1; Start = 1;
//repeat (32)
repeat (38)          // Modify for test of reset_b on-the-fly
begin
    #5 clock = ~ clock;    // Clock generator
end
end
initial
$monitor ("A = %b E = %b F = %b time = %0d", A, E, F, $time);
endmodule

```

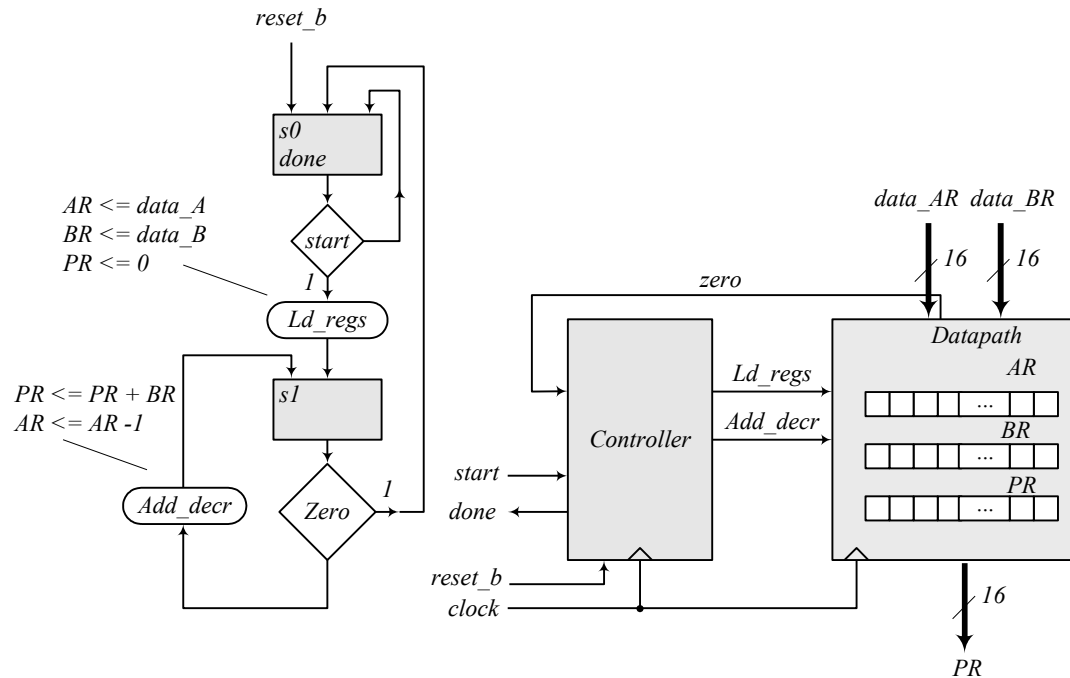


8.16 RTL notation:

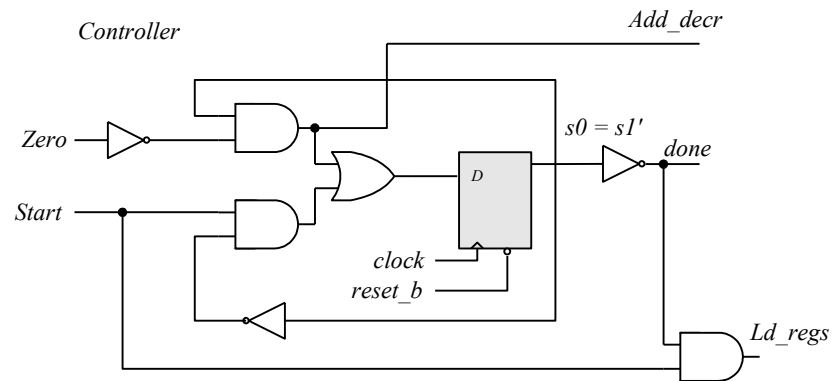
s0: (initial state) If *start* = 0 go back to state *s0*, If (*start* = 1) then $BR \leftarrow multiplicand$, $AR \leftarrow multiplier$, $PR \leftarrow 0$, go to *s1*.

s1: (check *AR* for Zero) *Zero* = 1 if *AR* = 0, if (*Zero* = 1) then go back to *s0* (*done*) If (*Zero* = 0) then go to *s1*, $PR \leftarrow PR + BR$, $AR \leftarrow AR - 1$.

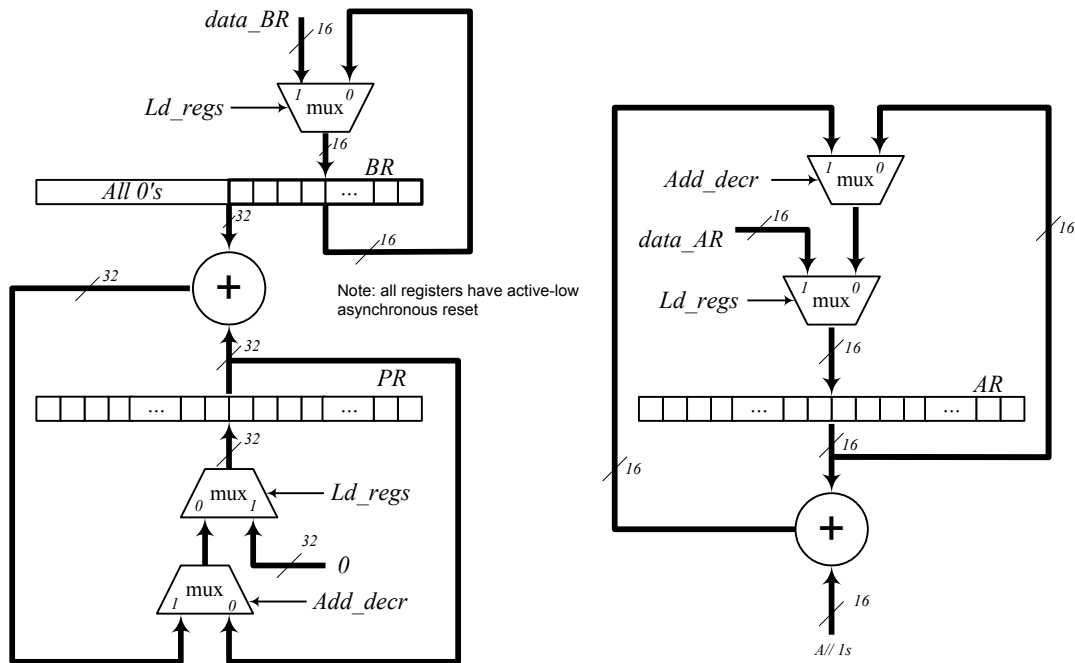
The internal architecture of the datapath consists of a double-width register to hold the product (*PR*), a register to hold the multiplier (*AR*), a register to hold the multiplicand (*BR*), a double-width parallel adder, and single-width parallel adder. The single-width adder is used to implement the operation of decrementing the multiplier unit. Adding a word consisting entirely of 1s to the multiplier accomplishes the 2's complement subtraction of 1 from the multiplier. Figure 8.16 (a) below shows the ASMD chart, block diagram, and controller of the circuit. Figure 8.16 (b) shows the internal architecture of the datapath. Figure 8.16 (c) shows the results of simulating the circuit.



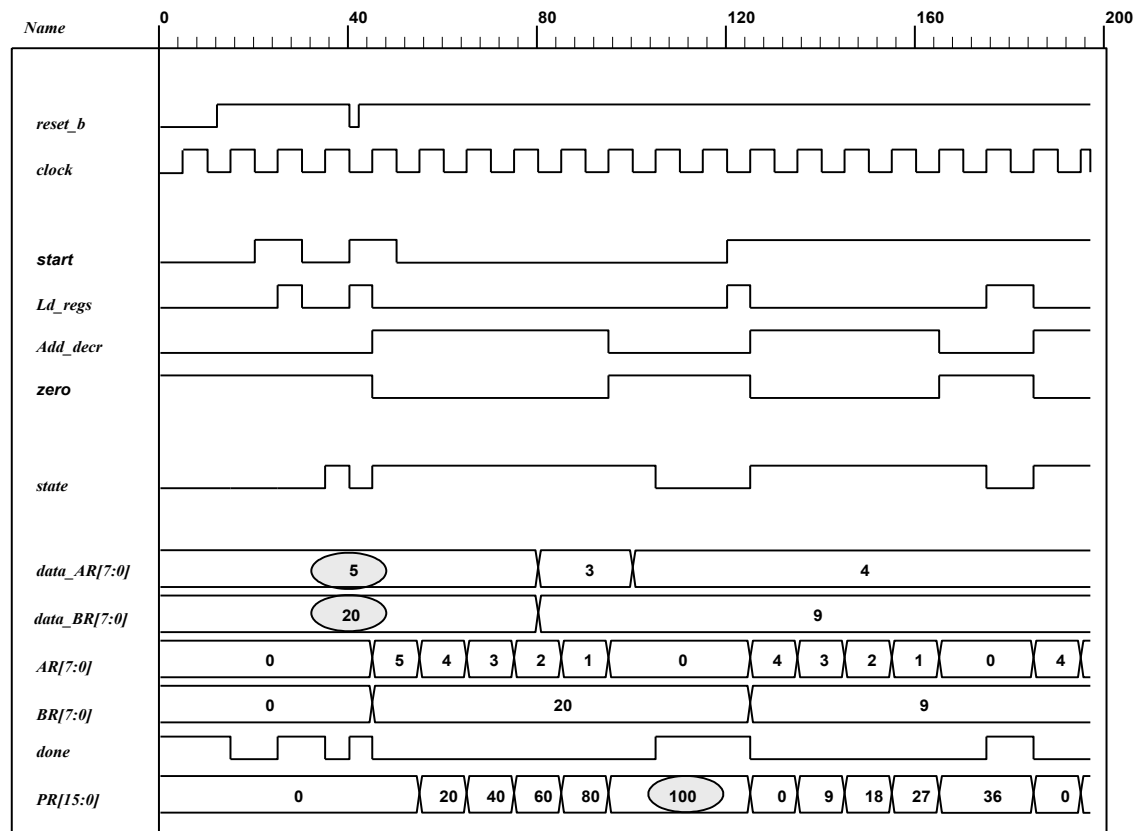
Note: Form Zero as the output of an OR gate whose inputs are the bits of the register AR.



(a) ASMD chart, block diagram, and controller



(b) Datapath



(c) Simulation results

```

module Prob_8_16_STR (
output [15: 0] PR, output done,
input [7: 0] data_AR, data_BR, input start, clock, reset_b
);

Controller_P8_16 M0 (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

Datapath_P8_16 M1 (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
endmodule

module Controller_P8_16 (output done, output reg Ld_regs, Add_decr, input start, zero, clock, reset_b);
parameter s0 = 1'b0, s1 = 1'b1;
reg state, next_state;
assign done = (state == s0);

always @ (posedge clock, negedge reset_b)
if (!reset_b) state <= s0; else state <= next_state;

always @ (state, start, zero) begin
Ld_regs = 0;
Add_decr = 0;
case (state)
s0: if (start) begin Ld_regs = 1; next_state = s1; end
s1: if (zero) next_state = s0; else begin next_state = s1; Add_decr = 1; end
default: next_state = s0;
endcase
end
endmodule

module Register_32 (output [31: 0] data_out, input [31: 0] data_in, input clock, reset_b);
Register_8 M3 (data_out [31: 24] , data_in [31: 24], clock, reset_b);
Register_8 M2 (data_out [23: 16] , data_in [23: 16], clock, reset_b);
Register_8 M1 (data_out [15: 8] , data_in [15: 8], clock, reset_b);
Register_8 M0 (data_out [7: 0] , data_in [7: 0], clock, reset_b);
endmodule

module Register_16 (output [15: 0] data_out, input [15: 0] data_in, input clock, reset_b);
Register_8 M1 (data_out [15: 8] , data_in [15: 8], clock, reset_b);
Register_8 M0 (data_out [7: 0] , data_in [7: 0], clock, reset_b);
endmodule

module Register_8 (output [7: 0] data_out, input [7: 0] data_in, input clock, reset_b);
D_flop M7 (data_out[7] data_in[7], clock, reset_b);
D_flop M6 (data_out[6] data_in[6], clock, reset_b);
D_flop M5 (data_out[5] data_in[5], clock, reset_b);
D_flop M4 (data_out[4] data_in[4], clock, reset_b);
D_flop M3 (data_out[3] data_in[3], clock, reset_b);
D_flop M2 (data_out[2] data_in[2], clock, reset_b);
D_flop M1 (data_out[1] data_in[1], clock, reset_b);
D_flop M0 (data_out[0] data_in[0], clock, reset_b);
endmodule

module Adder_32 (output c_out, output [31: 0] sum, input [31: 0] a, b);
assign {c_out, sum} = a + b;
endmodule

module Adder_16 (output c_out, output [15: 0] sum, input [15: 0] a, b);
assign {c_out, sum} = a + b;
endmodule

```

```

module D_flop (output q, input data, clock, reset_b);
always @ (posedge clock, negedge reset_b)
if (!reset_b) q <= 0; else q <= data;
endmodule

module Datapath_P8_16 (
output reg [15: 0] PR, output zero,
input [7: 0] data_AR, data_BR, input Ld_regs, Add_decr, clock, reset_b
);

reg [7: 0] AR, BR;
assign zero = ~( | AR);

always @ (posedge clock, negedge reset_b)
if (!reset_b) begin AR <= 8'b0; BR <= 8'b0; PR <= 16'b0; end
else begin
if (Ld_regs) begin AR <= data_AR; BR <= data_BR; PR <= 0; end
else if (Add_decr) begin PR <= PR + BR; AR <= AR -1; end
end
endmodule

// Test plan – Verify;
// Power-up reset
// Data is loaded correctly
// Control signals assert correctly
// Status signals assert correctly
// start is ignored while multiplying
// Multiplication is correct
// Recovery from reset on-the-fly

module t_Prob_P8_16;
wire done;
wire [15: 0] PR;
reg [7: 0] data_AR, data_BR;
reg start, clock, reset_b;

Prob_8_16_STR M0 (PR, done, data_AR, data_BR, start, clock, reset_b);

initial #500 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
reset_b = 0;
#12 reset_b = 1;
#40 reset_b = 0;
#42 reset_b = 1;
#90 reset_b = 1;
#92 reset_b = 1;
join

initial fork
#20 start = 1;
#30 start = 0;
#40 start = 1;
#50 start = 0;
#120 start = 1;
#120 start = 0;
join

```



```
initial fork  
data_AR = 8'd5;    // AR > 0  
data_BR = 8'd20;  
  
#80 data_AR = 8'd3;  
#80 data_BR = 8'd9;  
  
#100 data_AR = 8'd4;  
#100 data_BR = 8'd9;  
join  
endmodule
```

VHDL

entity Prob_8_16_STR **is**

port (PR: **out** Std_Logic_Vector (15 **downto** 0); done: **out** Std_Logic; data_AR, data_BR: **in** Std_Logic_Vector (7 **downto** 0); start, clock, reset_b: **in** Std_Logic);
end Prob_8_16_STR;

Architecture Structural **of** Prob_8_16_STR **is**

component Controller_P8_16 **port** (done, Ld_regs, Add_decr: **out** Std_Logic ; start, zero, clock, reset_b: **in** Std_Logic);

 component Datapath_P8_16 **port** (PR: **out** Std_Logic_Vector (15 **downto** 0); zero: **out** Std_Logic; data_AR, data_BR: **in** Std_Logic_Vector (7 **downto** 0); Ld_regs, Add_decr, clock, reset_b: **in** Std_Logic);
 begin
 zero <= !
 M0 Controller_P8_16 **port map** (done, Ld_regs, Add_decr, start, zero, clock, reset_b);
 M1 Datapath_P8_16 **port map** (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
end Structural;

entity Controller_P8_16 **is**

port (done, Ld_regs, Add_decr: **out** Std_Logic; start, zero, clock, reset_b: **in** Std_Logic);
end Controller_P8_16;

architecture Behavioral of Controller_P8_16 **is**

constant s0 = 1'b0, s1 = 1'b1;
 signal state, next_state: Std_Logic;
begin
 done <= (state = s0);

process (clock, reset_b)
 if reset_b'event and reset_b = 0 **then** state <= s0; **else** state <= next_state; **end if**;

process @ (state, start, zero) **begin**
 Ld_regs <= 0;
 Add_decr <= 0;
 case state **is**
 when s0 => **if** (start = 1) **then** **begin** Ld_regs <= 1; next_state <= s1; **end if**;
 when s1 => **if** (zero = 1) **then** next_state <= s0; **else** **begin**
 next_state = s1; Add_decr = 1; **end if**;
 others: next_state <= s0;
 end case
end Behavioral;

entity Register_32 **is**

port (data_out: **out** Std_Logic_Vector (31 **downto** 0);
 data_in: **in** Std_Logic_Vector (31 **downto** 0); clock, reset_b: **in** Std_Logic);
end Register_32;

architecture Structural **of** Register_32 **is**

component Register_8 **port** (Dout: **out** Std_Logic_Vector (7 **downto** 0);
 Din: **in** Std_Logic_Vector (7 **downto** 0);
 clock, reset_b: **in** Std_Logic);
begin
 M0 Register_8 **port map** (Dout => data_out(31:24);
 Din => data_in(31:24); clock => clock, reset_b => reset_b);
 M1 Register_8 **port map** (Dout => data_out(23:16);
 Din => data_in(23:16); clock => clock, reset_b => reset_b);
 M2 Register_8 **port map** (Dout => data_out(15:8);
 Din => data_in(15:8); clock => clock, reset_b => reset_b);
 M3 Register_8 **port map** (Dout => data_out(7:0);
 Din => data_in(7:0); clock => clock, reset_b => reset_b);
end Structural;

```

entity Register_16 is
  port (output [15: 0] data_out: out Std_Logic_Vector (15 downto 0);
        data_in: in Std_Logic_Vector (15 downto 0);
        clock, reset_b: in Std_Logic);
end Register_16;

architecture Structural of Register_16 is
  component Register_8 port (data_out: out Std_Logic_Vector (7 downto 0);
        data_in: in Std_Logic_Vector (7 downto 0); clock, reset_b: in Std_Logic);
  begin
    M0 Register_8 port map (data_out(15 downto 8), data_in (15 downto 8), clock, reset_b);
    M1 Register_8 port map (data_out(7 downto 0), data_in (7 downto 0), clock, reset_b);
  end Structural;

entity Register_8 is
  port (data_out: out Std_Logic_Vector (7 downto 0); data_in: in Std_Logic_Vector (7 downto 0);
        clock, reset_b: in Std_Logic);
end Register_8;

architecture Structural of Register_8 is
  component D_flop port (q: out Std_Logic; data, clock, reset_b: in Std_Logic);
  begin
    M7 D_flop M7 (data_out[7] data_in[7], clock, reset_b);
    M6 D_flop M6 (data_out[6] data_in[6], clock, reset_b);
    M5 D_flop M5 (data_out[5] data_in[5], clock, reset_b);
    M4 D_flop M4 (data_out[4] data_in[4], clock, reset_b);
    M3 D_flop M3 (data_out[3] data_in[3], clock, reset_b);
    M2 D_flop M2 (data_out[2] data_in[2], clock, reset_b);
    M1 D_flop M1 (data_out[1] data_in[1], clock, reset_b);
    M0 D_flop M0 (data_out[0] data_in[0], clock, reset_b);
  end Structural;

entity Adder_32 is
  port (c_out: out Std_Logic; sum: out Std_Logic_Vector (31 downto 0);
        a, b: in Std_Logic_Vector (31 downto 0);
end Adder_32;

architecture Behavioral of adder_32 is
  (c_out & sum) <= a + b;
end Behavioral;

module Adder_16 (output c_out, output [15: 0] sum, input [15: 0] a, b);
assign {c_out, sum} = a + b;
endmodule

entity Adder_16 is
  port (c_out: out Std_Logic; sum: out Std_Logic_Vector (15 downto 0);
        a, b: in Std_Logic_Vector (15 downto 0);
end Adder_16;

architecture Behavioral of adder_16 is
  (c_out & sum) <= a + b;
end Behavioral;

```

```

entity D_flop is
  port (q: out Std_Logic; data, clock, reset_b: in Std_Logic);
end D_flop;

architecture Behavioral of D_flop is
begin
  process (clock, reset_b)
    if reset_b'event and reset_b = 0 then q <= 0;
    elsif clock'event and clock = 1 then q <= data; end if;
  end process;
end Behavioral;

entity Datapath_P8_16 is
  port (PR: out Std_Logic_Vector (15 downto 0);
        zero: out std_Logic;
        data_AR, data_BR: in Std_Logic_Vector (7 downto 0);
        Ld_regs, Add_decr, clock, reset_b: in Std_Logic);
end Datapath_P8_16;

archtiecture Behavioral of Datapath_P8_16 is
  zero <= not (AR(7) or AR(6) or AR(5) or AR(4) or AR(3) or AR(2) or AR(1) or AR(0));
  process (clock, reset_b)
    if reset_b'event and reset_b = 0 then begin
      AR <= 8'b0; BR <= 8'b0; PR <= 16'b0; end
    elsif clock'event and clock = 1 then begin
      if (Ld_regs) begin AR <= data_AR; BR <= data_BR; PR <= 0; end;
      elsif Add_decr = 1 then begin PR <= PR + BR; AR <= AR -1; end
    end if;
  end process;
end Behavioral;

// Test plan – Verify;
// Power-up reset
// Data is loaded correctly
// Control signals assert correctly
// Status signals assert correctly
// start is ignored while multiplying
// Multiplication is correct
// Recovery from reset on-the-fly

entity t_Prob_P8_16 is
end t_Prob_P8_16

architecture Test_Bench of t_Prob_P8_16 is ;
  signal t_done: Std_Logic;
  signal t_PR: Std_Logic_Vector (15 downto 0);
  signal t_atata_AR, t_data_BR: Std_Logic_Vector (7 downto 0);
  signal t_start, t_clock, t_reset_b;
  component Prob_8_16_STR port (PR: out Std_Logic_Vector (15 downto 0);
    done: out Std_Logic;
    data_AR, data_BR: Std_Logic_Vector (7 downto 0);
    start, clock, reset_b: in Std_Logic);

begin
  M_UUT Prob_8_16_STR M0 (PR => t_PR, done => t_done, data_AR => t_data_AR,
    data_BR => t_data_BR, start => t_start, clock => t_clock
    reset_b => t_reset_b);

  process (clock)
    t_clock <= 0;
    wait for 5 ns;
    clock <= '1';
  
```

```

    wait for 5 ns;
end process;

reset_b <= 0;
reset_b <= 1 after 12 ns;
reset_b <= 0 after 40 ns;
reset_b <= 1 after 42 ns;
reset_b <= 1 after 90 ns;
reset_b <= 1 after 92 ns;

start <= 1 after 20 ns;
start <= 0 after 30 ns;
start <= 1 after 40 ns;
start <= 0 after 50 ns;
start <= 1 after 120 ns;
start <= 0 after 120 ns;

data_AR <= 8'd5;    // AR > 0
data_BR <= 8'd20;

data_AR = 8'd3 after 80 ns;
data_BR = 8'd9 after 80 ns;

data_AR = 8'd4 after 100 ns;
data_BR = 8'd9 after 100 ns;

end Test_Bench;

```

8.17 $(2^n - 1)(2^n - 1) < (2^{2n} - 1)$ for $n \geq 1$

- 8.18** (a) The maximum product size is 40-bits available in registers A and Q .
 (b) P counter must have 5-bits to load 16 (binary 10000) initially.
 (c) Z (zero) detection is generated with a 5-input NOR gate.

8.19

Multiplicand $B = 11011_2 = 27_{10}$

Multiplier $Q = 10111_2 = 23_{10}$

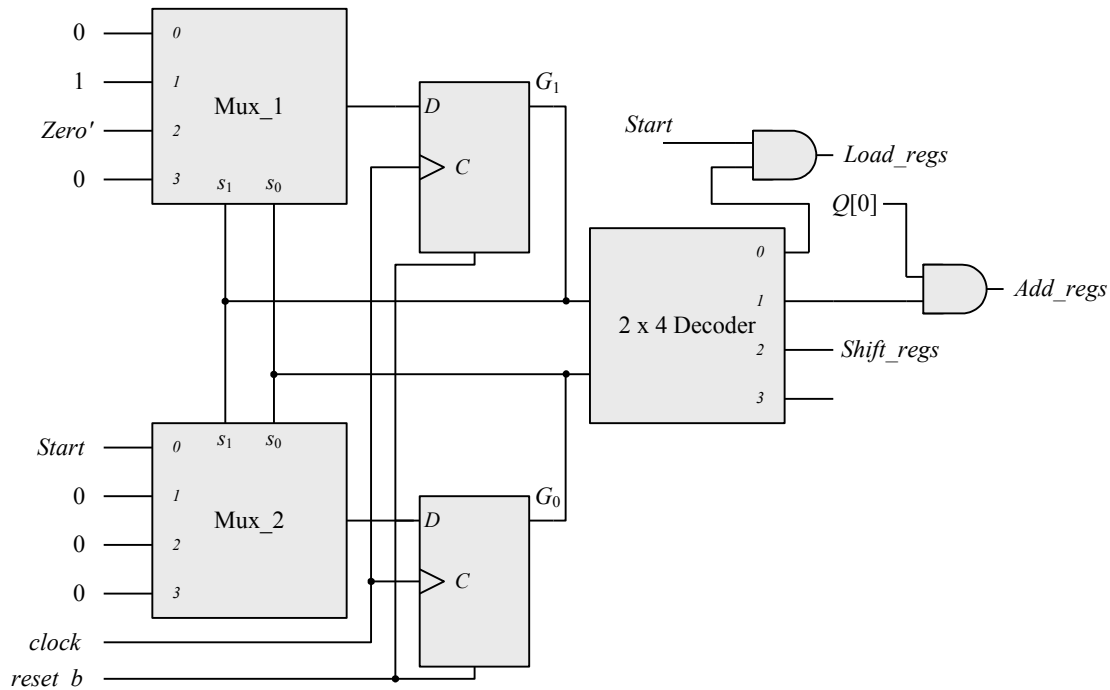
Product: $CAQ = 621_{10}$

	C	A	Q	P
Multipplier in Q	0	00000	10111	101
$Q0 = 1$; add B		11011		
First partial product	0	11011	10111	100
Shift right CAQ	0	01101	11011	
$Q0 = 1$; add B		11011		
Second partial product	1	01000	11011	011
Shift right CAQ	0	10100	01101	
$Q0 = 1$; add B		11011		
Third partial product	1	01111	01101	010
Shift right CAQ	0	10111	10110	
Shift right CAQ	0	01011	11011	
Fourth partial product	0	01011	11011	001
$Q0 = 1$; add B		11011		
Fifth partial product	1	00110	11011	000
Shift right CAQ	0	10011	01101	
Final product in AQ :				
$AQ = 10011\ 01101 = 621_{10}$				

- 8.20** Clock frequency = 20 MHz
 Clock cycle = $1/20\text{ MHz} = 50\text{ ns}$
 Total time to multiply = $(2n + 1)t = (2 \times 8 + 1) \times 50 = 850\text{ ns}$

8.21

State codes:	G_1	G_0
S_idle	0	0
S_add	0	1
S_shift	1	0
unused	0	0



- 8.22 Note that the machine described by Fig. P8.22 requires four states, but the machine described by Fig. 8.15 (b) requires only three. Also, observe that the sample simulation results show a case where the carry bit register, C, is needed to support the addition operation. The datapath is 8 bits wide.

Verilog

```

module Prob_8_22 # (parameter m_size = 9)
(
  output [2*m_size-1: 0] Product,
  output Ready,
  input [m_size-1: 0] Multiplicand, Multiplier,
  input Start, clock, reset_b
);
  wire [m_size-1: 0] A, Q;

  assign Product = {A, Q};
  wire Q0, Zero, Load_regs, Decr_P, Add_regs, Shift_regs;

  Datapath_Unit M0 (A, Q, Q0, Zero, Multiplicand, Multiplier, Load_regs, Decr_P, Add_regs, Shift_regs,
  clock, reset_b);
  Control_Unit M1 (Ready, Decr_P, Load_regs, Add_regs, Shift_regs, Start, Q0, Zero, clock, reset_b);
endmodule

```

```

module Datapath_Unit # (parameter m_size = 9, BC_size = 4)
(
  output reg [m_size -1: 0] A, Q,
  output Q0, Zero,
  input [m_size -1: 0] Multiplicand, Multiplier,
  input Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b
);
  reg C;
  reg [BC_size -1: 0] P;
  reg [m_size -1: 0] B;

  assign Q0 = Q[0];
  assign Zero = (P == 0);

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) begin
      B <= 0; C <= 0;
      A <= 0;
      Q <= 0;
      P <= m_size;
    end
    else begin
      if (Load_regs) begin
        A <= 0;
        C <= 0;
        Q <= Multiplier;
        B <= Multiplicand;
        P <= m_size;
      end
      if (Decr_P) P <= P -1;
      if (Add_regs) {C, A} <= A + B;
      if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
    end
  endmodule

module Control_Unit (
  output Ready, Decr_P, output reg Load_regs, Add_regs, Shift_regs, input Start, Q0, Zero, clock,
  reset_b
);
  reg [ 1: 0] state, next_state;
  parameter S_idle = 2'b00, S_loaded = 2'b01, S_sum = 2'b10, S_shifted = 2'b11;
  assign Ready = (state == S_idle);
  assign Decr_P = (state == S_loaded);

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= S_idle; else state <= next_state;

  always @ (state, Start, Q0, Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
      S_idle: if (Start == 0) next_state = S_idle; else begin next_state = S_loaded; Load_regs = 1; end
      S_loaded: if (Q0) begin next_state = S_sum; Add_regs = 1; end
      else begin next_state = S_shifted; Shift_regs = 1; end
      S_sum: begin next_state = S_shifted; Shift_regs = 1; end
      S_shifted: if (Zero) next_state = S_idle; else next_state = S_loaded;
    endcase
  end
endmodule

```



```

module t_Prob_8_22 ();
  parameter m_size = 9;           // Width of datapath
  wire [2 * m_size - 1: 0] Product;
  wire Ready;
  reg [m_size - 1: 0] Multiplicand, Multiplier;
  reg Start, clock, reset_b;
  integer Exp_Value;
  reg Error;

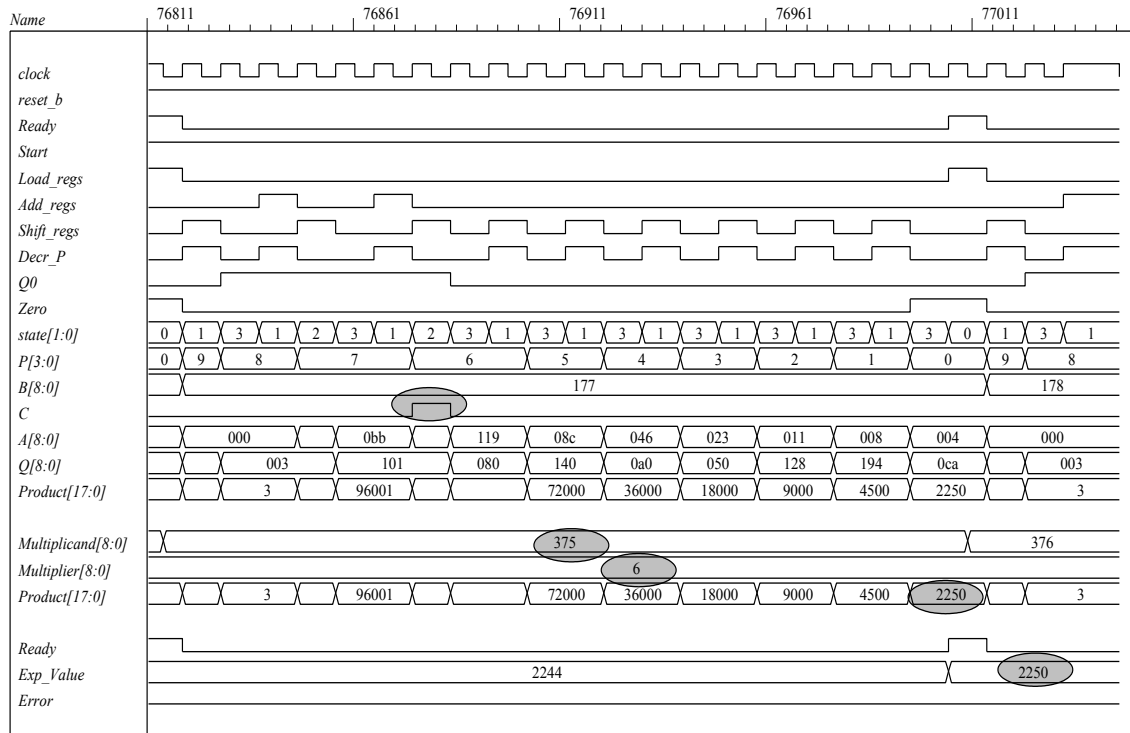
  Prob_8_22 M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

  initial #140000 $finish;
  initial begin clock = 0; #5 forever #5 clock = ~clock; end
  initial fork
    reset_b = 1;

    #2 reset_b = 0;
    #3 reset_b = 1;
  join
  initial begin #5 Start = 1; end
    always @ (posedge Ready) begin
      Exp_Value = Multiplier * Multiplicand;
      //Exp_Value = Multiplier * Multiplicand + 1; // Inject error to confirm detection
    end
    always @ (negedge Ready) begin
      Error = (Exp_Value ^ Product);
    end

  initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (64) #10 begin Multiplier = Multiplier + 1;
      repeat (64) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
  end
endmodule

```



VHDL

Library IEEE;

use IEEE.Std_Logic_1164.all;

entity Prob_8_22_vhdl **is**

generic (m_size: **integer** := 9);

port (Product: **out** Std_Logic_vector (2*m_size -1 **downto** 0);

 Ready: **out** Std_Logic;

 Multiplicand, Multiplier: **in** Std_Logic_vector (m_size -1 **downto** 0);

 Start, clock, reset_b: **in** Std_Logic);

end Prob_8_22_vhdl;

architecture Structural **of** Prob_8_22_vhdl **is**

signal A, Q: Std_Logic_vector (m_size -1 **downto** 0);

signal Q0, Zero, Load_regs, Decr_P, Add_regs, Shift_regs: Std_Logic;

component Datapath_Unit

generic (m_size, BC_size: **integer**);

port (A, Q: **out** Std_Logic_vector (m_size -1 **downto** 0);

 Q0, Zero: **out** Std_Logic;

 Multiplicand, Multiplier: **in** Std_Logic_vector (m_size -1 **downto** 0);

 Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: **in** Std_Logic);

end component;

component Control_Unit

port (Ready, Decr_P, Load_regs, Add_regs, Shift_regs: **out** Std_Logic; Start, Q0, Zero, clock, reset_b: **in** Std_Logic);

end component;

begin

 Product <= A & Q;

 M0: Datapath_Unit **generic map** (m_size => 9, BC_size => 3)

```

port map (A, Q, Q0, Zero, Multiplicand, Multiplier, Load_regs, Decr_P, Add_regs, Shift_regs, clock,
reset_b);

M1: Control_Unit port map (Ready, Decr_P, Load_regs, Add_regs, Shift_regs, Start, Q0, Zero, clock,
reset_b);
end Structural;

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Datapath_Unit is
    generic (m_size , BC_size: integer);
    port (A: buffer Unsigned (m_size -1 downto 0); Q: buffer Unsigned (m_size-1 downto 0);
        Q0, Zero: out Std_Logic;
        Multiplicand, Multiplier: in Unsigned (m_size -1 downto 0);
        Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
end DataPath_Unit;

architecture Behavioral of Datapath_Unit is
    signal C: Std_Logic;
    signal P: integer;
    signal B: Unsigned (m_size-1 downto 0);
    signal sum: Unsigned (m_size downto 0);
begin

    Q0 <= Q(0);
    Zero <= '1' when (P = 0) else '0';
    Sum <= ('0' & A) + ('0' & B);
    process (clock, reset_b) begin
        if (reset_b = '0') then
            for k in 0 to m_size-1 loop A(k) <= '0'; end loop;
            for k in 0 to m_size-1 loop B(k) <= '0'; end loop;
            for k in 0 to m_size-1 loop Q(k) <= '0'; end loop;
            C <= '0';
            P <= m_size;
        elsif clock'event and clock = '1' then
            if (Load_regs = '1') then
                for k in 0 to m_size-1 loop A(k) <= '0'; end loop;
                C <= '0';
                Q <= Multiplier;
                B <= Multiplicand;
                P <= m_size;
            end if;
            if (Decr_P = '1') then P <= P -1; end if;

            if (Add_regs = '1') then
                C <= Sum(m_size);
                A <= Sum (m_size-1 downto 0);
            end if;

            if (Shift_regs = '1') then
                C <= '0';
                A <= C & A(m_size-1 downto 1);
                Q <= A(0) & Q(m_size-1 downto 1);
            end if;
        end if;
    end process;
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity Control_Unit is
  port (Ready, Decr_P, Load_regs, Add_regs, Shift_regs: out Std_Logic; Start, Q0, Zero, clock, reset_b:
in Std_Logic);
end Control_Unit;

architecture Behavioral of Control_Unit is
  signal state, next_state: Std_Logic_vector (1 downto 0);
  constant S_idle: Std_Logic_vector (1 downto 0) := "00";
  constant S_loaded: Std_Logic_vector (1 downto 0) := "01";
  constant S_sum: Std_Logic_vector (1 downto 0) := "10";
  constant S_shifted: Std_Logic_vector (1 downto 0) := "11";

begin
  Ready <= '1' when (state = S_idle) else '0';
  Decr_P <= '1' when (state = S_loaded) else '0';

  process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_idle;
    elsif clock'event and clock = '1' then state <= next_state; end if;
  end process;

  process (state, Start, Q0, Zero) begin
    next_state <= S_idle;
    Load_regs <= '0';
    Add_regs <= '0';
    Shift_regs <= '0';
    case (state) is
      when S_idle => if (Start = '0') then next_state <= S_idle;
                     else next_state <= S_loaded; Load_regs <= '1';
                     end if;

      when S_loaded => if (Q0 = '1') then next_state <= S_sum; Add_regs <= '1';
                      else next_state <= S_shifted; Shift_regs <= '1'; end if;

      when S_sum => next_state <= S_shifted; Shift_regs <= '1';

      when S_shifted => if (Zero = '1') then next_state <= S_idle;
                      else next_state <= S_loaded; end if;

    end case;
  end process;
end Behavioral;

```

- 8.23** As shown in Fig. P8.23 the machine asserts *Load_regs* in state *S_load*. This will cause the machine to operate incorrectly. Once *Load_regs* is removed from *S_load* the machine operates correctly. The state *S_load* is a wasted state. Its removal leads to the same machine as shown in Fig. P8.15b.

```

module Prob_8_23 # (parameter m_size = 9)
(
  output [2*m_size-1: 0] Product,
  output Ready,
  input [m_size-1: 0] Multiplicand, Multiplier,
  input Start, clock, reset_b
);
  wire [m_size-1: 0] A, Q;

```

```

assign Product = {A, Q};
wire Q0, Zero, Load_regs, Decr_P, Add_regs, Shift_regs;

```

```

Datapath_Unit M0 (A, Q, Q0, Zero, Multiplicand, Multiplier, Load_regs, Decr_P, Add_regs, Shift_regs,
clock, reset_b);
Control_Unit M1 (Ready, Decr_P, Shift_regs, Add_regs, Load_regs, Start, Q0, Zero, clock, reset_b);
endmodule

```

```

module Datapath_Unit # (parameter m_size = 9, BC_size = 4)
(
  output reg [m_size -1: 0] A, Q,
  output Q0, Zero,
  input [m_size -1: 0] Multiplicand, Multiplier,
  input Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b
);

```

```

  reg C;
  reg [BC_size -1: 0] P;
  reg [m_size -1: 0] B;

```

```

assign Q0 = Q[0];
assign Zero = (P == 0);

```

```

always @ (posedge clock, negedge reset_b)

```

```

  if (reset_b == 0) begin
    A <= 0;
    C <= 0;
    Q <= 0;
    B <= 0;
    P <= m_size;
  end
  else begin
    if (Load_regs) begin
      A <= 0;
      C <= 0;
      Q <= Multiplier;
      B <= Multiplicand;
      P <= m_size;
    end
    if (Decr_P) P <= P -1;
    if (Add_regs) {C, A} <= A + B;
    if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
  end
endmodule

```

```

module Control_Unit (
  output Ready, Decr_P, Shift_regs, output reg Add_regs, Load_regs, input Start, Q0, Zero, clock,
  reset_b
);
  reg [ 1: 0] state, next_state;
  parameter S_idle = 2'b00, S_load = 2'b01, S_decr = 2'b10, S_shift = 2'b11;

```

```

assign Ready = (state == S_idle);
assign Shift_regs = (state == S_shift);
assign Decr_P = (state == S_decr);

```

```

always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) state <= S_idle; else state <= next_state;

```

```

always @ (state, Start, Q0, Zero) begin
  next_state = S_idle;
  Load_regs = 0;
  Add_regs = 0;

```

```

case (state)
  S_idle: if (Start == 0) next_state = S_idle; else begin next_state = S_load; Load_regs = 1; end
  S_load: begin next_state = S_decr; end
  S_decr: begin next_state = S_shift; if (Q0) Add_regs = 1; end
  S_shift: if (Zero) next_state = S_idle; else next_state = S_load;
endcase
end
endmodule

module t_Prob_8_23 ();
  parameter m_size = 9; // Width of datapath
  wire [2 * m_size - 1: 0] Product;
  wire Ready;
  reg [m_size - 1: 0] Multiplicand, Multiplier;
  reg Start, clock, reset_b;
  integer Exp_Value;
  reg Error;

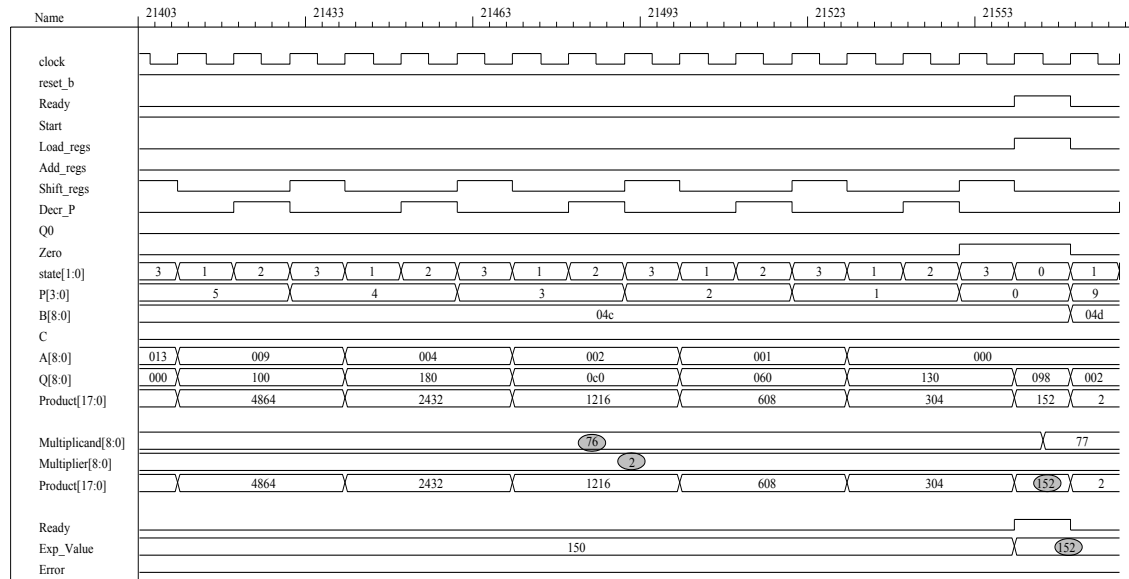
  Prob_8_23 M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

  initial #140000 $finish;
  initial begin clock = 0; #5 forever #5 clock = ~clock; end
  initial fork
    reset_b = 1;

    #2 reset_b = 0;
    #3 reset_b = 1;
  join
  initial begin #5 Start = 1; end
  always @ (posedge Ready) begin
    Exp_Value = Multiplier * Multiplicand;
    //Exp_Value = Multiplier * Multiplicand + 1; // Inject error to confirm detection
  end
  always @ (negedge Ready) begin
    Error = (Exp_Value ^ Product);
  end

  initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (64) #10 begin Multiplier = Multiplier + 1;
      repeat (64) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
  end
endmodule

```



VHDL

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
```

```
entity Prob_8_23_vhdl is
    generic (m_size: integer := 9);
```

```
    port (Product: buffer Unsigned (2*m_size -1 downto 0);
          Ready: buffer Std_Logic;
          Multiplicand, Multiplier: in Unsigned (m_size -1 downto 0);
          Start, clock, reset_b: in Std_Logic);
```

```
end entity Prob_8_23_vhdl;
```

```
architecture Partitioned of Prob_8_23_vhdl is
```

```
    signal A, Q: Unsigned (m_size -1 downto 0);
```

```
    signal Q0, Zero, Load_regs, Decr_P, Add_regs, Shift_regs: Std_Logic;
```

```
    component Datapath_Unit
```

```
        generic (m_size, BC_size: integer);
```

```
        port (A, Q: out Unsigned (m_size-1 downto 0);
```

```
              Q0, Zero: out Std_Logic;
```

```
              Multiplicand, Multiplier: in Unsigned (m_size-1 downto 0);
```

```
              Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
```

```
    end component;
```

```
    component Control_Unit
```

```
        generic (m_size, BC_size: integer);
```

```
        port (Ready, Decr_P, Shift_regs, Add_regs, Load_regs: out Std_Logic;
```

```
              Start, Q0, Zero, clock, reset_b: in Std_Logic);
```

```
    end component;
```

```
begin
```

```
    Product <= A & Q;
```

```
    M0: Datapath_Unit generic map (m_size => 9, BC_size => 4) port map (A, Q, Q0, Zero, Multiplicand,
        Multiplier, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b);
```

```

        M1: Control_Unit generic map (m_size => 9, BC_size => 4) port map (Ready, Decr_P, Shift_regs,
        Add_regs, Load_regs, Start, Q0, Zero, clock, reset_b);
end Partitioned;

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Datapath_Unit is
    generic (m_size, BC_size: integer);
    port (A: buffer Unsigned (m_size-1 downto 0); Q: buffer Unsigned (m_size-1 downto 0);
          Q0, Zero: out Std_Logic;
          Multiplicand, Multiplier: in Unsigned (m_size-1 downto 0);
          Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
end entity Datapath_Unit;

architecture Behavioral of Datapath_Unit is
    signal P: integer;
    signal B: Unsigned (m_size-1 downto 0);
    signal S: Unsigned (m_size downto 0);
    signal C: Std_Logic;
begin
    Q0 <= Q(0);
    Zero <= '1' when P = '0' else '0';
    S <= ('0' & A) + ('0' & B);

    process (clock, reset_b) begin
        if (reset_b = '0') then
            C <= '0';
            for k in 0 to m_size-1 loop
                B(k) <= '0';
                A(k) <= '0';
                Q(k) <= '0';
            end loop;
        elsif clock'event and clock = '1' then
            if Load_regs = '1' then
                for k in 0 to m_size-1 loop
                    A(k) <= '0';
                end loop;
                C <= '0';
                Q <= Multiplier;
                B <= Multiplicand;
                P <= m_size;
            end if;
            if (Decr_P = '1') then P <= P - 1; end if;
            if (Add_regs = '1') then
                C <= S(m_size);
                A <= S(m_size-1 downto 0);
            end if;
            if (Shift_regs = '1') then
                Q <= A(0) & Q(m_size-1 downto 1);
                A <= C & Q(m_size-1 downto 1);
                C <= '0';
            end if;
        end if;
    end process;
end Behavioral;

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
Library Synopsys;

```



```

use Synopsys.attributes.all;

entity Control_Unit is
  generic (m_size, BC_size: integer);
  port (Ready, Decr_P, Shift_regs, Add_regs, Load_regs: out bit; Start, Q0, Zero, clock, reset_b: in bit);
end entity Control_Unit;

architecture Behavioral of Control_Unit is
  type State_type is (S_idle, S_load, S_decr, S_shift);
  attribute enum_encoding of State_type : type is "00 01 10 11";
  signal state, next_state: State_type;
  Ready <= '1' when (state = S_idle) else '0';
  Shift_regs <= '1' when (state = S_shift) else '0';
  Decr_P <= '1' when (state = S_decr) else '0';

  process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_idle;
    elsif clock'event and clock = '1' then state <= next_state;
    end if;
  end process;

  process (state, Start, Q0, Zero) begin
    next_state <= S_idle;

    Load_regs <= '0';
    Add_regs <= '0';
    case (state) is
      when S_idle => if (Start = '0') then next_state <= S_idle;
                     else next_state <= S_load; Load_regs <= '1'; end if;
      when S_load => next_state <= S_decr;
      when S_decr => next_state <= S_shift; if (Q0 = '1') then Add_regs <= '1'; end if;
      when S_shift => if (Zero = '1') then next_state <= S_idle; else next_state <= S_load; end if;
    end case;
  end process;
end behavioral;

```

8.24

```

module Prob_8_24 # (parameter dp_width = 5)
(
  output    [2*dp_width - 1: 0]    Product,
  output    Ready,
  input     [dp_width - 1: 0]      Multiplicand, Multiplier,
  input     Start, clock, reset_b
);
  wire Load_regs, Decr_P, Add_regs, Shift_regs, Zero, Q0;

  Controller M0 (
    Ready, Load_regs, Decr_P, Add_regs, Shift_regs, Start, Zero, Q0,
    clock, reset_b
  );

  Datapath M1(Product, Q0, Zero, Multiplicand, Multiplier,
    Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b);
endmodule

module Controller (
  output Ready,
  output reg Load_regs, Decr_P, Add_regs, Shift_regs,
  input Start, Zero, Q0, clock, reset_b
);

  parameter    S_idle = 3'b001,      // one-hot code
               S_add = 3'b010,
               S_shift = 3'b100;

  reg [2: 0]    state, next_state;    // sized for one-hot
  assign       Ready = (state == S_idle);

  always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;

  always @ (state, Start, Q0, Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Decr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
      S_idle: if (Start) begin next_state = S_add; Load_regs = 1; end
      S_add: begin next_state = S_shift; Decr_P = 1; if (Q0) Add_regs = 1; end
      S_shift: begin
        Shift_regs = 1;
        if (Zero) next_state = S_idle;
        else next_state = S_add;
      end
      default: next_state = S_idle;
    endcase
  end
endmodule

```

```

module Datapath #(parameter dp_width = 5, BC_size = 3) (
    output [2*dp_width - 1: 0] Product, output Q0, output Zero,
    input [dp_width - 1: 0] Multiplicand, Multiplier,
    input Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b
);
// Default configuration: 5-bit datapath
reg [dp_width - 1: 0] A, B, Q; // Sized for datapath
reg C;
reg [BC_size - 1: 0] P; // Bit counter

assign Q0 = Q[0];
assign Zero = (P == 0); // Counter is zero
assign Product = {C, A, Q};
always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) begin // Added to this solution, but
        P <= dp_width; // not really necessary since Load_regs
        B <= 0; // initializes the datapath
        C <= 0;
        A <= 0;
        Q <= 0;
    end
    else begin
        if (Load_regs) begin
            P <= dp_width;
            A <= 0;
            C <= 0;
            B <= Multiplicand;
            Q <= Multiplier;
        end
        if (Add_regs) {C, A} <= A + B;
        if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
        if (Decr_P) P <= P - 1;
    end
endmodule

module t_Prob_8_24;
    parameter dp_width = 5; // Width of datapath
    wire [2 * dp_width - 1: 0] Product;
    wire Ready;
    reg [dp_width - 1: 0] Multiplicand, Multiplier;
    reg Start, clock, reset_b;
    integer Exp_Value;
    reg Error;

    Prob_8_24 M0(Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

    initial #115000 $finish;
    initial begin clock = 0; #5 forever #5 clock = ~clock; end
    initial fork
        reset_b = 1;
        #2 reset_b = 0;
        #3 reset_b = 1;
    join
    always @ (negedge Start) begin
        Exp_Value = Multiplier * Multiplicand;
        //Exp_Value = Multiplier * Multiplicand + 1; // Inject error to confirm detection
    end
    always @ (posedge Ready) begin
        # 1 Error <= (Exp_Value ^ Product) ;
    end

    initial begin

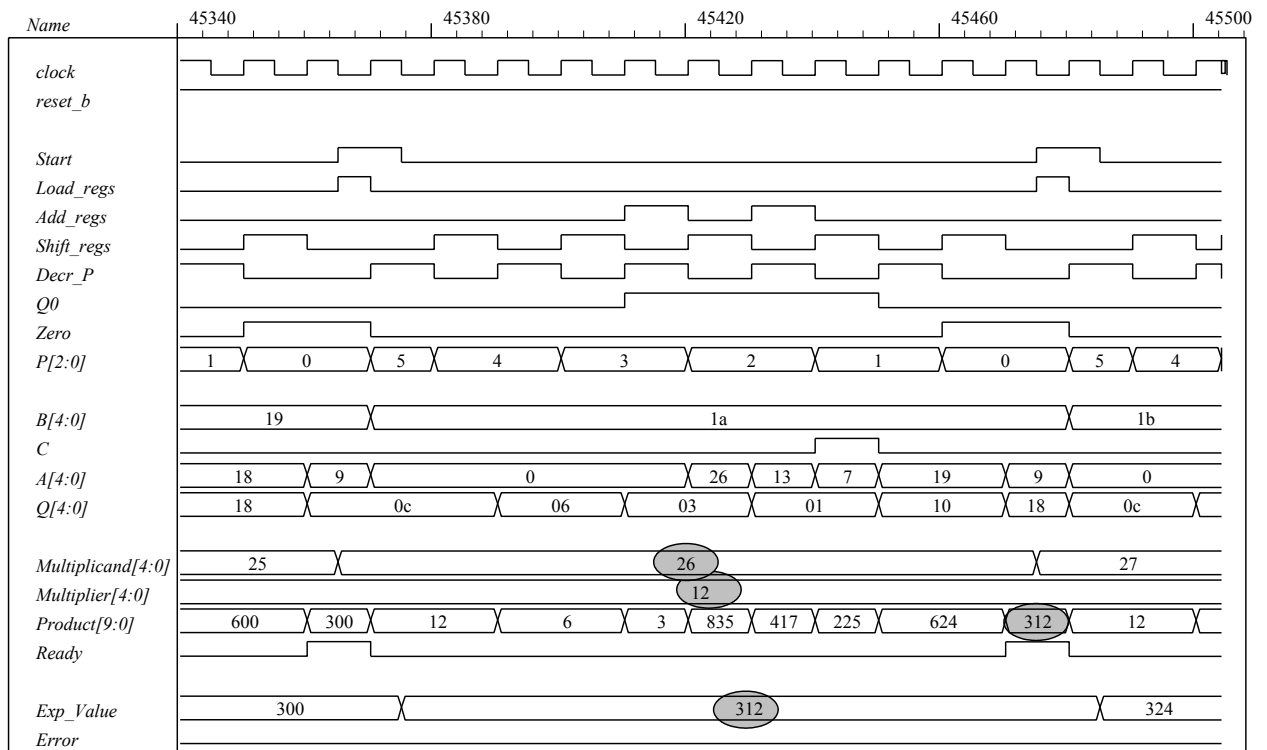
```

```

#5 Multiplicand = 0;
Multiplier = 0;

repeat (32) #10 begin
  Start = 1;
  #10 Start = 0;
  repeat (32) begin
    Start = 1;
    #10 Start = 0;
    #100 Multiplicand = Multiplicand + 1;
  end
  Multiplier = Multiplier + 1;
end
end
endmodule

```



VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Arith.all;

entity Prob_8_24_vhdl is
  generic (dp_width: integer := 5);
  port (Product: out Std_Logic_vector(2*dp_width downto 0);
        Ready: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector(dp_width - 1 downto 0);
        Start, clock, reset_b: in Std_Logic);
end Prob_8_24_vhdl;

architecture Partitioned of Prob_8_24_vhdl is
  signal Load_regs, Decr_P, Add_regs, Shift_regs, Zero, Q0: Std_Logic;
  component Controller
    generic (dp_width, BC_width: integer);

```

```

    port (Ready: out Std_Logic;
          Load_regs, Decr_P, Add_regs, Shift_regs: out Std_Logic;
          Start, Zero, Q0, clock, reset_b: in Std_Logic);
end component;

component Datapath
  generic (dp_width, BC_width: integer);
  port (Product: out Std_Logic_Vector (2*dp_width downto 0);
        Q0, Zero: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector (dp_width-1 downto 0);
        Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
  end component;

begin
M0: Controller
  generic map (dp_width => 5, BC_width => 3)
  port map (Ready, Load_regs, Decr_P, Add_regs, Shift_regs, Start, Zero, Q0, clock, reset_b);

M1: Datapath
  generic map (dp_width => 5, BC_width => 3)
  port map (Product, Q0, Zero, Multiplicand, Multiplier, Start, Load_regs, Decr_P, Add_regs, Shift_regs,
clock, reset_b);
end Partitioned;

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Arith.all;
library Synopsys;
use Synopsys.attributes.all; -- used for one-hot encoding of states

entity Controller is
  generic (dp_width, BC_width: integer);
  port (Ready, Load_regs, Decr_P, Add_regs, Shift_regs: out Std_Logic;
        Start, Zero, Q0, clock, reset_b: in Std_Logic);
end Controller;

architecture Behavioral of Controller is
  type State_type is (S_idle, S_add, s_shift);
  attribute enum_encoding of State_type: type is "001 010 100"; -- one-hot

  signal state, next_state: State_type;

begin
  Ready <= '1' when (state = S_idle) else '0';

  process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_idle; else state <= next_state; end if;
  end process;

  process (state, Start, Q0, Zero) begin
    next_state <= S_idle;
    Load_regs <= '0';
    Decr_P <= '0';
    Add_regs <= '0';
    Shift_regs <= '0';
    case (state) is
      when S_idle => if (Start = '1') then next_state <= S_add; Load_regs <= '1'; end if;
      when S_add => next_state <= S_shift; Decr_P <= '1'; if (Q0 = '1') then Add_regs <= '1'; end if;
      when S_shift => Shift_regs <= '1'; if (zero = '1') then next_state <= S_idle;
      else next_state <= S_add; end if;
      when others => next_state <= S_idle;
    end case;
  end process;

```

```

end Behavioral;

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Arith.all;
use IEEE.Std_Logic_unsigned.all;

entity Datapath is
  generic (dp_width, BC_width: integer);
  port ( Product: buffer Std_Logic_vector (2*dp_width downto 0);
        Q0: out Std_Logic;
        Zero: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector (dp_width - 1 downto 0);
        Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic); begin
end Datapath;
architecture Behavioral of Datapath is
  signal C: Std_Logic;
  signal A, B, Q: Std_Logic_vector (dp_width - 1 downto 0);      -- Sized for datapath
  signal P: integer; signal S: Std_Logic_vector (dp_width downto 0);
begin

  Q0 <= Q(0);
  Product <= C & A & Q;

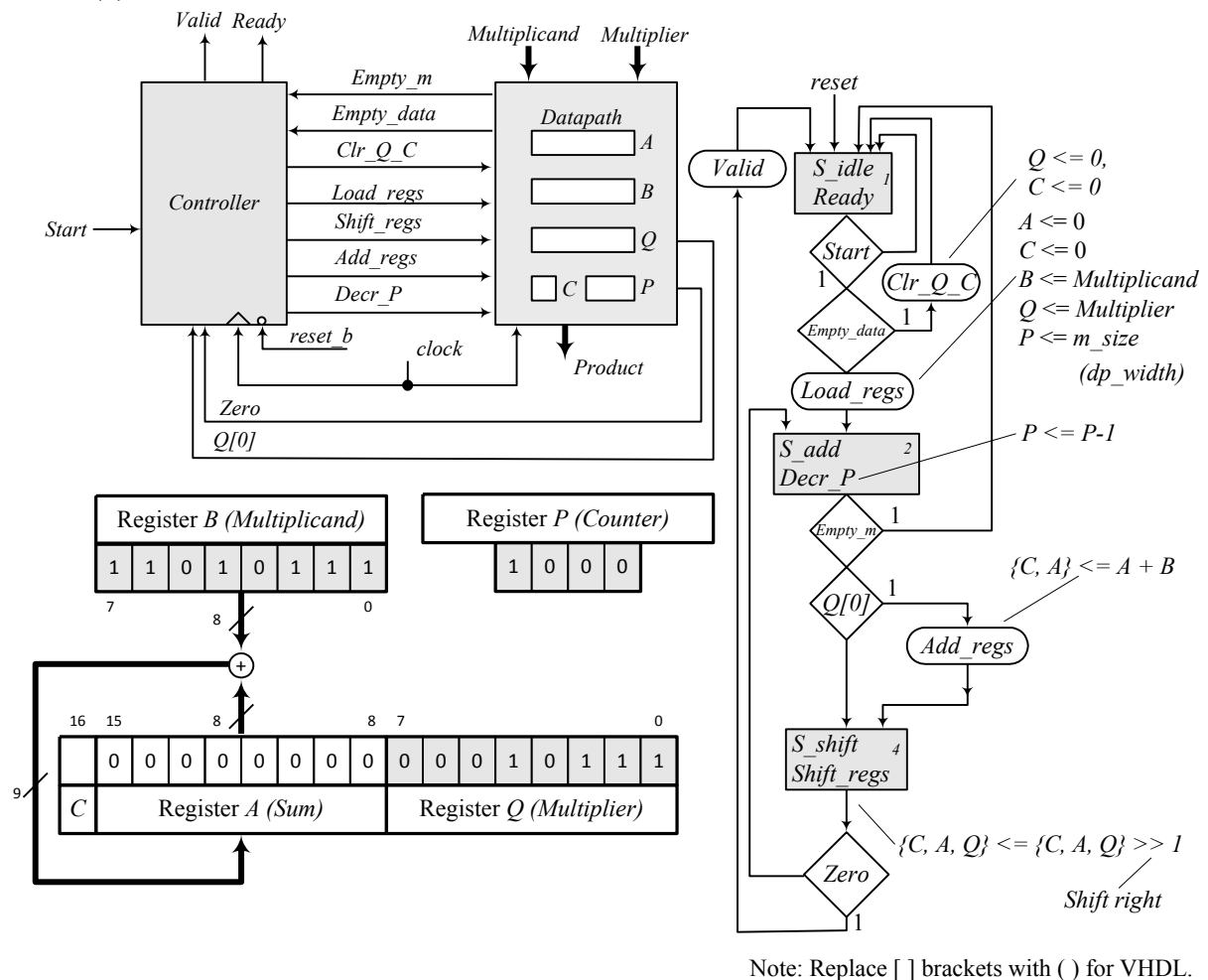
  process(P) begin          -- Check for empty P (Bit counter)
    if P = 0 then Zero <= '1'; else Zero <= '0'; end if;
  end process;

  process (clock, reset_b) begin
    if (reset_b = '0') then
      C <= '0';
      for k in 0 to dp_width-1 loop
        B(k) <= '0';
        A(k) <= '0';
        Q(k) <= '0';
      end loop;
    elsif clock'event and clock = '1' then
      if (Load_regs = '1') then
        P <= dp_width;
        C <= '0';
        B <= Multiplicand;
        Q <= Multiplier;
        for k in 0 to dp_width-1 loop
          A(k) <= '0';
        end loop;
      end if;
      if (Add_regs = '1') then
        S <= ('0' & A) + ('0' & B);
        C <= S(dp_width);
        A <= S(dp_width-1 downto 0);
      end if;

      if (Shift_regs = '1') then
        Q <= A(0) & Q(dp_width-1 downto 1);
        A <= C & A(dp_width - 1 downto 1);
        C <= '0';
      end if;
      if (Decr_P = '1') then P <= P - 1; end if;
    end if;
  end process;
end Behavioral;

```

8.25 (a)



(b)

The binary multiplier of Fig. 8.15 is modified to detect whether the multiplier or multiplicand are initially zero, and to detect whether the shifted multiplier becomes zero before the entire multiplier has been applied to the multiplicand.

Status signal *Empty_data* indicates that either *Multiplier* or *Multiplicand* are 0, in which case there is no need to load the data or to multiply. *Clr_Q_C* assures that *Q* and *C* are empty so that *CAQ* does not mistakenly use residual data and present a bogus value of *Product*. *Zero* indicates that the count of bits of *Multiplier* that remain to be processed is 0. *Empty_m* indicates that the unprocessed bits of *Multiplier* are all 0s (early termination). *P* indicates how many bits of the multiplier word in *Q* remain to be processed. A Mealy-type signal *Valid* asserts for one cycle during a transition from *S_shift* to *S_idle*, and indicates that the value of *Product* is valid. *Product* will remain valid until the machine asserts *Load_regs*. Signal *Clr_Q_C* is necessary because *Empty_data* does not imply that *Q* is empty. In the situation where *Empty_data* asserts because *Multiplicand* alone is empty, *Clr_Q_C* assures that *QAC* will be empty, presenting a correct value of *Product*.

Verilog

```

module Prob_8_25 #(parameter dp_width = 5)
(
  output [2*dp_width - 1: 0] Product,
  output Ready,
  input [dp_width - 1: 0] Multiplicand, Multiplier,
  input Start, clock, reset_b
);
  wire Load_regs, Clr_Q, Decr_P, Add_regs, Shift_regs, Empty_m, Empty_data, Zero, Q0;

  Controller M0 (
    Ready, Load_regs, Clr_Q, Decr_P, Add_regs, Shift_regs, Start, Empty_m, Empty_data, Zero, Q0,
    clock, reset_b
  );

  Datapath M1(Product, Q0, Empty_m, Empty_data, Zero, Multiplicand, Multiplier,
    Start, Load_regs, Clr_Q, Decr_P, Add_regs, Shift_regs, clock, reset_b);

endmodule

module Controller (
  output Ready,
  output reg Load_regs, Clr_Q, Decr_P, Add_regs, Shift_regs,
  input Start, Empty, Zero, Q0, clock, reset_b
);

  parameter BC_size = 3; // Size of bit counter
  parameter S_idle = 3'b001, // one-hot code
             S_add = 3'b010,
             S_shift = 3'b100;

  reg [2: 0] state, next_state; // sized for one-hot
  assign Ready = (state == S_idle);

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 1'b0) state <= S_idle; else state <= next_state;

  always @ (state, Start, Q0, Empty_m, Empty_data, Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Clr_Q = 0;
    Decr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
      S_idle: if (Start == 1'b0) begin next_state = S_idle
              else if (Empty_data == 1) begin Clr_Q = 1; next_state = S_idle end
              else begin Load_regs = 1; next_state = S_add; end

      S_add: if (Empty_m) next_state = S_idle;
             else begin next_state = S_shift; Decr_P = 1;
                     if (Q0) Add_regs = 1;
             end

      S_shift: begin
                Shift_regs = 1;
                if (Zero) next_state = S_idle;
                else next_state = S_add; end
             end

      default: next_state = S_idle;
    endcase
  end
endmodule

```



```

module Datapath #(parameter dp_width = 5, BC_size = 3) (
  output reg [2*dp_width : 0] Product, output Q0, output Empty_m, Empty_data, output Zero,
  input [dp_width - 1: 0] Multiplicand, Multiplier,
  input Start, Load_regs, Clr_Q, Decr_P, Add_regs, Shift_regs, clock, reset_b
);
// Default configuration: 5-bit datapath
parameter      S_idle = 3'b001,      // one-hot code
                S_add = 3'b010,
                S_shift = 3'b100;
reg [dp_width - 1: 0]  A, B, Q;      // Sized for datapath
reg                C;
reg [BC_size - 1: 0]  P;            // Bit counter
wire [2*dp_width: 0]  Pre_Product = {C, A, Q};

assign      Q0 = Q[0];
assign      Zero = (P == 0);        // Bit counter is zero

always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) begin            // Added to this solution, but
    P <= dp_width;                  // not really necessary since Load_regs
    B <= 0;                          // initializes the datapath
    C <= 0;
    A <= 0;
    Q <= 0;
  end
  else begin
    if (Clr_Q) Q <= 0;
    if (Load_regs) begin
      P <= dp_width;
      A <= 0;
      C <= 0;
      B <= Multiplicand;
      Q <= Multiplier;
    end
    if (Add_regs) {C, A} <= A + B;
    if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
    if (Decr_P) P <= P - 1;
  end
  // Status signals
  wire Empty_m = (Q[P: 0] == 0);
  wire Empty_data = (Multiplicand == 0) || (Multiplier == 0);

  assign Product = Pre_Product [2*dp_width: P];

  {C, A[dp_width-1: 0], Q[dp_width-1: P]};

always @ (P, Internal_Product) begin // Note: hardwired for dp_width 5
  Product = 0;
  case (P) // Examine multiplier bit counter/pointer
    0: Product = Internal_Product [2*dp_width : 0];
    1: Product = Internal_Product [2*dp_width : 1];
    2: Product = Internal_Product [2*dp_width : 2];
    3: Product = Internal_Product [2*dp_width : 3];
    4: Product = Internal_Product [2*dp_width : 4];
    5: Product = 0
    default: Product = 0;
  endcase
end

```

```

always @ (P, Q) begin           // Note: hardwired for dp_width 5
    Empty_m = 0;
    case (P)
        0: Empty_m = 1;
        1: if (Q[1] == 0)      Empty_multiplier = 1;
        2: if (Q[2: 1] == 0)   Empty_multiplier = 1;
        3: if (Q[3: 1] == 0)   Empty_multiplier = 1;
        4: if (Q[4: 1] == 0)   Empty_multiplier = 1;
        5: if (Q[5: 1] == 0)   Empty_multiplier = 1;
        default: Empty_multiplier = 0;
    endcase
end
endmodule

module t_Prob_8_25;
    parameter      dp_width = 5;           // Width of datapath
    wire [2 * dp_width - 1: 0] Product;
    wire Ready;
    reg [dp_width - 1: 0] Multiplicand, Multiplier;
    reg Start, clock, reset_b;
    integer Exp_Value;
    reg Error;

    Prob_8_25 M0(Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

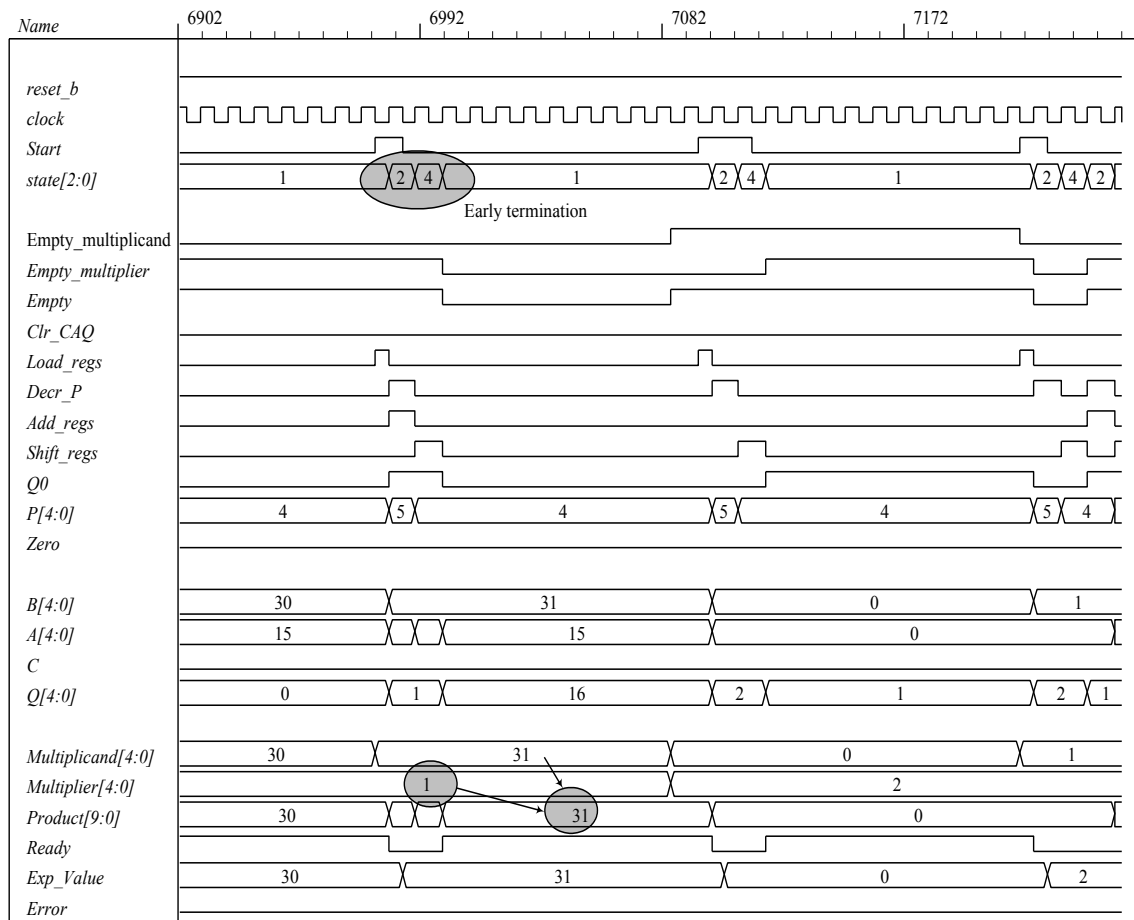
    initial #115000 $finish;
    initial begin clock = 0; #5 forever #5 clock = ~clock; end
    initial fork
        reset_b = 1;
        #2 reset_b = 0;
        #3 reset_b = 1;
    join
    always @ (negedge Start) begin
        Exp_Value = Multiplier * Multiplicand;
        //Exp_Value = Multiplier * Multiplicand + 1; // Inject error to confirm detection
    end
    always @ (posedge Ready) begin
        # 1 Error <= (Exp_Value ^ Product) ;
    end

    initial begin
        #5 Multiplicand = 0;
        Multiplier = 0;

        repeat (32) #10 begin
            Start = 1;
            #10 Start = 0;
            repeat (32) begin
                Start = 1;
                #10 Start = 0;
                #100 Multiplicand = Multiplicand + 1;
            end
            Multiplier = Multiplier + 1;
        end
    end
endmodule

```

(c) Test plan: Exhaustively test all combinations of multiplier and multiplicand, using automatic error checking. Verify that early termination is implemented. Sample of simulation results is shown below.



VHDL

The width of the datapath in the original counter determines the number of times the multiplier word (Q) is shifted to the right, with bits of the Product being shifted into Q from the left. The modified machine dynamically detects whether the unprocessed portion of the multiplier word is devoid of 1s. If so, early termination is possible.

If early termination occurs no additional copies of the multiplicand will be added to the partial product. However, if the process terminates before Q has been shifted by the width of the datapath, the LSB of the product will not occupy the LSB position of Q, leading to a misformed value of C & A & Q. Caution is advised because Q contains bits of the multiplier and bits of the product, with the distribution changing with each shift. The solution is to extract the product from the proper segment of CAQ, depending on the number of shifts that have occurred in forming the partial product. P always indicates how many bits of A are associated with the multiplier.

The design methodology created separate files for the datapath and controller. Each was verified separately. Then the integration of the datapath with the controller was verified.

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_8_25_vhdl is
generic dp_width;
port (Product out Std_Logic (2*dp_width downto 0),
      Ready, Valid: out Std_Logic; Multiplicand, Multiplier in (dp_width - 1: 0);
      Start, clock, reset_b: in Std_Logic);
```

```

end Prob_8_25_vhdl;

architecture Behavioral of Prob_8_25_vhdl is
    signal Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs,
           Empty_data, Empty_m, Zero, Q0: Std_Logic;
    component Controller generic dp_width;
        port (Ready, Valid, Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs: out Std_Logic;
              Start, Zero, Empty_data, Empty_m, Q0, clock, reset_b: in Std_Logic);

    component Datapath generic dp_width;
        port (Product: out Std_Logic_vector (2*dp_width downto 0); Q0, Empty_data, Empty_m,
              Zero: out Std_Logic; Empty_data, Empty_m: out Std_Logic; Multiplicand,
              Multiplier: in Std_Logic_vector (dp_width - 1 downto 0); Start, Load_regs, Clr_Q_C,
              Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
begin
    M0: Controller
        generic map (dp_width = 5, BC_size = 3)
        port map (Ready, Valid, Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs, Start,
                  Empty_data, Empty_m, Zero, Q0, clock, reset_b);

    M1: Datapath
        generic map (dp_width = 5, BC_size = 3)
        port map (Product, Q0, Zero, Empty_data, Empty_m, Multiplicand, Multiplier,
                  Start, Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs, clock, reset_b);
end Behavioral;

-- Control Unit
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
Library Synopsys;
use Synopsys.attributes.all;
-- File: Prob_8_25b_Controller_vhdl.vhd

entity Prob_8_25b_Controller_vhdl is
    generic (dp_width: integer := 5; BC_size: integer := 3);
    port (Ready, Valid, Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs: out Std_Logic;
          Start, Empty_data, Empty_m, Zero, Q0, clock, reset_b: in Std_Logic);
end Prob_8_25b_Controller_vhdl;

architecture Behavioral of Prob_8_25b_Controller_vhdl is -- one-hot
    type State_type is (S_idle, S_add, S_shift);
    attribute enum_encoding of State_type: type is "001 010 100";
    signal state, next_state: State_type;

begin
    -- State transitions
    process (clock, reset_b) begin
        if (reset_b = '0') then state <= S_idle;
        elsif clock'event and clock = '1' then state <= next_state; end if;
    end process;

    -- Mealy Output
    Valid <= '1' when state = S_shift and Zero = '1' else '0';

    -- Next state logic
    process (state, Start, Empty_data, Empty_m, Q0, Zero) begin
        next_state <= S_idle;
        Ready <= '0';
        Load_regs <= '0';
        Clr_Q_C <= '0';
    end process;
end architecture;

```

```

Decr_P      <= '0';
Add_regs    <= '0';
Shift_regs  <= '0';

case (state) is
when S_idle => Ready <= '1';
               if Start = '0' then next_state <= S_idle;
               elsif Empty_data = '1' then Clr_Q_C <= '1'; next_state <= S_idle;
               else Load_regs <= '1'; next_state <= S_add;
               end if;

when S_add => Decr_P <= '1';
               if Empty_m = '1' then next_state <= S_idle;
               elsif Q0 = '1' then Add_regs <= '1'; next_state <= S_shift;
               else next_state <= S_shift;
               end if;

when S_shift => Shift_regs <= '1';
                 if Zero = '1' then next_state <= S_idle;
                 else next_state <= S_add;
                 end if;

when others => next_state <= S_idle;
end case;
end process;
end Behavioral;

-- Datapath Unit
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
use IEEE.Std_Logic_unsigned.all;

entity Prob_8_25b_Datapath_vhdl is
generic (dp_width: integer := 5; BC_size: integer := 3);
port (Product: out Std_Logic_vector (2*dp_width downto 0);
       Q0, Empty_data, Empty_m: out Std_Logic; Zero: buffer Std_Logic;
       Multiplicand, Multiplier: in Std_Logic_vector (dp_width - 1 downto 0);
       Start, Load_regs, Clr_Q_C, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
end Prob_8_25b_Datapath_vhdl;

architecture Behavioral of Prob_8_25b_Datapath_vhdl is
  signal Sum: Std_Logic_vector (dp_width downto 0);
  signal A, B, Q: Std_Logic_vector (dp_width - 1 downto 0);      -- Sized for datapath
  signal C: Std_Logic;
  signal P: integer;      -- Bit counter
  signal Q0: Std_Logic;
  signal multiplier_0, multiplicand_0: Std_Logic;
begin
  Q0 <= Q(0);
  Sum <= ('0' & A) + ('0' & B);
  Zero <= '1' when P = 0 else '0';      -- Counter is zero
  Product <= C & A & Q;
  Empty_data <= '1' when multiplier_0 = '1' and multiplier_0 = '1' else '0';

  process (multiplier) begin      -- Detect empty multiplier word
    multiplier_0 <= '1';
    for k in dp_width-1 downto 0 loop
      if multiplier(k) = '1' then multiplier_0 <= '0';
      end if;
    end loop;

```

```

end process;

process (multiplicand) begin    -- Detect empty multiplicand word
    multiplicand_0 <= '1';
    for k in dp_width-1 downto 0 loop
        if multiplicand(k) = '1' then multiplicand_0 <= '0';
        end if;
    end loop;
end process;

-- Nested loops (below) work around the constraint that that left range bound of a loop must be a constant.

process (P) begin            -- Detect whether remaining multiplier bits are all 0
    Empty_m <= '1';
    for k in dp_width to 0 loop
        for m in P-1 to 0 loop if Q(m) = '1' then Empty_m <= '0'; end if; end loop;
    end loop;
end process;

process (clock, reset_b)
begin
    if (reset_b = '0') then
        P <= 0;
        C <= '0';
        for k in 0 to dp_width-1 loop
            B(k) <= '0'; A(k) <= '0'; Q(k) <= '0';
        end loop;

        elsif clock'event and clock = '1' then
            if (Load_regs = '1') then
                P <= dp_width;
                C <= '0';
                for k in 0 to dp_width-1 loop A(k) <= '0'; end loop;
                B <= Multiplicand;
                Q <= Multiplier;
            end if;

            if Clr_Q_C = '1' then
                C <= '0';
                for k in dp_width-1 downto 0 loop Q(k) <= '0'; end loop;
            end if;

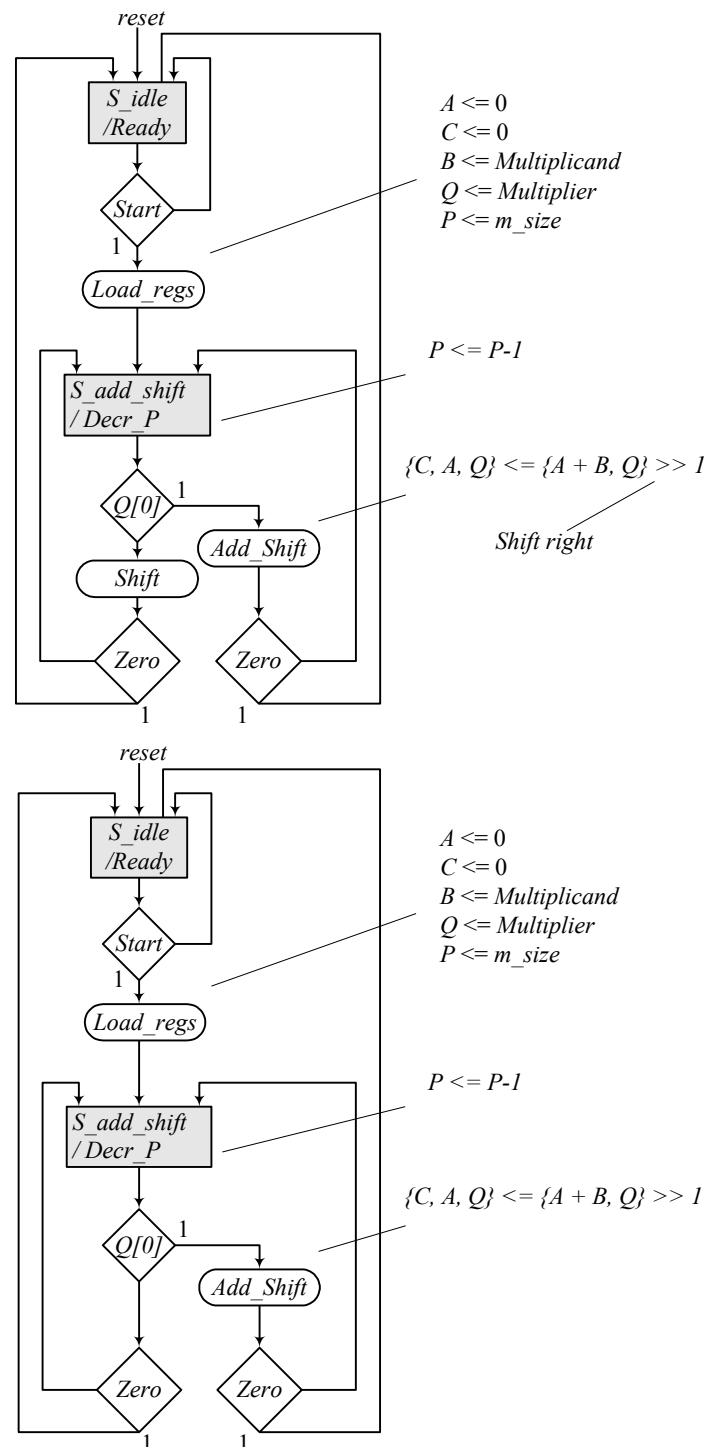
            if (Add_regs = '1') then
                C <= Sum(dp_width);
                A <= Sum(dp_width-1 downto 0);
            end if;

            if (Shift_regs = '1') then
                C <= '0';
                A <= C & A(dp_width-1 downto 1);
                Q <= A(0) & Q(dp_width-1 downto 1);
            end if;

            if (Decr_P = '1') then
                P <= P - 1;
            end if;
        end if;
    end process;
end Behavioral;

```

8.26

**Verilog**

```

module Prob_8_26 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);
// Default configuration: 5-bit datapath
parameter dp_width = 5; // Set to width of datapath
output [2*dp_width - 1: 0] Product;
output Ready;

```

```

input      [dp_width - 1: 0] Multiplicand, Multiplier;
input      Start, clock, reset_b;
parameter BC_size = 3;      // Size of bit counter
parameter S_idle = 2'b01,   // one-hot code
           S_add_shift = 2'b10;

reg [2: 0] state, next_state;
reg [dp_width - 1: 0] A, B, Q;      // Sized for datapath
reg C;
reg [BC_size - 1: 0] P;
reg Load_regs, Decr_P, Add_shift, Shift;
assign Product = {C, A, Q};
wire Zero = (P == 0);      // counter is zero
wire Ready = (state == S_idle); // controller status

// control unit
always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;

always @ (state, Start, Q[0], Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Decr_P = 0;
    Add_shift = 0;
    Shift = 0;
    case (state)
        S_idle:      begin if (Start) next_state = S_add_shift; Load_regs = 1; end
        S_add_shift: begin
            Decr_P = 1;
            if (Zero) next_state = S_idle;
            else begin
                next_state = S_add_shift;
                if (Q[0]) Add_shift = 1; else Shift = 1;
            end
        end
        default:      next_state = S_idle;
    endcase
end

// datapath unit
always @ (posedge clock) begin
    if (Load_regs) begin
        P <= dp_width;
        A <= 0;
        C <= 0;
        B <= Multiplicand;
        Q <= Multiplier;
    end
    if (Decr_P) P <= P - 1;
    if (Add_shift) {C, A, Q} <= {C, A+B, Q} >> 1;
    if (Shift) {C, A, Q} <= {C, A, Q} >> 1;
end
endmodule

module t_Prob_8_26;
parameter dp_width = 5;      // Width of datapath
wire [2 * dp_width - 1: 0] Product;
wire Ready;
reg [dp_width - 1: 0] Multiplicand, Multiplier;
reg Start, clock, reset_b;
integer Exp_Value;
wire Error;

```



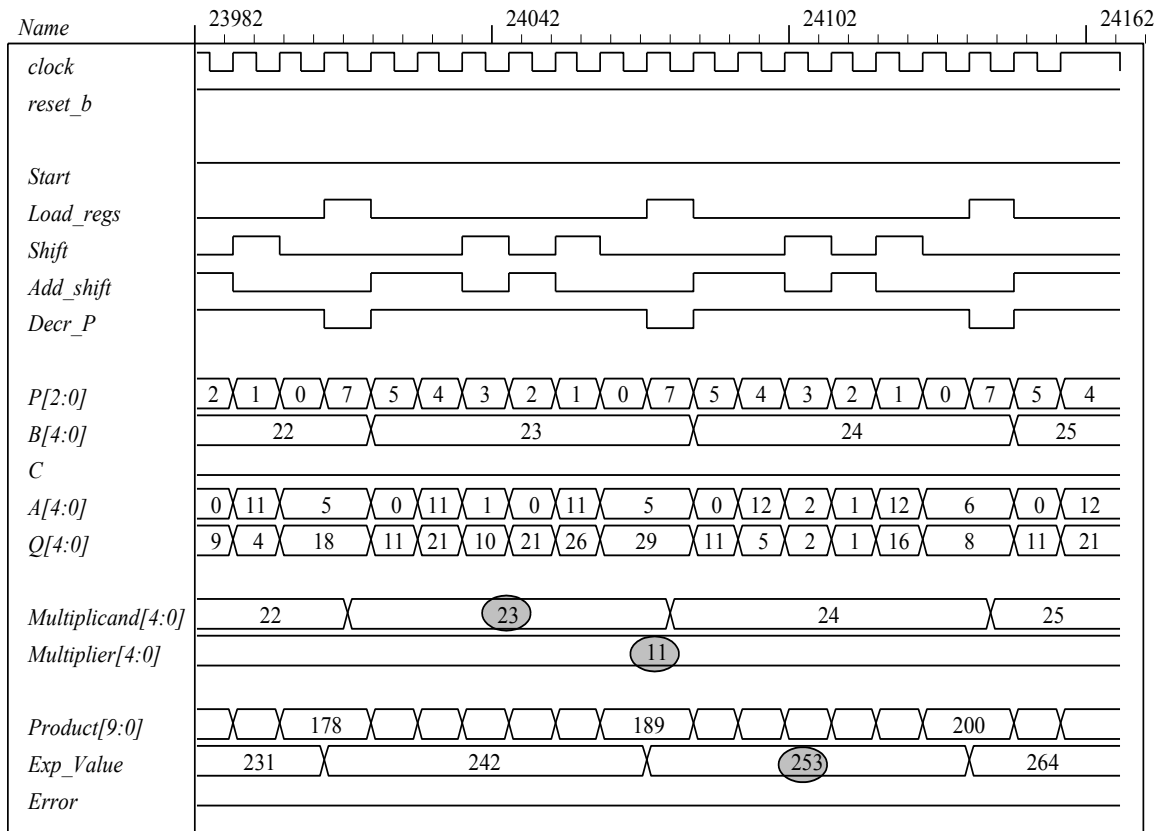
```

Prob_8_26 M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

initial #70000 $finish;
initial begin clock = 0; #5 forever #5 clock = ~clock; end
initial fork
    reset_b = 1;
    #2 reset_b = 0;
    #3 reset_b = 1;
join
initial begin #5 Start = 1; end
always @ (posedge Ready) begin
    Exp_Value = Multiplier * Multiplicand;
end
assign Error = Ready & (Exp_Value ^ Product);
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (32) #10 begin Multiplier = Multiplier + 1;
    repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
end
endmodule

```

Sample of simulation results.

**VHDL**

Library IEEE;

use IEEE.Std_Logic_1164.all;

use IEEE.Std_Logic_Unsigned.all;

Library Synopsys;

use Synopsys.attributes.all;

entity Prob_8_26_vhdl **is**

port (Product: **out** Std_Logic_vector (9 **downto** 0); Ready: **out** Std_Logic;

Multiplicand, Multiplier: **out** Std_Logic_vector (4 **downto** 0);

Start, clock, reset_b: **in** Std_Logic);

end Prob_8_26_vhdl;

architecture Behavioral **of** Prob_8_26_vhdl **is**

-- Default configuration: 5-bit datapath

constant dp_width: **integer** := 5; --Set to width of datapath

constant BC_size: **integer** := 3; -- Size of bit counter

type State_type **is** (S_idle, S_add_shift);

attribute enum_encoding **of** State_type: **type is** "01 10";

signal state, next_state: State_type;

signal A, B, Q: Std_Logic_vector (dp_width-1 **downto** 0); -- Sized for datapath

signal C: Std_Logic;

signal P: **integer**;

signal Load_regs, Decr_P, Add_shift, Shift: Std_Logic;

signal Sum: Std_logic_vector (dp_width **downto** 0);

begin

Product <= C & A & Q;

```

Sum <= A + B;
Zero <= '1' when P = 0 else '0';          -- Counter is zero
Ready <= '1' when state = S_idle else '0'; -- Controller status

-- Datapath unit RTL (Included asynch, active-low reset)
process (clock, reset_b) begin
    if (reset_b = '0') then
        P <= dp_width;
        C <= '0';
        for k in 0 to dp_width-1 loop
            B(k) <= '0'; A(k) <= '0'; Q(k) <= '0';
        end loop;
    elsif clock'event and clock = '1' then
        case (Load_regs & Decr_P & Add_shift & Shift) is
            when "1000" => --if Load_regs = '1' then
                C <= '0';
                P <= dp_width;
                for k in dp_width-1 downto 0 loop
                    A(k) <= '0';
                end loop;
                B <= Multiplicand;
                Q <= Multiplier;

            when "0100" => --elsif (Decr_P = '1') then
                P <= P -1;

            when "0010" => --elsif (Add_shift = '1') then
                Q <= Sum(0) & Q(dp_width -1 downto 1);
                A <= C & Sum(dp_width-1 downto 1);
                C <= Sum(dp_width);

            when "0001" => --elsif Shift = '1' then
                Q <= A(0) & Q(dp_width -1 downto 1);
                A <= C & A(dp_width-1 downto 1);
                C <= '0';

        end case;
    end if;
end process;

-- Control unit RTL
process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_idle;
    elsif clock'event and clock = '1' then state <= next_state;
    end if;
end process;

process (state, Start, Q(0), Zero) begin
    next_state <= S_idle;
    Load_regs <= '0';
    Decr_P <= '0';
    Add_shift <= '0';
    Shift <= '0';
    case (state) is
        when S_idle => if (Start = '1') then
            next_state <= S_add_shift;
            Load_regs <= '1';
        end if;

        when S_add_shift => Decr_P <= '1';
            if zero = '1' then next_state <= S_idle;
            else next_state <= S_add_shift;
        end if;
    end case;
end process;

```

```

        if Q(0) = '1' then
            Add_shift <= '1';
            Shift <= '1';
        end if;

        when others => next_state <= S_idle;
    end case;
end process;
end Behavioral;

```

Testbench template for Problem 8.26.

```

entity t_Prob_8_26_vhdl is
end t_Prob_8_26_vhdl;

architecture Behavioral of t_Prob_8_26_vhdl is
    signal Product: Std_Logic_vector (9 downto 0);
    signal Multiplicand, Multiplier: Std_Logic_vector (4 downto 0);
    signal Ready: Std_Logic;
    signal Start, clock, reset_b: Std_Logic;

    component Prob_8_26_vhdl
        port (Product: out Std_Logic_vector (9 downto 0); Ready: out Std_Logic;
            Multiplicand, Multiplier: out Std_Logic_vector (4 downto 0);
            Start, clock, reset_b: in Std_Logic);
    end component;
begin
    -- Instantiate Prob_8_26_vhdl
    UUT: Prob_8_26_vhdl port map (Product, Ready, Multiplicand, Multiplier,
        Start, clock, reset_b);

    -- Clock generation
    process -- clock for testbench
    begin
        t_clk <= '0';
        wait for 5 ns;
        t_clk <= '1';
        wait for 5 ns;
    end process;

    -- Input stimulus -- left as an exercise
end Behavioral;

```

8.27 (a)

```

// Verilog test bench for exhaustive simulation
module t_Sequential_Binary_Multiplier;
    parameter dp_width = 5;           // Width of datapath
    wire [2 * dp_width - 1: 0] Product;
    wire Ready;
    reg [dp_width - 1: 0] Multiplicand, Multiplier;
    reg Start, clock, reset_b;

    Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

    initial #109200 $finish;
    initial begin clock = 0; #5 forever #5 clock = ~clock; end
    initial fork
        reset_b = 1;
        #2 reset_b = 0;
        #3 reset_b = 1;
    join

```

```

join
initial begin #5 Start = 1; end
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (31) #10 begin Multiplier = Multiplier + 1;
        repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
    Start = 0;
end

// Error Checker
reg Error;
reg [2*dp_width - 1: 0] Exp_Value;
always @ (posedge Ready) begin
    Exp_Value = Multiplier * Multiplicand;
    //Exp_Value = Multiplier * Multiplicand + 1;    // Inject error to verify detection
    Error = (Exp_Value ^ Product);
end
endmodule

module Sequential_Binary_Multiplier (Product, Ready, Multiplicand,
    Multiplier, Start, clock, reset_b);
// Default configuration: 5-bit datapath
parameter dp_width = 5;    // Set to width of datapath
output [2*dp_width - 1: 0] Product;
output Ready;
input [dp_width - 1: 0] Multiplicand, Multiplier;
input Start, clock, reset_b;

parameter BC_size = 3; // Size of bit counter
parameter S_idle = 3'b001, // one-hot code
           S_add = 3'b010,
           S_shift = 3'b100;

reg [2: 0] state, next_state;
reg [dp_width - 1: 0] A, B, Q;    // Sized for datapath
reg C;
reg [BC_size - 1: 0] P;
reg Load_regs, Decr_P, Add_regs, Shift_regs;

// Miscellaneous combinational logic
assign Product = {C, A, Q};
wire Zero = (P == 0);    // counter is zero
wire Ready = (state == S_idle);    // controller status

// control unit
always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;

always @ (state, Start, Q[0], Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Decr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
        S_idle: begin if (Start) next_state = S_add; Load_regs = 1; end
        S_add: begin next_state = S_shift; Decr_P = 1; if (Q[0]) Add_regs = 1; end
        S_shift: begin Shift_regs = 1; if (Zero) next_state = S_idle;

```

```

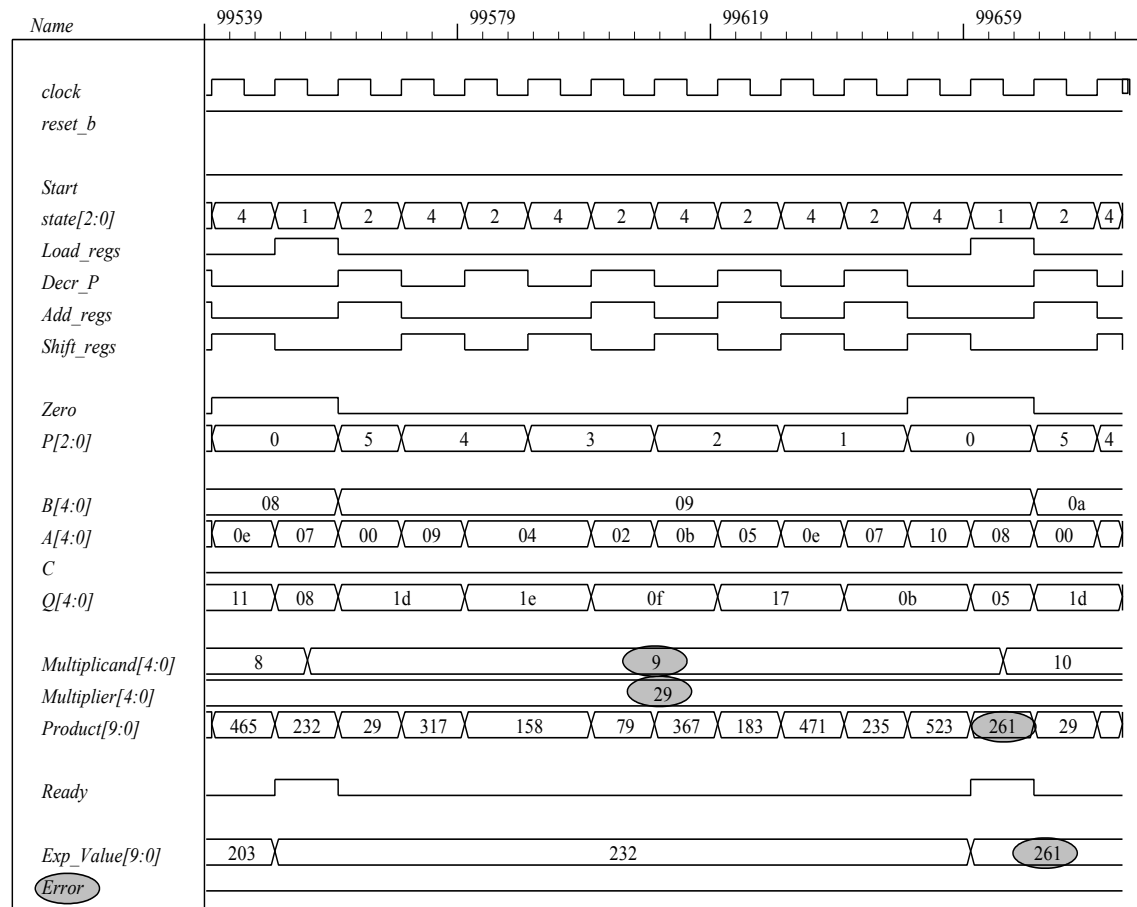
    else next_state = S_add; end
    default: next_state = S_idle;
  endcase
end

// datapath unit

always @ (posedge clock) begin
  if (Load_regs) begin
    P <= dp_width;
    A <= 0;
    C <= 0;
    B <= Multiplicand;
    Q <= Multiplier;
  end
  if (Add_regs) {C, A} <= A + B;
  if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
  if (Decr_P) P <= P - 1;
end
endmodule

```

Sample of simulation results:



```

-- VHDL test bench for exhaustive simulation

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Unsigned.all;

entity t_Prob_8_27_Sequential_Binary_Multiplier_vhdl is
    generic ( dp_width: integer := 5);          -- Width of datapath
end t_Prob_8_27_Sequential_Binary_Multiplier_vhdl;

architecture Behavioral of t_Prob_8_27_Sequential_Binary_Multiplier_vhdl is
    signal t_Product: Std_Logic_vector (2*dp_width downto 0);
    signal t_Ready: Std_Logic;
    signal t_Multiplicand, t_Multiplier: Std_Logic_vector (dp_width - 1 downto 0);
    signal t_Start, t_clock, t_reset_b: Std_Logic;
    signal Error: Std_Logic;
    signal Exp_Value: Std_Logic_vector (2*dp_width-1 downto 0);

    component Prob_8_27_Sequential_Binary_Multiplier_vhdl port (Product: out Std_Logic_vector
        (2*dp_width downto 0); Ready: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector (dp_width- 1 downto 0);
        Start, clock, reset_b: in Std_Logic);
    end component;
begin
    UUT: Prob_8_27_Sequential_Binary_Multiplier_vhdl port map (t_Product, t_Ready,
        t_Multiplicand, t_Multiplier, t_Start, t_clock, t_reset_b);

process begin
    -- clock for testbench
    t_clock <= '0';
    wait for 5 ns;
    t_clock <= '1';
    wait for 5 ns;
end process;
process begin
    t_reset_b <= '1';
    wait for 2 ns;
    t_reset_b <= '0';
    wait for 3 ns;
    t_reset_b <= '1';
    wait for 5 ns;
    t_Start <= '1';
end process;
-- Generate data values

-- Error Checker - Check whether result matches expected result of multiplication
-- Assertion of Ready indicates that computation of product is complete

process (t_Ready) begin
    if t_Ready'event and t_Ready = '1' then
        Exp_Value <= t_Multiplier * t_Multiplicand;          -- Normal operation
        -- Exp_value <= t_Multiplier * t_Multiplicand + 1;      -- Inject error to confirm detection
        Error <= '0';
        for k in 0 to 2*dp_width-1 loop
            if Exp_Value(k) /= t_Product(k) then Error <= '1'; end if;
        end loop;
    end if;
end process;
end Behavioral;

```

Library IEEE;

```

use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Unsigned.all;
Library Synopsys;      -- For one-hot encoding of states
use Synopsys.attributes.all;

entity Prob_8_27_Sequential_Binary_Multiplier_vhdl is
    generic (dp_width: integer := 5; BC_size: integer := 3); -- Width of datapath, counter
    port (Product: out Std_Logic_vector (2*dp_width downto 0);
        Ready: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector (dp_width - 1 downto 0);
        Start, clock, reset_b: in Std_Logic);
end Prob_8_27_Sequential_Binary_Multiplier_vhdl;

architecture Behavioral of Prob_8_27_Sequential_Binary_Multiplier_vhdl is
    type State_type is (S_idle, S_add, S_shift);
    attribute enum_encoding of State_type: type is "001 010 100"; -- one-hot

    signal state, next_state: State_type;
    signal A, B, Q: Std_Logic_vector (dp_width - 1 downto 0);      -- Sized for datapath
    signal C: Std_Logic;
    signal P: integer; -- Std_Logic_vector (BC_size- 1 downto 0);
    signal Load_regs, Decr_P, Add_regs, Shift_regs: Std_Logic;
    signal Sum: Std_Logic_Vector (dp_width downto 0);
    signal Zero: Std_Logic;
begin
    -- Miscellaneous combinational logic
    Ready <= '1' when (state = S_idle) else '0';    -- controller status

    Product <= C & A & Q;
    Sum <= ('0' & A) + ('0' & B); -- Forms Sum from combinational logic

    process (P) begin
        if P = 0 then Zero <= '1'; else Zero <= '0'; end if;
    end process;

    -- control unit

    process (clock, reset_b) begin
        if (reset_b = '0') then state <= S_idle;
        elsif clock'event and clock = '1' then state <= next_state;
        end if;
    end process;

    process (state, Start, Q(0), Zero) begin
        net_state <= S_idle;
        Lod_regs <= '0';
        Decr_P <= '0';
        Ad_regs <= '0';
        Shift_regs <= '0';
        case (state) is
            when S_idle => if (Start = '1') then
                next_state <= S_add;
                Load_regs <= '1';
            end if;

            when S_add => next_state <= S_shift;
                Decr_P <= '1';
                if (Q(0) = '1') then Add_regs <= '1';
            end if;

            when S_shift => Shift_regs <= '1';
                if (Zero = '1') then next_state <= S_idle;

```



```

        else next_state <= S_add; end if;

    when others => next_state <= S_idle;
end case;
end process;

-- datapath unit

process (clock) begin
    if (clock'event and clock = '1') then
        if (Load_regs = '1') then
            P <= dp_width;
            for k in dp_width-1 downto 0 loop A(k) <= '0'; end loop;
            C <= '0';
            B <= Multiplicand;
            Q <= Multiplier;
        elsif Add_regs = '1' then
            C <= Sum(dp_width);
            A <= Sum(dp_width-1 downto 0);
        elsif (Shift_regs = '1') then
            C <= '0';
            A <= C & A(dp_width-1 downto 1) ;
            Q <= A(0) & Q(dp_width-1 downto 1);
        end if;
    end if;
end process;
end Behavioral;

```

(b) In this part the controller is described by Fig. 8.18. The test bench includes probes to display the state of the controller.

Verilog

```
// Test bench for exhaustive simulation
module t_Sequential_Binary_Multiplier;
  parameter dp_width = 5;           // Width of datapath
  wire [2 * dp_width - 1: 0] Product;
  wire Ready;
  reg [dp_width - 1: 0] Multiplicand, Multiplier;
  reg Start, clock, reset_b;

  Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

  initial #109200 $finish;
  initial begin clock = 0; #5 forever #5 clock = ~clock; end
  initial fork
    reset_b = 1;
    #2 reset_b = 0;
    #3 reset_b = 1;
  join
  initial begin #5 Start = 1; end
  initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (31) #10 begin Multiplier = Multiplier + 1;
    repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
    Start = 0;
  end

  // Error Checker
  reg Error;
  reg [2*dp_width - 1: 0] Exp_Value;
  always @ (posedge Ready) begin
    Exp_Value = Multiplier * Multiplicand;
    //Exp_Value = Multiplier * Multiplicand + 1;    // Inject error to verify detection
    Error = (Exp_Value ^ Product);
  end

  wire [2: 0] state = {M0.G2, M0.G1, M0.G0};
endmodule

module Sequential_Binary_Multiplier (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);
// Default configuration: 5-bit datapath
  parameter dp_width = 5;           // Set to width of datapath
  output [2*dp_width - 1: 0] Product;
  output Ready;
  input [dp_width - 1: 0] Multiplicand, Multiplier;
  input Start, clock, reset_b;

  parameter BC_size = 3; // Size of bit counter
  reg [dp_width - 1: 0] A, B, Q;    // Sized for datapath
  reg C;
  reg [BC_size - 1: 0] P;
  wire Load_regs, Decr_P, Add_regs, Shift_regs;
```

```

// Status signals

assign      Product = {C, A, Q};
wire       Zero = (P == 0);      // counter is zero
wire       Q0 = Q[0];

// One-Hot Control unit (See Fig. 8.18)
DFF_S M0 (G0, D0, clock, Set);
DFF M1 (G1, D1, clock, reset_b);
DFF M2 (G2, G1, clock, reset_b);
or (D0, w1, w2);
and (w1, G0, Start_b);
and (w2, Zero, G2);
not (Start_b, Start);
not (Zero_b, Zero);
or (D1, w3, w4);
and (w3, Start, G0);
and (w4, Zero_b, G2);

and (Load_regs, G0, Start);
and (Add_regs, Q0, G1);
assign Ready = G0;
assign Decr_P = G1;
assign Shift_regs = G2;
not (Set, reset_b);

// datapath unit

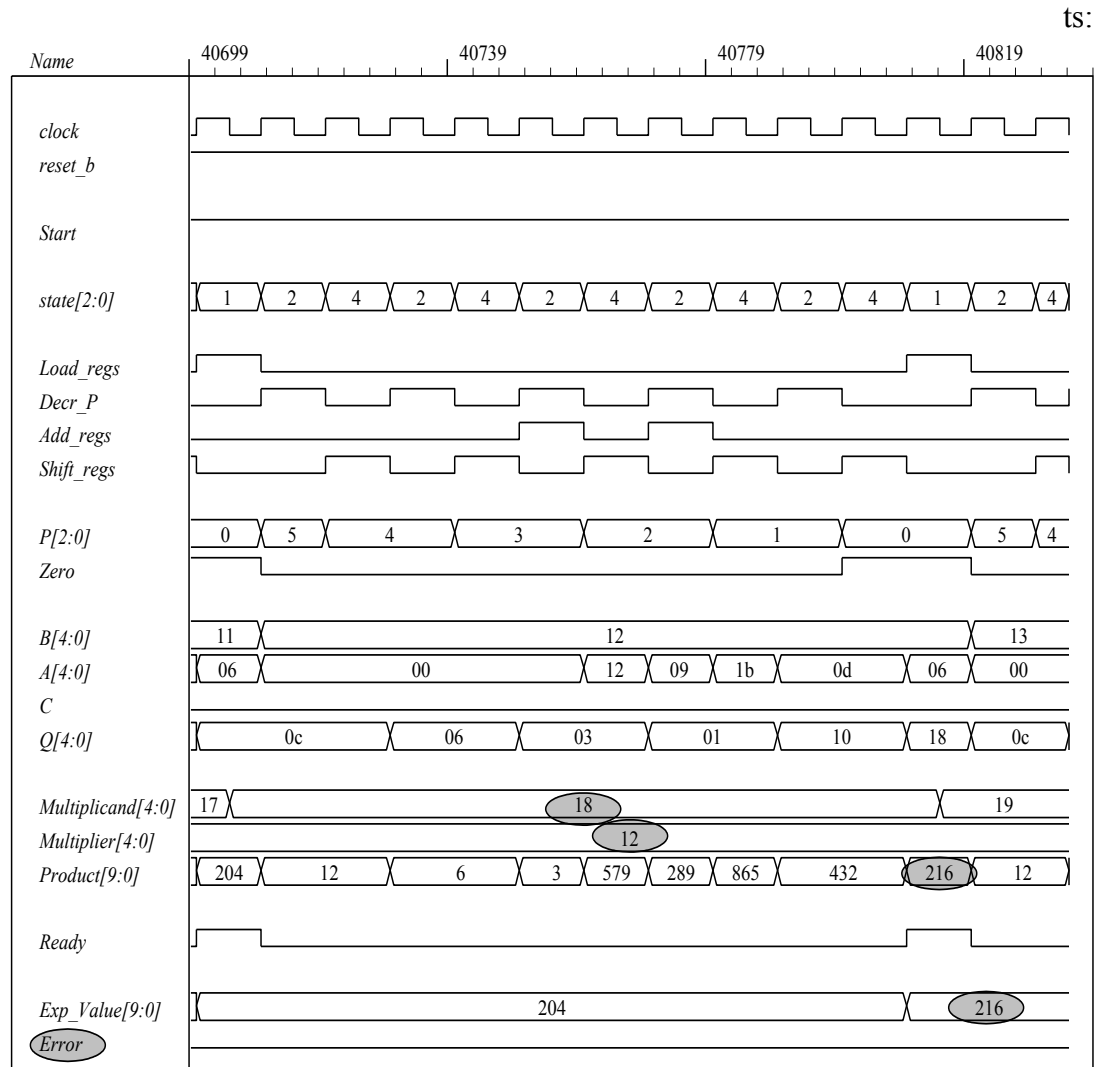
always @ (posedge clock) begin
  if (Load_regs) begin
    P <= dp_width;
    A <= 0;
    C <= 0;
    B <= Multiplicand;
    Q <= Multiplier;
  end
  if (Add_regs) {C, A} <= A + B;
  if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
  if (Decr_P) P <= P - 1;
end
endmodule

module DFF_S (output reg Q, input data, clock, Set);
  always @ (posedge clock, posedge Set)
    if (Set) Q <= 1'b1; else Q <= data;
endmodule

module DFF (output reg Q, input data, clock, reset_b);
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) Q <= 1'b0; else Q <= data;
endmodule

```

Sample of simulation results:



-- VHDL test bench for exhaustive simulation

Library IEEE;

use IEEE.Std_Logic_1164.all;

use IEEE.Std_Logic_Unsigned.all;

entity t_Prob_8_27_Sequential_Binary_Multiplier_vhdl **is**

generic (dp_width: **integer** := 5); -- Width of datapath

end t_Prob_8_27_Sequential_Binary_Multiplier_vhdl;

architecture Behavioral **of** t_Prob_8_27_Sequential_Binary_Multiplier_vhdl **is**

signal t_Product: Std_Logic_vector (2*dp_width **downto** 0);

signal t_Ready: Std_Logic;

signal t_Multiplicand, t_Multiplier: Std_Logic_vector (dp_width - 1 **downto** 0);

signal t_Start, t_clock, t_reset_b: Std_Logic;

signal Error: Std_Logic;

signal Exp_Value: Std_Logic_vector (2*dp_width-1 **downto** 0);

component Prob_8_27_Sequential_Binary_Multiplier_vhdl **port** (Product: **out** Std_Logic_vector (2*dp_width **downto** 0); Ready: **out** Std_Logic;

```

        Multiplicand, Multiplier: in Std_Logic_vector (dp_width- 1 downto 0); Start, clock, reset_b: in Std_Logic);
    end component;
begin
    UUT: Prob_8_27_Sequential_Binary_Multiplier_vhdl port map (t_Product, t_Ready, t_Multiplicand,
        t_Multiplier, t_Start, t_clock, t_reset_b);

    process begin
        -- clock for testbench
        t_clock <= '0';
        wait for 5 ns;
        t_clock <= '1';
        wait for 5 ns;
    end process;

    process begin
        t_reset_b <= '1';
        wait for 2 ns;
        t_reset_b <= '0';
        wait for 3 ns;
        t_reset_b <= '1';
        wait for 5 ns;
        t_Start <= '1';
    end process;
    -- Generate data values

    -- Error Checker - Check whether result matches expected result of multiplication
    -- Assertion of Ready indicates that computation of product is complete

    process (t_Ready) begin
        if t_Ready'event and t_Ready = '1' then
            Exp_Value <= t_Multiplier * t_Multiplicand;           -- Normal operation
            -- Exp_value <= t_Multiplier * t_Multiplicand + 1;      -- Inject error to confirm detection
            Error <= '0';
            for k in 0 to 2*dp_width-1 loop
                if Exp_Value(k) /= t_Product(k) then Error <= '1'; end if;
            end loop;
        end if;
    end process;
end Behavioral;

```

8.28 Verilog

```

// Test bench for exhaustive simulation
module t_Sequential_Binary_Multiplier;
    parameter dp_width = 5;           // Width of datapath
    wire [2 * dp_width - 1: 0] Product;
    wire Ready;
    reg [dp_width - 1: 0] Multiplicand, Multiplier;
    reg Start, clock, reset_b;

    Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

    initial #109200 $finish;
    initial begin clock = 0; #5 forever #5 clock = ~clock; end
    initial fork
        reset_b = 1;
        #2 reset_b = 0;
        #3 reset_b = 1;
    join
    initial begin #5 Start = 1; end

```

```

initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (31) #10 begin Multiplier = Multiplier + 1;
        repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
    Start = 0;
end

// Error Checker
reg Error;
reg [2*dp_width -1: 0] Exp_Value;
always @ (posedge Ready) begin
    Exp_Value = Multiplier * Multiplicand;
    //Exp_Value = Multiplier * Multiplicand + 1;    // Inject error to verify detection
    Error = (Exp_Value ^ Product);
end
wire [2: 0] state = {M0.M0.G2, M0.M0.G1, M0.M0.G0}; // Watch state
endmodule

module Sequential_Binary_Multiplier
    #(parameter dp_width = 5)
    (
        output [2*dp_width -1: 0] Product,
        output Ready,
        input [dp_width -1: 0] Multiplicand, Multiplier,
        input Start, clock, reset_b
    );
    wire Load_regs, Decr_P, Add_regs, Shift_regs, Zero, Q0;

    Controller M0 (Ready, Load_regs, Decr_P, Add_regs, Shift_regs, Start, Zero, Q0, clock, reset_b);
    Datapath M1(Product, Q0, Zero, Multiplicand, Multiplier, Start, Load_regs, Decr_P, Add_regs,
        Shift_regs, clock, reset_b);
endmodule

module Controller (
    output Ready,
    output Load_regs, Decr_P, Add_regs, Shift_regs,
    input Start, Zero, Q0, clock, reset_b
);
// One-Hot Control unit (See Fig. 8.18)
wire G0, G1, G2, D0, w1, w2, D1, w3, w4;

DFF_S M0 (G0, D0, clock, Set);
DFF M1 (G1, D1, clock, reset_b);
DFF M2 (G2, G1, clock, reset_b);
or (D0, w1, w2);
and (w1, G0, Start_b);
and (w2, Zero, G2);
not (Start_b, Start);
not (Zero_b, Zero);
or (D1, w3, w4);
and (w3, Start, G0);
and (w4, Zero_b, G2);

and (Load_regs, G0, Start);
and (Add_regs, Q0, G1);
assign Ready = G0;
assign Decr_P = G1;
assign Shift_regs = G2;
not (Set, reset_b);
endmodule

```

```

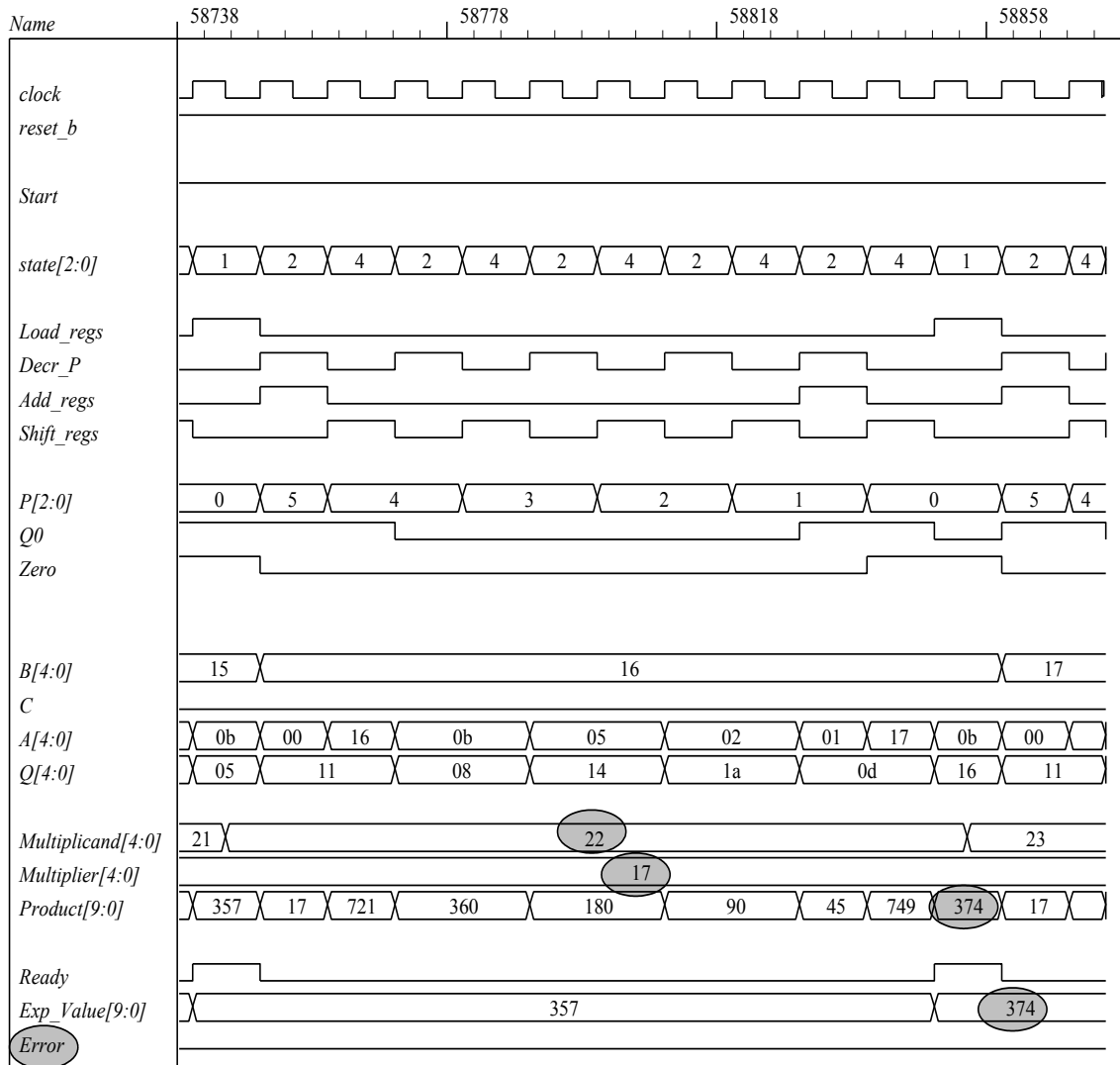
module Datapath #(parameter dp_width = 5, BC_size = 3) (
  output [2*dp_width - 1: 0] Product, output Q0, output Zero,
  input [dp_width - 1: 0] Multiplicand, Multiplier,
  input Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b
);
  reg [dp_width - 1: 0] A, B, Q; // Sized for datapath
  reg C;
  reg [BC_size - 1: 0] P;
  assign Product = {C, A, Q};
  // Status signals
  assign Zero = (P == 0); // counter is zero
  assign Q0 = Q[0];

  always @ (posedge clock) begin
    if (Load_regs) begin
      P <= dp_width;
      A <= 0;
      C <= 0;
      B <= Multiplicand;
      Q <= Multiplier;
    end
    if (Add_regs) {C, A} <= A + B;
    if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
    if (Decr_P) P <= P - 1;
  end
endmodule

module DFF_S (output reg Q, input data, clock, Set);
  always @ ( posedge clock, posedge Set)
    if (Set) Q <= 1'b1; else Q <= data;
endmodule

module DFF (output reg Q, input data, clock, reset_b);
  always @ ( posedge clock, negedge reset_b)
    if (reset_b == 0) Q <= 1'b0; else Q <= data;
endmodule

```



VHDL

Need IEEE.Std_Logic_unsigned.all to support addition of standard logic vectors. It defines +, -, and * or operating on std_logic_vectors.

library IEEE;

use IEEE.Std_Logic_1164.all;

use IEEE.Std_Logic_arith.all;

use IEEE.Std_Logic_unsigned.all;

entity Prob_8_28_Sequential_Binary_Multiplier_vhdl **is**

generic (dp_width: **integer** := 5; BC_size: **integer** := 3);

port (Product: **out** Std_Logic_vector (2*dp_width **downto** 0);

Ready: **out** Std_Logic;

Multiplicand, Multiplier: **in** Std_Logic_vector (dp_width - 1 **downto** 0);

Start, clock, reset_b: **in** Std_Logic;

end Prob_8_28_Sequential_Binary_Multiplier_vhdl;

architecture Structural of Prob_8_28_Sequential_Binary_Multiplier_vhdl **is**

signal Load_regs, Decr_P, Add_regs, Shift_regs, Q0, Zero: Std_Logic;

component Controller


```

        generic (dp_width, BC_size: integer);
        port (Ready, Load_regs, Decr_P, Add_regs, Shift_regs: out Std_Logic;
              Start, Zero, Q0, clock, reset_b: in Std_Logic);
    end component;

    component Datapath
        generic (dp_width, BC_size: integer);
        port (Product: out Std_Logic_vector (2*dp_width downto 0);
              Q0, Zero: out Std_Logic;
              Multiplicand, Multiplier: in Std_Logic_vector (dp_width - 1 downto 0);
              Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
    end component;

begin
    M0: Controller generic map (dp_width => 5, BC_size => 3)
        port map (Ready, Load_regs, Decr_P, Add_regs, Shift_regs, Start, Zero, Q0, clock,
                  reset_b);

    M1: Datapath generic map (dp_width => 5, BC_size => 3)
        port map (Product, Q0, Zero, Multiplicand, Multiplier, Start, Load_regs, Decr_P,
                  Add_regs,
                  Shift_regs, clock, reset_b);

end Structural;

library ieee;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Controller is
    generic (dp_width: integer := 5; BC_size: integer := 3);
    port (Ready, Load_regs, Decr_P, Add_regs, Shift_regs: out Std_Logic;
          Start, Zero, Q0, clock, reset_b: in Std_Logic);
end Controller;

architecture Structural of Controller is
    -- One-Hot Control unit (See Fig. 8.18)
    signal G0, G1, G2, D0, w1, w2, D1, w3, w4, Set, Start_b, Zero_b: Std_Logic;

    component D_FF_S port (Q: out Std_Logic; D, C, Set: in Std_Logic);
    end component;

    component D_FF port (Q: out Std_Logic; D, C, Clear_b: in Std_Logic);
    end component;

    component or2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    end component;

    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    end component;

    component not_gate port (y: out Std_Logic; xin: in Std_Logic);
    end component;

begin
    M_0: D_FF_S port map (G0, D0, clock, Set);
    M_1: D_FF port map (G1, D1, clock, reset_b);
    M_2: D_FF port map (G2, G1, clock, reset_b);
    G_1: or2_gate port map (D0, w1, w2);
    G_2: and2_gate port map (w1, G0, Start_b);
    G_3: and2_gate port map (w2, Zero, G2);
    G_4: not_gate port map (Start_b, Start);
    G_5: not_gate port map (Zero_b, Zero);

```

```

G_6: or2_gate port map (D1, w3, w4);
G_7: and2_gate port map (w3, Start, G0);
G_8: and2_gate port map (w4, Zero_b, G2);
G_9: and2_gate port map (Load_regs, G0, Start);
G_10: and2_gate port map (Add_regs, Q0, G1);
G_11: not_gate port map (Set, reset_b);

Ready <= G0;
Decr_P <= G1;
Shift_regs <= G2;
end Structural;

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;

entity Datapath is
  generic (dp_width: integer := 5; BC_size: integer := 3);
  port (Product: buffer Std_Logic_vector (2*dp_width downto 0);
        Q0, Zero: out Std_Logic;
        Multiplicand, Multiplier: in Std_Logic_vector (dp_width - 1 downto 0);
        Start, Load_regs, Decr_P, Add_regs, Shift_regs, clock, reset_b: in Std_Logic);
end Datapath;

architecture Behavioral of Datapath is
  signal A, B, Q: Std_Logic_vector (dp_width-1 downto 0); -- Sized for datapath
  signal C: Std_Logic;
  signal P: integer; --Std_Logic_vector (BC_size - 1 downto 0);
  signal Sum: Std_Logic_vector (dp_width downto 0);
begin
  Product <= C & A & Q;
  -- Status signals
  Zero <= '1' when P = 0 else '0'; -- counter is zero
  Q0 <= Q(0);

  process (clock) begin
    if(clock'event and clock = '1') then
      if (Load_regs = '1') then
        P <= dp_width;
        for k in dp_width-1 downto 0 loop A(k) <= '0'; end loop;
        C <= '0';
        B <= Multiplicand;
        Q <= Multiplier;
      end if;
      if Add_regs = '1' then
        --S <= ('0' & A) + ('0' & B) + S(2*dp_width);
        --(C & A) <= ('0' & A) + ('0' & B) + C;
        C <= Sum(dp_width);
        A <= Sum(dp_width-1 downto 0);
      end if;
      if Shift_regs = '1' then
        C <= '0';
        A <= C & A(dp_width-1 downto 1);
        Q <= A(0) & Q(dp_width-1 downto 1);
      end if;
      if Decr_P = '1' then
        P <= P -1;
      end if;
    end if;
  end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;

```

```

entity D_FF is
  port (Q: out Std_Logic; D, C, Clear_b: in Std_Logic);
end D_FF;

```

```

architecture Behavioral of D_FF is
begin

```

```

    process (C) begin
      if C'event and C = '1' then
        if(Clear_b = '0') then Q <= '0';
        else Q <= D;
        end if;
      end if;
    end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;
entity D_FF_S is
  port (Q: out Std_Logic; D, C, Set: in Std_Logic);
end D_FF_S;

```

```

architecture Behavioral of D_FF_S is
begin

```

```

    process (C) begin
      if C'event and C = '1' then
        if(Set = '0') then Q <= '1';
        else Q <= D;
        end if;
      end if;
    end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;

```

```

entity or2_gate is
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end or2_gate;

```

```

architecture Behavioral of or2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;
entity and2_gate is
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

```

```

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

```

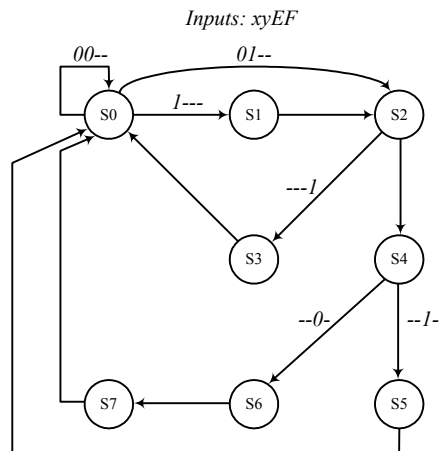
Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;

entity not_gate is
  port (y: out Std_Logic; xin: in Std_Logic);
end not_gate;

architecture Behavioral of not_gate is
begin
  y <= not xin;
end Behavioral;

```

8.29 (a)



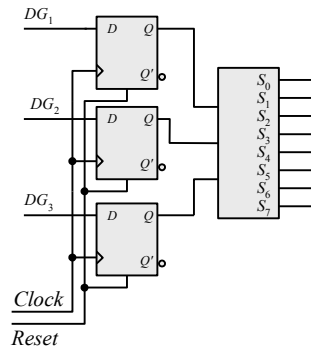
(b)

$$\begin{aligned}
 DS_0 &= x'y'S_0 + S_3 + S_5 + S_7 \\
 DS_1 &= xS_0 \\
 DS_2 &= x'yS_0 + S_1 \\
 DS_3 &= FS_2 \\
 DS_4 &= F'S_2 \\
 DS_5 &= E'S_5 \\
 DS_6 &= E'S_4 \\
 DS_7 &= S_6
 \end{aligned}$$

(c)

Output	Present state $G_1 G_2 G_3$	Inputs $x y E F$	Next state $G_1 G_2 G_3$
S_0	0 0 0	0 0 x x	0 0 0
S_0	0 0 0	1 x x x	0 0 1
S_0	0 0 0	0 1 x x	0 1 0
S_1	0 0 1	x x x x	0 1 0
S_2	0 1 0	x x 0 x	1 0 0
S_2	0 1 0	x x 1 x	0 1 1
S_3	0 1 1	x x x x	0 0 0
S_4	1 0 0	x x x 0	1 1 0
S_4	1 0 0	x x x 1	1 0 1
S_5	1 0 1	x x x x	0 0 0
S_6	1 1 0	x x x x	1 1 0
S_7	1 1 1	x x x x	0 0 0

(d)



$$DG_1 = F'S_2 + S_4 + S_6$$

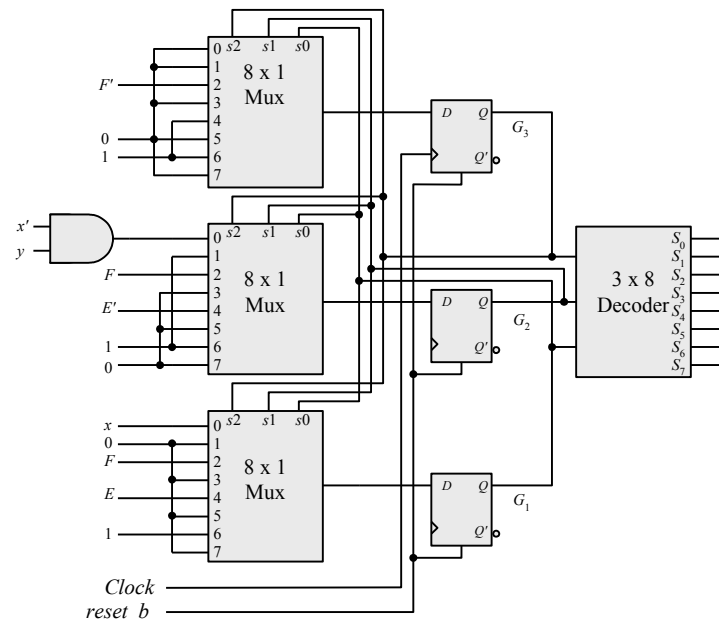
$$DG_2 = x'yS_0 + S_1 + FS_2 + E'S_4 + S_6$$

$$DG_3 = xS_0 + FS_2 + ES_4 + S_6$$

(e)

Present state $G_1 G_2 G_3$	Next state $G_1 G_2 G_3$	Input conditions	Mux1	Mux2	Mux3
0 0 0 0 0 0 0 0 0	0 0 0 0 0 1 0 1 0	$x'y'$ x $x'y$	0	$x'y$	x
0 0 1	0 1 0	None	0	1	0
0 1 0 0 1 0	1 0 0 0 1 1	F' F'	F'	F	F
0 1 1	0 0 0	None	0	0	0
1 0 0 1 0 0	1 1 0 1 0 1	E' E'	1	E'	E
1 0 1	0 0 0	None	0	0	0
1 1 0	1 1 0	None	1	1	1
1 1 1	0 0 0	None	0	0	0

(f)



(g) Verilog

```
module Controller_8_29g (input x, y, E, F, clock, reset_b);
```

```
    supply0 GND;
```

```
    supply1 VCC;
```

```
    mux_8x1 M3 (m3, GND, GND, F_bar, GND, VCC, GND, VCC, GND, G3, G2, G1);
```

```
    mux_8x1 M2 (m2, w1, VCC, F, GND, E_bar, GND, VCC, GND, G3, G2, G1);
```

```
    mux_8x1 M1 (m1, x, GND, F, GND, E, GND, VCC, GND, G3, G2, G1);
```

```
    DFF_8_29g DM3 (G3, m3, clock, reset_b);
```

```

DFF_8_29g DM2 (G2, m2, clock, reset_b);
DFF_8_29g DM1 (G1, m1, clock, reset_b);
decoder_3x8 M0_D (y0, y1, y2, y3, y4, y5, y6, y7, G3, G2, G1);
and (w1, x_bar, y);
not (F_bar, F);
not (E_bar, E);
not (x_bar, x);
endmodule

// Test plan: Exercise all paths of the ASM chart

module t_Controller_8_29g ();
reg x, y, E, F, clock, reset_b;
Controller_8_29g M0 (x, y, E, F, clock, reset_b);
wire [2: 0] state = {M0.G3, M0.G2, M0.G1};

initial #500 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin end
initial fork
    reset_b = 0; #2 reset_b = 1;
    #0 begin x = 1; y = 1; E = 1; F = 1; end // Path: S_0, S_1, S_2, S_34
    #80 reset_b = 0; #92 reset_b = 1;
    #90 begin x = 1; y = 1; E = 1; F = 0; end
    #150 reset_b = 0;
    #152 reset_b = 1;
    #150 begin x = 1; y = 1; E = 0; F = 0; end // Path: S_0, S_1, S_2, S_4, S_5
    #200 reset_b = 0;
    #202 reset_b = 1;
    #190 begin x = 1; y = 1; E = 0; F = 0; end // Path: S_0, S_1, S_2, S_4, S_6, S_7
    #250 reset_b = 0;
    #252 reset_b = 1;
    #240 begin x = 0; y = 0; E = 0; F = 0; end // Path: S_0
    #290 reset_b = 0;
    #292 reset_b = 1;
    #280 begin x = 0; y = 1; E = 0; F = 0; end // Path: S_0, S_2, S_4, S_6, S_7
    #360 reset_b = 0;
    #362 reset_b = 1;
    #350 begin x = 0; y = 1; E = 1; F = 0; end // Path: S_0, S_2, S_4, S_5
    #420 reset_b = 0;
    #422 reset_b = 1;
    #410 begin x = 0; y = 1; E = 0; F = 1; end // Path: S_0, S_2, S_3
join
endmodule

module mux_8x1 (output reg y, input x0, x1, x2, x3, x4, x5, x6, x7, s2, s1, s0);
always @ (x0, x1, x2, x3, x4, x5, x6, x7, s0, s1, s2)
    case ({s2, s1, s0})
        3'b000: y = x0;
        3'b001: y = x1;
        3'b010: y = x2;
        3'b011: y = x3;
        3'b100: y = x4;
        3'b101: y = x5;
        3'b110: y = x6;
        3'b111: y = x7;
    endcase
endmodule

module DFF_8_29g (output reg q, input data, clock, reset_b);
always @ (posedge clock, negedge reset_b)
    if (!reset_b) q <= 1'b0; else q <= data;

```

endmodule

```

module decoder_3x8 (output reg y0, y1, y2, y3, y4, y5, y6, y7, input x2, x1, x0);
always @ (x0, x1, x2) begin
    {y7, y6, y5, y4, y3, y2, y1, y0} = 8'b0;
    case ({x2, x1, x0})
        3'b000: y0= 1'b1;
        3'b001: y1= 1'b1;
        3'b010: y2= 1'b1;
        3'b011: y3= 1'b1;
        3'b100: y4= 1'b1;
        3'b101: y5= 1'b1;
        3'b110: y6= 1'b1;
        3'b111: y7= 1'b1;
    endcase;
end
endmodule

```

VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity Prob_8_29g_Controller_vhdl is
    port (x, y, E, F, clock, reset_b: in Std_Logic);
end Prob_8_29g_Controller_vhdl;

```

architecture Structural **of** Prob_8_29g_Controller_vhdl **is**

```

    signal w1, m1, m2, m3: Std_Logic;
    signal G1, G2, G3: Std_Logic;
    signal F_bar, E_bar, x_bar: Std_Logic;
    signal y0, y1, y2, y3, y4, y5, y6, y7: Std_Logic;
    signal GND: Std_Logic;
    signal VCC: Std_Logic;

```

```

    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end component;

```

```

    component DFF_8_29g port (q: out Std_Logic; data: in Std_Logic; clock, reset_b: in Std_Logic);
end component;

```

```

    component mux_8x1 port (y: out Std_Logic; x0, x1, x2, x3, x4, x5, x6, x7: in Std_Logic; s2, s1, s0: in Std_Logic);
end component;

```

```

    component not_gate port (y: out Std_Logic; xin1: in Std_Logic);
end component;

```

```

    component decoder_3x8 port (y0, y1, y2, y3, y4, y5, y6, y7: out Std_Logic; x2, x1, x0: in Std_Logic);
end component;

```

begin

```

    GND <= '0';
    VCC <= '1';
    M_3: mux_8x1          port map (m3, GND, GND, F_bar, GND, VCC, GND, VCC, GND, G3, G2, G1);
    M_2: mux_8x1          port map (m2, w1, VCC, F, GND, E_bar, GND, VCC, GND, G3, G2, G1);
    M_1: mux_8x1          port map (m1, x, GND, F, GND, E, GND, VCC, GND, G3, G2, G1);
    DFF_M3:DFF_8_29g      port map (G3, m3, clock, reset_b);
    DFF_M2:DFF_8_29g      port map (G2, m2, clock, reset_b);
    DFF_M1:DFF_8_29g      port map (G1, m1, clock, reset_b);
    M0_D: decoder_3x8      port map (y0, y1, y2, y3, y4, y5, y6, y7, G3, G2, G1);
    M_4: and2_gate        port map (w1, x_bar, y);

```



```

        M_5: not_gate      port map (F_bar, F);
        M_6: not_gate      port map (E_bar, E);
        M_7: not_gate      port map (x_bar, x);
    end Structural;

    library IEEE;
    use IEEE.Std_Logic_1164.all;

    entity mux_8x1 is
        port (y: out Std_Logic; x0, x1, x2, x3, x4, x5, x6, x7: in Std_Logic; s2, s1, s0: in Std_Logic);
    end mux_8x1;

    architecture Behavioral of mux_8x1 is
        signal sel_mux: Std_Logic_vector (2 downto 0);

    begin
        sel_mux <= s2 & s1 & s0;
        process (x0, x1, x2, x3, x4, x5, x6, x7, s0, s1, s2) begin
            case (sel_mux) is
                when "000" => y <= x0;
                when "001" => y <= x1;
                when "010" => y <= x2;
                when "011" => y <= x3;
                when "100" => y <= x4;
                when "101" => y <= x5;
                when "110" => y <= x6;
                when "111" => y <= x7;
            end case;
        end process;
    end Behavioral;

    library IEEE;
    use IEEE.Std_Logic_1164.all;

    entity DFF_8_29g is
        port (q: out Std_Logic; data, clock, reset_b: in Std_Logic);
    end DFF_8_29g;

    architecture Behavioral of DFF_8_29g is
    begin
        process (clock, reset_b) begin
            if (reset_b = '0') then q <= '0';
            elsif clock'event and clock = '1' then q <= data;
            end if;
        end process;
    end Behavioral;

    library IEEE;
    use IEEE.Std_Logic_1164.all;

    entity decoder_3x8 is
        port (y0, y1, y2, y3, y4, y5, y6, y7: out Std_Logic; x2, x1, x0: in Std_Logic);
    end decoder_3x8;

    architecture Behavioral of decoder_3x8 is
        signal sel_dec: Std_Logic_vector (2 downto 0);
    begin
        sel_dec <= x2 & x1 & x0;
        process (x0, x1, x2) begin

            y7 <= '0'; y6 <= '0'; y5 <= '0'; y4 <= '0';
            y3 <= '0'; y2 <= '0'; y1 <= '0'; y0 <= '0';
        end process;
    end Behavioral;

```

```

        case (sel_dec) is
        when "000" => y0 <= '1';
        when "001" => y1 <= '1';
        when "010" => y2 <= '1';
        when "011" => y3 <= '1';
        when "100" => y4 <= '1';
        when "101" => y5 <= '1';
        when "110" => y6 <= '1';
        when "111" => y7 <= '1';
        end case;
    end process;
end Behavioral;

library IEEE;
use IEEE.Std_Logic_1164.all;

entity and2_gate is
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavioral of and2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;

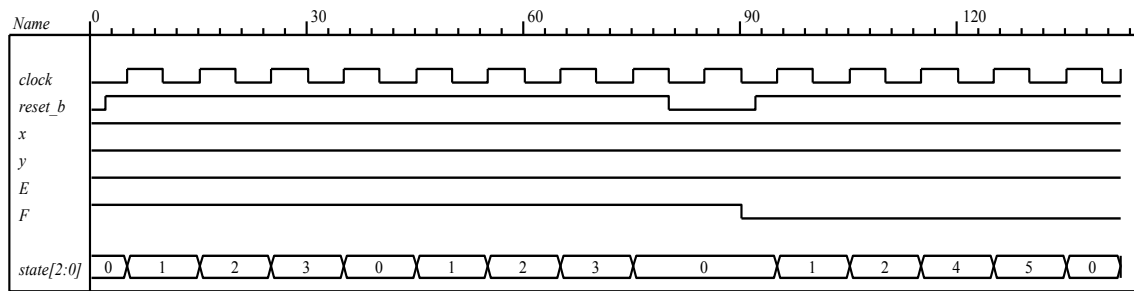
library IEEE;
use IEEE.Std_Logic_1164.all;

entity not_gate is
    port (y: out Std_Logic; xin1: in Std_Logic);
end not_gate;

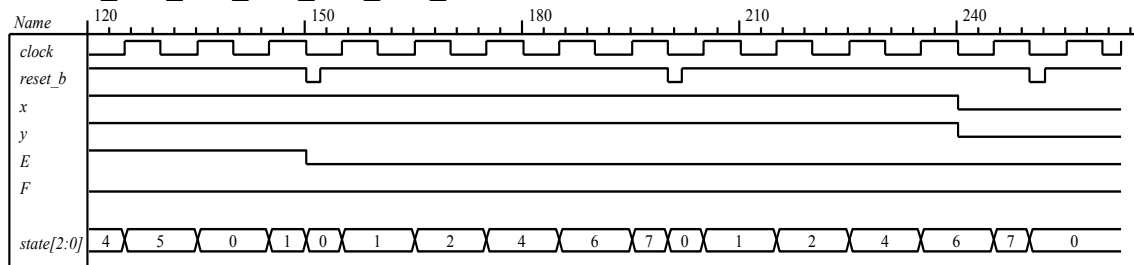
architecture Behavioral of not_gate is
begin
    y <= not xin1;
end Behavioral;

```

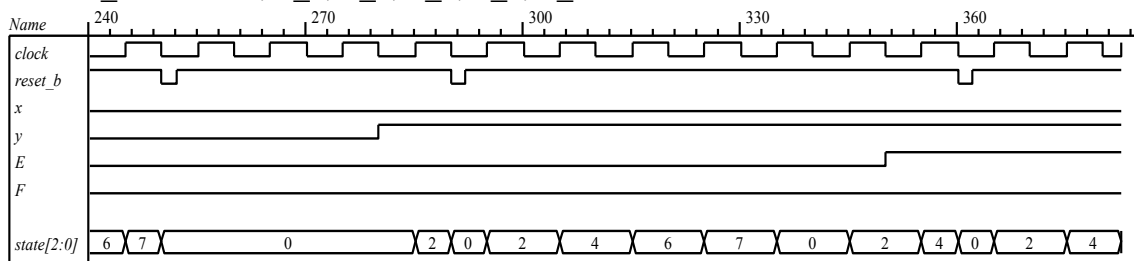
Path: S_0, S_1, S_2, S_3 and Path: S_0, S_1, S_2, S_4, S_5



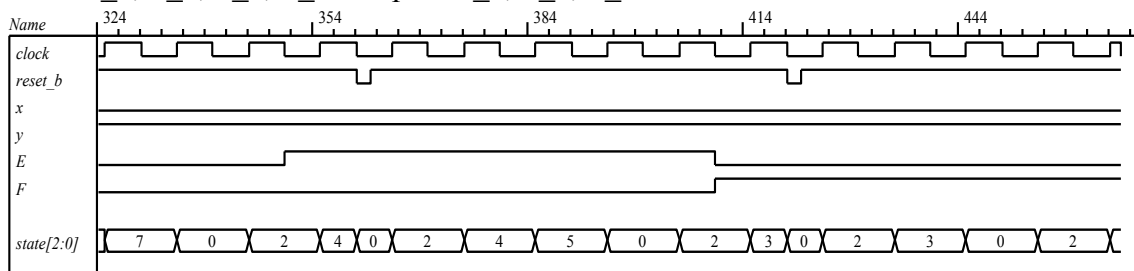
Path: S_0, S_1, S_2, S_4, S_6, S_7



Path: S_0 and Path , S_0, S_2, S_4, S_6, S_7



Path: S_0, S_2, S_4, S_5 and path S_0, S_2, S_3



(h) Verilog

```

module Controller_8_29h (input x, y, E, F, clock, reset_b);
  parameter S_0 = 3'b000, S_1 = 3'b001, S_2 = 3'b010,
    S_3 = 3'b011, S_4 = 3'b100, S_5 = 3'b101, S_6 = 3'b110, S_7 = 3'b111;
  reg [2:0] state, next_state;

  always @ (posedge clock, negedge reset_b)

```

```

if (!reset_b) state <= S_0; else state <= next_state; always @ (state, x, y, E, F) begin
case (state)
S_0:          if (x) next_state = S_1;
               else next_state = y ? S_2: S_0;
S_1:          next_state = S_2;
S_2:          if (F) next_state = S_3; else next_state = S_4;
S_3, S_5, S_7: next_state = S_0;
S_4:          if (E) next_state = S_5; else next_state = S_6;
S_6:          next_state = S_7;
default:      next_state = S_0;
endcase
end
endmodule

```

// Test plan: Exercise all paths of the ASM chart

```

module t_Controller_8_29h ();
reg x, y, E, F, clock, reset_b;

Controller_8_29h M0 (x, y, E, F, clock, reset_b);

initial #500 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin end
initial fork
reset_b = 0; #2 reset_b = 1;
#20 begin x = 1; y = 1; E = 1; F = 1; end // Path: S_0, S_1, S_2, S_34
#80 reset_b = 0; #92 reset_b = 1;
#90 begin x = 1; y = 1; E = 1; F = 0; end
#150 reset_b = 0;
#152 reset_b = 1;
#150 begin x = 1; y = 1; E = 0; F = 0; end // Path: S_0, S_1, S_2, S_4, S_5
#200 reset_b = 0;
#202 reset_b = 1;
#190 begin x = 1; y = 1; E = 0; F = 0; end // Path: S_0, S_1, S_2, S_4, S_6, S_7
#250 reset_b = 0;
#252 reset_b = 1;
#240 begin x = 0; y = 0; E = 0; F = 0; end // Path: S_0
#290 reset_b = 0;
#292 reset_b = 1;
#280 begin x = 0; y = 1; E = 0; F = 0; end // Path: S_0, S_2, S_4, S_6, S_7
#360 reset_b = 0;
#362 reset_b = 1;
#350 begin x = 0; y = 1; E = 1; F = 0; end // Path: S_0, S_2, S_4, S_5
#420 reset_b = 0;
#422 reset_b = 1;
#410 begin x = 0; y = 1; E = 0; F = 1; end // Path: S_0, S_2, S_3
join
endmodule

```

Note: Simulation results match those for 8.29g.

VHDL

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity Prob_8_29h_Controller_vhdl is
port (x, y, E, F, clock, reset_b: in Std_Logic);
end Prob_8_29h_Controller_vhdl;

```

```

architecture Behavioral of Prob_8_29h_Controller_vhdl is
  constant S_0: Std_Logic_vector (2 downto 0) := "000";
  constant S_1: Std_Logic_vector (2 downto 0) := "001";
  constant S_2: Std_Logic_vector (2 downto 0) := "010";
  constant S_3: Std_Logic_vector (2 downto 0) := "011";
  constant S_4: Std_Logic_vector (2 downto 0) := "100";
  constant S_5: Std_Logic_vector (2 downto 0) := "101";
  constant S_6: Std_Logic_vector (2 downto 0) := "110";
  constant S_7: Std_Logic_vector (2 downto 0) := "111";
  signal state, next_state: Std_Logic_vector (2 downto 0);
begin
  process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_0;
    elsif clock'event and clock = '1' then state <= next_state;
    end if;
  end process;

  process (state, x, y, E, F) begin
    next_state <= S_0;
    case (state) is
      when S_0 => if (x = '1') then next_state <= S_1;
        elsif y = '1' then next_state <= S_2;
        else next_state <= S_0; end if;
      when S_1 => next_state <= S_2;
      when S_2 => if (F = '1') then next_state <= S_3;
        else next_state <= S_4; end if;
      when S_4 => if (E = '1') then next_state <= S_5;
        else next_state <= S_6; end if;
      when S_3 => next_state <= S_0;
      when S_5 => next_state <= S_0;
      when S_6 => next_state <= S_7;
      when S_7 => next_state <= S_0;
      when others => next_state <= S_0;
    end case;
  end process;
end Behavioral;

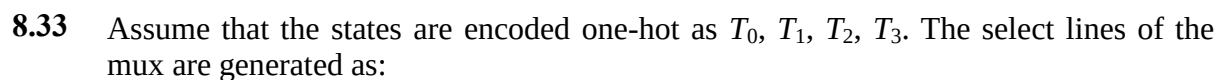
```

8.30 (a) $E = 1$

(b) $E = 0$

8.31 $A = 0110$, $B = 0010$, and $C = 0000$.

$A * B = 1100$	$A + B = 1000$	$A - B = 0100$
$\sim C = 1111$	$A \& B = 0010$	$A B = 0110$
$A \wedge B = 0100$	$\&A = 0$	$\sim C = 1$
$A B = 1$	$A \&\& C = 0$	$ A = 1$
$A < B = 0$	$A > B = 1$	$A ! B = 1$



$$S0 = T_1 + T_3$$

$$\text{load} = T_0 + T_1 + T_2 + T_3$$


```
module Datapath BEH
```

(

Digital Design With An Introduction to the Verilog HDL, VHDL, and SystemVerilog, Sixth Edition – Solution Manual.
M. Mano, M.D. Ciletti, Copyright 2018 .

```

reg [dp_width -1: 0] R1;
reg [R2_width -1: 0] R2;

assign count = R2;
assign Zero = ~(| R1);
always @ (posedge clock) begin
    E <= R1[dp_width -1] & Shift_left;
    if (Load_regs) begin R1 <= data; R2 <= {R2_width{1'b1}}; end
    if (Shift_left) {E, R1} <= {E, R1} << 1;
    if (Incr_R2) R2 <= R2 + 1;
end
endmodule

// Test Plan for Datapath Unit:
// Demonstrate action of Load_regs
//   R1 gets data, R2 gets all ones
// Demonstrate action of Incr_R2
// Demonstrate action of Shift_left and detect E

// Test bench for datapath

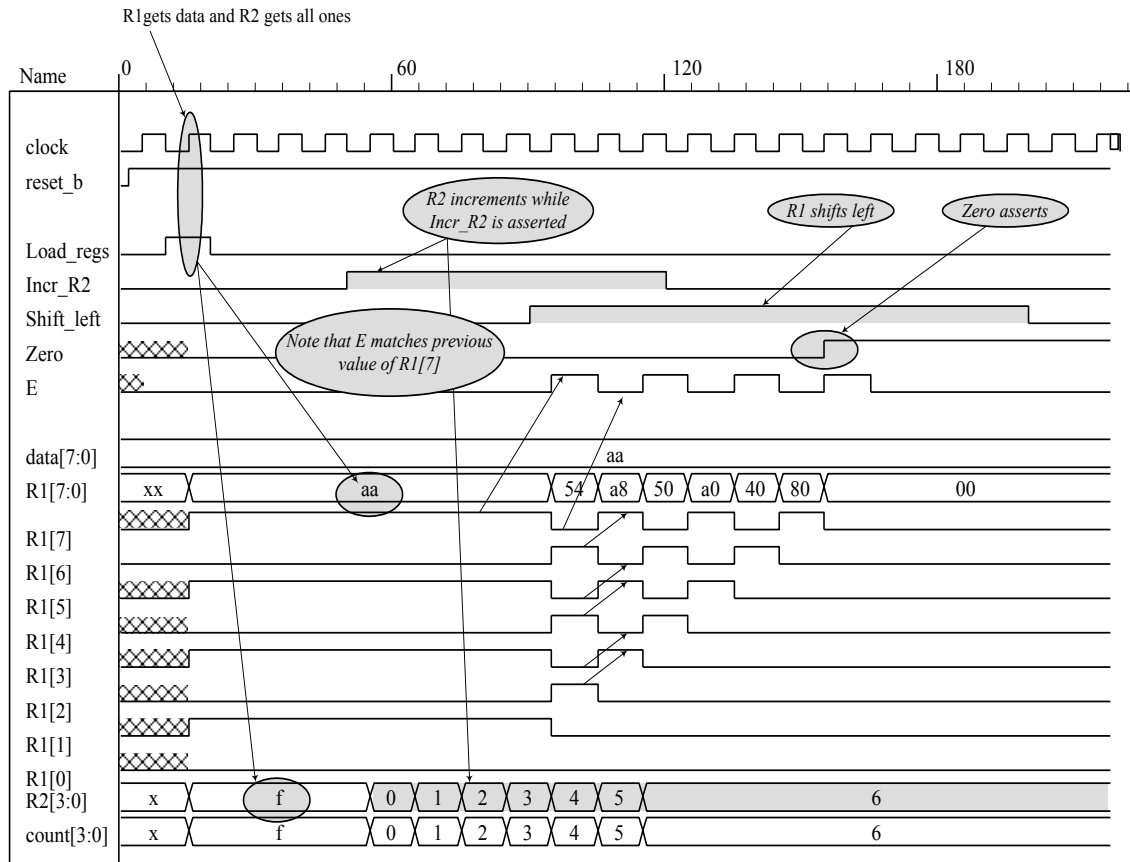
module t_Datapath_Unit
#(parameter dp_width = 8, R2_width = 4)
( );
    wire [R2_width -1: 0] count;
    wire E, Zero;
    reg [dp_width -1: 0] data;
    reg Load_regs, Shift_left, Incr_R2, clock, reset_b;

    Datapath_BEH M0 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);

    initial #250 $finish;
    initial begin clock = 0; forever #5 clock = ~clock; end
    initial begin reset_b = 0; #2 reset_b = 1; end
    initial fork
        data = 8'haa;
        Load_regs = 0;
        Incr_R2 = 0;
        Shift_left = 0;
        #10 Load_regs = 1;
        #20 Load_regs = 0;
        #50 Incr_R2 = 1;
        #120 Incr_R2 = 0;
        #90 Shift_left = 1;
        #200 Shift_left = 0;
    join
endmodule

```

Note: The simulation results show tests of the operations of the datapath independent of the control unit, so *count* does not represent the number of ones in the *data*.



VHDL

```
entity Prob_8_34a_Datapath_BEH_vhdl
  generic (dp_width: integer := 8; R2_width: integer := 4);
  port (count: out Std_Logic_vector (R2_width - 1 downto 0); E, Zero: out Std_Logic;
        data: in Std_Logic_vector (dp_width - 1 downto 0);
        Load_regs, Shift_left, Incr_R2, clock, reset_b: in Std_Logic);
end Prob_8_34_Datapath_BEH_vhdl;
```

architecture Behavioral of Prob_8_34_Datapath_BEH_vhdl is

```
  signal R1: Std_Logic_vector (dp_width - 1 downto 0);
  signal R2: Std_Logic_vector (R2_width - 1 downto 0);
```

begin

```
  count <= R2;
  process (R1) begin
    Zero <= 1;
    for k = dp_width - 1 downto 0 loop
      if R1(k) /= '0' then Zero = '0'; end if;
    end loop;
  end process;
```

```
  process (clock) begin
    if clock'event and clock = '1' then
      E <= R1(dp_width - 1) and Shift_left; end if;
      if (Load_regs = '1') then
        R1 <= data;
        for dp_width - 1 downto 0 loop
          R2(k) = '1';
```



```

        end loop; end if;
    if (Shift_left = '1') then
        E <= R1(dp_width-1);
        R1 <= R1(dp_width-2 downto 0) & '0';
    end if;
    if (Incr_R2= '1') then R2 <= R2 + '1';
    end if;
end if;
end process;
endmodule

```

Test plan: Verify assertion of *Ready* in *S_idle*
 Verify assertion of *Incr_R2* in *S_1*
 Verify assertion of *Shift_left* in *S_2*
 Separately assert and verify the actions of *Load_regs*, *Shift_left*, *Incr_R2_regs*
 Verify recovery from random *reset_b*
 confirm that all activity occurs on rising edge of clock
 Other?

:

(b) // Control Unit

Verilog

```

module Controller_BEH (
    output Ready,
    output reg Load_regs,
    output Incr_R2, Shift_left,
    input Start, Zero, E, clock, reset_b
);
    parameter S_idle = 0, S_1 = 1, S_2 = 2, S_3 = 3;
    reg [1:0] state, next_state;

    assign Ready = (state == S_idle);
    assign Incr_R2 = (state == S_1);
    assign Shift_left = (state == S_2);

    always @ (posedge clock, negedge reset_b)
        if (reset_b == 0) state <= S_idle;
        else state <= next_state;

    always @ (state, Start, Zero, E) begin
        Load_regs = 0;
        case (state)
            S_idle: if (Start) begin Load_regs = 1; next_state = S_1; end
                    else next_state = S_idle;
            S_1: if (Zero) next_state = S_idle; else next_state = S_2;

            S_2: next_state = S_3;
            S_3: if (E) next_state = S_1; else next_state = S_2;
        endcase
    end
endmodule

```

```

// Test plan for Control Unit
// Verify that state enters S_idle with reset_b asserted.
// With reset_b de-asserted, verify that state enters S_1 and asserts Load_Regs when
// Start is asserted.
// Verify that Incr_R2 is asserted in S_1.
// Verify that state returns to S_idle from S_1 if Zero is asserted.
// Verify that state goes to S_2 if Zero is not asserted.

```

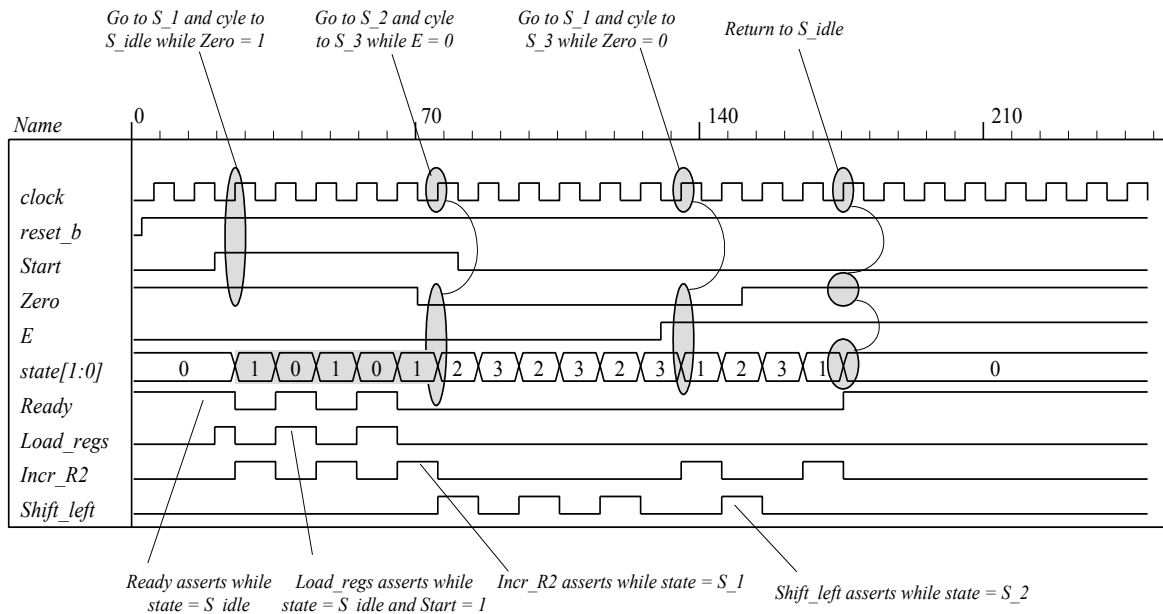
```
// Verify that Shift_left is asserted in S_2.
// Verify that state goes to S_3 from S_2 unconditionally.
// Verify that state returns to S_2 from S_3 if E is not asserted.
// Verify that state goes to S_1 from S_3 if E is asserted.
```

```
// Test bench for Control Unit
```

```
module t_Control_Unit ();
wire Ready, Load_regs, Incr_R2, Shift_left;
reg Start, Zero, E, clock, reset_b;
```

```
Controller_BEH M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);
```

```
initial #250 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin reset_b = 0; #2 reset_b = 1; end
initial fork
  Zero = 1;
  E = 0;
  Start = 0;
  #20 Start = 1; // Cycle from S_idle to S_1
  #80 Start = 0;
  #70 Zero = 0; // S_idle to S_1 to S_2 to S_3 and cycle to S_2.
  #130 E = 1; // Cycle to S_3 to S_1 to S_2 to S_3
  #150 Zero = 1; // Return to S_idle
join
endmodule
```



VHDL

```
entity Prob_8_34b_Controller_BEH_vhdl is
  port (Ready: out Std_Logic; Load_regs: out Std_Logic; Incr_R2, Shift_left: out Std_Logic,
        Start, Zero, E, clock, reset_b: in Std_Logic);
end Prob_8_34_Controller_BEH_vhdl;
```

```
architecture Behavioral of Prob_8_34_Controller_BEH_vhdl is
```

```

type State_type is (S_idle, S_1, S_2, S_3);
attribute enum_encoding of State_type: type is "0 1 2 3";
signal state, next_state: State_type;
begin
  Ready <= '1' when state = S_idle else "0";
  Incr_R2 <= '1' when state = S_1 else '0';
  Shift_left <= '1' when state = S_2 else '0';

  process (clock, reset_b) begin
    if (reset_b = '0') then state <= S_idle;
    elsif clock.event and clock = '1' then state <= next_state;
    end if;

  process (state, Start, Zero, E) begin
    Load_regs <= '0';
    case (state) is
      when S_idle => if (Start = '1') then Load_regs <= 1; next_state <= S_1;
                     else next_state <= S_idle; end if;

      when S_1 =>    if (Zero = '1') then next_state = S_idle; else next_state = S_2;
                     end if;

      when S_2 =>    next_state <= S_3;

      when S_3 =>    if (E = '1') then next_state <= S_1; else next_state <= S_2;
                     end if;

    end case;
  end process;
end Behavioral;

```

(c) Verilog

```

// Integrated system
module Count_Ones_BEH_BEH
# (parameter dp_width = 8, R2_width = 4)
(
  output [R2_width -1: 0] count,
  input [dp_width -1: 0] data,
  input Start, clock, reset_b
);
  wire Load_regs, Incr_R2, Shift_left, Zero, E;
  Controller_BEH M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);
  Datapath_BEH M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);
endmodule

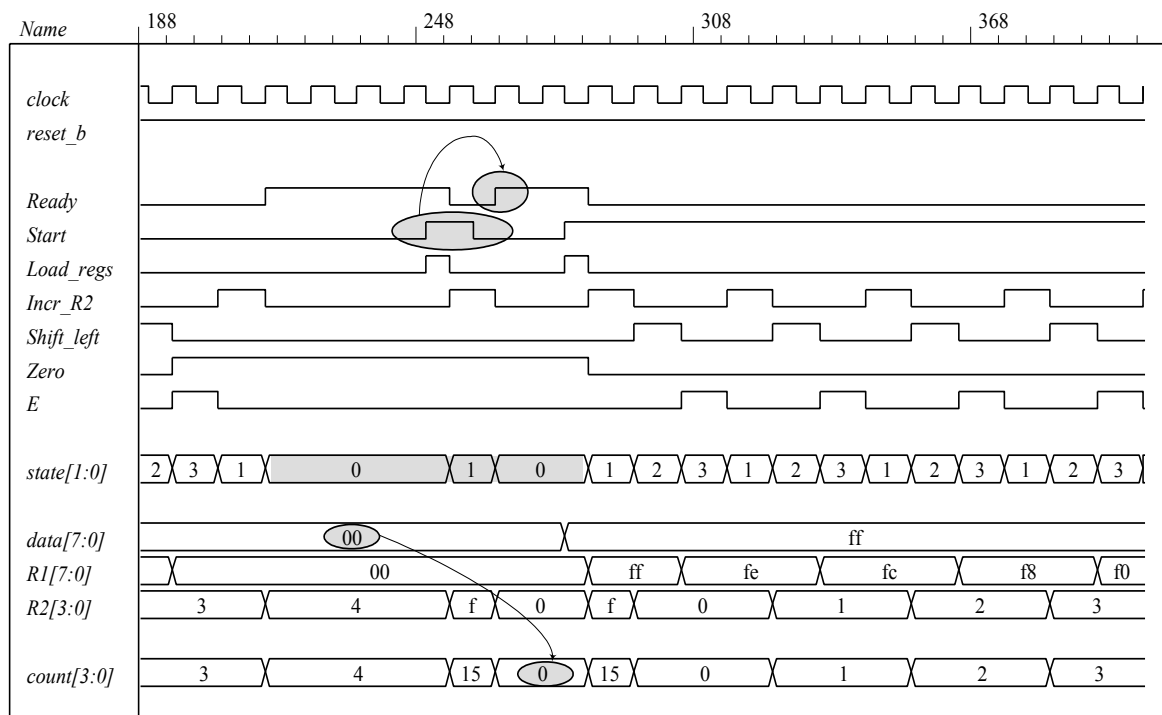
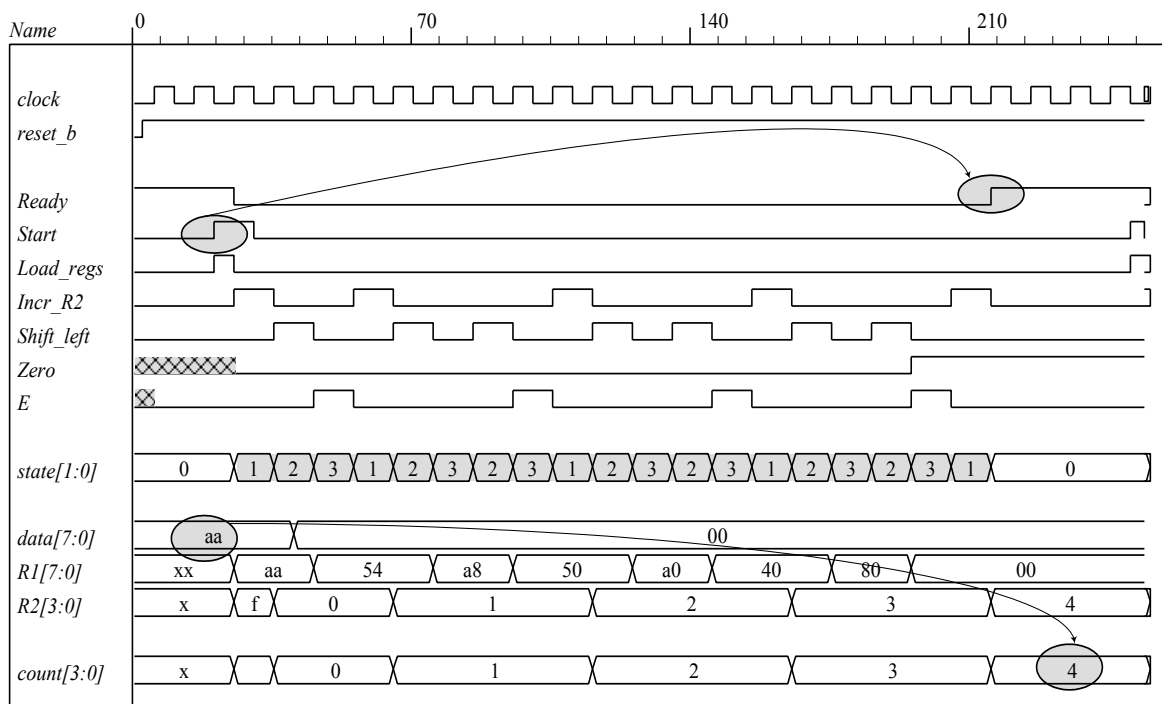
// Test plan for integrated system
// Test for data values of 8'haa, 8'h00, 8'hff.

// Test bench for integrated system

module t_count_Ones_BEH_BEH ();
parameter dp_width = 8, R2_width = 4;
wire [R2_width -1: 0] count;
reg [dp_width -1: 0] data;
reg Start, clock, reset_b;

Count_Ones_BEH_BEH M0 (count, data, Start, clock, reset_b);
initial #700 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin reset_b = 0; #2 reset_b = 1; end
initial fork
  data = 8'haa;      // Expect count = 4
  Start = 0;
  #20 Start = 1;
  #30 Start = 0;
  #40 data = 8'b00; // Expect count = 0
  #250 Start = 1;
  #260 Start = 0;
  #280 data = 8'hff;
  #280 Start = 1;
  #290 Start = 0;
join
endmodule

```




```

assign Shift_left = (state == S_2);

always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) state <= S_idle;
  else state <= next_state;

always @ (state, Start, Zero, E) begin
  Load_regs = 0;
  case (state)
    S_idle: if (Start) begin Load_regs = 1; next_state = S_1; end
           else next_state = S_idle;
    S_1:   if (Zero) next_state = S_idle; else next_state = S_2;

    S_2:   next_state = S_3;
    S_3:   if (E) next_state = S_1; else next_state = S_2;
  endcase
end
endmodule

```

Note: Test plan, test bench and simulation results are same as (b), but with states numbered with one-hot codes.

VHDL

-- One-Hot Control unit

Library Synopsys;

use Synopsys.attributes.all;

entity Prob_8_34d_Controller_BEH_1Hot_vhdl **is**

port (Ready: out Std_Logic; Load_regs, Incr_R2, Shift_left: **out** Std_Logic;
 Start, Zero, E, clock, reset_b: **in** Std_Logic;

end Prob_8_34d_Controller_BEH_1Hot_vhdl;

architecture Behavioral **of** Prob_8_34d_Controller_BEH_1Hot_vhdl **is**

type State_type **is** (S_idle, S_1, S_2, S_3);

attribute enum_encoding **of** State_type: **type is** "0001 0010 0100 1000";

signal state, next_state: State_type;

begin

 Ready = '1' **when** (state = S_idle) **else** '0';

 Incr_R2 = '1' **when** (state = S_1) **else** '0';

 Shift_left = '1' **when** (state = S_2) **else** '0';

process (clock, reset_b) **begin**

if (reset_b = '0') **then** state <= S_idle;

elsif clock'event **and** clock = '1' **then** state <= next_state; **end if**;

process (state, Start, Zero, E) **begin**

 Load_regs <= '0';

case (state) **is**

when S_idle => **if** (Start) **begin** Load_regs <= '1'; next_state <= S_1; **end**
 else next_state <= S_idle; **end if**;

when S_1 => **if** (Zero = '1') **then** next_state <= S_idle;
 else next_state <= S_2; **end if**;

when S_2 => next_state <= S_3;

when S_3 => **if** (E = '1') **then** next_state <= S_1;
 else next_state <= S_2; **end if**;

```

        when others => next_state <= S_idle;
    end case;
end process;
end Behavioral;

```

(e) Verilog

// Integrated system with one-hot controller

```

module Prob_8_34e_Count_Ones_BEH_1Hot
# (parameter dp_width = 8, R2_width = 4)
(
    output [R2_width -1: 0]    count,
    input [dp_width -1: 0]     data,
    input                      Start, clock, reset_b
);
    wire Load_regs, Incr_R2, Shift_left, Zero, E;
    Controller_BEH_1Hot M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);
    Datapath_BEH M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);
endmodule

```

Note: Test plan, test bench and simulation results are same as (c), but with states numbered with one-hot codes.

VHDL

-- Integrated system with one-hot controller

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

```

entity Prob_8_34e_Count_Ones_BEH_1Hot_vhdl **is**

constant dp_width: integer := 8;

constant R2_width: integer := 4;

port (count: **out** Std_Logic_vector (R2_width -1 **downto** 0);

 data **in** (dp_width -1 **downto** 0);

 Start, clock, reset_b: **in** Std_Logic);

end Prob_8_34e_Count_Ones_BEH_1Hot_vhdl;

architecture Structural **of** Prob_8_34e_Count_Ones_BEH_1Hot_vhdl **is**

component Controller_BEH_1Hot **port** (Ready, Load_regs, Incr_R2, Shift_left, Start: **in** Std_Logic;
 Zero, E, clock, reset_b: **in** Std_Logic); **end component**;

component Datapath_BEH **port** (count: **out** Std_Logic_vector (dp_width-1 **downto** 0); E, Zero: **out** Std_Logic;
 data: **in** Std_Logic_vector (dp_width-1 **downto** 0); Load_regs, Shift_left, Incr_R2, clock, reset_b: **in** Std_Logic);

signal Load_regs, Incr_R2, Shift_left, Zero, E: Std_Logic;

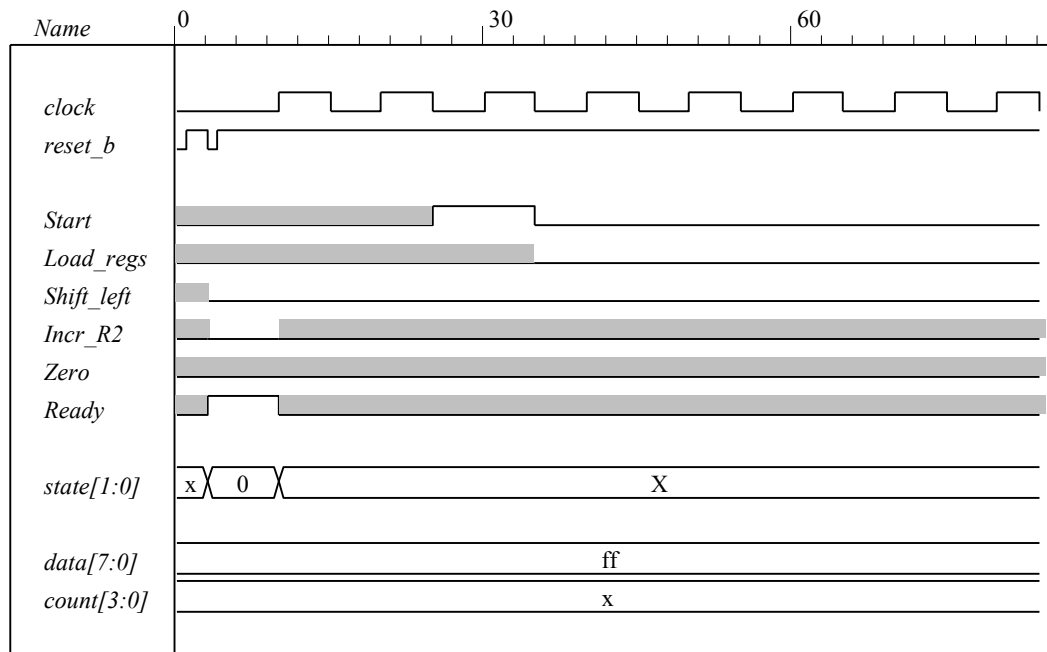
begin

 M0Controller_BEH_1Hot **port map** (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);

 M1Datapath_BEH **port map** (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);

end Structural;

- 8.35** See block diagram, ASMD chart, and logic diagram in Figs. 8.22, 8.23. Note: Signal *Start* is initialized to 0 when the simulation begins. Otherwise, the state of the structural model will become *X* at the first clock after the reset condition is deasserted, with *Start* and *Load_Regs* having unknown values. In this condition the structural model cannot operate correctly.



Verilog

```

module Count_Ones_STR_STR (count, Ready, data, Start, clock, reset_b);
// Mux – decoder implementation of control logic
// controller is structural
// datapath is structural

```

```

parameter R1_size = 8, R2_size = 4;
output [R2_size - 1: 0] count;
output Ready;
input [R1_size - 1: 0] data;
input Start, clock, reset_b;
wire Load_regs, Shift_left, Incr_R2, Zero, E;

```

```

Controller_STR M0 (Ready, Load_regs, Shift_left, Incr_R2, Start, E, Zero, clock, reset_b);
Datapath_STR M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock);

```

endmodule

```

module Controller_STR (Ready, Load_regs, Shift_left, Incr_R2, Start, E, Zero, clock, reset_b);
output Ready;
output Load_regs, Shift_left, Incr_R2;
input Start;
input E, Zero;
input clock, reset_b;

```

```

supply0    GND;
supply1    PWR;
parameter  S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11; // Binary code
wire       Load_regs, Shift_left, Incr_R2;
wire       G0, G0_b, D_in0, D_in1, G1, G1_b;
wire       Zero_b = ~Zero;

wire       E_b = ~E;
wire [1:0]  select = {G1, G0};
wire [0:3]  Decoder_out;

assign     Ready = ~Decoder_out[0];
assign     Incr_R2 = ~Decoder_out[1];
assign     Shift_left = ~Decoder_out[2];
and        (Load_regs, Ready, Start);
mux_4x1_beh Mux_1 (D_in1, GND, Zero_b, PWR, E_b, select);
mux_4x1_beh Mux_0  (D_in0, Start, GND, PWR, E, select);
D_flip_flop_AR_b M1  (G1, G1_b, D_in1, clock, reset_b);
D_flip_flop_AR_b M0  (G0, G0_b, D_in0, clock, reset_b);
decoder_2x4_df M2    (Decoder_out, G1, G0, GND);

endmodule

module Datapath_STR (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock);
parameter    R1_size = 8, R2_size = 4;
output [R2_size -1: 0] count;
output       E, Zero;
input [R1_size -1: 0] data;
input       Load_regs, Shift_left, Incr_R2, clock;
wire [R1_size -1: 0] R1;
supply0     Gnd;
supply1     Pwr;
assign      Zero = (R1 == 0);

Shift_Reg     M1    (R1, data, Gnd, Shift_left, Load_regs, clock, Pwr);
Counter       M2    (count, Load_regs, Incr_R2, clock, Pwr);
D_flip_flop_AR M3    (E, w1, clock, Pwr);
and         (      w1, R1[R1_size -1], Shift_left);
endmodule

module Shift_Reg (R1, data, SI_0, Shift_left, Load_regs, clock, reset_b);
parameter    R1_size = 8;
output [R1_size -1: 0] R1;
input [R1_size -1: 0] data;
input       SI_0, Shift_left, Load_regs;
input       clock, reset_b;
reg [R1_size -1: 0] R1;

always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) R1 <= 0;
  else begin
    if (Load_regs) R1 <= data; else
      if (Shift_left) R1 <= {R1[R1_size -2:0], SI_0}; end
endmodule

module Counter (R2, Load_regs, Incr_R2, clock, reset_b);
parameter    R2_size = 4;
output [R2_size -1: 0] R2;
input       Load_regs, Incr_R2;
input       clock, reset_b;
reg [R2_size -1: 0] R2;

```

```

always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) R2 <= 0;
  else if (Load_regs) R2 <= {R2_size {1'b1}}; // Fill with 1
  else if (Incr_R2 == 1) R2 <= R2 + 1;
endmodule

module D_flip_flop_AR (Q, D, CLK, RST);
  output   Q;
  input    D, CLK, RST;
  reg      Q;

  always @ (posedge CLK, negedge RST)
    if (RST == 0) Q <= 1'b0;
    else Q <= D;
endmodule

module D_flip_flop_AR_b (Q, Q_b, D, CLK, RST);
  output   Q, Q_b;
  input    D, CLK, RST;
  reg      Q;
  assign    Q_b = ~Q;
  always @ (posedge CLK, negedge RST)
    if (RST == 0) Q <= 1'b0;
    else Q <= D;
endmodule

// Behavioral description of 4-to-1 line multiplexer
// Verilog 2005 port syntax

module mux_4x1_beh
(output reg   m_out,
 input       in_0, in_1, in_2, in_3,
 input [1: 0] select
);

  always @ (in_0, in_1, in_2, in_3, select) // Verilog 2005 syntax
    case (select)
      2'b00: m_out = in_0;
      2'b01: m_out = in_1;
      2'b10: m_out = in_2;
      2'b11: m_out = in_3;
    endcase
endmodule

// Dataflow description of 2-to-4-line decoder
// See Fig. 4.19. Note: The figure uses symbol E, but the
// Verilog model uses enable to clearly indicate functionality.

module decoder_2x4_df (D, A, B, enable);
  output   [0: 3] D;

  input     A, B;
  input     enable;

  assign    D[0] = ~(~A & ~B & ~enable),
             D[1] = ~(~A & B & ~enable),
             D[2] = ~(A & ~B & ~enable),
             D[3] = ~(A & B & ~enable);
endmodule

module t_Count_Ones;
  parameter R1_size = 8, R2_size = 4;

```

```

wire [R2_size -1: 0]   R2;

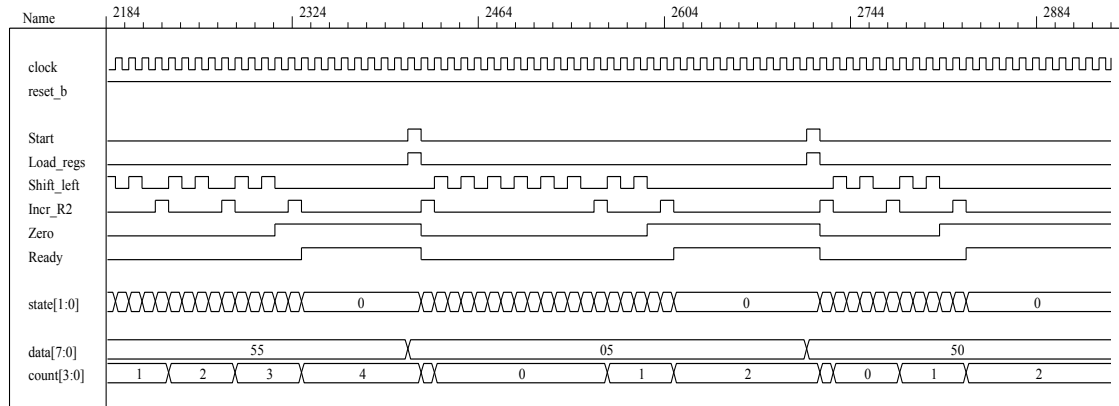
wire [R2_size -1: 0]   count;
wire                                Ready;
reg [R1_size -1: 0]   data;
reg                                Start, clock, reset_b;
wire [1: 0] state;      // Use only for debug
assign state = {M0.M0.G1, M0.M0.G0};

Count_Ones_STR_STR M0 (count, Ready, data, Start, clock, reset_b);

initial #4000 $finish;
initial begin clock = 0; #5 forever #5 clock = ~clock; end
initial fork
    Start = 0;

    #1 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
        data = 8'hff;
    # 25 Start = 1;
    # 35 Start = 0;
    #310 data = 8'h0f;
    #310 Start = 1;
    #320 Start = 0;
    #610 data = 8'hf0;
    #610 Start = 1;
    #620 Start = 0;
    #910 data = 8'h00;
    #910 Start = 1;
    #920 Start = 0;
    #1210 data = 8'haa;
    #1210 Start = 1;
    #1220 Start = 0;
    #1510 data = 8'h0a;
    #1510 Start = 1;
    #1520 Start = 0;
    #1810 data = 8'ha0;
    #1810 Start = 1;
    #1820 Start = 0;
    #2110 data = 8'h55;
    #2110 Start = 1;
    #2120 Start = 0;
    #2410 data = 8'h05;
    #2410 Start = 1;
    #2420 Start = 0;
    #2710 data = 8'h50;
    #2710 Start = 1;
    #2720 Start = 0;
    #3010 data = 8'ha5;
    #3010 Start = 1;
    #3020 Start = 0;
    #3310 data = 8'h5a;
    #3310 Start = 1;
    #3320 Start = 0;
join
endmodule

```



VHDL

```
-- Mux/decoder implementation of control logic
-- controller is structural
-- datapath is structural
```

Library IEEE;

use IEEE.Std_Logic_1164.all;

entity PROB_8_35_COUNT_ONES_STR_STR_VHDL **is**

generic (R1_size: **integer** := 8; R2_size: **integer** := 4);

port (count: **out** Std_Logic_vector (R2_size-1 **downto** 0);

 Ready: **out** Std_Logic; data: **in** Std_Logic_vector (R1_size-1 **downto** 0);

 Start, clock, reset_b: **in** Std_Logic);

end PROB_8_35_COUNT_ONES_STR_STR_VHDL;

architecture Structural **of** PROB_8_35_COUNT_ONES_STR_STR_VHDL **is**

signal Load_regs, Shift_left, Incr_R2, Zero, E: Std_Logic;

component Controller_STR **port** (Ready: **out** Std_Logic;

 Load_regs, Shift_left, Incr_R2: **out** Std_Logic;

 Start, E, Zero, clock, reset_b: **in** Std_Logic);

end component;

component Datapath_STR **port** (count: **out** Std_Logic_vector

 (R2_size-1 **downto** 0); E, Zero: **out** Std_Logic;

 data: Std_Logic_vector (R1_size-1 **downto** 0);

 Load_regs, Shift_left, Incr_R2, clock: **in** Std_Logic);

end component;

begin

 M0: Controller_STR **port map** (Ready, Load_regs, Shift_left, Incr_R2, Start, E, Zero, clock, reset_b);

 M1: Datapath_STR **port map** (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock);

end Structural;

Library IEEE;

use IEEE.Std_Logic_1164.all;

entity Controller_STR **is**

port (Ready: **buffer** Std_Logic; Load_regs, Shift_left, Incr_R2: **out** Std_Logic;

 Start, E, Zero, clock, reset_b: **in** Std_Logic);

end Controller_STR;

architecture Behavioral **of** Controller_STR **is**

signal GND: Std_Logic;

signal PWR: Std_Logic;

type State_type **is** (S0, S1, S2, S3);

signal state, next_state: State_type;

signal G0, D_in0, D_in1, G1: Std_Logic;

```

signal Decoder_out: Std_Logic_vector (0 to 3);
signal Zero_b, E_b: Std_Logic;
signal sel: Std_Logic_vector (1 downto 0);

component mux_4x1_beh port (m_out: out Std_Logic;
    in_0, in_1, in_2, in_3: in Std_Logic;
    sel: in Std_Logic_vector (1 downto 0));
end component;

component D_flip_flop_AR
    port (Q: out Std_Logic; D, CLK, RST: in Std_Logic);
end component;

component decoder_2x4_df
    port (D: out Std_Logic_vector (3 downto 0);
    A, B, enable: in Std_Logic);
end component;

begin
    PWR <= '1';
    GND <= '0';
    Zero_b <= not Zero;
    E_b <= not E ;
    sel <= G1 & G0;
    Ready <= not Decoder_out(0);
    Incr_R2 <= not Decoder_out(1);
    Shift_left <= not Decoder_out(2);
    Load_regs <= Ready and Start;

    Mux_1: mux_4x1_beh    port map (D_in1, GND, Zero_b, PWR, E_b, sel);
    Mux_0: mux_4x1_beh    port map (D_in0, Start, GND, PWR, E, sel);
    M1: D_flip_flop_AR    port map (G1, D_in1, clock, reset_b);
    M0: D_flip_flop_AR    port map (G0, D_in0, clock, reset_b);
    M2: decoder_2x4_df    port map (Decoder_out, G1, G0, GND);
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Datapath_STR is
    generic (R1_size: integer := 8; R2_size: integer := 4);
    port (count: out Std_Logic_vector (R2_size-1 downto 0);
    E, Zero: out Std_Logic; data: Std_Logic_vector
    (R1_size-1 downto 0);
    Load_regs, Shift_left, Incr_R2, clock: in Std_Logic);
end Datapath_STR;

architecture Structural of Datapath_STR is
    signal GND: Std_Logic;
    signal PWR: Std_Logic;
    signal R1: Std_Logic_vector (R1_size -1 downto 0);
    signal w1: Std_Logic;

    component Shft_Reg port (R1: out Std_Logic_vector (R1_size-1 downto 0);
    data: in Std_Logic_vector (R1_size-1 downto 0);
    SI_0, Shift_left, Load_regs, clock, reset_b: in Std_Logic);
    end component;

    component Counter port (R2: out Std_Logic_vector (R2_size -1 downto 0);
    Load_regs, Incr_R2, clock, reset_b: in Std_Logic);
    end component;

```

```

component D_flip_flop_AR port (Q: out Std_Logic;
                               D, CLK, RST: in Std_Logic);
end component;

component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end component;

begin
  PWR <= '1';
  GND <= '0';
  process (R1) begin      -- Detect empty R1
    Zero <= '1';
    for k in R1_size-1 downto 0 loop if R1(k) = '1' then Zero <= '0'; end if;
    end loop;
  end process;

  M_1: Shft_Reg port map (R1, data, Gnd, Shift_left, Load_regs, clock, PWR);
  M_2: Counter port map (R2 => count, Load_regs => Load_regs,
                          Incr_R2 => Incr_R2, clock => clock, reset_b => PWR);
  M_3: D_flip_flop_AR port map (E, w1, clock, Pwr);
  G_1: and2_gate port map (w1, R1(R1_size -1), Shift_left);
end Structural;

Library IEEE;
use IEEE.Std_Logic_unsigned.all;      -- Needed for + operator in Incr_R2
use IEEE.Std_Logic_1164.all;

entity Counter is
  generic (R2_size: integer := 4);
  port (R2: buffer Std_Logic_vector (R2_size -1 downto 0);
        Load_regs, Incr_R2: in Std_Logic; clock, reset_b: in Std_Logic);
end Counter;

architecture Behavioral of Counter is
begin
  process (clock, reset_b) begin
    if reset_b = '0' then for k in 0 to R2_size -1 loop R2(k) <= '0';
    end loop;
    elsif clock'event and clock = '1' then
      if Load_regs = '1' then for k in 0 to R2_size -1 loop
        R2(k) <= '1';
      end loop;
      elsif Incr_R2 = '1' then R2 <= R2 + 1;
      end if;
    end if;
  end process;
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Shft_Reg is
  generic (R1_size: integer := 8);
  port (R1: buffer Std_Logic_vector (R1_size-1 downto 0);
        data: in Std_Logic_vector (R1_size-1 downto 0);
        SI_0, Shift_left, Load_regs, clock, reset_b: in Std_Logic);
end Shft_Reg;

architecture Behavioral of Shft_Reg is
begin
  process (clock, reset_b) begin
    if (reset_b = '0') then for k in 0 to R1_size-1 loop

```

```

        R1(k) <= '0';
    end loop;
    elsif clock'event and clock = '1' then
        if (Load_regs = '1') then R1 <= data;
    elsif (Shift_left = '1') then R1 <= R1(R1_size -2 downto 0) & SI_0;
    end if;
    end if;
end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity D_flip_flop_AR is
    port (Q: out Std_Logic; D, CLK, RST: in Std_Logic);
end D_flip_flop_AR;

```

```

architecture Behavioral of D_flip_flop_AR is
begin
    process (CLK, RST) begin
        if (RST = '0') then Q <= '0';
        elsif CLK'event and CLK = '1' then Q <= D;
        end if;
    end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
-- Behavioral description of 4-to-1 line multiplexer
entity mux_4x1_beh is
    port (m_out: out Std_Logic;
        in_0, in_1, in_2, in_3: in Std_Logic;
        sel: in Std_Logic_vector (1 downto 0));
end mux_4x1_beh;

```

```

architecture Behavioral of mux_4x1_beh is
begin
    process (in_0, in_1, in_2, in_3, sel) begin
        case (sel) is
            when "00" => m_out <= in_0;
            when "01" => m_out <= in_1;

            when "10" => m_out <= in_2;
            when "11" => m_out <= in_3;
            when others => m_out <= in_0;
        end case;
    end process;
end Behavioral;

```

-- Dataflow description of 2-to-4-line decoder
-- See Fig. 4.19. Note: The figure uses symbol E, but the
-- VHDL model uses enable to clearly indicate functionality.

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
entity decoder_2x4_df is
    port (D: out Std_Logic_vector (0 to 3); A, B, enable: in Std_Logic);
end decoder_2x4_df;

```

```

architecture Behavioral of decoder_2x4_df is
begin
    D(0) <= not ((not A) and (not B) and (not enable));

```



```

    D(1) <= not ((not A) and B and (not enable));
    D(2) <= not (A and (not B) and (not enable));
    D(3) <= not (A and B and (not enable));
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity and2_gate is
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavioral of and2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;

-- Testbench
Library IEEE;
use IEEE.Std_Logic_1164.all;
entity t_Count_Ones is
    generic (R1_size: integer := 8; R2_size: integer := 4);
end t_Count_ones;

architecture Behavioral of t_Count_ones is
    signal t_R2: Std_Logic_vector (R2_size -1 downto 0);
    signal t_count: integer;
    signal t_Ready: Std_Logic;
    signal t_data : Std_Logic_vector (R1_size -1 downto 0);
    signal t_Start, t_clk, t_reset_b: Std_Logic;
    component Prob_8_35_Count_Ones_STR_STR_vhdl
        generic (R1_size: integer := 8; R2_size: integer := 4);
        port (count: out Std_Logic_vector (R2_size-1 downto 0);
              Ready: out Std_Logic;
              data: in Std_Logic_vector (R1_size-1 downto 0);
              Start, clock, reset_b: in Std_Logic);
    end component;
begin
    UUT: Prob_8_35_Count_Ones_STR_STR_vhdl
        generic map (R1_size => 8, R2_size => 4)
        port map (t_count, t_Ready, t_data, t_Start, t_clk, t_reset_b);

    process -- clock for testbench
    begin
        t_clk <= '0';
        wait for 5 ns;
        t_clk <= '1';
        wait for 5 ns;
    end process;

    t_Start <= '0';
    t_reset_b <= '1' after 1 ns;
    t_reset_b <= '0' after 3 ns;
    t_reset_b <= '1' after 4 ns;
    t_data <= "11111111" after 4 ns;
    t_Start <= '1' after 4 ns;
    t_Start <= '0' after 300 ns;
    t_data <= "00001111" after 310 ns;
    t_Start <= '1' after 310 ns;
    t_Start <= '0' after 320 ns;
    t_data <= "11110000" after 610 ns;
    t_Start <= '1' after 610 ns;
    t_Start <= '0' after 620 ns;

```

```

t_data <= "00000000" after 910 ns;
t_Start <= '1' after 910 ns;
t_Start <= '0' after 920 ns;
t_data <= "10101010" after 1210 ns;
t_Start <= '1' after 1210 ns;
t_Start <= '0' after 1220 ns;
t_data <= "00001010" after 1510 ns;
t_Start <= '1' after 1510 ns;
t_Start <= '0' after 1520 ns;
t_data <= "10100000" after 1810 ns;
t_Start <= '1' after 1810 ns;
t_Start <= '0' after 1820 ns;
t_data <= "01010101" after 2110 ns;
t_Start <= '1' after 2110 ns;
t_Start <= '0' after 2120 ns;
t_data <= "00000101" after 2410 ns;
t_Start <= '1' after 2410 ns;
t_Start <= '0' after 2420 ns;
t_data <= "0101" after 2710 ns;
t_Start <= '1' after 2710ns;
t_Start <= '0' after 2720 ns;
t_data <= "10100101" after 3010 ns;
t_Start <= '1' after 3010 ns;
t_Start <= '0' after 3020 ns;
t_data <= "01011010" after 3310 ns;
t_Start <= '1' after 3310 ns;
t_Start <= '0' after 3320 ns;
end Behavioral;

```

8.36 Note: See Prob. for a behavioral model of the datapath unit, Prob. 8.36d for a one-hot control unit.

- (a) T_0, T_1, T_2, T_3 be asserted when the state is in S_idle, S_1, S_2 , and S_3 , respectively. Let D_0, D_1, D_2 , and D_3 denote the inputs to the one-hot flip-flops.

$$D_0 = T_0 \text{ Start}' + T_1 \text{ Zero}$$

$$D_1 = T_0 \text{ Start} + T_3 E$$

$$D_2 = T_1 \text{ Zero}' + T_3 E'$$

$$D_3 = T_2$$

- (b) Gate-level one-hot controller

Verilog

```

module Controller_Gates_1Hot
(
  output    Ready,
  output    Load_regs, Incr_R2, Shift_left,
  input     Start, Zero, E, clock, reset_b
);
  wire w1, w2, w3, w4, w5, w6;
  wire T0, T1, T2, T3;
  wire set;
  assign Ready = T0;
  assign Incr_R2 = T1;
  assign Shift_left = T2;
  and (Load_regs, T0, Start);
  not (set, reset_b);
  DFF_S M0 (T0, D0, clock, set);    // Note: reset action must initialize S_idle = 4'b0001
  DFF M1 (T1, D1, clock, reset_b);
  DFF M2 (T2, D2, clock, reset_b);
  DFF M3 (T3, D3, clock, reset_b);

  not (Start_b, Start);
  and (w1, T0, Start_b);
  and (w2, T1, Zero);
  or (D0, w1, w2);

```

```

and (w3, T0, Start);
and (w4, T3, E);
or (D1, w3, w4);

not (Zero_b, Zero);
not (E_b, E);
and (w5, T1, Zero_b);
and (w6, T3, E_b);
or (D2, w5, w6);

buf (D3, T2);
endmodule

```

```

module DFF (output reg Q, input D, clock, reset_b);
always @ (posedge clock, negedge reset_b)
  if (reset_b == 0) Q <= 0;
  else Q <= D;
endmodule

module DFF_S (output reg Q, input D, clock, set);
always @ (posedge clock, posedge set)
  if (set == 1) Q <= 1;
  else Q <= D;
endmodule

```

VHDL

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Prob_8_36b_Controller_Gates_1Hot_vhdl is
  port (Ready: out Std_Logic; Load_regs, Incr_R2, Shft_left: out Std_Logic;
        Start, Zero, E, clock, reset_b: in Std_Logic);
end Prob_8_36b_Controller_Gates_1Hot_vhdl;

architecture Structural of Prob_8_36b_Controller_Gates_1Hot_vhdl is
  signal w1, w2, w3, w4, w5, w6: Std_Logic;
  signal D0, D1, D2, D3, T0, T1, T2, T3: Std_Logic;
  signal Zero_b, E_b, Start_b, set: Std_Logic;

  component D_FF_S
    port (Q: out Std_Logic; D, clock, set: in Std_Logic);
  end component;

  component D_FF
    port (Q: out Std_Logic; D, clock, reset_b: in Std_Logic);
  end component;

  component not_gate
    port (y: out Std_Logic; xin: in Std_Logic);
  end component;

  component and2_gate
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
  end component;

  component or2_gate
    port (y: out Std_Logic; xin1, xin2: in Std_Logic);
  end component;

  component buf_gate
    port (y: out Std_Logic; xin: in Std_Logic);
  end component;

```

```

begin
  Ready <= T0;
  Incr_R2 <= T1;
  Shft_left <= T2;

  M0: D_FF_S port map (T0, D0, clock, set); -- Note: reset action must initialize S_idle = 4'b0001
  D_FF_1: D_FF port map (T1, D1, clock, reset_b);
  D_FF_2: D_FF port map (T2, D2, clock, reset_b);
  D_FF_3: D_FF port map (T3, D3, clock, reset_b);
  G_1: not_gate port map (Start_b, Start);
  G_2: and2_gate port map (w1, T0, Start_b);
  G_3: and2_gate port map (w2, T1, Zero);
  G_4: or2_gate port map (D0, w1, w2);
  G_5: and2_gate port map (w3, T0, Start);
  G_6: and2_gate port map (w4, T3, E);
  G_7: or2_gate port map (D1, w3, w4);
  G_8: not_gate port map (Zero_b, Zero);
  G_9: not_gate port map (E_b, E);
  G_10: and2_gate port map (w5, T1, Zero_b);
  G_11: and2_gate port map (w6, T3, E_b);
  G_12: or2_gate port map (D2, w5, w6);
  G_13: buf_gate port map (D3, T2);
  G_14: and2_gate port map (Load_regs, T0, Start);
  G_15: not_gate port map (set, reset_b);
end Structural;

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

entity or2_gate is
  port (y: out bit; xin1, xin2: in bit);
end or2_gate;

architecture Behavioral of or2_gate is
begin
  y <= xin1 or xin2;
end Behavioral;

entity not_gate is
  port (y: out bit; xin: in bit);
end not_gate;

architecture Behavioral of not_gate is
begin
  y <= not (xin);
end Behavioral;

entity buf_gate is
  port (y: out bit; xin: in bit);
end buf_gate;

architecture Behavioral of buf_gate is
begin
  y <= xin;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity D_FF is
  port (Q: out Std_Logic; D, clock, reset_b: in Std_Logic);
end D_FF;

architecture Behavioral of D_FF is
begin
  process (clock, reset_b) begin
    if (reset_b = '0') then Q <= '0'; elsif clock'event and clock = '1' then Q <= D;
    end if;
  end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity D_FF_S is
  port (Q: out Std_Logic; D, clock, set: in Std_Logic);
end D_FF_S;

```

```

architecture Behavioral of D_FF_S is
begin
  process (clock, set) begin
    if (set = '1') then Q <= '1';
    elsif clock'event and clock = '1' then Q <= D;
    end if;
  end process;
end Behavioral;

```

(c)

```

// Test plan for Control Unit
// Verify that state enters S_idle with reset_b asserted.
// With reset_b de-asserted, verify that state enters S_1 and asserts Load_Regs when
// Start is asserted.
// Verify that Incr_R2 is asserted in S_1.
// Verify that state returns to S_idle from S_1 if Zero is asserted.
// Verify that state goes to S_2 if Zero is not asserted.
// Verify that Shift_left is asserted in S_2.
// Verify that state goes to S_3 from S_2 unconditionally.
// Verify that state returns to S_2 from S_3 if E is not asserted.
// Verify that state goes to S_1 from S_3 if E is asserted.

```

// Test bench for One-Hot Control Unit

```

module t_Control_Unit ();
  wire Ready, Load_regs, Incr_R2, Shift_left;
  reg Start, Zero, E, clock, reset_b;
  wire [3: 0] state = {M0.T3, M0.T2, M0.T1, M0.T0}; // Observe one-hot state bits
  Controller_Gates_1Hot M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);

  initial #250 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin reset_b = 0; #2 reset_b = 1; end
  initial fork
    Zero = 1;
    E = 0;
    Start = 0;
    #20 Start = 1; // Cycle from S_idle to S_1
    #80 Start = 0;
  end fork

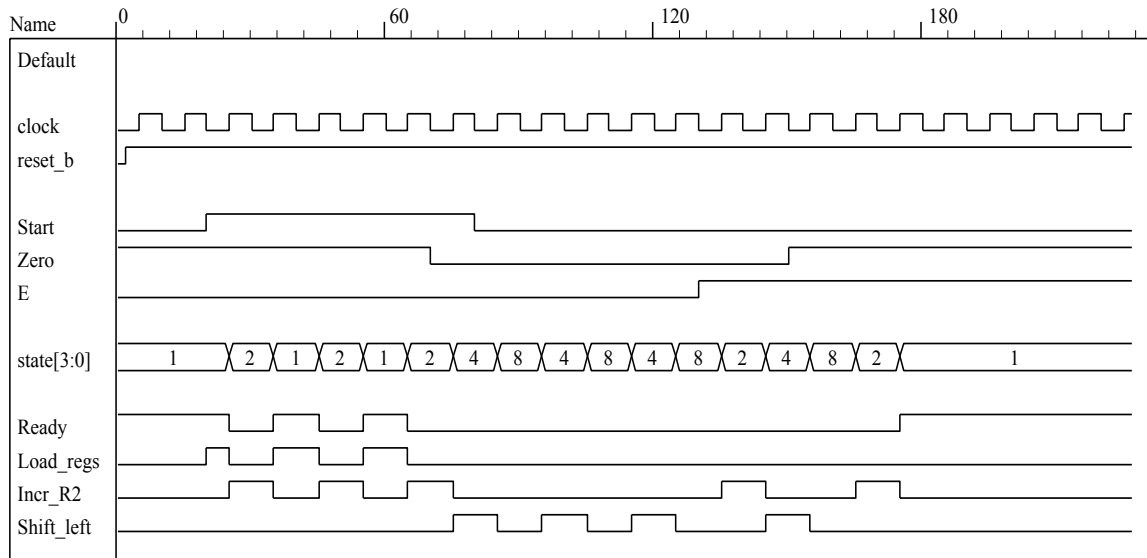
```

```

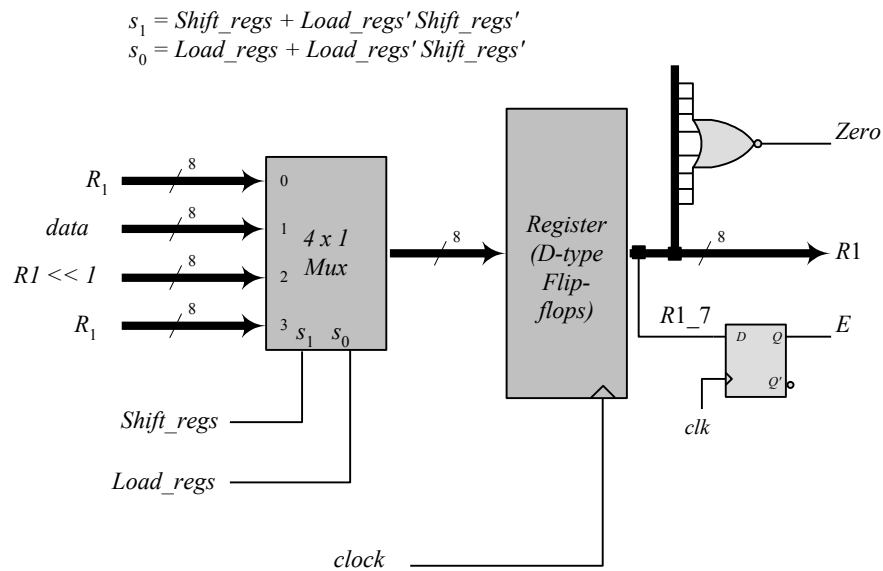
#70 Zero = 0;    // S_idle to S_1 to S_2 to S_3 and cycle to S_2.
#130 E = 1;      // Cycle to S_3 to S_1 to S_2 to S_3
#150 Zero = 1;   // Return to S_idle
join
endmodule

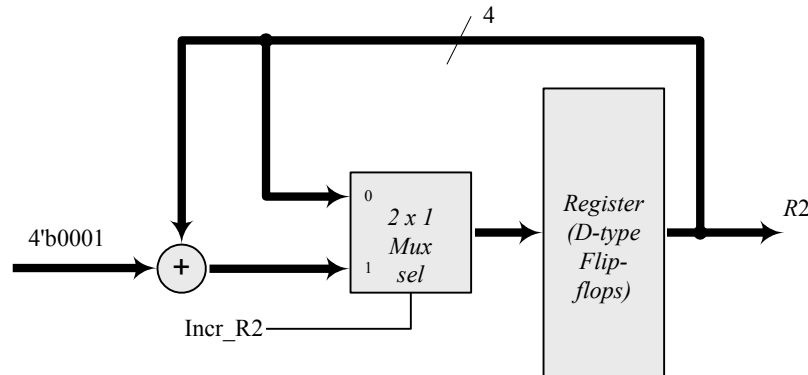
```

Note: simulation results match those for Prob. 8.34(d). See Prob. (c) for annotations.



(d) Datapath unit detail:





Verilog

```
// Datapath unit – structural model
module Datapath_STR
#(parameter dp_width = 8, R2_width = 4)
(
    output [R2_width - 1: 0] count, output E, output Zero, input [dp_width - 1: 0] data,
    input Load_regs, Shift_left, Incr_R2, clock, reset_b);
    supply1 pwr;
    supply0 gnd;
    wire [dp_width - 1: 0] R1_Dbus, R1;
    wire [R2_width - 1: 0] R2_Dbus;
    wire DR1_0, DR1_1, DR1_2, DR1_3, DR1_4, DR1_5, DR1_6, DR1_7;
    wire R1_0, R1_1, R1_2, R1_3, R1_4, R1_5, R1_6, R1_7;
    wire R2_0, R2_1, R2_2, R2_3;
    wire [R2_width - 1: 0] R2 = {R2_3, R2_2, R2_1, R2_0};
    assign count = {R2_3, R2_2, R2_1, R2_0};
    assign R1 = { R1_7, R1_6, R1_5, R1_4, R1_3, R1_2, R1_1, R1_0};
    assign DR1_0 = R1_Dbus[0];
    assign DR1_1 = R1_Dbus[1];
    assign DR1_2 = R1_Dbus[2];
    assign DR1_3 = R1_Dbus[3];
    assign DR1_4 = R1_Dbus[4];
    assign DR1_5 = R1_Dbus[5];
    assign DR1_6 = R1_Dbus[6];
    assign DR1_7 = R1_Dbus[7];

    nor (Zero, R1_0, R1_1, R1_2, R1_3, R1_4, R1_5, R1_6, R1_7);
    DFF D_E (E, R1_7, clock, pwr);

    DFF DF_0 (R1_0, DR1_0, clock, pwr);    // Disable reset
    DFF DF_1 (R1_1, DR1_1, clock, pwr);
    DFF DF_2 (R1_2, DR1_2, clock, pwr);
    DFF DF_3 (R1_3, DR1_3, clock, pwr);
    DFF DF_4 (R1_4, DR1_4, clock, pwr);
    DFF DF_5 (R1_5, DR1_5, clock, pwr);
    DFF DF_6 (R1_6, DR1_6, clock, pwr);
    DFF DF_7 (R1_7, DR1_7, clock, pwr);

    DFF_S DR_0 (R2_0, DR2_0, clock, Load_regs); // Load_regs (set) drives R2 to all ones
    DFF_S DR_1 (R2_1, DR2_1, clock, Load_regs);
    DFF_S DR_2 (R2_2, DR2_2, clock, Load_regs);
    DFF_S DR_3 (R2_3, DR2_3, clock, Load_regs);

    assign DR2_0 = R2_Dbus[0];
    assign DR2_1 = R2_Dbus[1];
    assign DR2_2 = R2_Dbus[2];
    assign DR2_3 = R2_Dbus[3];
```



```

wire [1: 0] sel = {Shift_left, Load_regs};
wire [dp_width -1: 0] R1_shifted = {R1_6, R1_5, R1_4, R1_3, R1_2, R1_1, R1_0, 1'b0};
wire [R2_width -1: 0] sum = R2 + 4'b0001;

Mux8_4_x_1 M0 (R1_Dbus, R1, data, R1_shifted, R1, sel);
Mux4_2_x_1 M1 (R2_Dbus, R2, sum, Incr_R2);
endmodule

module Mux8_4_x_1 #(parameter dp_width = 8) (output reg [dp_width -1: 0] mux_out,
input [dp_width -1: 0] in0, in1, in2, in3, input [1: 0] sel);
always @ (in0, in1, in2, in3, sel)
case (sel)
2'b00: mux_out = in0;
2'b01: mux_out = in1;
2'b10: mux_out = in2;
2'b11: mux_out = in3;
endcase
endmodule

module Mux4_2_x_1 #(parameter dp_width = 4) (output [dp_width -1: 0] mux_out,
input [dp_width -1: 0] in0, in1, input sel);
assign mux_out = sel ? in1: in0;
endmodule

// Test Plan for Datapath Unit:
// Demonstrate action of Load_regs
//   R1 gets data, R2 gets all ones
// Demonstrate action of Incr_R2
// Demonstrate action of Shift_left and detect E

// Test bench for datapath
module t_Datapath_Unit
#(parameter dp_width = 8, R2_width = 4)
();
wire [R2_width -1: 0] count;
wire E, Zero;
reg [dp_width -1: 0] data;
reg Load_regs, Shift_left, Incr_R2, clock, reset_b;
Datapath_STR M0 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);

initial #250 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin reset_b = 0; #2 reset_b = 1; end
initial fork
data = 8'haa;
Load_regs = 0;
Incr_R2 = 0;
Shift_left = 0;
#10 Load_regs = 1;
#20 Load_regs = 0;
#50 Incr_R2 = 1;
#120 Incr_R2 = 0;
#90 Shift_left = 1;
#200 Shift_left = 0;
join
endmodule

// Integrated system
module Count_Ones_Gates_1_Hot_STR
#(parameter dp_width = 8, R2_width = 4)
(

```

```

    output [R2_width-1: 0] count,
    input [dp_width-1: 0] data,
    input      Start, clock, reset_b
);
    wire Load_regs, Incr_R2, Shift_left, Zero, E;
    Controller_Gates_1Hot M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);
    Datapath_STR M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);
endmodule

// Test plan for integrated system
// Test for data values of 8'haa, 8'h00, 8'hff.

// Test bench for integrated system

module t_count_Ones_Gates_1_Hot_STR ();
    parameter dp_width = 8, R2_width = 4;
    wire [R2_width-1: 0] count;
    reg [dp_width-1: 0] data;
    reg Start, clock, reset_b;
    wire [3: 0] state = {M0.M0.T3, M0.M0.T2, M0.M0.T1, M0.M0.T0};

    Count_Ones_Gates_1_Hot_STR M0 (count, data, Start, clock, reset_b);
    initial #700 $finish;
    initial begin clock = 0; forever #5 clock = ~clock; end
    initial begin reset_b = 0; #2 reset_b = 1; end
    initial fork
        data = 8'haa;    // Expect count = 4
        Start = 0;
        #20 Start = 1;
        #30 Start = 0;
        #40 data = 8'b00;    // Expect count = 0
        #250 Start = 1;
        #260 Start = 0;
        #280 data = 8'hff;
        #280 Start = 1;
        #290 Start = 0;
    join
endmodule

```

Note: The simulation results show tests of the operations of the datapath independent of the control unit, so count does not represent the number of ones in the data.

[illegible]

All rights reserved.

```

entity Prob_8_36d_Datapath_STR_vhdl is
generic (R1_size: integer := 8; R2_size: integer := 4);

port (count: out Std_Logic_Vector (R2_size -1 downto 0);
      E, Zero: out Std_Logic;
      data: in Std_Logic_Vector (R1_size -1 downto 0);
      Load_regs, Shft_left, Incr_R2, clock: in Std_Logic);
end Prob_8_36d_Datapath_STR_vhdl;

architecture Structural of Prob_8_36d_Datapath_STR_vhdl is
  signal R1: Std_Logic_Vector (R1_size -1 downto 0);
  signal R2: Std_Logic_Vector (R2_size -1 downto 0);
  signal PWR: Std_Logic;
  signal GND: Std_Logic;
  signal w1: Std_Logic;
  component Shft_Reg port (R1: out Std_Logic_Vector (R1_size-1 downto 0);
    data: in Std_Logic_Vector (R1_size -1 downto 0);
    SI_0, Shft_left, Load_regs: in Std_Logic;
    clock, reset_b: in Std_Logic);
end component;

  component Counter port (R2: buffer Std_Logic_Vector (R2_size -1 downto 0);
    Load_regs, Incr_R2: in Std_Logic;
    clock, reset_b: in Std_Logic);
end component;

  component D_flip_flop_AR port (Q: out Std_Logic; D: in Std_Logic;
    CLK, RST_b : in Std_Logic);
end component;

  component and2_gate port (y: out Std_Logic; xin1, xin2 : in Std_Logic);
end component;

begin -- concurrent statements

  PWR <= '1';
  GND <= '0';

  process (R1) begin      -- Check for empty R1
    Zero <= '1';
    for k in 0 to R1_size -1 loop
      if R1(k) = '1' then Zero <= '0'; end if;
    end loop;
  end process;

  -- Instantiate components

  M1: Shft_Reg port map (R1, data, GND, Shft_left, Load_regs, clock, PWR);

  M2: Counter port map (R2 => count, Load_regs => Load_regs, Incr_R2 => Incr_R2,
    clock => clock, reset_b => PWR);

  M3: D_flip_flop_AR port map (Q => E, D => w1, CLK => clock, RST_b => PWR);

  G0: and2_gate port map (y => w1, xin1 => R1(R1_size-1), xin2 => Shft_left);

end Structural;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Shft_Reg is

```

```

generic (R1_size: integer := 8);
port (R1: buffer Std_Logic_Vector (R1_size-1 downto 0);
      data: in Std_Logic_Vector (R1_size-1 downto 0);
      SI_0, Shift_left, Load_regs: in Std_Logic;
      clock, reset_b: in Std_Logic);
end Shft_Reg;

architecture Behavioral of Shft_Reg is
  signal PWR: Std_Logic;
  signal GND: Std_Logic;
begin
  PWR <= '1';
  GND <= '0';

  process (clock, reset_b)
  begin
    if reset_b = '0' then for k in 0 to R1_size-1 loop R1(k) <= '0'; end loop;
    elsif clock'event and clock = '1' then
      if Load_regs = '1' then R1 <= data;
      elsif Shift_left = '1' then R1 <= R1(R1_size-2 downto 0) & SI_0;
      end if;
    end if;
  end process;
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_Unsigned.all;
use IEEE.Std_Logic_1164.all;

entity Counter is
  generic (R2_size: integer:= 4);
  port (R2: buffer Std_Logic_Vector (R2_size-1 downto 0);
      Load_regs, Incr_R2: in Std_Logic;
      clock, reset_b: in Std_Logic);
end Counter;

architecture Behavioral of Counter is
begin
  process (clock, reset_b) begin
    if reset_b = '0' then for k in R2_size-1 downto 0 loop R2(k) <= '0'; end loop;
    elsif clock'event and clock = '1' then -- Fill with 1s
      if Load_regs = '1' then for k in 0 to R2_size-1 loop R2(k) <= '1'; end loop;
      elsif Incr_R2 = '1' then R2 <= R2 + '1';
      end if;
    end if;
  end process;
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity D_flip_flop_AR is
  generic (R1_size: integer := 8);
  port (Q: out Std_Logic; D, CLK, RST_b: in Std_Logic);
end D_flip_flop_AR;

architecture Behavioral of D_flip_flop_AR is
begin
  process (CLK, RST_b) begin
    if RST_b = '0' then Q <= '0';
    elsif CLK'event and CLK = '1' then Q <= D;
    end if;
  end process;
end Behavioral;

```

```

end process;
end Behavioral;

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;

architecture Behavioral of and2_gate is
begin
  y <= xin1 and xin2;
end Behavioral;

```

(e)VHDL

```

-- Integrated system with one-hot controller
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_8_34e_Count_Ones_BEH_1Hot_vhdl is
  constant dp_width: integer := 8;
  constant R2_width: integer := 4;
  port (count: out Std_Logic_vector (R2_width-1 downto 0);
        data: in (dp_width-1 downto 0);
        Start, clock, reset_b: in Std_Logic);
end Prob_8_34e_Count_Ones_BEH_1Hot_vhdl;

architecture Structural of Prob_8_34e_Count_Ones_BEH_1Hot_vhdl is
  component Controller_BEH_1Hot port (Ready, Load_regs, Incr_R2, Shift_left, Start: in Std_Logic;
    Zero, E, clock, reset_b: in Std_Logic); end component;

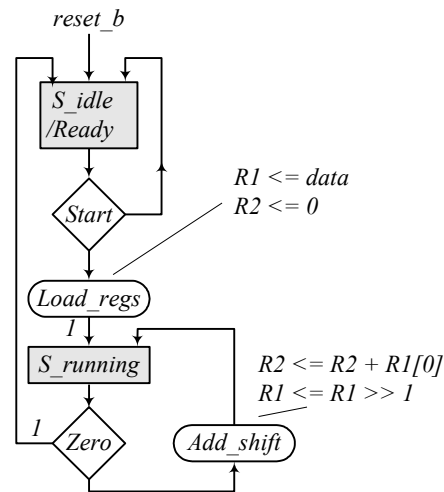
  component Datapath_BEH port (count: out Std_Logic_vector (dp_width-1 downto 0);
    E, Zero: out Std_Logic; data: in Std_Logic_vector (dp_width-1 downto 0);
    Load_regs, Shift_left,
    Incr_R2, clock, reset_b: in Std_Logic);
  signal Load_regs, Incr_R2, Shift_left, Zero, E: Std_Logic;
begin
  M0: Controller_BEH_1Hot port map (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);

  M1: Datapath_BEH port map (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);

end Structural;

```

8.37 (a) ASMD chart:



(b) Verilog - RTL model - Datapath:

```

module Datapath_Unit_2_Beh #(parameter dp_width = 8, R2_width = 4)
(
  output [R2_width -1: 0]    count,
  output                    Zero,
  input [dp_width -1: 0]     data,
  input                    Load_regs, Add_shift, clock, reset_b
);
  reg [dp_width -1: 0]      R1;
  reg [R2_width -1: 0]     R2;
  assign count = R2;
  assign Zero = ~|R1;
  always @ (posedge clock, negedge reset_b)
  begin
    if (reset_b == 0) begin R1 <= 0; R2 <= 0; end else begin
      if (Load_regs) begin R1 <= data; R2 <= 0; end
      if (Add_shift) begin R1 <= R1 >> 1; R2 <= R2 + R1[0]; end // concurrent operations
    end
  end
endmodule

// Test plan for datapath unit
// Verify active-low reset action
// Test for action of Add_shift
// Test for action of Load_regs

module t_Datapath_Unit_2_Beh();
  parameter R1_size = 8, R2_size = 4;
  wire [R2_size -1: 0] count;
  wire Zero;
  reg [R1_size -1: 0] data;
  reg Load_regs, Add_shift, clock, reset_b;

  Datapath_Unit_2_Beh M0 (count, Zero, data, Load_regs, Add_shift, clock, reset_b);

  initial #1000 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork

```

```

#1 reset_b = 1;
#3 reset_b = 0;
#4 reset_b = 1;
join
initial fork
  data = 8'haa;
  Load_regs = 0;
  Add_shift = 0;
#10 Load_regs = 1;
#20 Load_regs = 0;
#50 Add_shift = 1;
#150 Add_shift = 0;
join
endmodule

```

VHDL – RTL Model – Datapath

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
entity Prob_8_37b_Datapath_Unit_2_Beh_vhdl is
  generic (dp_width: integer := 8; R2_width: integer := 4);
  port (count: out Std_Logic_vector (R2_width-1 downto 0);
        Zero: out Std_Logic;
        data: in Std_Logic_vector (dp_width-1 downto 0);
        Load_regs, Add_shift, clock, reset_b: in Std_Logic);
end Prob_8_37b_Datapath_Unit_2_Beh_vhdl;

architecture Behavioral of Prob_8_37b_Datapath_Unit_2_Beh_vhdl is
  R1: Std_Logic_vector (dp_width-1 downto 0);
  R2: Std_Logic_vector (R2_width-1 downto 0);
begin
  count <= R2;
  process (R1)
    Zero <= '1';
    for k in 0 to dp_width-1 loop
      if R1(k) = '1' then Zero <= '0'; end if;
    end loop;

    process (clock, reset_b) begin
      if (reset_b = '0') then
        for k in 0 to dp_width-1 loop R1(k) <= '0'; end loop;
        for k in 0 to R2_width-1 loop R2(k) <= '0'; end loop;
        if (Load_regs) begin R1 <= data; R2 <= 0; end
        elsif clock'event and clock = '1' then
          if (Add_shift = '1') then
            R1 <= {0} & R1(dp_width-1 downto 1);
            R2 <= R2 + R1(0); // concurrent operations
          end if;
        end if;
      end if;
    end Behavioral;

    -- VHDL Test plan for datapath unit
    -- Verify active-low reset action
    -- Test for action of Add_shift
    -- Test for action of Load_regs

  entity t_Datapath_Unit_2_Beh_vhdl is
    generic (R1_size: integer := 8; R2_size: integer := 4);
    signal t_count: Std_Logic_vector (R2_size-1 downto 0);
    signal t_Zero: Std_Logic;

```



```

signal t_data: Std_Logic_vector (R1_size -1 downto 0);
signal t_Load_regs, t_Add_shift, t_clock, t_reset_b: Std_Logic;

    UUT: Prob_8_37b_Datapath_Unit_2_Beh_vhdl
        port map (t_count, t_Zero, t_data, t_Load_regs, t_Add_shift, t_clock, t_reset_b);

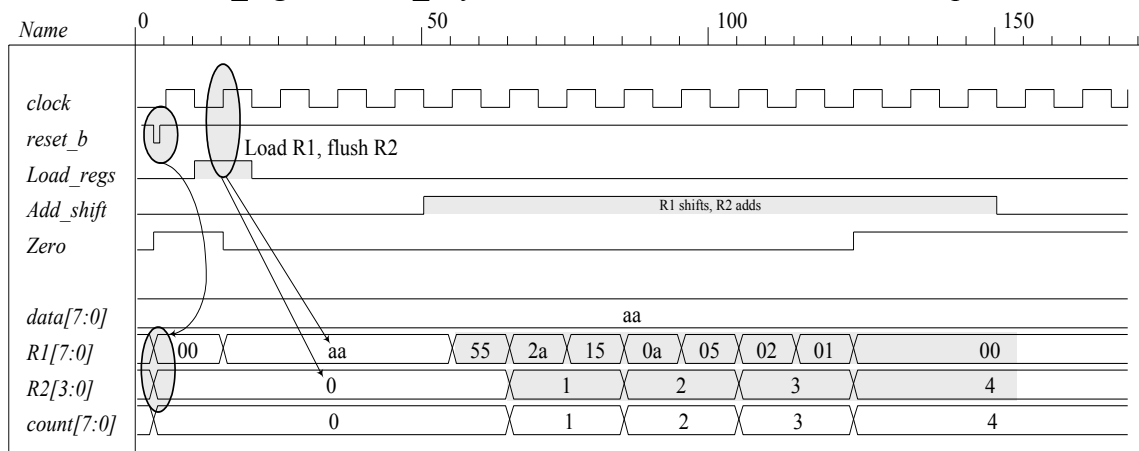
process – clock for testbench
begin
    t_clock <= '0';
    wait for 5 ns;
    t_clock <= '1';
    wait for 5 ns;
end process;

t_data <= "10101010";
t_Load_regs <= 0;
t_Add_shift <= 0;
t_Load_regs <= '1' after 10 ns;
t_Load_regs <= '0' after 20 ns;
t_Add_shift <= '1' after 50 ns;
t_Add_shift <= '0' after 150 ns;

wait for 1 ns; t_reset_b <= '1';
    t_reset_b <= '0' after 3 ns;
    t_reset_b = '1' after 7 ns;
end Behavioral;

```

Note that the operations of the datapath unit are tested independent of the controller, so the actions of *Load_regs* and *add_shift* and the value of *count* do not correspond to *data*.



Verilog – RTL Model – Control Unit

```

module Controller_2_Beh (
    output    Ready,
    output reg Load_regs,
              Add_shift,
    input     Start, Zero, clock, reset_b
);
    parameter S_idle = 0, S_running = 1;
    reg state, next_state;
    assign    Ready = (state == S_idle);

```

always @ (posedge clock, negedge reset_b)

```

    if (reset_b == 0) state <= S_idle;
    else state <= next_state;

always @ (state, Start, Zero) begin
    next_state = S_idle;
    Load_regs = 0;
    Add_shift = 0;

    case (state)
        S_idle:      if (Start) begin Load_regs = 1; next_state = S_running; end
        S_running:   if (Zero) next_state = S_idle;
                    else begin Add_shift = 1; next_state = S_running; end
    endcase
end
endmodule

```

VHDL – RTL Model – Control Unit

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Prob_8_37b_Controller_2_Beh_vhdl is
    port (Ready: out Std_Logic;
          Load_regs, Add_shift: out Std_Logic;
          Start, Zero, clock, reset_b: in Std_Logic);
end Prob_8_37b_Controller_2_Beh_vhdl;

architecture Behavioral of Prob_8_37b_Controller_2_Beh_vhdl is
    type State_type is (S_idle, S_running);
    signal state, next_state: State_type;
begin
    Ready <= '1' when state = S_idle else '0';

    process (clock, reset_b) begin -- State Transitions
        if (reset_b = '0') then state <= S_idle;
        elsif clock'event and clock = '1' then state <= next_state;
        end if;
    end process;

    process (state, Start, Zero) begin
        next_state <= S_idle;
        Load_regs <= '0';
        Add_shift <= '0';

        case (state) is
            when S_idle => if (Start = '1') then
                            Load_regs <= '1';
                            next_state <= S_running;
                        end if;

            when S_running => if (Zero = '1') then
                               next_state <= S_idle;
                           else
                               Add_shift <= '1';
                               next_state <= S_running;
                           end if;

        end case
    end process;
end Behavioral;

```

Testbench (VHDL):

```
Library IEEE;
```

```

use IEEE.Std_Logic_1164.all;

entity Prob_8_37b_t_Controller_2_Beh_vhdl is
end Prob_8_37b_t_controller_2_Beh_vhdl;

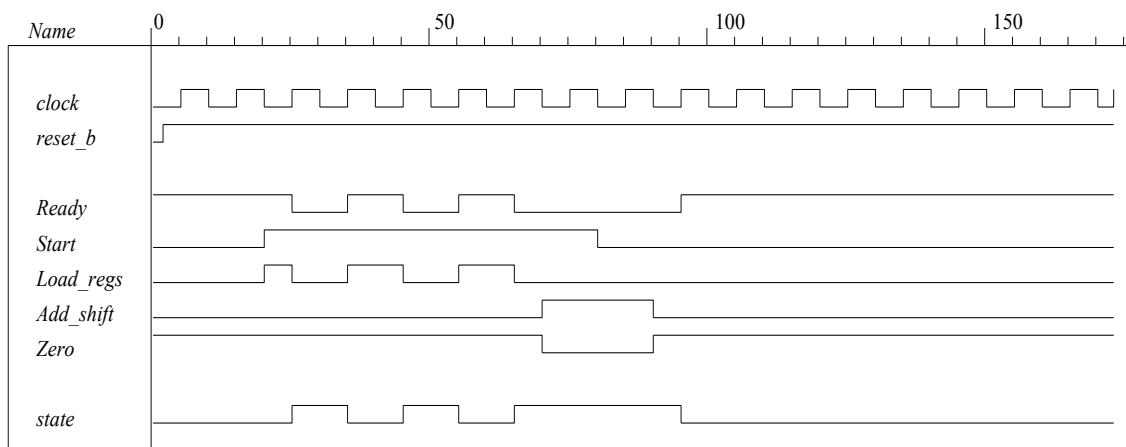
architecture Behavioral of Prob_8_37b_t_Controller_2_Beh_vhdl is
    signal t_Ready, t_Load_regs, t_Add_shift: Std_Logic;
    signal t_Start, t_Zero, t_clock, t_reset_b: Std_Logic;
    component Prob_8_37b_Controller_2_Beh_vhdl port (Ready: out Std_Logic;
        Load_regs, Add_shift: out Std_Logic;
        Start, Zero, clock, reset_b: in Std_Logic);
    end component;
begin
    M0: Prob_8_37b_Controller_2_Beh_vhdl port map (t_Ready, t_Load_regs, t_Add_shift,
        t_Start, t_Zero, t_clock, t_reset_b);

    process begin
        t_clock <= '0';
        wait for 5 ns;
        t_clock <= '1';
        wait for 5 ns;
    end process;

    t_reset_b <= '0';
    t_reset_b <= '1' after 2 ns;
    t_zero <= '1';
    t_Start <= '0';
    t_Start <= '1' after 20 ns; -- Cycle from S_idle to S_1
    t_Start <= '0' after 80 ns;
    t_Zero <= '0' after 70 ns; -- S_idle to S_1 to S_idle
    t_Zero <= '1' after 90 ns; -- Return to S_idle
end Behavioral;

```

Note: The state transitions and outputs of the controller match the ASMD chart.



Verilog - Integrated Design:

```

module Count_of_Ones_2_Beh #(parameter dp_width = 8, R2_width = 4)
(
  output [R2_width -1: 0] count,
  output Ready,
  input [dp_width -1: 0] data,
  input Start, clock, reset_b
);
  wire Load_regs, Add_shift, Zero;

  Controller_2_Beh M0 (Ready, Load_regs, Add_shift, Start, Zero, clock, reset_b);
  Datapath_Unit_2_Beh M1 (count, Zero, data, Load_regs, Add_shift, clock, reset_b);
endmodule

// Test plan for integrated system
// Test for data values of 8'haa, 8'h00, 8'hff.

// Test bench for integrated system

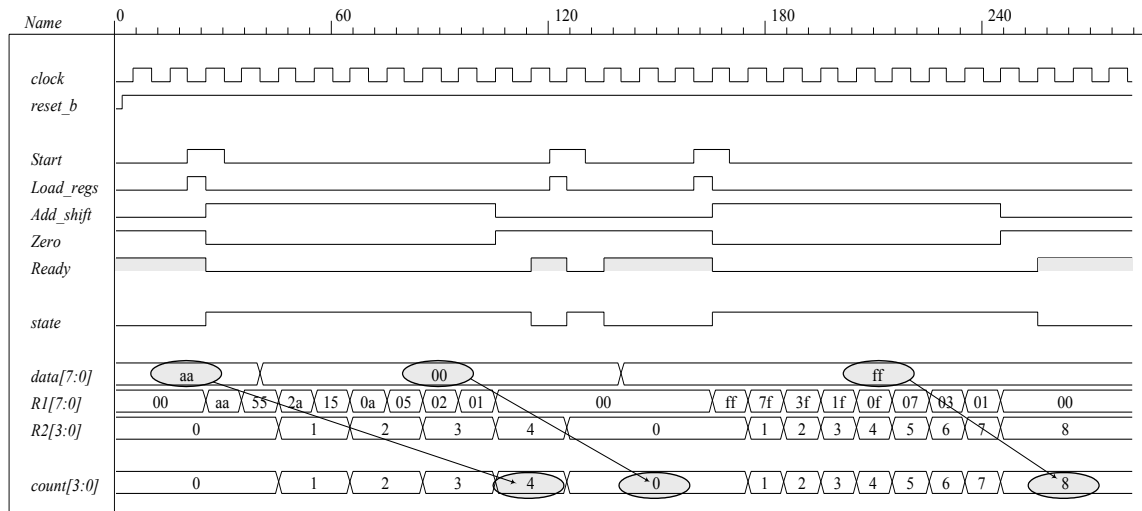
module t_Count_Ones_2_Beh ();
  parameter dp_width = 8, R2_width = 4;
  wire [R2_width -1: 0] count;
  reg [dp_width -1: 0] data;
  reg Start, clock, reset_b;

  Count_of_Ones_2_Beh M0 (count, Ready, data, Start, clock, reset_b);

  initial #700 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin reset_b = 0; #2 reset_b = 1; end
  initial fork
    data = 8'haa;      // Expect count = 4
    Start = 0;
    #20 Start = 1;
    #30 Start = 0;
    #40 data = 8'b00; // Expect count = 0
    #120 Start = 1;
    #130 Start = 0;
    #140 data = 8'hff;
    #160 Start = 1;
    #170 Start = 0;

  join
endmodule

```



VHDL – Integrated Design

The control unit and the datapath are instantiated and connected in Count_of_Ones_2_Beh. The individual units have been verified; it remains to verify that the integration of the units is correct.

entity Count_of_Ones_2_Beh **is**

```

generic ( dp_width: integer := 8; R2_width: integer := 4);
port (count: buffer Std_Logic_vector (R2_width -1 downto 0);
      Ready: out Std_Logic;
      data: in Std_Logic_vector (dp_width -1 downto 0);
      Start, clock, reset_b: in Std_Logic);

```

end Count_of_Ones_2_Beh;

architecture Structural **of** Count_of_Ones_2_Beh **is**

```

signal Load_regs, Add_shift, Zero: Std_Logic;
component Prob_8_37b_Controller_2_Beh_vhdl port (Ready: out Std_Logic;
      Load_regs, Add_shift: out Std_Logic;
      Start, Zero, clock, reset_b: in Std_Logic);
end component;

```

```

component Prob_8_37b_Datapath_Unit_2_Beh_vhdl port (count: out Std_Logic_vector
      (R2_width -1 downto 0);
      Zero: out Std_Logic;
      data: in Std_Logic_vector (dp_width -1 downto 0);
      Load_regs, Add_shift, clock, reset_b: in Std_Logic);
end component;

```

begin

```

UUT_1: Prob_8_37b_Controller_2_Beh_vhdl
      port map (Ready, Load_regs, Add_shift, Start, Zero, clock, reset_b);

```

```

UUT_2: Prob_8_37b_Datapath_Unit_2_Beh_vhdl
      port map (count, Zero, data, Load_regs, Add_shift, clock, reset_b);

```

end Structural;

-- Test plan for integrated system

-- Test for data values of 8'haa, 8'h00, 8'hff.

entity t_Count_Ones_2_Beh **is**

```

generic (dp_width: integer := 8; R2_width: integer := 4);
end t_Count_Ones_2_Beh

```

architecture Behavioral **of** t_Count_Ones_2_Beh **is**

```

signal t_count: Std_Logic_vector (R2_width -1 downto 0);
signal t_data: Std_Logic_vector (dp_width -1 downto 0);
signal t_Start, t_clock, t_reset_b: Std_Logic;
component Count_of_Ones_2_Beh port (count: buffer Std_Logic_vector
    (R2_width -1 downto 0);
    Ready: out Std_Logic;
    data: in Std_Logic_vector (dp_width -1 downto 0);
    Start, clock, reset_b: in Std_Logic);
end component;
begin
    UUT: Count_of_Ones_2_Beh
        port map (t_count, t_Ready, t_data, t_Start, t_clock, t_reset_b);
    process -- clock for testbench
    begin
        t_clk <= '0';
        wait for 5 ns;
        t_clk <= '1';
        wait for 5 ns;
    end process;

    reset_b <= '0' after 2 ns;
    reset_b <= '1' after 4 ns;

    data <= "10101010";    -- Expect count = 4
    Start <= '0';
    Start <= '1' after 20 ns;
    Start <= '0' after 30 ns;
    data <= "00000000" after 40 ns; // Expect count = 0
    Start <= '1' after 120 ns;
    Start <= '0' after 130 ns;
    data <= "11111111" after 140 ns;
    Start <= '1' after 160 ns;
    Start <= '0' after 170 ns;
end Behavioral;

```

- (c) T_0, T_1 are to be asserted when the state is in $S_idle, S_running$, respectively. Let $D0, D1$ denote the inputs to the one-hot flip-flops.

$$D_0 = T_0 Start' + T_1 Zero$$

$$D_1 = T_0 Start + T_1 E'$$

- (d) Verilog - Gate-level one-hot controller

```

module Controller_2_Gates_1Hot
(
    output    Ready, Load_regs, Add_shift,
    input     Start, Zero, clock, reset_b
);
    wire w1, w2, w3, w4;
    wire T0, T1;
    wire set;
    assign Ready = T0;
    assign Add_shift = T1;
    and (Load_regs, T0, Start);
    not (set, reset_b);
    DFF_S M0 (T0, D0, clock, set);    // Note: reset action must initialize S_idle = 2'b01
    DFF M1 (T1, D1, clock, reset_b);

```

```

not (Start_b, Start);
not (Zero_b, Zero);
and (w1, T0, Start_b);
and (w2, T1, Zero);
or (D0, w1, w2);
and (w3, T0, Start);
and (w4, T1, Zero_b);
or (D1, w3, w4);
endmodule

```

```

module DFF (output reg Q, input D, clock, reset_b);
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) Q <= 0;
    else Q <= D;
endmodule

module DFF_S (output reg Q, input D, clock, set);
  always @ (posedge clock, posedge set)
    if (set == 1) Q <= 1;
    else Q <= D;
endmodule

```

VHDL - Gate-level one-hot controller

```

Library IEEE;
use IEEE>Std_Logic_1164.all;

entity Prob_8_37d_Controller_2_Gates_1Hot is
  port (Ready, Load_regs, Add_shift: out Std_Logic;
        Start, Zero, clock, reset_b: in Std_Logic);
end Prob_8_37d_Controller_2_Gates_1Hot;

architecture Structural of Prob_8_37d_Controller_2_Gates_1Hot is
  signal w1, w2, w3, w4: Std_Logic;
  signal T0, T1: Std_Logic;
  signal set: Std_Logic;
  signal D0, D1: Std_Logic;
  signal Zero_b, Start_b: Std_Logic;

  component D_FF port (Q: out Std_Logic, D, clock, reset_b: in Std_Logic);
  end component;

  component D_FF_S port (Q: out Std_Logic, D, clock, set: in Std_Logic);
  end component;

begin

  Ready <= T0;
  Add_shift <= T1;
  Load_regs <= T0 and Start;
  set <= not reset_b;

  M0: D_FF_S port map (T0, D0, clock, set); -- Note: reset action must
  M1: D_FF port map (T1, D1, clock, reset_b); -- initialize S_idle = 2'b01

  Start_b <= not Start;
  Zero_b <= not Zero;
  w1 <= T0 and Start_b;
  w2 <= T1 and Zero;
  D0 <= w1 or w2;
  w3 <= T0 and Start;
  w4 <= T1 and Zero_b;
  D1 <= w3 or w4;
end Structural;

```

```

entity D_FF is
  port (Q: out Std_Logic; D, clock, reset_b: in Std_Logic);
end D_FF;
architecture Behavioral of D_FF is
begin
  process (clock, reset_b) begin
    if (reset_b = '0') then Q <= '0';
    elsif clock'event and clock = '1' then Q <= D; end if;
  end process;
end Behavioral;

entity D_FF_S is
  port (Q: out Std_Logic; D, clock, set: in Std_Logic);
end D_FF_S;

architecture Behavioral of D_FF_S is
begin
  process (clock, set) begin
    if (set = '1') then Q <= '1';
    elsif clock'event and clock = '1' then Q <= D; end if;
  end process;
end Behavioral;

-- Test plan for Control Unit
-- Verify that state enters S_idle with reset_b asserted.
-- With reset_b de-asserted, verify that state enters S_running and asserts Load_Regs when
-- Start is asserted.
-- Verify that state returns to S_idle from S_running if Zero is asserted.
-- Verify that state goes to S_running if Zero is not asserted.

entity t_Control_Unit (); -- VHDL Test bench for One-Hot Control Unit
end t_Control_Unit;

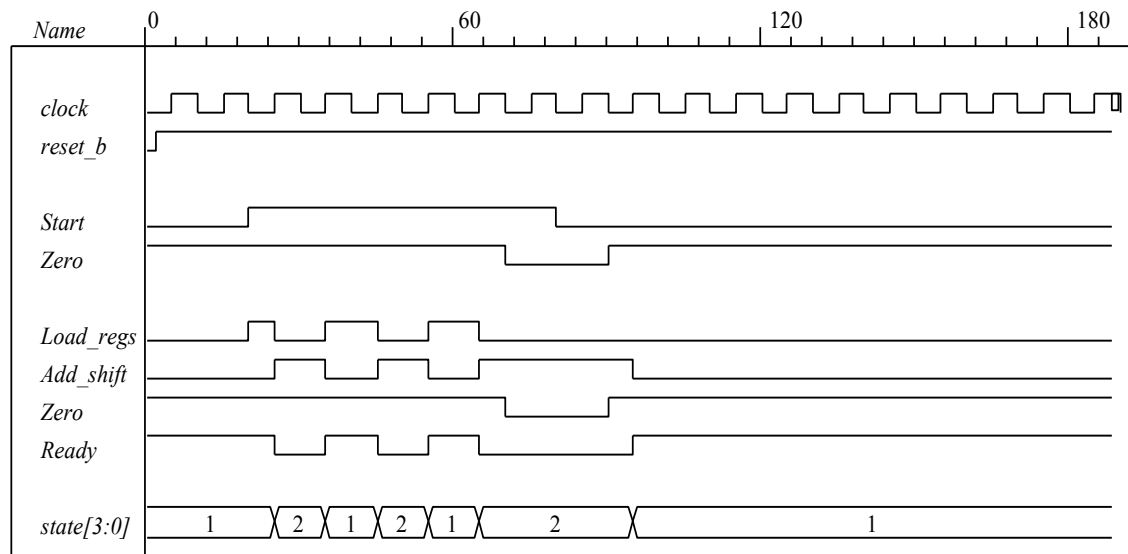
architecture Behavioral of t_Control_Unit;
  signal t_Ready, t_Load_regs, t_Add_shift: Std_Logic;
  signal t_Start, t_Zero, t_clock, t_reset_b: Std_Logic;
  component Prob_8_37d_Controller_2_Gates_1Hot
    port (Ready, Load_regs, Add_shift: out Std_Logic;
          Start, Zero, clock, reset_b: in Std_Logic);
  end component;
begin
  UUT: Prob_8_37d_Controller_2_Gates_1Hot port map (t_Ready, t_Load_regs,
    t_Add_shift, t_Start, t_Zero, t_clock, t_reset_b);
  process -- clock for testbench
  begin
    t_clock <= '0';
    wait for 5 ns;
    t_clock <= '1';
    wait for 5 ns;
  end process;

  reset_b <= '0';
  reset_b <= '1' after 2ns;

  t_Zero <= '1';
  t_Start <= '0';
  t_Start <= '1' after 20 ns; -- Cycle from S_idle to S_1
  t_Start <= '0' after 80 ns;
  t_Zero <= '0' after 70 ns; -- S_idle to S_1 to S_idle
  t_Zero <= '1' after 90 ns; -- Return to S_idle
end Behavioral;

```


Simulation results show that the controller matches the ASMD chart.



// Datapath unit – structural model

```

module Datapath_2_STR
#(parameter dp_width = 8, R2_width = 4)
(
    output [R2_width -1: 0]    count,
    output                    Zero,
    input [dp_width -1: 0]    data,
    input                    Load_regs, Add_shift, clock, reset_b);
    supply1                    pwr;
    supply0                    gnd;
    wire [dp_width -1: 0] R1_Dbus, R1;
    wire [R2_width -1: 0] R2_Dbus;
    wire DR1_0, DR1_1, DR1_2, DR1_3, DR1_4, DR1_5, DR1_6, DR1_7;
    wire R1_0, R1_1, R1_2, R1_3, R1_4, R1_5, R1_6, R1_7;
    wire R2_0, R2_1, R2_2, R2_3;
    wire [R2_width -1: 0] R2 = {R2_3, R2_2, R2_1, R2_0};
    assign count = {R2_3, R2_2, R2_1, R2_0};
    assign R1 = { R1_7, R1_6, R1_5, R1_4, R1_3, R1_2, R1_1, R1_0};
    assign DR1_0 = R1_Dbus[0];
    assign DR1_1 = R1_Dbus[1];
    assign DR1_2 = R1_Dbus[2];
    assign DR1_3 = R1_Dbus[3];
    assign DR1_4 = R1_Dbus[4];
    assign DR1_5 = R1_Dbus[5];
    assign DR1_6 = R1_Dbus[6];
    assign DR1_7 = R1_Dbus[7];

    nor (Zero, R1_0, R1_1, R1_2, R1_3, R1_4, R1_5, R1_6, R1_7);
    not (Load_regs_b, Load_regs);

    DFF DF_0 (R1_0, DR1_0, clock, pwr);    // Disable reset
    DFF DF_1 (R1_1, DR1_1, clock, pwr);
    DFF DF_2 (R1_2, DR1_2, clock, pwr);
    DFF DF_3 (R1_3, DR1_3, clock, pwr);
    DFF DF_4 (R1_4, DR1_4, clock, pwr);

```

```

DFF DF_5 (R1_5, DR1_5, clock, pwr);
DFF DF_6 (R1_6, DR1_6, clock, pwr);
DFF DF_7 (R1_7, DR1_7, clock, pwr);

DFF DR_0 (R2_0, DR2_0, clock, Load_regs_b); // Load_regs (set) drives R2 to all ones
DFF DR_1 (R2_1, DR2_1, clock, Load_regs_b);
DFF DR_2 (R2_2, DR2_2, clock, Load_regs_b);
DFF DR_3 (R2_3, DR2_3, clock, Load_regs_b);

assign DR2_0 = R2_Dbus[0];
assign DR2_1 = R2_Dbus[1];
assign DR2_2 = R2_Dbus[2];
assign DR2_3 = R2_Dbus[3];

wire [1: 0]          sel = {Add_shift, Load_regs};
wire [dp_width-1: 0] R1_shifted = {1'b0, R1_7, R1_6, R1_5, R1_4, R1_3, R1_2, R1_1};
wire [R2_width-1: 0] sum = R2 + {3'b000, R1[0]};

Mux8_4_x_1 M0 (R1_Dbus, R1, data, R1_shifted, R1, sel);
Mux4_2_x_1 M1 (R2_Dbus, R2, sum, Add_shift);
endmodule

```

```

module Mux8_4_x_1 #(parameter dp_width = 8) (output reg [dp_width -1: 0] mux_out,
input [dp_width -1: 0] in0, in1, in2, in3, input [1: 0] sel);
always @ (in0, in1, in2, in3, sel)
    case (sel)
        2'b00: mux_out = in0;
        2'b01: mux_out = in1;
        2'b10: mux_out = in2;
        2'b11: mux_out = in3;
    endcase
endmodule

module Mux4_2_x_1 #(parameter dp_width = 4) (output [dp_width -1: 0] mux_out,
input [dp_width -1: 0] in0, in1, input sel);
    assign mux_out = sel ? in1: in0;
endmodule

// Test Plan for Datapath Unit:
// Demonstrate action of Load_regs
// R1 gets data, R2 gets all ones
// Demonstrate action of Incr_R2
// Demonstrate action of Add_shift and detect Zero

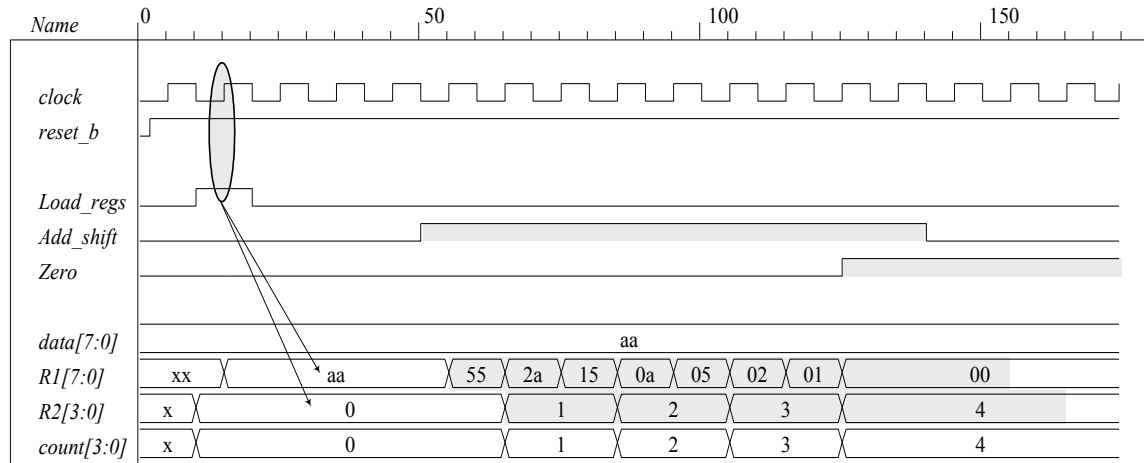
// Test bench for datapath

module t_Datapath_Unit
#(parameter dp_width = 8, R2_width = 4)
();
wire [R2_width -1: 0] count;
wire Zero;
reg [dp_width -1: 0] data;
    reg Load_regs, Add_shift, clock, reset_b;

Datapath_2_STR M0 (count, Zero, data, Load_regs, Add_shift, clock, reset_b);

initial #250 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin reset_b = 0; #2 reset_b = 1; end
initial fork
    data = 8'haa;
    Load_regs = 0;
    Add_shift = 0;
    #10 Load_regs = 1;
    #20 Load_regs = 0;
    #50 Add_shift = 1;
    #140 Add_shift = 0;
join
endmodule

```



// Integrated system

```

module Count_Ones_2_Gates_1Hot_STR
# (parameter dp_width = 8, R2_width = 4)
(
  output [R2_width -1: 0] count,
  input [dp_width -1: 0] data,
  input Start, clock, reset_b
);
  wire Load_regs, Add_shift, Zero;
  Controller_2_Gates_1Hot M0 (Ready, Load_regs, Add_shift, Start, Zero, clock, reset_b);
  Datapath_2_STR M1 (count, Zero, data, Load_regs, Add_shift, clock, reset_b);
endmodule

```

// Test plan for integrated system
 // Test for data values of 8'haa, 8'h00, 8'hff.

// Test bench for integrated system

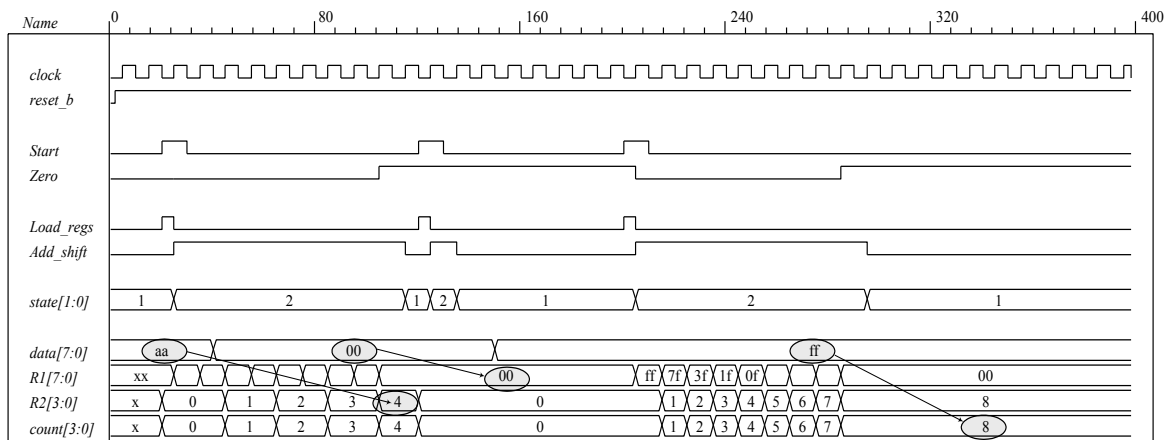
```

module t_Count_Ones_2_Gates_1Hot_STR ();
  parameter dp_width = 8, R2_width = 4;
  wire [R2_width -1: 0] count;
  reg [dp_width -1: 0] data;
  reg Start, clock, reset_b;
  wire [1: 0] state = {M0.M0.T1, M0.M0.T0};

  Count_Ones_2_Gates_1Hot_STR M0 (count, data, Start, clock, reset_b);

  initial #700 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin reset_b = 0; #2 reset_b = 1; end
  initial fork
    data = 8'haa; // Expect count = 4
    Start = 0;
    #20 Start = 1;
    #30 Start = 0;
    #40 data = 8'b00; // Expect count = 0
    #120 Start = 1;
    #130 Start = 0;
    #150 data = 8'hff; // Expect count = 8
    #200 Start = 1;
    #210 Start = 0;
  join
endmodule

```



(d)

```
Library IEEE;
use IEEE.Std_Logic_1164.all;
```

```
entity Prob_8_37d_Controller_2_Gates_1Hot is
    port (Ready, Load_regs, Add_shift: out Std_Logic;
          Start, Zero, clock, reset_b: in Std_Logic);
end Prob_8_37d_Controller_2_Gates_1Hot;
```

```
architecture Structural of Prob_8_37d_Controller_2_Gates_1Hot is
```

```
    signal w1, w2, w3, w4: Std_Logic;
    signal T0, T1: Std_Logic;
    signal set: Std_Logic;
    signal D0, D1: Std_Logic;
    signal Zero_b, Start_b: Std_Logic;
```

```
    component D_FF port (Q: out Std_Logic; D, clock, reset_b: in Std_Logic);
    end component;
```

```
    component D_FF_S port (Q: out Std_Logic; D, clock, set: in Std_Logic);
    end component;
```

```
begin
```

```
    Ready <= T0;
    Add_shift <= T1;
    Load_regs <= T0 and Start;
    set <= not reset_b;
```

```
    M0: D_FF_S port map (T0, D0, clock, set);    -- Note: reset action must
    M1: D_FF port map (T1, D1, clock, reset_b);  -- initialize S_idle = 2'b01
```

```
    Start_b <= not Start;
    Zero_b <= not Zero;
    w1 <= T0 and Start_b;
    w2 <= T1 and Zero;
    D0 <= w1 or w2;
    w3 <= T0 and Start;
    w4 <= T1 and Zero_b;
    D1 <= w3 or w4;
```

```
end Structural;
```

```
Library IEEE;
use IEEE.Std_Logic_1164.all;
```

```

entity D_FF is
  port (Q: out Std_Logic; D, clock, reset_b: in Std_Logic);
end D_FF;
architecture Behavioral of D_FF is
begin
  process (clock, reset_b) begin
    if (reset_b = '0') then Q <= '0';
    elsif clock'event and clock = '1' then Q <= D; end if;
  end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity D_FF_S is
  port (Q: out Std_Logic; D, clock, set: in Std_Logic);
end D_FF_S;

architecture Behavioral of D_FF_S is
begin
  process (clock, set) begin
    if (set = '1') then Q <= '1';
    elsif clock'event and clock = '1' then Q <= D; end if;
  end process;
end Behavioral;

```

8.38 Verilog

```

module Prob_8_38 (
  output reg [7: 0]   Sum,
  output reg         Car_Bor,
  input [7: 0]       Data_A, Data_B;
  reg [7: 0]         Reg_A, Reg_B;

  always @ (Data_A, Data_B)
    case ({Data_A[7], Data_B[7]})
      2'b00, 2'b11: begin                                // ++, --
                    {Car_Bor, Sum[6: 0]} = Data_A[6: 0] + Data_B[6: 0];
                    Sum[7] = Data_A[7];
                    end

      default:      if (Data_A[6: 0] >= Data_B[6: 0]) begin    // +-, -+
                    {Car_Bor, Sum[6: 0]} = Data_A[6: 0] - Data_B[6: 0];
                    Sum[7] = Data_A[7];
                    end
                    else begin
                      {Car_Bor, Sum[6: 0]} = Data_B[6: 0] - Data_A[6: 0];
                      Sum[7] = Data_B[7];
                    end
    endcase
endmodule

module t_Prob_8_38 ();
  wire [7: 0]   Sum;
  wire         Car_Bor;
  reg [7: 0]   Data_A, Data_B;
  wire [6: 0]   Mag_A, Mag_B;
  assign       Mag_A = M0.Data_A[6: 0];           // Hierarchical dereferencing
  assign       Mag_B = M0.Data_B[6: 0];
  wire         Sign_A = M0.Data_A[7];
  wire         Sign_B = M0.Data_B[7];
  wire         Sign = Sum[7];
  wire [7: 0]   Mag = Sum[6: 0];

  Prob_8_38 M0 (Sum, Car_Bor, Data_A, Data_B);

  initial #650 $finish;

```

initial fork

```

// Addition
// A      B
#0 begin Data_A = {1'b0, 7'd25}; Data_B = {1'b0, 7'd10}; end // +25, +10
#40 begin Data_A = {1'b1, 7'd25}; Data_B = {1'b1, 7'd10}; end // -25, -10
#80 begin Data_A = {1'b1, 7'd25}; Data_B = {1'b0, 7'd10}; end // -25, +10
#120 begin Data_A = {1'b0, 7'd25}; Data_B = {1'b1, 7'd10}; end // 25, -10
// B      A
#160 begin Data_B = {1'b0, 7'd25}; Data_A = {1'b0, 7'd10}; end // +25, +10
#200 begin Data_B = {1'b1, 7'd25}; Data_A = {1'b1, 7'd10}; end // -25, -10

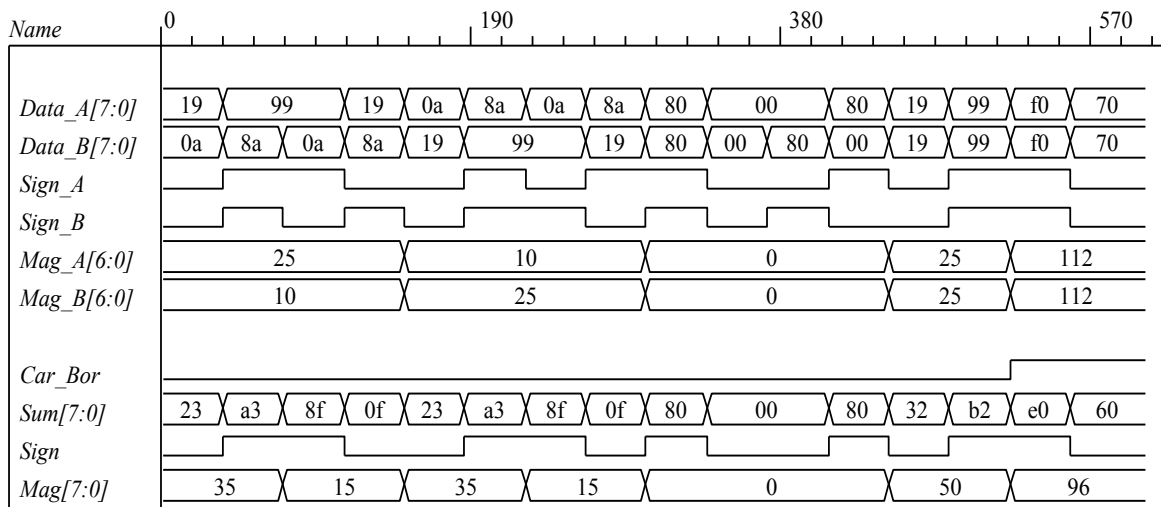
#240 begin Data_B = {1'b1, 7'd25}; Data_A = {1'b0, 7'd10}; end // -25, +10
#280 begin Data_B = {1'b0, 7'd25}; Data_A = {1'b1, 7'd10}; end // +25, -10
// Addition of matching numbers

#320 begin Data_A = {1'b1, 7'd0}; Data_B = {1'b1, 7'd0}; end // -0, -0
#360 begin Data_A = {1'b0, 7'd0}; Data_B = {1'b0, 7'd0}; end // +0, +0
#400 begin Data_A = {1'b0, 7'd0}; Data_B = {1'b1, 7'd0}; end // +0, -0
#440 begin Data_A = {1'b1, 7'd0}; Data_B = {1'b0, 7'd0}; end // -0, +0

#480 begin Data_B = {1'b0, 7'd25}; Data_A = {1'b0, 7'd25}; end // matching +
#520 begin Data_B = {1'b1, 7'd25}; Data_A = {1'b1, 7'd25}; end // matching -

// Test of carry (negative numbers)
#560 begin Data_A = 8'hf0; Data_B = 8'hf0; end // carry - -
// Test of carry (positive numbers)
#600 begin Data_A = 8'h70; Data_B = 8'h70; end // carry ++
join
endmodule

```

**VHDL**

```

entity Prob_8_38_vhdl is
    port (Sum: out Std_Logic_vector (7 downto 0); Car_Bor: out Std_Logic;
          Data_A, Data_B: in Std_Logic_vector (7 downto 0 ));
end Prob_8_38_vhdl;

architecture Behavioral of Prob_8_38_vhdl is
    signal Reg_A, Reg_B: Std_Logic_vector (7 downto 0);
begin

```



```

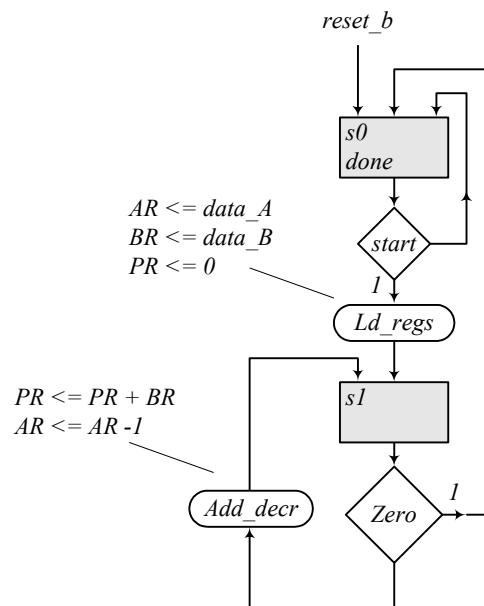
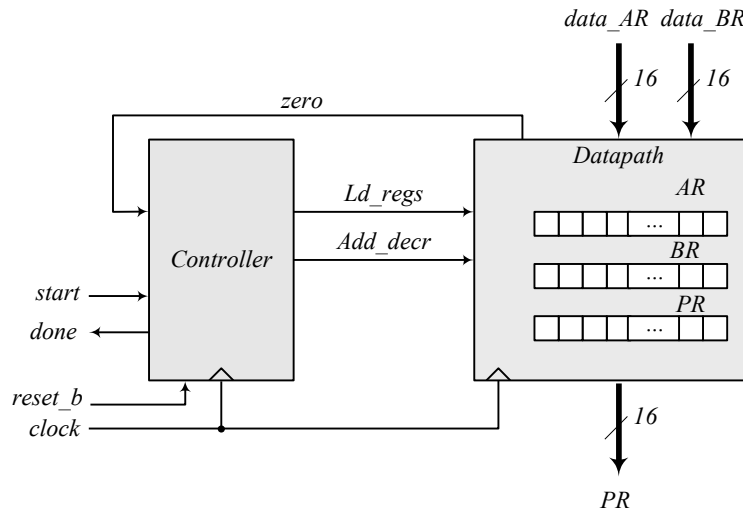
process (Data_A, Data_B)
  case (Data_A(7) & Data_B(7))
    -- ++, --
    when "00", "11" =>
      Car_Bor & Sum(6 downto 0) <= Data_A(6 downto 0] + Data_B(6 : 0);
      Sum(7) = Data_A(7);

    -- +-, -+
    when others => if Data_A(6 downto 0) >= Data_B(6 : 0) then
      Car_Bor & Sum(6 downto 0) <= Data_A(6 : 0) - Data_B(6 : 0);
      Sum(7) <= Data_A(7);
    else
      Car_Bor & Sum(6 : 0) <= Data_B(6 : 0) - Data_A(6 : 0);
      Sum(7) <= Data_B(7);
    end if;

  end case;
end Behavioral;

```

8.39 Block diagram and ASMD chart:



Verilog

```

module Prob_8_39 (
  output [15: 0] PR, output done,
  input [7: 0] data_AR, data_BR, input start, clock, reset_b
);

  Controller_P8_39 M0 (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

  Datapath_P8_39 M1 (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
endmodule

module Controller_P8_16 (output done, output reg Ld_regs, Add_decr, input start, zero, clock, reset_b);
  parameter s0 = 1'b0, s1 = 1'b1;
  reg state, next_state;
  assign done = (state == s0);

```

```

always @ (posedge clock, negedge reset_b)
  if (!reset_b) state <= s0; else state <= next_state;

always @ (state, start, zero) begin
  Ld_regs = 0;
  Add_decr = 0;
  case (state)
    s0: if (start) begin Ld_regs = 1; next_state = s1; end
    s1: if (zero) next_state = s0; else begin next_state = s1; Add_decr = 1; end
    default: next_state = s0;
  endcase
end
endmodule

module Datapath_P8_16 (
  output reg [15: 0] PR, output zero,
  input [7: 0] data_AR, data_BR, input Ld_regs, Add_decr, clock, reset_b
);

  reg [7: 0] AR, BR;
  assign zero = ~( | AR);

  always @ (posedge clock, negedge reset_b)
    if (!reset_b) begin AR <= 8'b0; BR <= 8'b0; PR <= 16'b0; end
    else begin
      if (Ld_regs) begin AR <= data_AR; BR <= data_BR; PR <= 0; end
      else if (Add_decr) begin PR <= PR + BR; AR <= AR -1; end
    end
  endmodule

// Test plan – Verify;
// Power-up reset
// Data is loaded correctly
// Control signals assert correctly
// Status signals assert correctly
// start is ignored while multiplying
// Multiplication is correct
// Recovery from reset on-the-fly

module t_Prob_P8_16;
  wire done;
  wire [15: 0] PR;
  reg [7: 0] data_AR, data_BR;
  reg start, clock, reset_b;

  Prob_8_16 M0 (PR, done, data_AR, data_BR, start, clock, reset_b);

  initial #500 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork
    reset_b = 0;
    #12 reset_b = 1;
    #40 reset_b = 0;
    #42 reset_b = 1;
    #90 reset_b = 1;
    #92 reset_b = 1;
  join

```

```

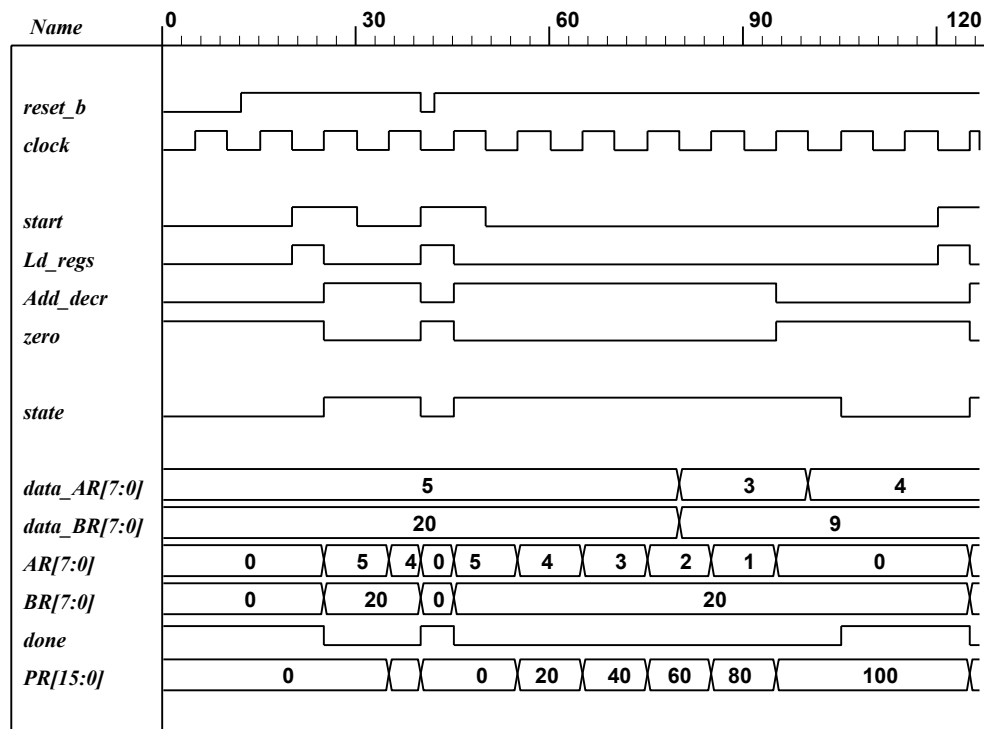
initial fork
  #20 start = 1;
  #30 start = 0;
  #40 start = 1;
  #50 start = 0;
  #120 start = 1;
  #120 start = 0;
join

initial fork
  data_AR = 8'd5;    // AR > 0
  data_BR = 8'd20;

  #80 data_AR = 8'd3;
  #80 data_BR = 8'd9;

  #100 data_AR = 8'd4;
  #100 data_BR = 8'd9;
join
endmodule

```



VHDL

```

Library IEEE;
use IEEE.Std_Logic_1164.all;

```

```

entity Prob_8_39_vhdl is
  port (PR: out Std_Logic_vector (16 downto 0); done: out Std_Logic; data_AR, data_BR:
    in Std_Logic_Vector (7 downto 0); start, clock, reset_b: in Std_Logic);
end Prob_8_39_vhdl;

```

```

architecture ASMD of Prob_8_39_vhdl is
  -- Interface signals
  signal Ld_regs, Add_decr, zero: Std_Logic;

```

```

-- Components
component Controller_P8_39 port (done, Ld_regs, Add_decr: out Std_Logic;
    Start, zero, clock, reset_b: in Std_Logic);
    end component;
component Datapath_P8_39 port(PR: out Std_Logic_Vector (16 downto 0);
    zero: out Std_Logic;
    data_AR, data_BR: in Std_Logic_Vector (7 downto 0);
    Ld_regs, Add_decr, clock, reset_b: in Std_Logic);
    end component;
begin
-- Instantiations of Components
    M0: Controller_P8_39 port map (done, Ld_regs, Add_decr, start, zero, clock, reset_b);

    M1: Datapath_P8_39 port map (PR, zero, data_AR, data_BR, Ld_regs, Add_decr, clock, reset_b);
end ASMD;

Library IEEE;
use IEEE.Std_Logic_1164.all;
entity Controller_P8_39 is
    port (done: out Std_Logic; Ld_regs, Add_decr: out Std_Logic; start, zero, clock, reset_b: in Std_Logic);
end Controller_P8_39;

architecture Behavioral of Controller_P8_39 is
    constant s0: Std_Logic := '0';
    constant s1: Std_Logic := '1';
    signal state, next_state: Std_Logic;
begin
    done <= '1' when state = s0 else '0';

    process (clock, reset_b) begin
        if reset_b = '0' then state <= s0;
        elsif clock'event and clock = '1' then state <= next_state; end if;
    end process;

    process (state, start, zero) begin
        Ld_regs <= '0';
        Add_decr <= '0';
        next_state <= s0;
        case state is
            when s0 => if (start = '1') then Ld_regs <= '1'; next_state <= s1; else next_state <= s0; end if;
            when s1 => if (zero = '1') then next_state <= s0; else next_state <= s1; Add_decr <= '1'; end if;
            when others => next_state <= s0;
        end case;
    end process;
end Behavioral;

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_Unsigned.all;

entity Datapath_P8_39 is
port (PR: buffer Std_Logic_vector (16 downto 0); zero: out Std_Logic; data_AR, data_BR: in Std_Logic_Vector (7
downto 0); Ld_regs, Add_decr, clock, reset_b: in Std_Logic);
end Datapath_P8_39;

architecture Behavioral of Datapath_P8_39 is
    signal Ar, BR: Std_Logic_vector (7 downto 0);
begin
    -- zero <= not( AR(7) or AR(6) or AR(5) or AR(4) or AR(3) or AR(2) or AR(1) or AR(0));
    process (AR) begin
        zero <= '1';

```

```

    for k in 0 to 7 loop
        if AR(k) = '1' then Zero <= '0'; end if;
    end loop;
end process;

process (clock, reset_b) begin
    if reset_b = '0' then AR <= "00000000"; BR <= "00000000"; PR <= "000000000000000000";
    elsif clock'event and clock = '1' then
        if Ld_regs = '1' then AR <= data_AR; BR <= data_BR; for k in 0 to 16 loop PR(k) <= '0'; end loop;
        elsif Add_decr = '1' then PR <= PR + ('0' & BR); AR <= AR -1; end if;
    end if;
end process;
end Behavioral;

-- Test plan (Operational features to verify)
-- Power-up reset
-- Data is loaded correctly
-- Control signals assert correctly
-- Status signals assert correctly
-- start is ignored while multiplying
-- Multiplication is correct
-- Recovery from reset on-the-fly

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity t_Prob_P8_39 is
end t_Prob_P8_39;

architecture Test_Bench of t_Prob_P8_39 is
    signal t_done: Std_Logic;
    signal t_PR: Std_Logic_Vector (15 downto 0);
    signal t_data_AR, t_data_BR: Std_Logic_Vector (7 downto 0);
    signal t_start, t_clock, t_reset_b: Std_Logic;
    component Prob_8_39 port (PR: out Std_Logic_Vector (15 downto 0); done: out Std_Logic;
        data_AR, data_BR: in Std_Logic_Vector (7 downto 0); start, clock, reset_b: in Std_Logic);
    end component;
begin

M_UUT: Prob_8_39 port map (PR => t_PR, done => t_done, data_AR => t_data_AR, data_BR => t_data_BR, start
=> t_start, clock => t_clock, reset_b => t_reset_b);

process begin
    t_clock <= '0';
    wait for 5ns;
    t_clock <= '1';
    wait for 5ns;
end process;

t_reset_b <= '0';
t_reset_b <= '1' after 12 ns;
t_reset_b <= '0' after 40 ns;
t_reset_b <= '1' after 42 ns;
t_reset_b <= '1' after 90 ns;
t_reset_b <= '1' after 92 ns;

t_start <= '1' after 20 ns;
t_start <= '0' after 30 ns;
t_start <= '1' after 40 ns;
t_start <= '0' after 50 ns;
t_start <= '1' after 120 ns;
t_start <= '0' after 120 ns;

```

```

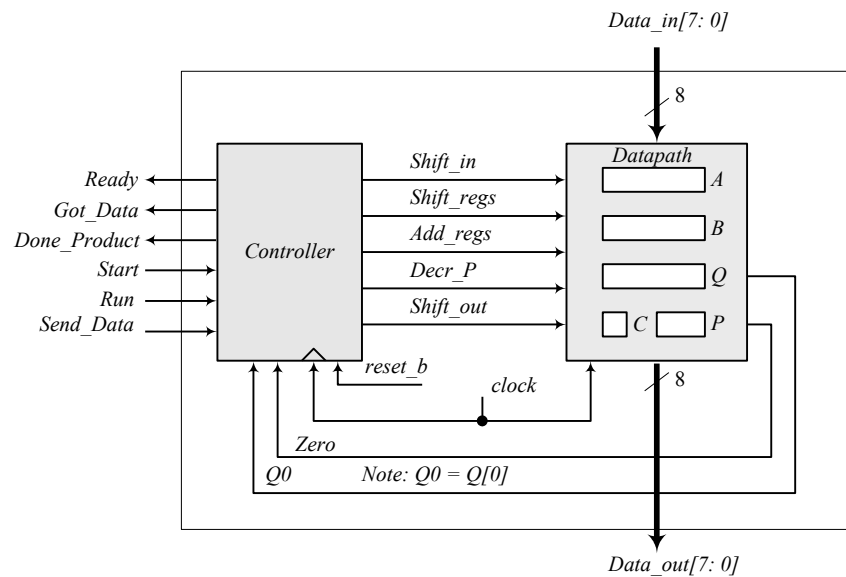
t_data_AR <= "00000101";
t_data_BR <= "00010100";

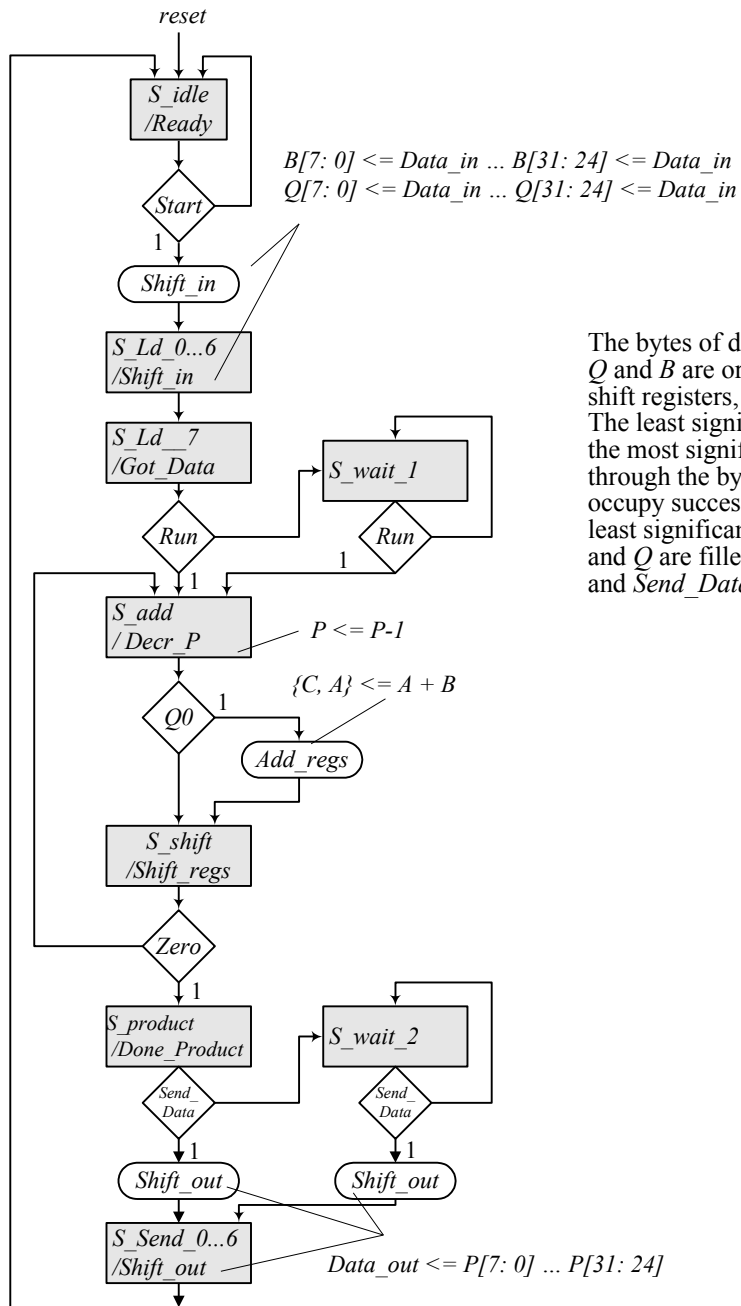
t_data_AR <= "00000011" after 80 ns;
t_data_BR <= "00001001" after 80 ns;

t_data_AR <= "00000100" after 100 ns;
t_data_BR <= "00001001" after 100 ns;
end Test_Bench;

```

8.40





The bytes of data will be read sequentially. Registers Q and B are organized to act as byte-wide parallel shift registers, taking 8 clock cycles to fill the pipe. The least significant byte of the multiplicand enters the most significant byte of Q and then moves through the bytes of Q to enter B , then proceed to occupy successive bytes of B until it occupies the least significant byte of B , and so forth until both B and Q are filled. Wait states are used to wait for Run and $Send_Data$.

Verilog

```

module Prob_8_40 (
  output [7: 0] Data_out,
  output Ready, Got_Data, Done_Product,
  input [7: 0] Data_in,
  input Start, Run, Send_Data, clock, reset_b
);

  Controller M0 (
    Ready, Shift_in, Got_Data, Done_Product, Decr_P, Add_regs, Shift_regs, Shift_out,
    Start, Run, Send_Data, Zero, Q0, clock, reset_b
  );
  Datapath M1(Data_out, Q0, Zero, Data_in,
    Start, Shift_in, Decr_P, Add_regs, Shift_regs, Shift_out, clock
  );
endmodule

module Controller (
  output reg Ready, Shift_in, Got_Data, Done_Product, Decr_P, Add_regs,
    Shift_regs, Shift_out,
  input Start, Run, Send_Data, Zero, Q0, clock, reset_b
);

  parameter S_idle = 5'd20,
    S_Ld_0 = 5'd0,
    S_Ld_1 = 5'd1,
    S_Ld_2 = 5'd2,
    S_Ld_3 = 5'd3,
    S_Ld_4 = 5'd4,
    S_Ld_5 = 5'd5,
    S_Ld_6 = 5'd6,
    S_Ld_7 = 5'd7,
    S_wait_1 = 5'd8, // Wait state
    S_add = 5'd9,
    S_Shift = 5'd10,
    S_product = 5'd11,
    S_wait_2 = 5'd12, // Wait state
    S_Send_0 = 5'd13,
    S_Send_1 = 5'd14,
    S_Send_2 = 5'd15,
    S_Send_3 = 5'd16,
    S_Send_4 = 5'd17,
    S_Send_5 = 5'd18,
    S_Send_6 = 5'd19;

  reg [4: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;

  always @ (state, Start, Run, Q0, Zero, Send_Data) begin
    next_state = S_idle; // Prevent accidental synthesis of latches
    Ready = 0;
    Shift_in = 0;
    Shift_regs = 0;
    Add_regs = 0;
    Decr_P = 0;
    Shift_out = 0;
    Got_Data = 0;
    Done_Product = 0;
  end

```

```

case (state) // Assign by exception to default values
  S_idle: begin
    Ready = 1;
    if (Start) begin next_state = S_Ld_0; Shift_in = 1; end
  end
  S_Ld_0: begin next_state = S_Ld_1; Shift_in = 1; end
  S_Ld_1: begin next_state = S_Ld_2; Shift_in = 1; end
  S_Ld_2: begin next_state = S_Ld_3; Shift_in = 1; end
  S_Ld_3: begin next_state = S_Ld_4; Shift_in = 1; end
  S_Ld_4: begin next_state = S_Ld_5; Shift_in = 1; end
  S_Ld_5: begin next_state = S_Ld_6; Shift_in = 1; end
  S_Ld_6: begin next_state = S_Ld_7; Shift_in = 1; end
  S_Ld_7: begin Got_Data = 1;
    if (Run) next_state = S_add;
    else next_state = S_wait_1;
  end
  S_wait_1: if (Run) next_state = S_add; else next_state = S_wait_1;
  S_add: begin next_state = S_Shift; Decr_P = 1; if (Q0) Add_regs = 1; end
  S_Shift: begin Shift_regs = 1; if (Zero) next_state = S_product;
    else next_state = S_add; end
  S_product: begin
    Done_Product = 1;
    if (Send_Data) begin next_state = S_Send_0; Shift_out = 1; end
    else next_state = S_wait_2; end
  S_wait_2: if (Send_Data) begin next_state = S_Send_0; Shift_out = 1; end
    else next_state = S_wait_2;
  S_Send_0: begin next_state = S_Send_1; Shift_out = 1; end
  S_Send_1: begin next_state = S_Send_2; Shift_out = 1; end
  S_Send_2: begin next_state = S_Send_3; Shift_out = 1; end
  S_Send_3: begin next_state = S_Send_4; Shift_out = 1; end
  S_Send_4: begin next_state = S_Send_5; Shift_out = 1; end
  S_Send_5: begin next_state = S_Send_6; Shift_out = 1; end
  S_Send_6: begin next_state = S_idle; Shift_out = 1; end
  default: next_state = S_idle;
endcase
end
endmodule

module Datapath #(parameter dp_width = 32, P_width = 6) (
  output [7: 0] Data_out,
  output Q0, Zero,
  input [7: 0] Data_in,
  input Start, Shift_in, Decr_P, Add_regs, Shift_regs, Shift_out, clock
);
reg [dp_width - 1: 0] A, B, Q; // Sized for datapath
reg C;
reg [P_width - 1: 0] P;
assign Q0 = Q[0];
assign Zero = (P == 0); // counter is zero
assign Data_out = {C, A, Q};

always @ (posedge clock) begin
  if (Shift_in) begin
    P <= dp_width;
    A <= 0;
    C <= 0;
    B[7: 0] <= B[15: 8]; // Treat B and Q registers as a pipeline to load data bytes
    B[15: 8] <= B[23: 16];
    B[23: 16] <= B[31: 24];
    B[31: 24] <= Q[7: 0];
    Q[7: 0] <= Q[15: 8];
    Q[15: 8] <= Q[23: 16];
  end

```

```

    Q[23: 16] <= Q[31: 24];
    Q[31: 24] <= Data_in;
end
if (Add_regs) {C, A} <= A + B;
if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
if (Decr_P) P <= P -1;
if (Shift_out) begin {C, A, Q} <= {C, A, Q} >> 8; end
end
endmodule

module t_Prob_8_40;
    parameter dp_width = 32;          // Width of datapath
    wire [7: 0] Data_out;
    wire Ready, Got_Data, Done_Product;
    reg Start, Run, Send_Data, clock, reset_b;

    integer Exp_Value;
    reg Error;
    wire [7: 0] Data_in;
    reg [dp_width -1: 0] Multiplicand, Multiplier;
    reg [2*dp_width -1: 0] Data_register;    // For test patterns
    assign Data_in = Data_register [7:0];
    wire [2*dp_width -1: 0] product;
    assign product = {M0.M1.C, M0.M1.A, M0.M1.Q};
    Prob_8_40 M0 (
        Data_out, Ready, Got_Data, Done_Product, Data_in, Start, Run, Send_Data, clock, reset_b
    );

    initial #2000 $finish;
    initial begin clock = 0; forever #5 clock = ~clock; end
    initial fork
        reset_b = 1;
        #2 reset_b = 0;
        #3 reset_b = 1;
    join
    initial fork
        Start = 0;
        Run = 0;
        Send_Data = 0;
        #10 Start = 1;
        #20 Start = 0;

        #50 Run = 1;          // Ignored by controller
        #60 Run = 0;
        #120 Run = 1;
        #130 Run = 0;

        #830 Send_Data = 1;
        #840 Send_Data = 0;
    join
    // Test patterns for multiplication

    initial begin
        Multiplicand = 32'h0f_00_00_aa;
        Multiplier = 32'h0a_00_00_ff;
        Data_register = {Multiplier, Multiplicand};
    end

    initial begin          // Synchronize input data bytes
        @ (posedge Start)
        repeat (15) begin

```

```

    @ (negedge clock)
      Data_register <= Data_register >> 8;
  end
end
endmodule

```

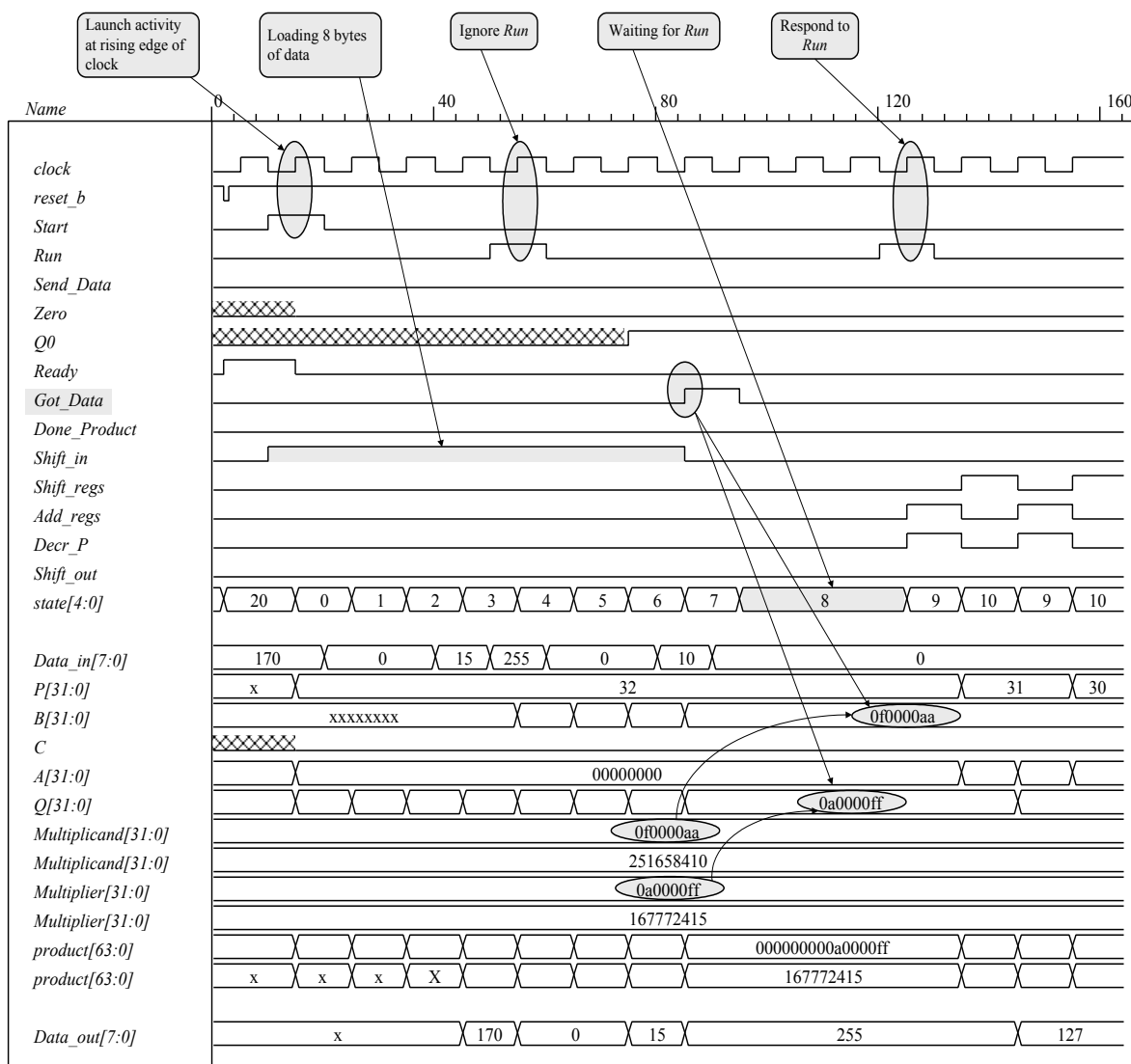
Simulation results: Loading multiplicand (0f0000aa_H) and multiplier (0a0000ff_H), 4 bytes each, in sequence, beginning with the least significant byte of the multiplicand.

Note: *Product* is not valid until *Done_Product* asserts. The value of *Product* shown here (255₁₀) reflects the contents of {*C*, *A*, *Q*} after the multiplier has been loaded, prior to multiplication.

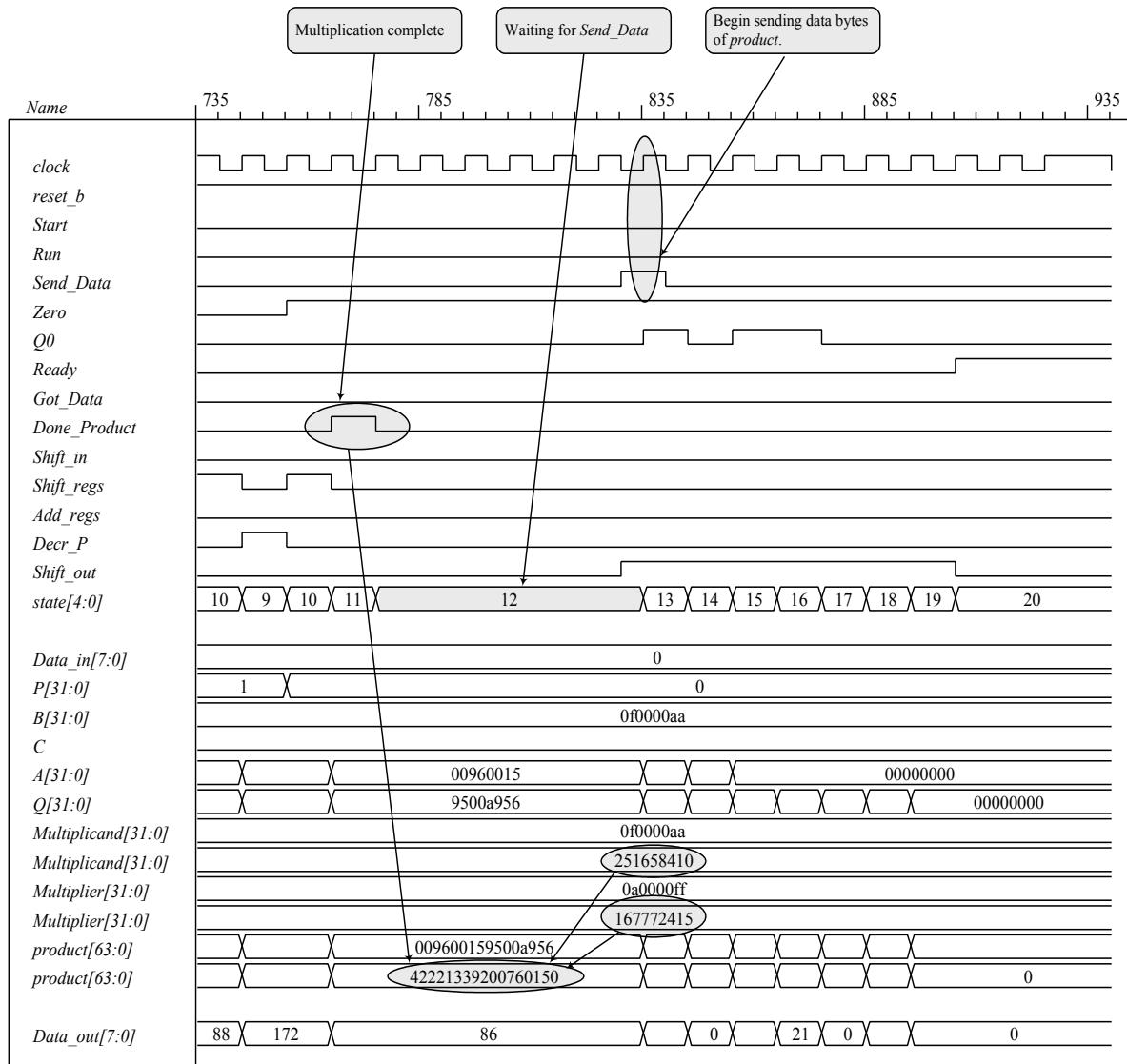
Note: The machine ignores a premature assertion of *Run*.

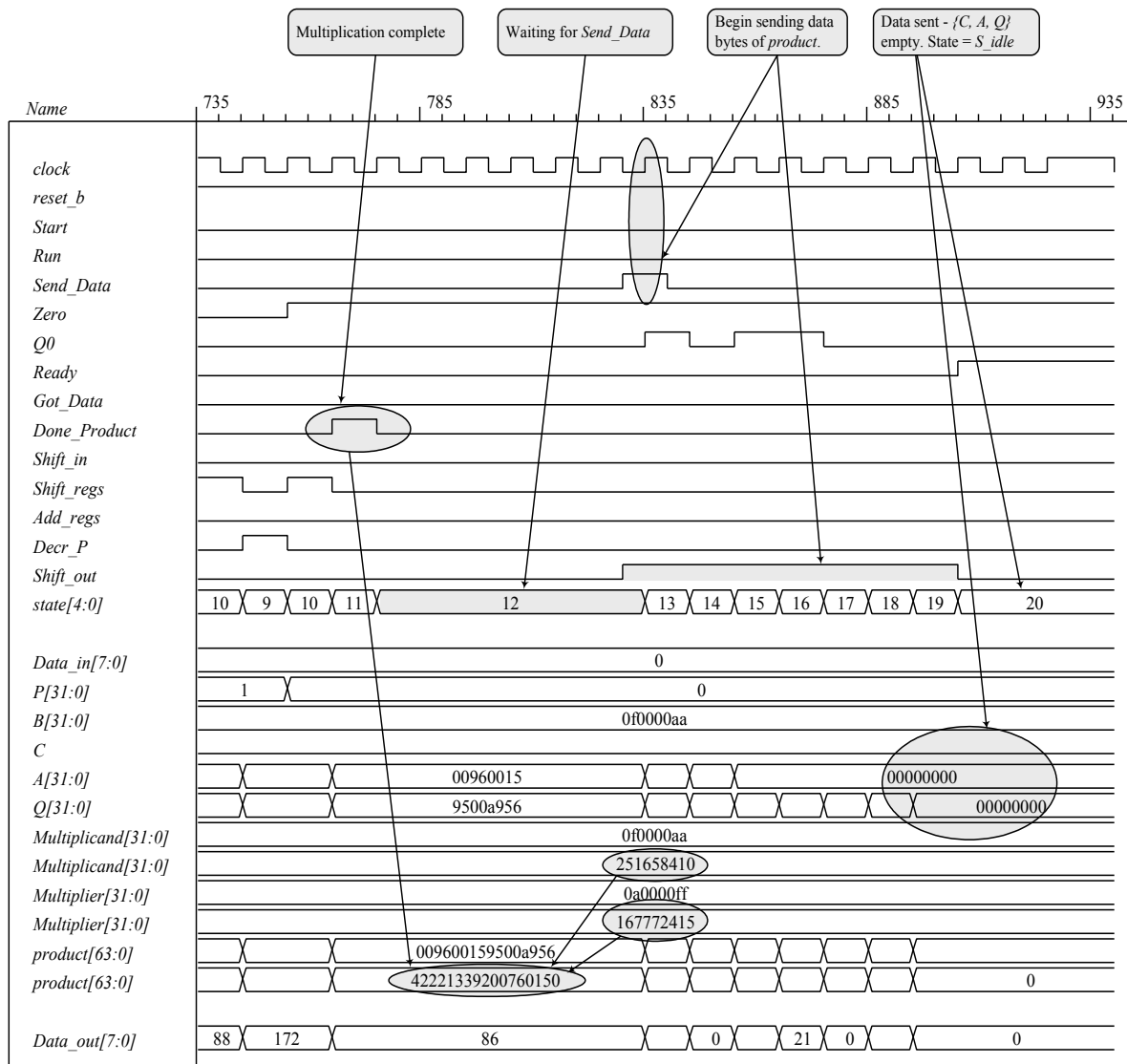
Note: *Got_Data* asserts at the 8th clock after *Start* asserts, i.e., 8 clocks to load the data.

Note: *Product*, *Multiplier*, and *Multiplicand* are formed in the test bench.



Note: Product (64 bits) is formed correctly





VHDL

Library IEEE;
use IEEE.Std_Logic_1164.all;

```
entity Prob_8_40 is
    port (Data_out: out Std_Logic_vector (7 downto 0);
          Ready, Got_Data, Done_Product: out Std_Logic;
          Data_in: in Std_Logic_vector (7 downto 0);
          Start, Run, Send_Data, clock, reset_b: in Std_Logic);
end Prob_8_40;
```

```
architecture Behavior of Prob_8_40 is
    signal Load_P, Shift_in, Shift_out, Shift_regs, Add_regs, Decr_P, Zero, Q0: Std_Logic;
```

```
    component Controller port (Ready, Load_P, Shift_in, Got_Data, Done_Product, Decr_P, Add_regs,
        Shift_regs, Shift_out: out Std_Logic; Start, Run, Send_Data, Zero, Q0,
        clock, reset_b: in Std_Logic);
    end component;
```

```

component Datapath
  generic dp_width: integer := 32; P_width: integer := 6;
  port (Data_out: out Std_Logic_vector (7 downto 0);
        Q0, Zero: out Std_Logic;
        Data_in: in Std_Logic_vector (7 downto 0);
        Start, Shift_in, Load_P, Decr_P, Add_regs, Shift_regs, Shift_out, clock: in std_Logic);
end component;
begin

M0: Controller port map (Ready, Load_P, Shift_in, Got_Data, Done_Product, Decr_P, Add_regs,
  Shift_regs, Shift_out, Start, Run, Send_Data, Zero, Q0, clock, reset_b);

M1: Datapath generic map (dp_width => 32, P_width => 6)
port map (Data_out, Q0, Zero, Data_in, Start, Shift_in, Load_P, Decr_P, Add_regs, Shift_regs, Shift_out,
  clock
);
end Behavior;

Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Controller is
  port (
    Ready, Load_P, Shift_in, Got_Data, Done_Product, Decr_P, Add_regs,
    Shift_regs, Shift_out: out Std_Logic;
    Start, Run, Send_Data, Zero, Q0, clock, reset_b: in Std_Logic
  );
end Controller;

architecture Behavioral of Controller is
  constant S_idle: Std_Logic_vector (4 downto 0) := "10100";
  constant S_Ld_0: Std_Logic_vector (4 downto 0) := "00000";
  constant S_Ld_1: Std_Logic_vector (4 downto 0) := "00001";
  constant S_Ld_2: Std_Logic_vector (4 downto 0) := "00010";
  constant S_Ld_3: Std_Logic_vector (4 downto 0) := "00011";
  constant S_Ld_4: Std_Logic_vector (4 downto 0) := "00100";
  constant S_Ld_5: Std_Logic_vector (4 downto 0) := "00101";
  constant S_Ld_6: Std_Logic_vector (4 downto 0) := "00110";
  constant S_Ld_7: Std_Logic_vector (4 downto 0) := "00111";
  constant S_wait_1: Std_Logic_vector (4 downto 0) := "01000"; -- wait state
  constant S_add: Std_Logic_vector (4 downto 0) := "01001";
  constant S_shift: Std_Logic_vector (4 downto 0) := "01010";
  constant S_product: Std_Logic_vector (4 downto 0) := "01011";
  constant S_wait_2: Std_Logic_vector (4 downto 0) := "11000";
  constant S_send_0: Std_Logic_vector (4 downto 0) := "01101"; -- wait state
  constant S_send_1: Std_Logic_vector (4 downto 0) := "01110";
  constant S_send_2: Std_Logic_vector (4 downto 0) := "01111";
  constant S_send_3: Std_Logic_vector (4 downto 0) := "10000";
  constant S_send_4: Std_Logic_vector (4 downto 0) := "10001";
  constant S_send_5: Std_Logic_vector (4 downto 0) := "10010";
  constant S_send_6: Std_Logic_vector (4 downto 0) := "10011";

  signal state, next_state: Std_Logic_vector (4 downto 0);
begin
  process (clock, reset_b) begin
    if reset_b = '0' then state <= S_idle;
    elsif clock'event and clock = '1' then state <= next_state;
    end if;
  end process;

  process (state, Start, Run, Q0, Zero, Send_Data) begin

```



```

next_state      <= S_idle; -- Initialize for assignment
Ready           <= '0';    -- by exception
Shift_in        <= '0';
Shift_regs      <= '0';
Add_regs        <= '0';
Load_P          <= '0';
Decr_P          <= '0';
Shift_out       <= '0';
Got_Data        <= '0';
Done_Product    <= '0';

case (state) is
when S_idle => Ready <= '1';
                if (Start = '1') then
                    next_state <= S_Ld_0;
                    Shift_in <= '1';
                    Load_P <= '1';
                end if;
when S_Ld_0 => next_state <= S_Ld_1; Shift_in <= '1';
when S_Ld_1 => next_state <= S_Ld_2; Shift_in <= '1';
when S_Ld_2 => next_state <= S_Ld_3; Shift_in <= '1';
when S_Ld_3 => next_state <= S_Ld_4; Shift_in <= '1';
when S_Ld_4 => next_state <= S_Ld_5; Shift_in <= '1';
when S_Ld_5 => next_state <= S_Ld_6; Shift_in <= '1';
when S_Ld_6 => next_state <= S_Ld_7; Shift_in <= '1';
when S_Ld_7 => Got_Data <= '1';
                if (Run = '1') then next_state <= S_add;
                else next_state <= S_wait_1;
                end if;
when S_wait_1 => if (Run = '1') then next_state <= S_add;
                else next_state <= S_wait_1;
when S_add => next_state <= S_Shift; Decr_P <= '1';
                if (Q0 = '1') then Add_regs <= '1'; end if;
when S_Shift => Shift_regs <= '1';
                if (Zero = '1') then next_state <= S_product;
                else next_state <= S_add; end if;
when S_product => Done_Product <= '1';
                if (Send_Data = '1') then
                    next_state <= S_Send_0;
                    Shift_out <= '1';
                else next_state <= S_wait_2;
                end if;
when S_wait_2 => if (Send_Data = '1') then
                    next_state <= S_Send_0; Shift_out <= '1';
                else next_state <= S_wait_2;
                end if;
when S_Send_0 => next_state <= S_Send_1; Shift_out <= '1';
when S_Send_1 => next_state <= S_Send_2; Shift_out <= '1';
when S_Send_2 => next_state <= S_Send_3; Shift_out <= '1';
when S_Send_3 => next_state <= S_Send_4; Shift_out <= '1';
when S_Send_4 => next_state <= S_Send_5; Shift_out <= '1';
when S_Send_5 => next_state <= S_Send_6; Shift_out <= '1';
when S_Send_6 => next_state <= S_idle; Shift_out <= '1';
when others => next_state <= S_idle;
end case;
end process;
end Behavioral;

```

```

Library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Unsigned.all

```

```

entity Datapath is
  generic (dp_width: integer := 32; P_width: integer := 6);
  port (
    Data_out: out Std_Logic_vector (7 downto 0);
    Q0, Zero: out Std_Logic;
    Data_in: in Std_Logic_vector (7 downto 0);
    Start, Shift_in, Load_P, Decr_P, Add_regs, Shift_regs, Shift_out, clock: in Std_Logic
  );
end Datapath;

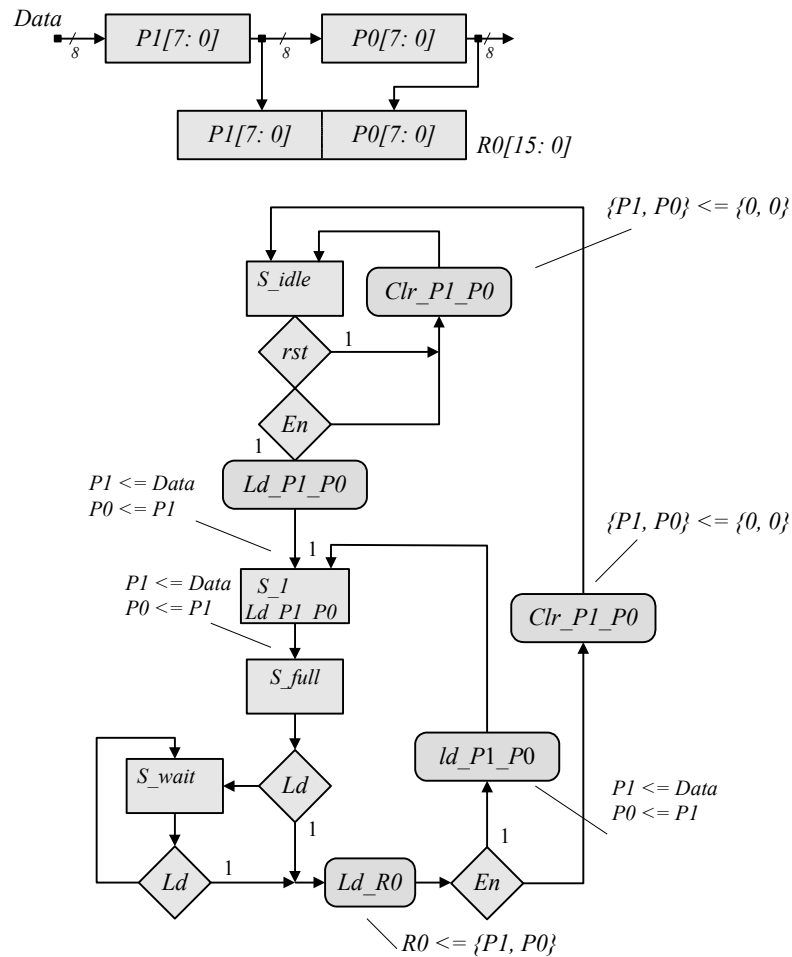
architecture Behavioral of Datapath is
  signal A, B, Q: Std_Logic_vector (dp_width - 1 downto 0);    -- Sized for datapath
  signal C: Std_Logic;
  signal P: integer;
  signal sum: Std_Logic_vector (dp_width downto 0);
begin
  Q0 <= Q(0);
  Zero <= '1' when P = 0 else '0';          -- counter is zero
  Data_out <= Q(7 downto 0);
  sum <= ('0' & A) + ('0' & B); -- Note: sum includes carry bit; bytes of Q arrive first

  process (clock) begin
    if Load_P = '1' then P <= dp_width; end if;
    if (Shift_in = '1') then -- Load pipe registers with data
      for k in 0 to dp_width-1 loop A(k) <= '0'; end loop; -- A <= 0;
      C <= '0';
      B(7 downto 0) <= B(15 downto 8); --Treat B and Q registers as a
      B(15 downto 8) <= B( 23 downto 16); -- pipeline to load data bytes
      B(23 downto 16) <= B(31 downto 24);
      B(31 downto 24) <= Q(7 downto 0);
      Q(7 downto 0) <= Q(15 downto 8);
      Q(15 downto 8) <= Q( 23 downto 16);
      Q(23 downto 16) <= Q(31 downto 24);
      Q(31 downto 24) <= Data_in;
    end if;
    if (Add_regs = '1') then
      C <= sum (dp_width);
      A < sum(dp_width-1 downto 0);
    end if;
    if (Shift_regs = '1') then -- Data_out <= (C & A & Q) srl 1;
      C <= '0';
      A <= C & A(dp_width-1 downto 1);
      Q <= A(0) & Q(dp_with-1 downto 1);
    end if;
    if (Decr_P = '1') then P <= P -1; end if;
    if (Shift_out = '1') then
      C <= '0';

      Q(7 downto 0) <= Q(15 downto 8);
      Q(15 downto 8) <= Q(23 downto 16);
      Q(23 downto 16) <= Q(31 downto 24);
      Q(31 downto 24) <= A(7 downto 0);
      A(7 downto 0) <= A(15 downto 8);
      A(15 downto 8) <= A(23 downto 16);
      A(23 downto 16) <= A(31 downto 24);
      A(31 downto 24) <= "0000000" & C;
    end if;
  end process;
end Behavioral;

```

8.41 (a) Complete ASMD chart:



(b) HDL model, test bench and simulation results for datapath unit.

Verilog

```
module Datapath_unit
```

```
(
```

```
  output reg [15: 0]  R0, input [7: 0] Data, input Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst);
```

```
  reg [7: 0]  P1, P0;
```

```
  always @ (posedge clock) begin
```

```
    if (Clr_P1_P0) begin P1 <= 0; P0 <= 0; end
```

```
    if (Ld_P1_P0) begin P1 <= Data; P0 <= P1; end
```

```
    if (Ld_R0) R0 <= {P1, P0};
```

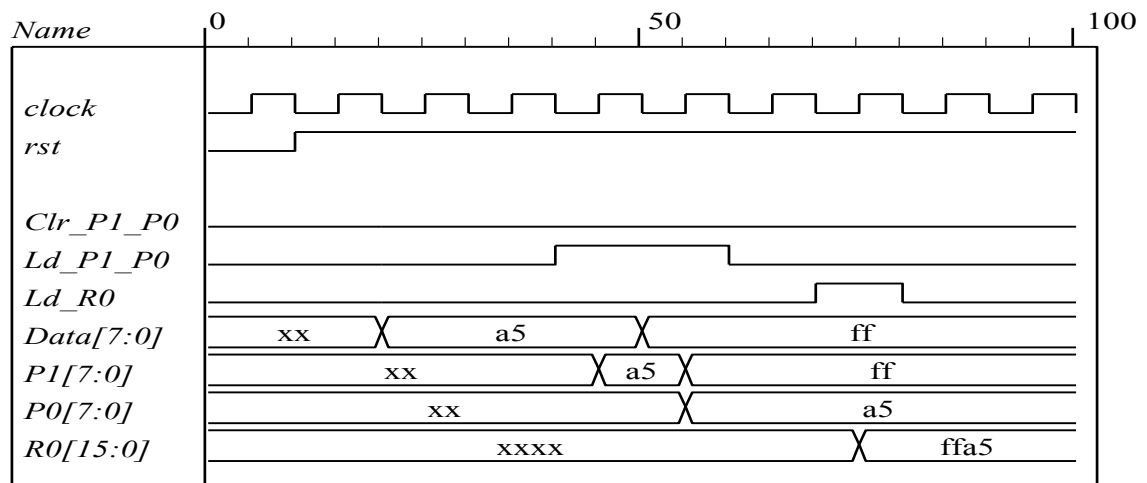
```
  end
```

```
endmodule
```

```
// Test bench for datapath
module t_Datapath_unit ();
  wire [15:0] R0;
  reg [7:0] Data;
  reg      Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst;

  Datapath_unit M0 (R0, Data, Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst);

  initial #100 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin rst = 0; #2 rst = 1; end
  initial fork
    #20 Clr_P1_P0 = 0;
    #20 Ld_P1_P0 = 0;
    #20 Ld_R0 = 0;
    #20 Data = 8'ha5;
    #40 Ld_P1_P0 = 1;
    #50 Data = 8'hff;
    #60 Ld_P1_P0 = 0;
    #70 Ld_R0 = 1;
    #80 Ld_R0 = 0;
  join
endmodule
```



VHDL

```
Library IEEE;
use IEEE.Std_Logic_1164.all;

entity Prob_8_41b_Datapath_unit_vhdl is
  port (R0: out Std_Logic_vector (15 downto 0); Data: in Std_Logic_vector (7 downto 0); Clr_P1_P0,
        Ld_P1_P0, Ld_R0, clock, rst: in Std_Logic);
end Prob_8_41b_Datapath_unit_vhdl;

architecture Behavioral of Prob_8_41b_Datapath_unit_vhdl is
  signal P1, P0: Std_Logic_vector (7 downto 0);
begin
  process (clock) begin
    if clock'event and clock = '1' then
      if (Clr_P1_P0 = '1') then
        for k in 7 downto 0 loop
          P1(k) <= '0'; P0(k) <= '0';
        end loop
      end if
    end if
  end process
```

```

        end loop;
    end if;
    if (Ld_P1_P0 = '1') then
        P1 <= Data;
        P0 <= P1;
    end if;
    if (Ld_R0 = '1') then R0 <= P1 & P0; end if;
end if;
end process;
end Behavioral;

architecture Behavioral of t_Prob_8_41b_Datapath_unit_vhdl is
    signal R0: Std_Logic_vector (15 downto 0);
    signal Data: Std_Logic_vector (7 downto 0);
    signal Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst: Std_Logic;
    component Datapath port (R0: out Std_Logic_vector (15 downto 0);
                           Data: in Std_Logic_vector (7 downto 0);
                           Clr_P1_P0, Ld_P1_P0, Ld_R0,
                           clock, rst: in Std_Logic);
begin
    M0: Datapath_unit port map (R0, Data, Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst);
    process -- clock for testbench
    begin
        t_clk <= '0';
        wait for 5 ns;
        t_clk <= '1';
        wait for 5 ns;
    end process;

    rst <= '0';
    rst <= '1' after 2 ns;
    Clr_P1_P0 <= '0' after 20 ns;
    Ld_P1_P0 <= '0' after 20 ns;
    Ld_R0 <= '0' after 20 ns;
    Data <= "0000101" <= '0' after 20 ns;;
    Ld_P1_P0 <= '1' after 40 ns;
    Data <= "11111111" after 50 ns;
    Ld_P1_P0 <= '0' after 60 ns;
    Ld_R0 <= '1' after 70 ns;
    Ld_R0 <= '0' after 80 ns;
end Behavioral;

```

(c) HDL model, test bench, and simulation results for the control unit.

Verilog

```
module Control_unit (output reg Clr_P1_P0, Ld_P1_P0, Ld_R0, input En, Ld, clock, rst);
parameter S_idle = 4'b0001, S_1 = 4'b0010, S_full = 4'b0100, S_wait = 4'b1000;
reg [3: 0] state, next_state;
```

```
always @ (posedge clock)
if (rst) state <= S_idle;
else state <= next_state;
```

```
always @ (state, Ld, En) begin
  Clr_P1_P0 = 0;           // Assign by exception
  Ld_P1_P0 = 0;
  Ld_R0 = 0;
  next_state = S_idle;
  case (state)
    S_idle: if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
           else next_state = S_idle;

    S_1: begin Ld_P1_P0 = 1; next_state = S_full; end

    S_full: if (!Ld) next_state = S_wait;
           else begin
             Ld_R0 = 1;
             if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
             else begin Clr_P1_P0 = 1; next_state = S_idle; end
           end

    S_wait: if (!Ld) next_state = S_wait;
           else begin
             Ld_R0 = 1;
             if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
             else begin Clr_P1_P0 = 1; next_state = S_idle; end
           end

    default: next_state = S_idle;
  endcase
end
endmodule
```

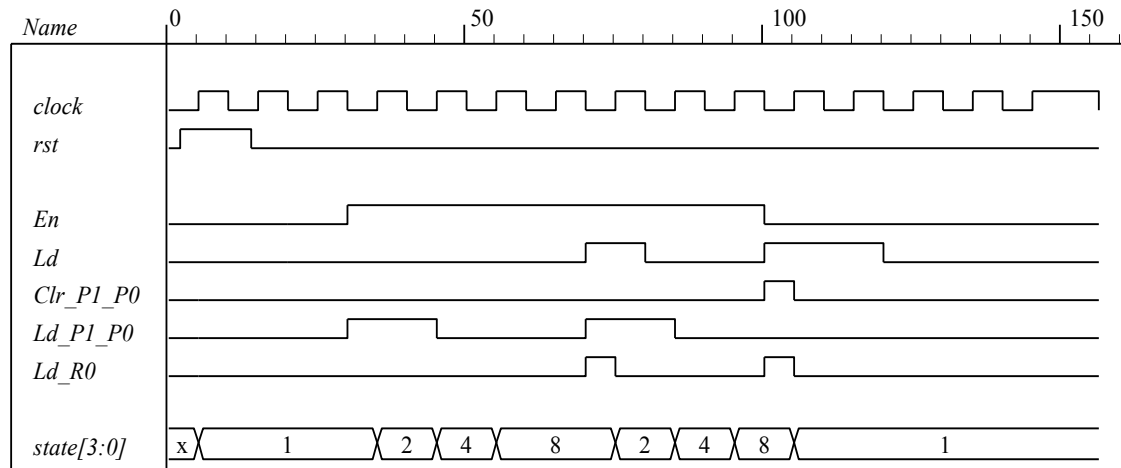
```
// Test bench for control unit
```

```
module t_Control_unit ();
wire Clr_P1_P0, Ld_P1_P0, Ld_R0;
reg En, Ld, clock, rst;
```

```
Control_unit M0 (Clr_P1_P0, Ld_P1_P0, Ld_R0, En, Ld, clock, rst);
```

```
initial #200 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial begin rst = 0; #2 rst = 1; #12 rst = 0; end
initial fork
  #20 Ld = 0;
  #20 En = 0;
  #30 En = 1; // Drive to S_wait
  #70 Ld = 1; // Return to S_1 to S_full tp S_wait
  #80 Ld = 0;
  #100 Ld = 1; // Drive to S_idle
  #100 En = 0;
  #110 En = 0;
  #120 Ld = 0;
join
```

endmodule



```
module Control_unit (output reg Clr_P1_P0, Ld_P1_P0, Ld_R0, input En, Ld, clock, rst);
parameter S_idle = 4'b0001, S_1 = 4'b0010, S_full = 4'b0100, S_wait = 4'b1000;
reg [3:0] state, next_state;
```

```
always @ (posedge clock)
if (rst) state <= S_idle;
else state <= next_state;
```

```
always @ (state, Ld, En) begin
  Clr_P1_P0 = 0;           // Assign by exception
  Ld_P1_P0 = 0;
  Ld_R0 = 0;
  next_state = S_idle;
  case (state)
    S_idle: if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
           else next_state = S_idle;

    S_1: begin Ld_P1_P0 = 1; next_state = S_full; end

    S_full: if (!Ld) next_state = S_wait;
           else begin
             Ld_R0 = 1;
             if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
             else begin Clr_P1_P0 = 1; next_state = S_idle; end
           end

    S_wait: if (!Ld) next_state = S_wait;
           else begin
             Ld_R0 = 1;
             if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
             else begin Clr_P1_P0 = 1; next_state = S_idle; end
           end

    default: next_state = S_idle;
  endcase
end
endmodule
```

VHDL**Library** IEEE;**use** IEEE.Std_Logic_1164.all;**entity** Prob_8_41c_Control_unit_vhdl **is****port** (Clr_P1_P0, Ld_P1_P0, Ld_R0: **out** Std_Logic; En, Ld, clock, rst: **in** Std_Logic);**end** Prob_8_41c_Control_unit_vhdl;**architecture** Behavioral **of** Prob_8_41c_Control_unit_vhdl **is****constant** S_idle: Std_Logic_vector (3 **downto** 0) := "0001";**constant** S_1: Std_Logic_vector (3 **downto** 0) := "0010";**constant** S_full: Std_Logic_vector (3 **downto** 0) := "0100";**constant** S_wait: Std_Logic_vector (3 **downto** 0) := "1000";**signal** state, next_state: Std_Logic_vector (3 **downto** 0);**begin****process** (clock) **begin****if** clock'event **and** clock = '1' **then****if** rst = '1' **then** state <= S_idle;**else** state <= next_state;**end if**;**end if**;**end process**;**process** (state, Ld, En) **begin**

Clr_P1_P0 <= '0'; -- Assign by exception

Ld_P1_P0 <= '0';

Ld_R0 <= '0';

next_state <= S_idle;

case (state) **is**

when S_1 => Ld_P1_P0 <= '1';
 next_state <= S_full;

when S_idle => **if** En = '1' **then**
 Ld_P1_P0 <= '1';
 next_state <= S_1;
else next_state <= S_idle;
end if;

when S_full => **if** Ld = '0' **then**
 next_state <= S_wait;
else
 Ld_R0 <= '1';
if En = '1' **then**
 Ld_P1_P0 <= '1';
 next_state <= S_1;
else
 Clr_P1_P0 <= '1';
 next_state <= S_idle;
end if;
end if;

when S_wait => **if** Ld = '0' **then** next_state <= S_wait;
else
 Ld_R0 <= '1';
if En = '1' **then**
 Ld_P1_P0 <= '1';
 next_state <= S_1;


```
        else
            Clr_P1_P0 <= '1';
            next_state <= S_idle;
        end if;
    end if;

    when others => next_state <= S_idle;
end case;
end process;
end Behavioral;
```

```

library IEEE;
use IEEE.Std_Logic_1164.all;

entity Prob_8_41b_Datapath_vhdl is
port ( R0: out Std_Logic_vector (15 downto 0);
        Data: in Std_Logic_vector (7 downto 0);
        Clr_P1_P0, Ld_P1_P0, Ld_R0, clock, rst: in Std_Logic);

end Prob_8_41b_Datapath_vhdl;

architecture Behavioral of Prob_8_41b_Datapath_vhdl is
    signal P1, P0: Std_Logic_vector (7 downto 0);
begin
    process (clock) begin
        if clock'event and clock = '1' then
            if (Clr_P1_P0 = '1') then
                for k in 7 downto 0 loop
                    P1(k) <= '0'; P0(k) <= '0';
                end loop;
            end if;
            if (Ld_P1_P0 = '1') then
                P1 <= Data;
                P0 <= P1;
            end if;
            if (Ld_R0 = '1') then R0 <= P1 & P0; end if;
        end if;
    end process;
end Behavioral;

```

(d) Integrated system Note that the test bench for the integrated system uses the input stimuli from the test bench for the control unit and displays the waveforms produced by the test bench for the datapath unit.:

```

module Prob_8_41d (output [15: 0] R0, input [7: 0] Data, input En, Ld, clock, rst);
    wire Clr_P1_P0, Ld_P1_P0, Ld_R0;

    Control_unit M0 (Clr_P1_P0, Ld_P1_P0, Ld_R0, En, Ld, clock, rst);
    Datapath_unit M1 (R0, Data, Clr_P1_P0, Ld_P1_P0, Ld_R0, clock);

endmodule

module Control_unit (output reg Clr_P1_P0, Ld_P1_P0, Ld_R0, input En, Ld, clock, rst);
    parameter S_idle = 4'b0001, S_1 = 4'b0010, S_full = 4'b0100, S_wait = 4'b1000;
    reg [3: 0] state, next_state;

    always @ (posedge clock)
        if (rst) state <= S_idle;
        else state <= next_state;

    always @ (state, Ld, En) begin
        Clr_P1_P0 = 0;           // Assign by exception
        Ld_P1_P0 = 0;
        Ld_R0 = 0;
        next_state = S_idle;
        case (state)
            S_idle:   if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
                     else next_state = S_idle;
            S_1:      begin Ld_P1_P0 = 1; next_state = S_full; end

```

```

S_full:   if (!Ld) next_state = S_wait;
          else begin
            Ld_R0 = 1;
            if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
            else begin Clr_P1_P0 = 1; next_state = S_idle; end
          end

S_wait:   if (!Ld) next_state = S_wait;
          else begin
            Ld_R0 = 1;
            if (En) begin Ld_P1_P0 = 1; next_state = S_1; end
            else begin Clr_P1_P0 = 1; next_state = S_idle; end
          end

default:  next_state = S_idle;
endcase
end
endmodule

```

```

module Datapath_unit
(
  output reg [15: 0] R0,
  input [7: 0] Data,
  input          Clr_P1_P0,
                Ld_P1_P0,
                Ld_R0,
                clock);
  reg [7: 0] P1, P0;

  always @ (posedge clock) begin
    if (Clr_P1_P0) begin P1 <= 0; P0 <= 0; end
    if (Ld_P1_P0) begin P1 <= Data; P0 <= P1; end
    if (Ld_R0) R0 <= {P1, P0};
  end
endmodule

```

```

// Test bench for integrated system
module t_Prob_8_41 ();
  wire [15: 0] R0;
  reg [7: 0] Data;
  reg En, Ld, clock, rst;

  Prob_8_41 M0 (R0, Data, En, Ld, clock, rst);

  initial #200 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial begin rst = 0; #10 rst = 1; #20 rst = 0; end
  initial fork

    #20 Data = 8'ha5;
    #50 Data = 8'hff;

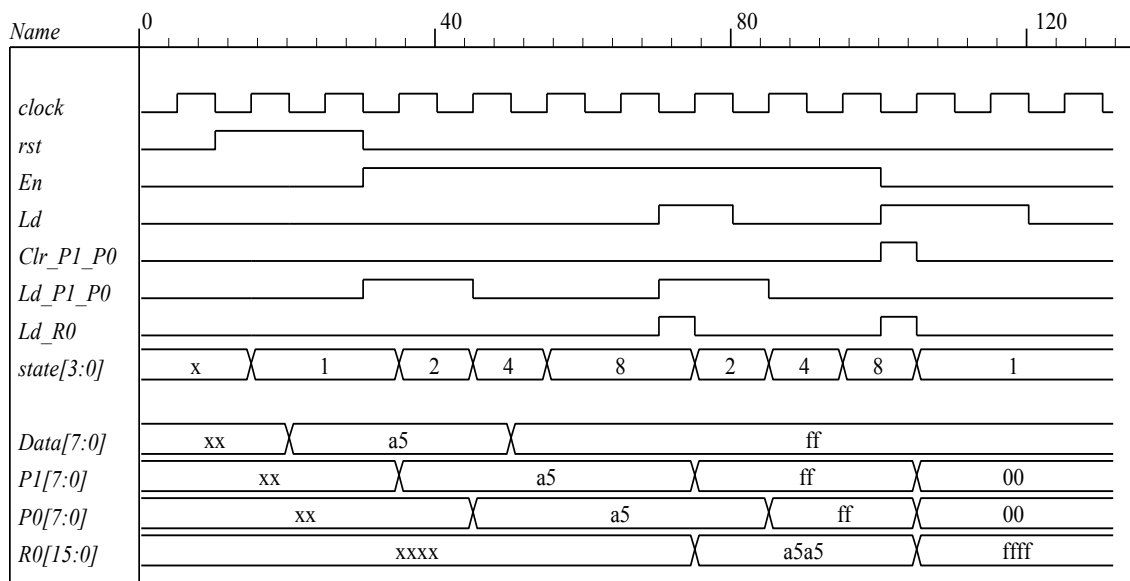
    #20 Ld = 0;
    #20 En = 0;
    #30 En = 1; // Drive to S_wait
    #70 Ld = 1; // Return to S_1 to S_full tp S_wait
    #80 Ld = 0;
    #100 Ld = 1; // Drive to S_idle
    #100 En = 0;
    #110 En = 0;
  end
endmodule

```

```

#120 Ld = 0;
join
endmodule

```



8.42 VERILOG

```

module Datapath_BEH
  #(parameter dp_width = 8, R2_width = 4)
  (
    output [R2_width -1: 0] count, output E, //output reg E,
    output Zero, input [dp_width -1: 0] data,
    input Load_regs, Shift_left, Incr_R2, clock, reset_b);
  reg [dp_width -1: 0] R1;
  reg [R2_width -1: 0] R2;

  assign E = R1[dp_width -1];
  assign count = R2;
  assign Zero = ~(| R1);

  always @ (posedge clock) begin
    // E <= R1[dp_width -1] & Shift_left;
    // if (Load_regs) begin R1 <= data; R2 <= {R2_width{1'b1}}; end
    if (Load_regs) begin R1 <= data; R2 <= {R2_width{1'b0}}; end
    if (Shift_left) R1 <= R1 << 1;
    //if (Shift_left) {E, R1} <= {E, R1} << 1;
    if (Incr_R2) R2 <= R2 + 1;
  end
endmodule

module Controller_BEH (
  output Ready,
  output reg Load_regs,
  output Incr_R2, Shift_left,
  input Start, Zero, E, clock, reset_b
);
  parameter S_idle = 0, S_1 = 1, S_2 = 2, S_3 = 3;
  reg [1:0] state, next_state;

  assign Ready = (state == S_idle);
  assign Incr_R2 = (state == S_1);
  assign Shift_left = (state == S_2);

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= S_idle;
    else state <= next_state;

  always @ (state, Start, Zero, E) begin
    Load_regs = 0;
    case (state)
      S_idle: if (Start) begin Load_regs = 1; next_state = S_1; end
        else next_state = S_idle;
      S_1: if (Zero) next_state = S_idle; else next_state = S_2;

      S_2: next_state = S_3;
      //S_3: if (E) next_state = S_1; else next_state = S_2;
      S_3: if (E) next_state = S_1; else if (Zero) next_state = S_idle; else next_state = S_2;

    endcase
  end
endmodule

```

// Integrated system

```

module Count_Ones_BEH_BEH
# (parameter dp_width = 8, R2_width = 4)
(
  output [R2_width -1: 0]    count,
  input [dp_width -1: 0]    data,
  input                      Start, clock, reset_b
);
  wire Load_regs, Incr_R2, Shift_left, Zero, E;
  Controller_BEH M0 (Ready, Load_regs, Incr_R2, Shift_left, Start, Zero, E, clock, reset_b);
  Datapath_BEH M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock, reset_b);
endmodule

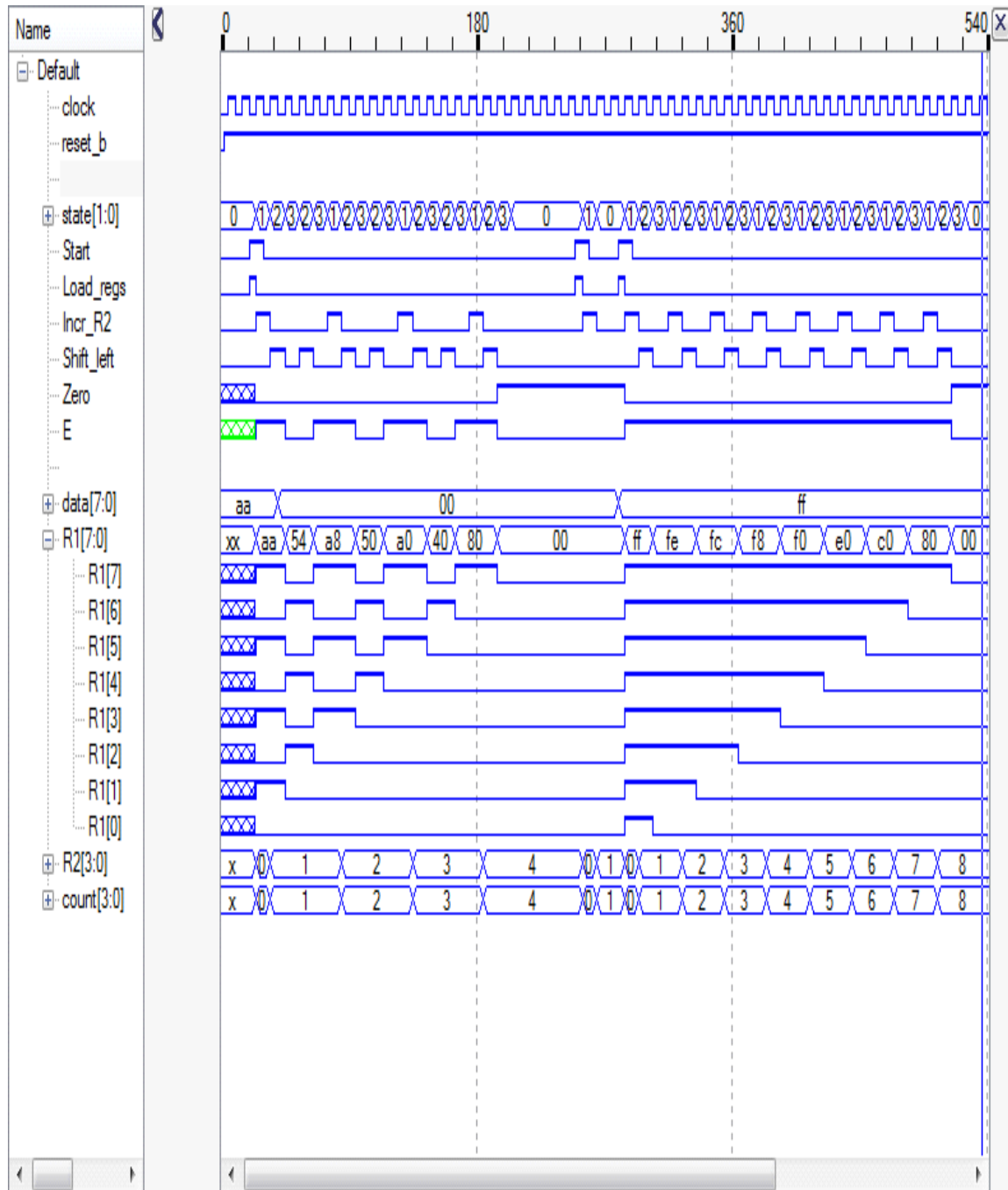
// Test plan for integrated system
// Test for data values of 8'haa, 8'h00, 8'hff.

// Test bench for integrated system

module t_count_Ones_BEH_BEH ();
parameter dp_width = 8, R2_width = 4;
wire [R2_width -1: 0] count;
reg [dp_width -1: 0] data;
reg Start, clock, reset_b;

Count_Ones_BEH_BEH M0 (count, data, Start, clock, reset_b);
initial #700 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial bebgin reset_b = 0; #2 reset_b = 1; enbd
initial fork
  data = 8'haa;      // Expect count = 4
  Start = 0;
  #20 Start = 1;
  #30 Start = 0;
  #40 data = 8'b00; // Expect count = 0
  #250 Start = 1;
  #260 Start = 0;
  #280 data = 8'hff;
  #280 Start = 1;
  #290 Start = 0;
join
endmodule

```



VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_8_42_Datapath_BEH_vhdl is
  generic (dp_width: integer := 8; R2_width: integer := 4);
  port ( count: out integer;
        E: out Std_Logic;
        Zero: out Std_Logic;
        data: in Std_Logic_vector(dp_width -1 downto 0);
        Load_regs, Shft_left, Incr_R2, clock, reset_b: in Std_Logic
      );
end Prob_8_42_Datapath_BEH_vhdl;

```

```

architecture Behavioral of Prob_8_42_Datapath_BEH_vhdl is
  signal R1: Std_Logic_vector (dp_width -1 downto 0);
  signal R2: integer;
begin
  E <= R1(dp_width -1);
  count <= R2;
  process (R1) begin
    Zero <= '1';
    for k in (dp_width -1) downto 0 loop
      if R1(k) /= '0' then Zero <= '0'; end if;
    end loop;
  end process;

```

```

process (clock) begin
  if clock'event and clock = '1' then
    if (Load_regs = '1') then
      R1 <= data;
      R2 <= 0;
    end if;

    if (Load_regs = '1') then
      R1 <= data;
      R2 <= 0;
    end if;

    if (Shft_left = '1') then
      R1 <= (R1((dp_width-2) downto 0)) & '0';
    end if;

    if (Incr_R2 = '1') then
      R2 <= R2 + 1;
    end if;
  end if;
end process; -- (clock)
end Behavioral;

```

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
library Synopsys;
use Synopsys.attributes.all;

```

```

entity Controller_BEH is
  port (Ready: out Std_Logic;
        Load_regs: out Std_Logic;

```



```

        Incr_R2, Shft_left: out Std_Logic;
        Start, Zero, E, clock, reset_b: in Std_Logic);
end Controller_BEH;

architecture Behavioral of Controller_BEH is
    type State_type is (s0, s1, s2, s3);
    attribute enum_encoding of State_type: type is "0 1 2 3";
    signal state, next_state: State_type;
begin
    Ready <= '1' when (state = s0) else '0';
    Incr_R2 <= '1' when (state = s1) else '0';
    Shft_left <= '1' when (state = s2) else '0';

    process (clock, reset_b) begin
        if (reset_b = '0') then
            state <= s0;
        elsif clock'event and clock = '1' then
            state <= next_state;
        end if;
    end process;

    process (state, Start, Zero, E) begin
        Load_regs <= '0';
        case (state) is
            when s0 => if (Start = '1') then
                        Load_regs <= '1'; next_state <= s1;
                        else next_state <= s0; end if;

            when s1 => if (Zero = '1') then
                        next_state <= s0;
                        else next_state <= s2;
                        end if;

            when s2 => next_state <= s3;

            when s3 => if (E = '1') then
                        next_state <= s1;
                        elsif (Zero = '1') then
                        next_state <= s0;
                        else next_state <= s2;
                        end if;

            when others => next_state <= s0;
        end case;
    end process;
end Behavioral;

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;
-- Integrated system
entity Count_Ones_BEH_BEH is
    generic (dp_width: integer := 8; R2_width: integer := 4);
    port (Ready: out Std_Logic; count: out Std_Logic_vector (R2_width downto 0);
        data: in Std_Logic_vector(dp_width -1 downto 0);
        Start, clock, reset_b: in Std_Logic);
end Count_Ones_BEH_BEH;

architecture Structural of Count_Ones_BEH_BEH is
    signal Load_regs, Incr_R2, Shft_left, Zero, E: Std_Logic;
    component Datapath_BEH port (count: out Std_Logic_vector (R2_width-1 downto 0); E: out Std_Logic;
        Zero: out Std_Logic; data: in Std_Logic_vector(dp_width -1 downto 0);
        Load_regs, Shft_left, Incr_R2, clock, reset_b: in Std_Logic);

```

```

end component;

component Controller_BEH port (Ready: out Std_Logic; Load_regs: out Std_Logic;
    Incr_R2, Shft_left: out Std_Logic; Start, Zero, E, clock, reset_b: in Std_Logic);
end component;

begin

    M0: Controller_BEH port map (Ready, Load_regs, Incr_R2, Shft_left, Start, Zero, E, clock, reset_b);
    M1: Datapath_BEH port map (count, E, Zero, data, Load_regs, Shft_left, Incr_R2, clock, reset_b);
end Structural;

```

8.43 Answer:

SystemVerilog

```

module divide_clock_by_4_SV (
    input logic clk, rst_b,
    output logic y
);
    typedef enum bit [3:0] {s0 = 4'b0001, s1 = 4'b0010, s2 = 4'b0100, s3 = 4'b1000} state_type_t;
    state_type_t [3:0] state, next_state;

    // Output
    assign y_out = (state == s3);

    // State transitions
    always_ff @ (posedge clk, negedge rst_b)
        if (!rst_b) state <= s0; else state <= next_state;

    always_comb @ ( state) // Next state logic
        case (state)
            s0: next_state = s1;
            s1: next_state = s2;
            s2: next_state = s3;
            s3: next_state = s0; // case is complete – no default
        endcase
endmodule

```

VHDL

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_arith.all;

entity Prob_8_43_divide_clock_by_4_SV_vhdl is
    port (clk, rst_b: in Std_Logic; y: out Std_Logic);
end Prob_8_43_divide_clock_by_4_SV_vhdl;

architecture Behavioral of Prob_8_43_divide_clock_by_4_SV_vhdl is
    subtype State_type is Std_Logic_vector (3 downto 0);
    constant s0: State_type := "0001";
    constant s1: State_type := "0010";
    constant s2: State_type := "0100";
    constant s3: State_type := "1000";

    signal state, next_state: State_type;
begin

    -- Output

```

```

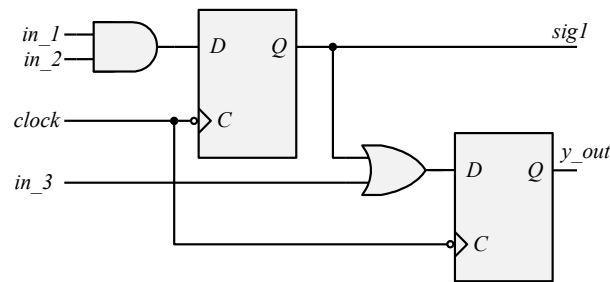
y <= '1' when (state = s3) else '0';

-- State transitions
process (clk, rst_b) begin
    if rst_b = '0' then state <= s0;
        elsif clk'event and clk = '1' then state <= next_state; end if;
    end process;

-- Next state logic

process (state) begin
    next_state <= s0;
    case (state) is
        when s0 => next_state <= s1;
        when s1 => next_state <= s2;
        when s2 => next_state <= s3;
        when s3 => next_state <= s0;
        when others => next_state <= s0;
    end case;
end process;
end Behavioral;

```

8.44 Answer:

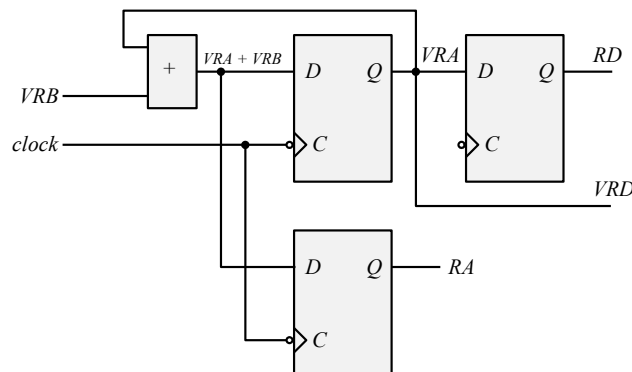
8.45 Answer: Yes, the result will be different in each case. In the first case, the result will be $x = 1, y = 0$, while in the second case, the result will be $x = 1, y = 1$.

8.46 Answer: The result will be same in both the cases.

8.47 Answer: The possible bit patterns of the case items are not exhausted, and there is no **default** assignment.

8.48 Answer: It is not an acceptable coding practice to assign value to the same signal (q) in multiple procedural blocks. Synthesis tools will reject such a description; SystemVerilog does not allow such code.

8.49 Answer: The latch responds to an event of *enable*. If *enable* is asserted, q gets the value of *data*. There is no change in q until a subsequent activating event of *enable* assigns a different value to q . Meanwhile, events of *data* are ignored because the sensitivity list does not include *data*. The behavior does not match that of a transparent latch. To repair the model, include *data* in the sensitivity list of the procedural block.

8.50 Answer:**8.51 Answer**

```
typedef enum {idle, start, finish, delay} state_FSM;
state_FSM current_state, next_state;
```

8.52 Answer:

```
assign y = B ? C : A ;
```

- 8.53** `module up_down_counter (`
 output reg [7:0] out, // Output of the counter
 input wire up_down, // up_down control for counter
 input wire clk, // clock input
 input wire reset // reset input
`);`

 always_ff @(posedge clk)
 if (reset) **begin** // active high reset
 out <= 8'b0;
 end else if (up_down) **begin**
 out ++;
 end else begin
 out --;
 end
`endmodule`
- 8.54** `module Prob_8_54_sv (`
 input logic [7:0] Data_in,
 input logic enable,
 output logic [7:0] Data_out
`);`
 always_latch (enable, Data_in) **begin**
 if (enable) Data_out <= Data_in;
 end
`endmodule`
- 8.55** `module 4_16_decoder (`
 input wire [3:0] binary_in, // 4 bit binary input
 output wire [15:0] decoder_out, //16-bit out
 input wire enable // Enable for the decoder
`);`

 assign decoder_out = (enable) ? (1 << binary_in) : 16'b0 ;
`endmodule`
- 8.56** `module Prob_8_56_sv (`
 input logic [3:0] x_in,
 output logic [3:0] y_out
`);`
 always_comb **begin**
 if (x_in(3)) y_out <= "1000";
 else if (x_in(2)) y_out <= "0100";
 else if (x_in(1)) y_out <= "0010";
 else if (x_in(0)) y_out <= "0001";
 else y_out <= "0000";
 end
`endmodule`

```

8.57 module Prob_8_57_sv (
    input logic clk, rst_b,
    output logic y_out
);
    typedef enum [2:0] {s0, s1, s2, s3, s4} state_type_t
    state_type_t state, next_state;

    always_ff (posedge clk, negedge_reset_b_)
        if (!rst_b) state <= s0;
        else state <= next_state;

    always_comb begin
        case (state)
            s0: next_state = s1;
            s1: next_state = s2;
            s2: next_state = s3;
            s3: next_state = s4;
            s4: next_state = s0;
            default: next_state = s0;
        endcase
    end

    assign y_out = (state == s0);
endmodule

```

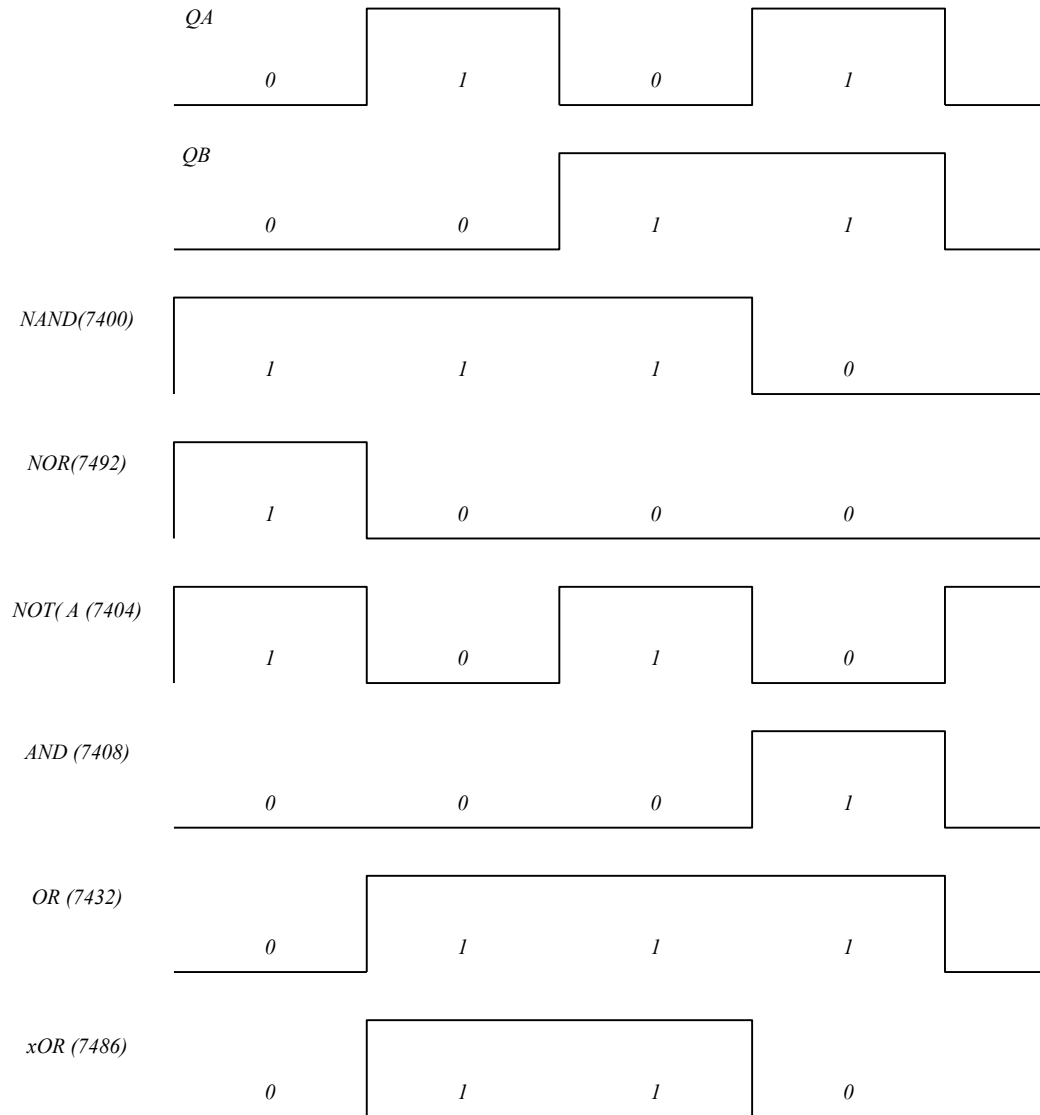
8.58 Answer:

```
enum logic [3:0] { s0 = 4'b0001, s1 = 4'b0010, s2 = 4'b0100, s3 = 4'b1000} state;
```


9.2 Truth table:

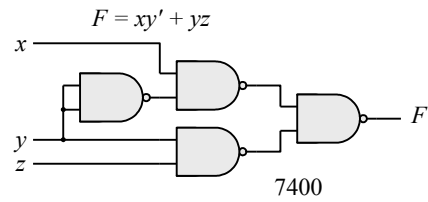
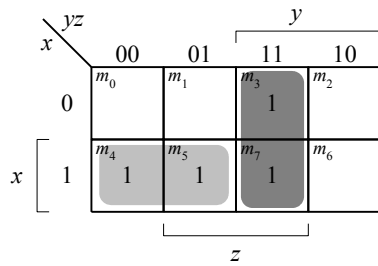
Inputs							
A	B	NAND	NOR	NOT(A)	AND	OR	XOR
0	0	1	0	1	0	0	0
0	1	1	1	0	0	1	1
1	0	1	1	1	0	1	1
1	1	0	1	0	1	1	0

Waveforms:

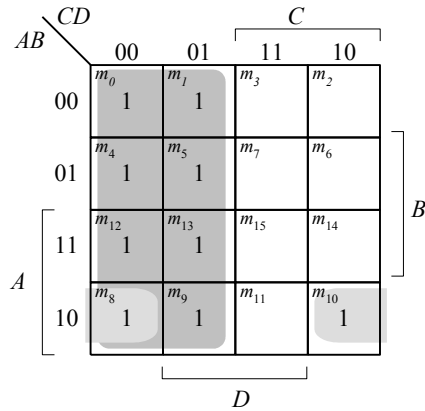


9.3

Logic Diagram

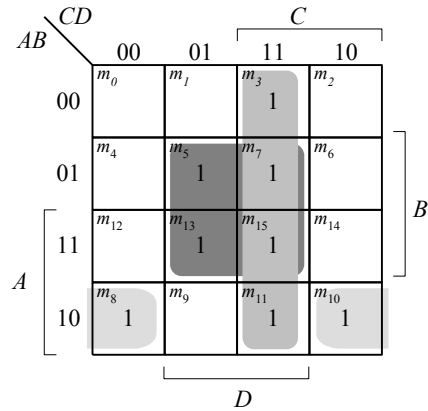


Boolean Functions:



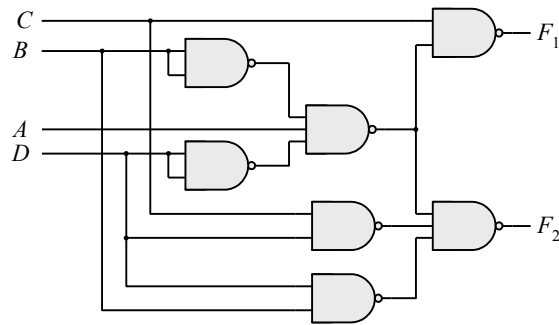
$$F_1 = C' + AB'D'$$

Boolean Functions:



$$F_2 = BD + CD + AB'D'$$

2 ICs: 7400, 7410

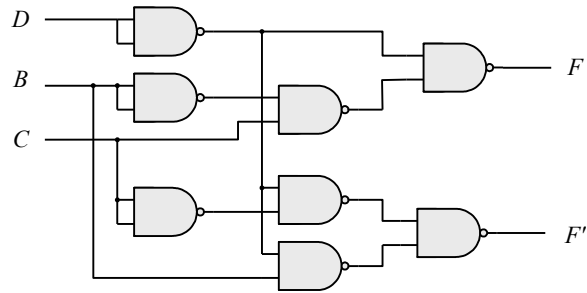


Complement:

AB \ CD		C			
		00	01	11	10
A	00	m_0 0	m_1 1	m_3 1	m_2 1
	01	m_4 0	m_5 1	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 1	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 1	m_{11} 1	m_{10} 1

$$F = D + B'C$$

$$F' = C'D' + BD'$$

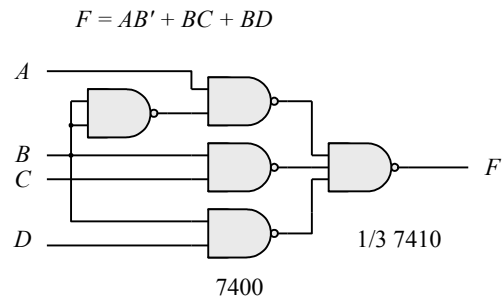


2 - 7400 ICs

9.4

Design Example:

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1	m_3	m_2
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12}	m_{13} 1	m_{15} 1	m_{14} 1
	10	m_8 1	m_9 1	m_{11} 1	m_{10} 1

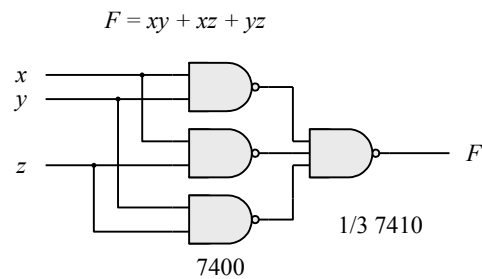


7400

1/3 7410

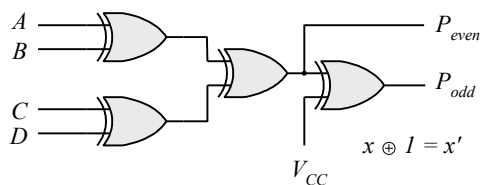
Majority Logic

yz \ x		y			
		00	01	11	10
x	0	m_0	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7 1	m_6 1



7400

1/3 7410



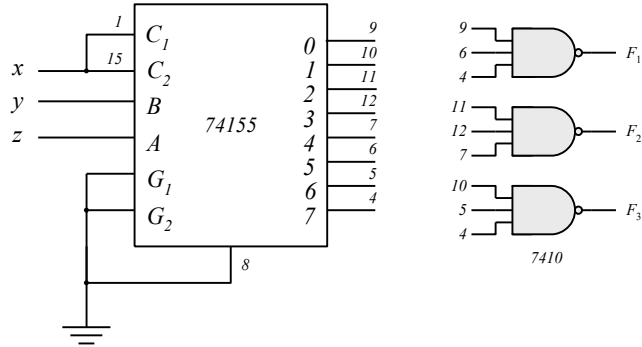
$$x \oplus 1 = x'$$

Decoder Implementation

$$F_1 = xz + x'y'z' = \Sigma(0, 5, 7)$$

$$F_2 = x'y + xy'z' = \Sigma(2, 3, 4)$$

$$F_3 = xy + x'y'z = \Sigma(1, 6, 7)$$

**9.5** Gray code to Binary – See solution to Prob. 4.7.

9's complements – See solution to Prob. 4.18.

$$w = A'B'C'$$

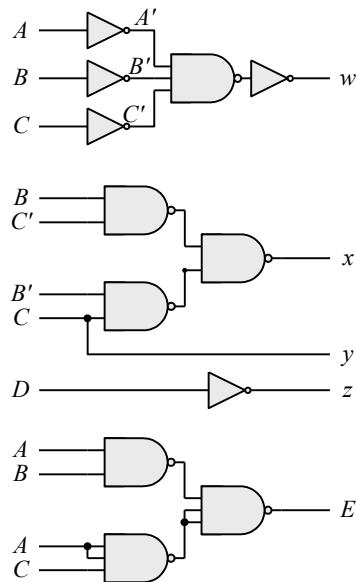
$$x = BC' + B'C$$

$$y = C$$

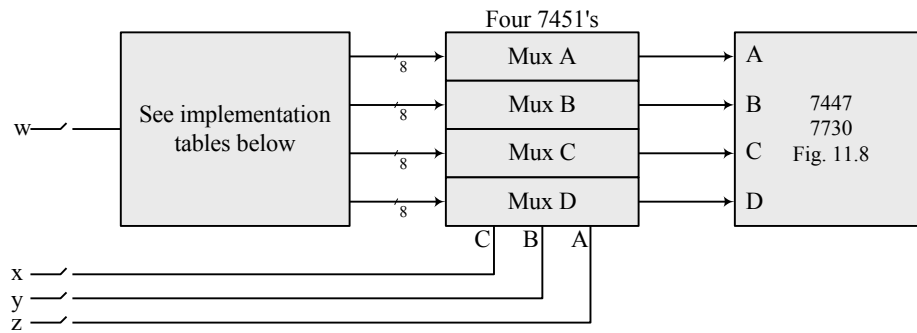
$$z = D'$$

$$E = AB + AC$$

3 ICs: 7400, 7404, 7410



9.6

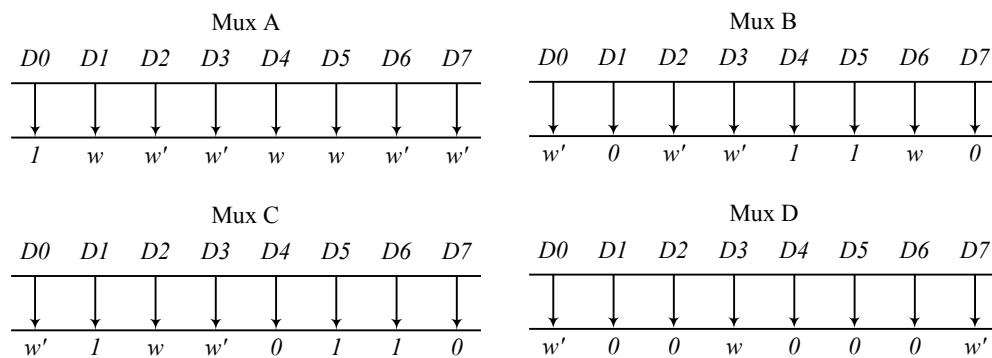


$$A = \sum (0, 2, 3, 6, 7, 8, 9, 12, 13)$$

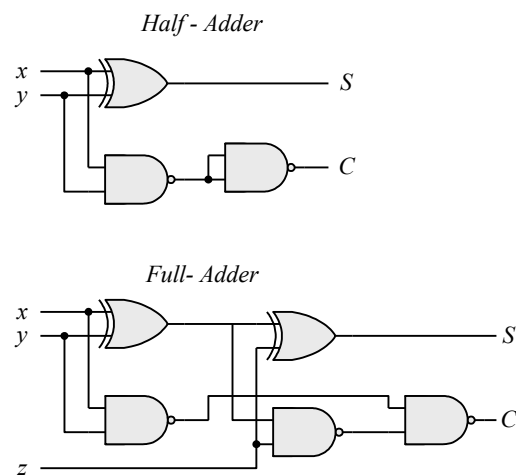
$$B = \sum (0, 2, 3, 4, 5, 12, 13, 14)$$

$$C = \sum (0, 1, 3, 5, 6, 9, 10, 13, 14)$$

$$D = \sum (0, 7, 11)$$



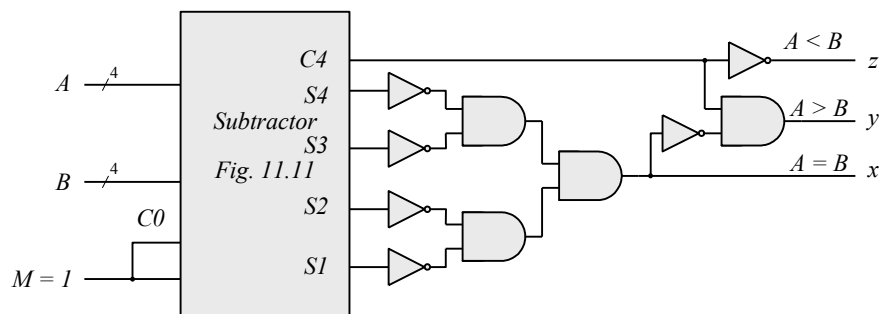
9.7



Parallel adder - See circuit of Fig. 9.10.

Adder-subtractor - See circuit of Fig. 9.11.

Operation	Inputs				Outputs		
	M	A	B	C ₀	S	C ₄	
9 + 5 = 14	0	1001	0101	0	1110	0	sum < 15
9 + 9 = 19 = 16 + 2	0	1001	1001	0	0010	1	sum > 15
9 + 15 = 24 = 16 + 8	0	1001	1111	0	1000	1	sum > 15
9 - 5 = 4	1	1001	0101	1	0100	1	A > B
9 - 9 = 0	1	1001	1001	1	0000	1	A = B
9 - 15 = -6	1	1001	1111	1	1010	0	A < B

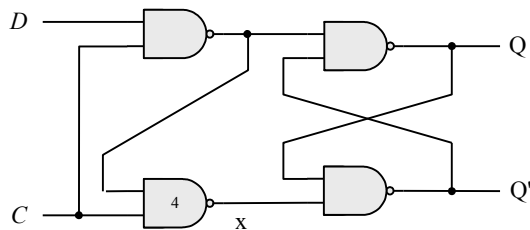


9.8 SR Latch: See Fig. 5.4.

D Latch:

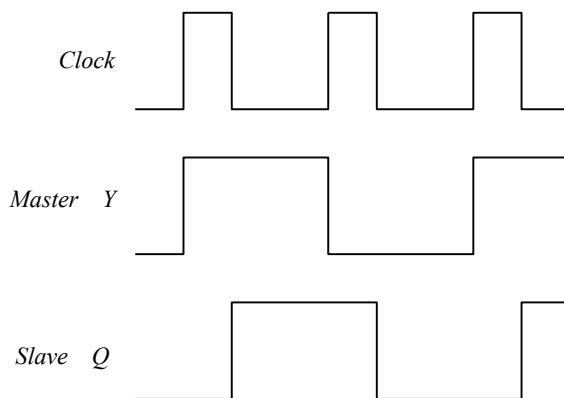
Let $CP = C$, x = output of gate 4.

$$x = [(DC)'C]' = (D'C)'$$

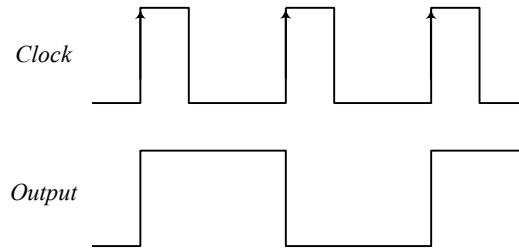


Master-Slave D Flip-Flop: The circuit is as in Fig. 5.9.

The oscilloscope display:



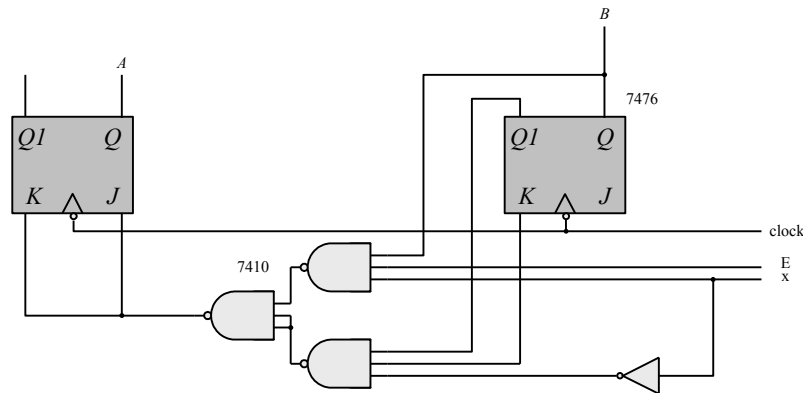
Edge-Triggered D Flip-Flop: Circuit is shown in Fig. 5.10.



IC Flip-Flops:

Connect all inputs to toggle switches, the clock to a pulser, and the outputs to indicator lamps.

9.9 Up-Down Counter with Enable:



$$J_B = K_B = E \text{ (Complement B when } E = 1\text{)}$$

$$J_A = K_A = E (Bx + B'x')$$

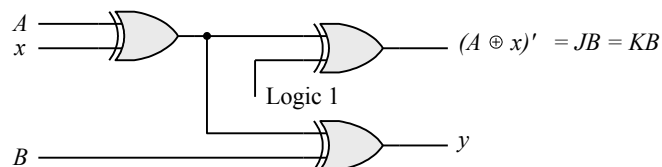
Complement A when $E = 1$ and:

$B = 1$ when $x = 1$ (Count up)

$B = 0$ when $x = 0$ (Count down)

State Diagram:

$$\begin{aligned} J_A &= B & J_B &= Ax + A'x' = (A \oplus x)' & Y &= A \oplus B \oplus x \\ K_A &= B' & K_B &= Ax + A'x' = (A \oplus x)' \end{aligned}$$



Design of Counter: ABCD

$$\begin{array}{ll}
 J_A = K_A = B(CD) & 0000 \rightarrow 0101 \rightarrow 090 \\
 J_B = K_B = CD & 1000 \rightarrow 1001 \rightarrow 1010 \\
 J_C = D & K_C = AD \\
 J_D = K_D = 1 &
 \end{array}$$

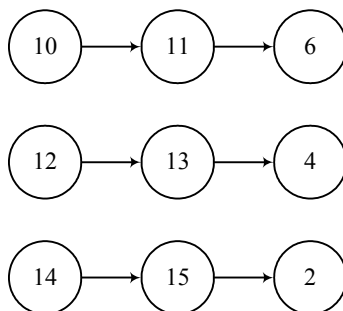
9.10 Ripple counter: See Fig. 6.8

Down counter: Either take outputs from Q' outputs or connect complement Q' to next clock input.

Synchronous counter: See Fig. 6.12.

BCD counter: See solution to Prob. 6.19.

Unused states:



Binary counter with parallel load:

Connect QA and QD through a NAND gate to the load. See Fig. 6.15.

9.11 Ring counter:

See Fig. 6.17(a).

States of register:

QA	QB	QC	QD
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Switch-tail ring counter: See Fig. 6.18(a). Connect $(QD)'$ at pin 12 to the serial input at pin 4. State sequence as in Fig. 6.18(b).

Feedback shift register: Serial input = $QC \oplus QD$ (Use 7486).

Sequence of states:

QA	QB	QC	QD
1	0	0	0
0	1	0	0
0	0	1	0
1	0	0	1
1	1	0	0

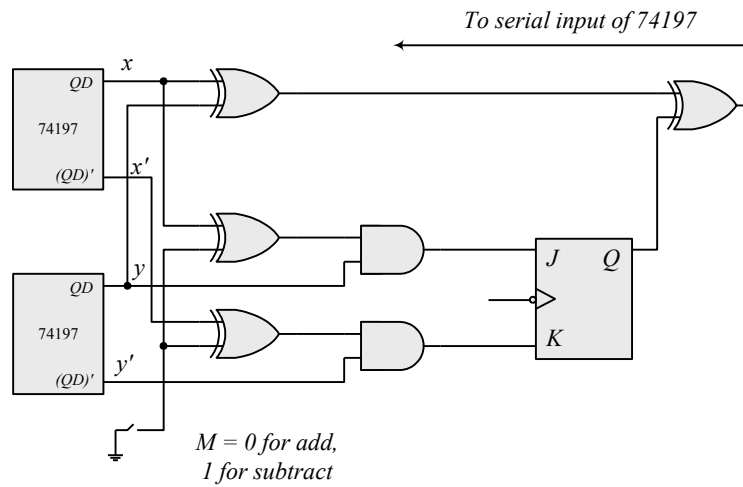
Bidirectional shift register with parallel load:

Function table:

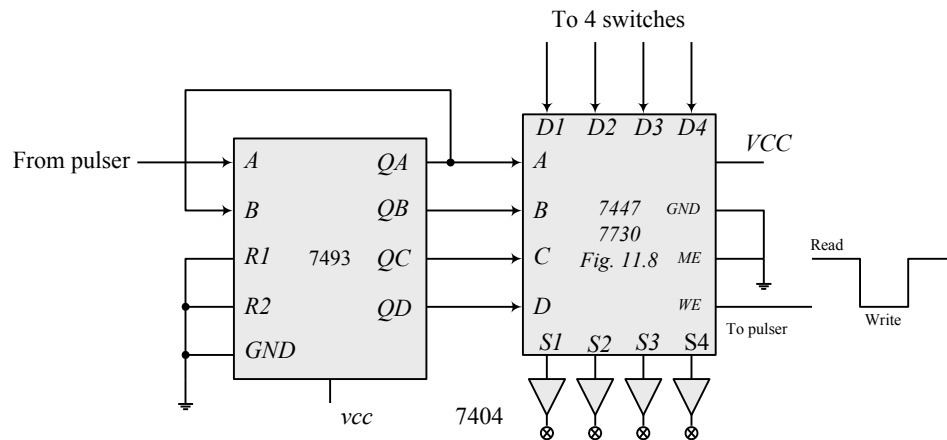
74195			74157		Function
Clear	Clock	SH/LD	STROBE	SELECT	
0	x	x	x	x	Async clear
1	$\uparrow$	1	x	x	Shift right ($QA \rightarrow QB$)
1	$\uparrow$	0	0	1	Shift left (Select B)*
1	$\uparrow$	0	0	0	Parallel Load (Select A)
1	$\uparrow$	0	1	x	Synchronous clear

* B inputs come from QA-QD shifted by one position.

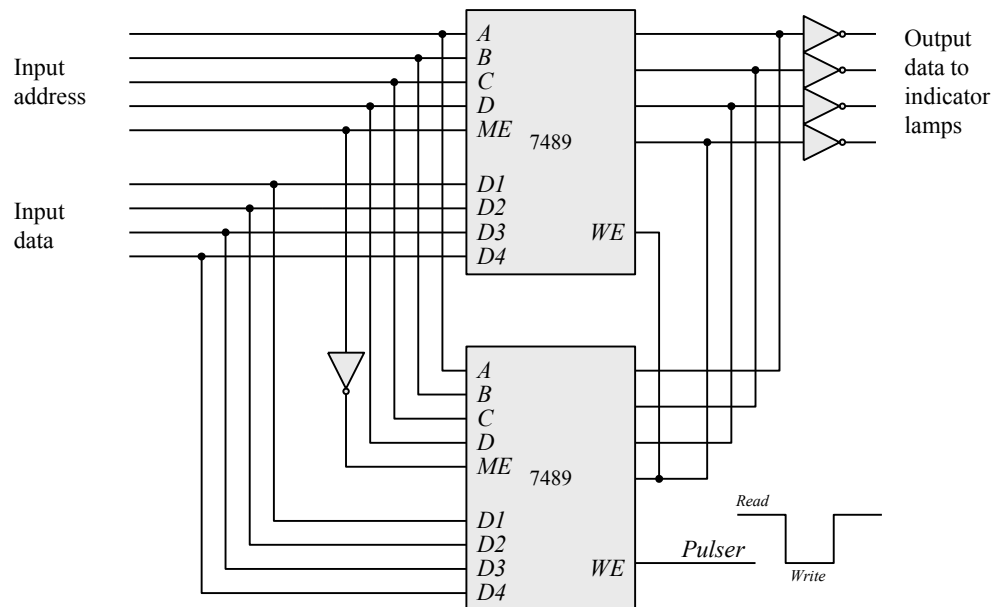
9.12



9.13 Testing the RAM:



Memory Expansion:



9.14 Circuit Analysis – Answers to questions:

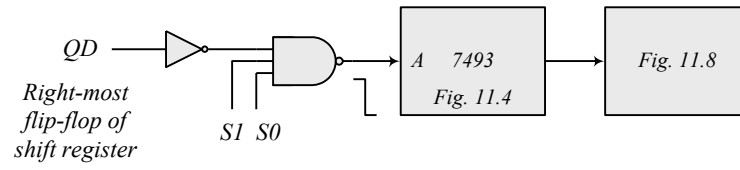
- 1) Resets to 0 the two 74194 ICs, the two D flip-flops, and the start SR latch. This makes $SIS0 = 11$ (parallel load).
- 2) The start switch sets the SR latch to 1. The clock pulses load 0000_0001 into the 8-bit register. If the start switch stays on, the register never clears to all 0s when $SIS0 = 11$ (right-most QD stays on).
- 3) Pressing the pulser makes $SIS0 = 10$ and the light shifts left. When QC becomes 1, the start SR latch is cleared to 0. When QA of the left 74194 becomes 1, it changes $S1$ to 0 (through the PR input) and $S0$ to 1 (through the CLR input. with $SIS0 = 01$, the single light shifts right).
- 4) If the pulser is pressed while the light is moving to the left or the right, $SIS0$ becomes 11 and all 0s are loaded into the register in parallel. The light goes off.
- 5) When the right-most QD becomes a 1, $SIS0$ changes from 01 (shift right) to 11 (parallel load). If the pulser is pressed before the next clock pulse, $SIS0$ goes to 10 (shift left). If not pressed, an all 0s value is loaded into the register in parallel. (Provided the start switch is in the logic 1 position.)

Lamp Ping-Pong

Add a left pulser. Three wire changes to the D flip-flop on the left:

- 1) Connect the clock input of the flip-flop to the pulser.
- 2) Connect the D input to the QA of the left 74197
- 3) Connect the input of the inverter (that goes to PR) to ground.

Counting the Losses



9.15 Clock Pulse Generator

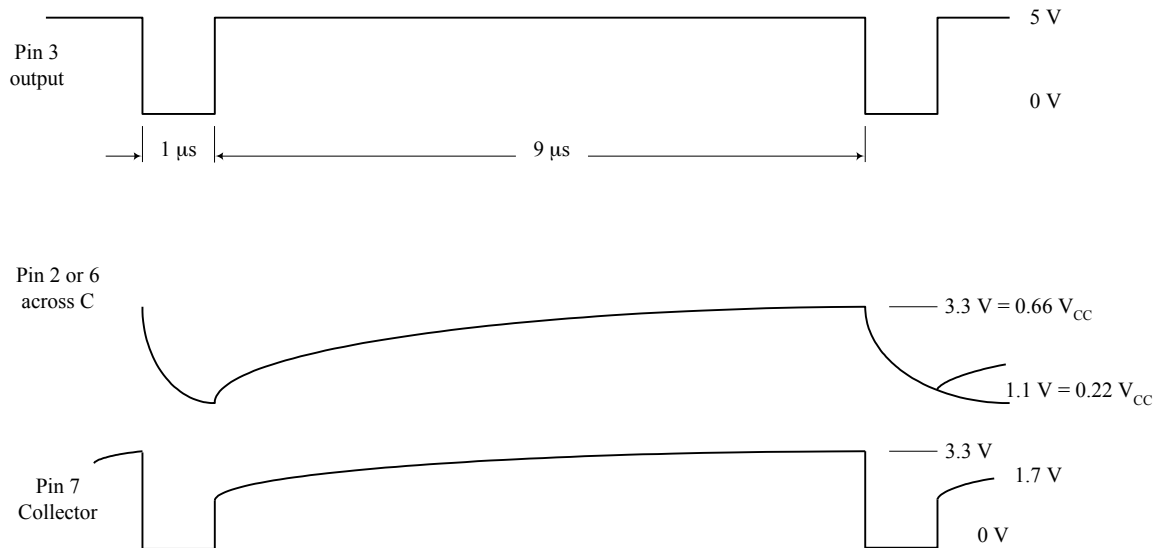
$$t_L = 0.693 R_B C = 10^{-6}$$

$$R_B = 10^{-6} / (0.693 \times 0.001 \times 10^{-6}) = 10^3 / 0.693 = 1.44 \text{ K}\Omega \text{ (Use } R_B = 1.5 \text{ K}\Omega)$$

$$t_H/t_L = 0.693 (R_A + R_B)C / (0.693 R_B C) = (R_A + R_B) / R_B = 9 / 1 = 9$$

$$9 R_B = R_A + R_B \quad R_A = 8 R_B = 8 \times 1.5 \text{ K}\Omega = 12 \text{ K}\Omega$$

Oscilloscope Waveforms (Actual results may be off by $\pm 20\%$.)



Variable Frequency Pulse Generator:

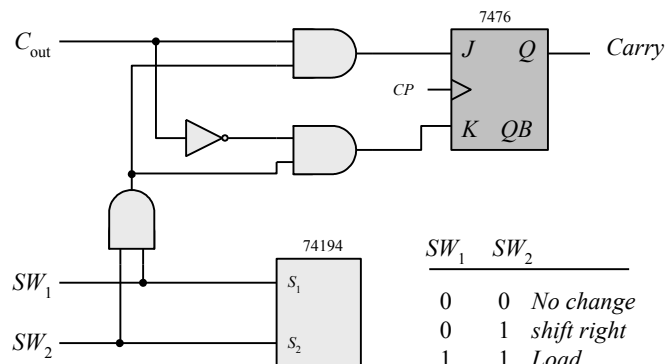
$$20 \text{ KHz: } 10^{-3} / 20 = 0.05 \times 10^{-3} = 50 \mu\text{s}$$

$$100 \text{ KHz: } 10^{-3} / 100 = 10^{-5} = 10 \mu\text{s}$$

$$t_H = 49 \mu\text{s: } (R_A + R_P + R_B) / R_B = 49 / 1 = 49$$

$$R_P = 48 R_B - R_A = 48 \times 1.5 - 11 = 60 \text{ K}\Omega$$

9.16 Control of Register



Checking the Circuit:

	Carry	Register
Initial	0	0 0 0 0
+ 0110	0	0110
+ 1110	1	0100
+ 1101	1	0001
+ 0101	0	0110
+ 0011	0	1001

Circuit Operation:

Address	Carry	RAM	
0	0	0110	RAM Value
1	0	0110	RAM + Register
2	0	0011	Shift Register
3		1110	RAM Value
4	1	0001	RAM + Register
5	1	1000	SHIFT
6		1101	RAM Value
7	1	0101	RAM + Register
8	1	1010	SHIFT
9		0101	RAM Value
10	0	1111	RAM + Register
11	0	0111	SHIFT
12		0011	RAM Value
13	0	1010	RAM + Register
14	0	0101	SHIFT

9.17 Multiplication Example (11 x 15 = 165)

Multiplicand B = 1111

		C	A	Q	P
Initial:	$T_1 = 1$	0	0000	1011	0000
$T_2 = 1$	Add B; $P \leq P+1$		1111		
		0	1111	1011	0001
$T_3 = 1$	Shift CAQ	0	0111	1101	0001
$T_2 = 1$	Add B; $P \leq P+1$		1111		
		1	0110	1101	0010
$T_3 = 1$	Shift CAQ	0	1011	0110	0010
$T_2 = 1$	$P \leq P+1$	0	1011	0110	0011
$T_3 = 1$	Shift CAQ	0	0101	1011	0011
$T_2 = 1$	Add B; $P \leq P+1$		1111		
		1	0100	1011	0100
$T_3 = 1$	Shift CAQ	0	1010	0101	0100
$T_0 = 1$	(Because $P_C = 1$)		1010	0101	= Product

Data Processor Design

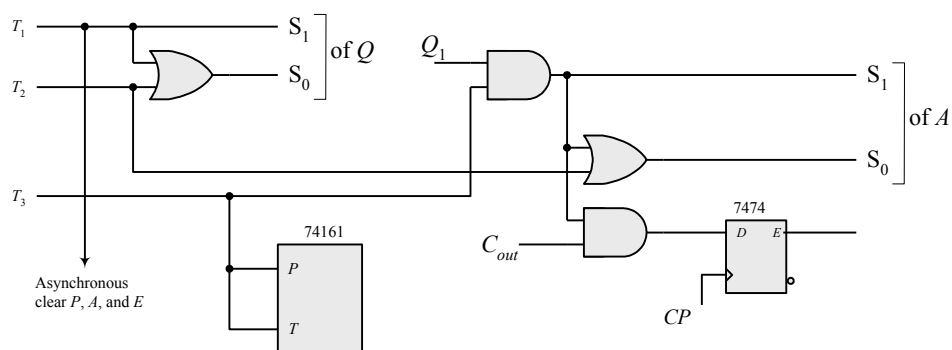
Load Q	Load A	Shift AQ	Register Q	Register A
T_1	T_2Q_1	T_3	$S_1 S_0$	$S_1 S_0$
0	0	0	0 0	0 0
1	0	0	1 1	0 0
0	1	0	0 0	1 1
0	0	1	0 1	0 1

$$S_1(Q) = T_1$$

$$S_0(Q) = T_1 + T_3$$

$$S_1(A) = T_2Q_1$$

$$S_0(A) = T_2Q_1 + T_3$$

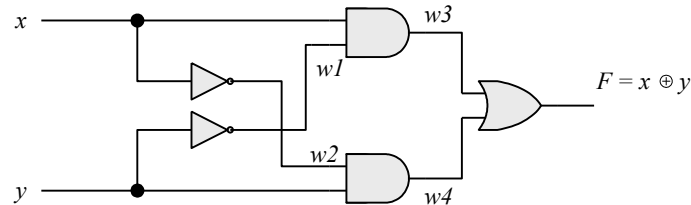


Design of Control: See Section 8.8.

SOLUTIONS FOR SECTION 9.19

Supplement to Experiment 2:

(a)



Initially, with $xy = 00$, $w1 = w2 = 1$, $w3 = w4 = 0$ and $F = 0$. $w1$ should change to 0 10ns after xy changes to 01. $w4$ should change to 1 20 ns after xy changes to 01. F should change from 0 to 1 30 ns after $w4$ changes from 0 to 1, i.e., 50 ns after xy changes from 00 to 01. $w3$ should remain unchanged because $x = 0$ for the entire simulation.

(b)

```
`timescale 1ns/1ps
```

```
module Prob_3_33 (output F, input x, y);
  wire w1, w2, w3, w4;
```

```
  and #20 (w3, x, w1);
  not #10 (w1, x);
  and #20 (w4, y, w2);
  not #10 (w2, y);
  or #30 (F, w3, w4);      A
```

```
endmodule
```

```
module t_Prob_3_33 ();
  reg x, y;
  wire F;
```

```
  Prob_3_33 M0 (F, x, y);
```

```
  initial #200 $finish;
```

```
  initial fork
```

```
    x = 0;
```

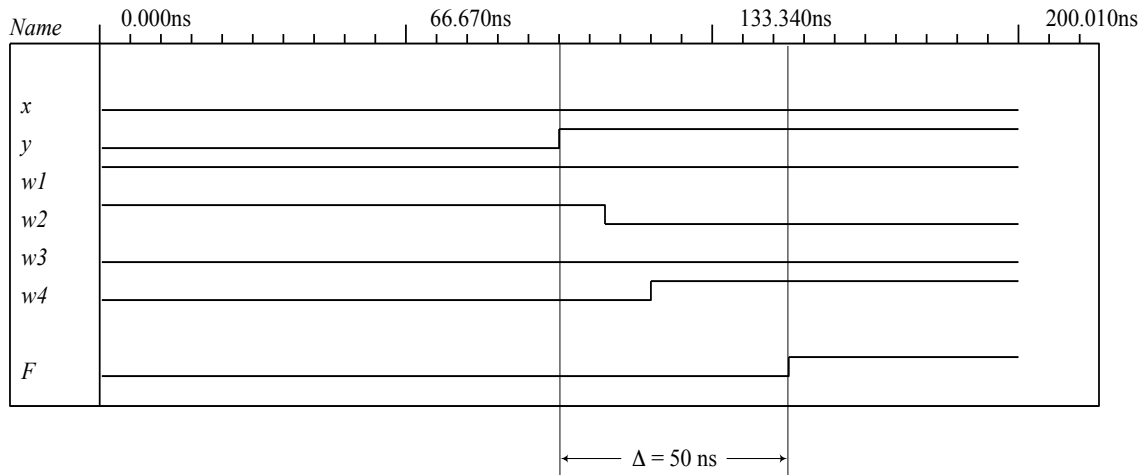
```
    y = 0;
```

```
    #100 y = 1;
```

```
  join
```

```
endmodule
```

(c) To simulate the circuit, it is assumed that the inputs $xy = 00$ have been applied sufficiently long for the circuit to be stable before $xy = 01$ is applied. The testbench sets $xy = 00$ at $t = 0$ ns, and $xy = 1$ at $t = 100$ ns. The simulator assumes that $xy = 00$ has been applied long enough for the circuit to be in a stable state at $t = 0$ ns, and shows $F = 0$ as the value of the output at $t = 0$. The waveforms show the response to $xy = 01$ applied at $t = 100$ ns.



Supplement to Experiment 4:

(a)

```
// Gate-level description of circuit in Fig. 4-2
module Circuit_of_Fig_4_2 (
  output F1, F2,
  input A, B, C);
  wire T1, T2, T3, F2_not, E1, E2, E3;
  orG1 (T1, A, B, C);
  and G2 (T2, A, B, C);
  and G3 (E1, A, B);
  and G4 (E2, A, C);
  and G5 (E3, B, C);
  orG6 (F2, E1, E2, E3);
  not G7 (F2_not, F2);
  and G8 (T3, T1, F2_not);
  orG9 (F1, T2, T3);
endmodule

module t_Circuit_of_Fig_4_2;
  reg [2:0] D;
  wire F1, F2;
  parameter stop_time = 100;

  Circuit_of_Fig_4_2 M1 (F1, F2, D[2], D[1], D[0]);

  initial # stop_time $finish;
  initial begin
    // Stimulus generator
    D = 3'b000;
    repeat (7)
      #10 D = D + 1'b1;
    end
  end

  initial begin
    $display ("A B C F1 F2");
    $monitor ("%b %b %b %b %b", D[2], D[1], D[0], F1, F2);
  end
endmodule
```

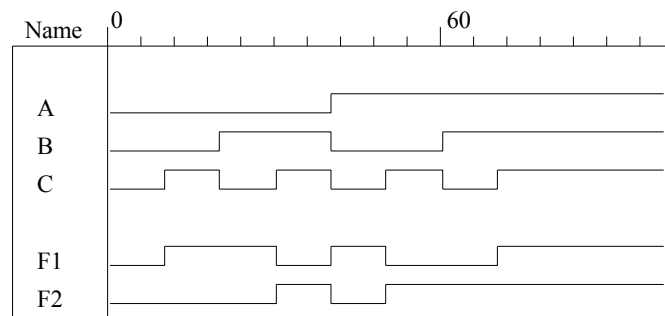


```

/*
A   B   C   F1  F2
0   0   0   0   0
0   0   1   1   0
0   1   0   1   0
0   1   1   0   1
1   0   0   1   0
1   0   1   0   1
1   1   0   0   1
1   1   1   1   1
*/

```

The simulation results demonstrate the behavior of a full adder, with F1 = sum, and F2 – carry.



(b)

```

// 3-INPUT MAJORITY DETECTOR CIRCUIT.
// Circuit implements  $F = xy + xz + yz$ .
module Majority_Detector (output F, input x, y, z);
    wire w1, w2, w3;
    nand    n1(w1, x, y),
            n2(w2, x, z),
            n3(w3, y, z),
            n4(F, w1, w2, w3);
endmodule

// Test bench
// Treating inputs to majority detector as a vector, reg [2:0]D; //D[2] = x, D[1] = y, D[0] = z. wire F;
module t_Majority_Detector ();
    wire F;
    reg [2: 0] D;
    wire x = D[2];
    wire y = D[1];
    wire z = D[0];

    Majority_Detector M0 (F, x, y, z);

    initial #100 $finish;
    initial $monitor ($time, "xyz = %b F = %b", D, F);

```

```

initial begin
    D = 0;
    repeat (7)
        #10 D = D + 1;
    end
endmodule

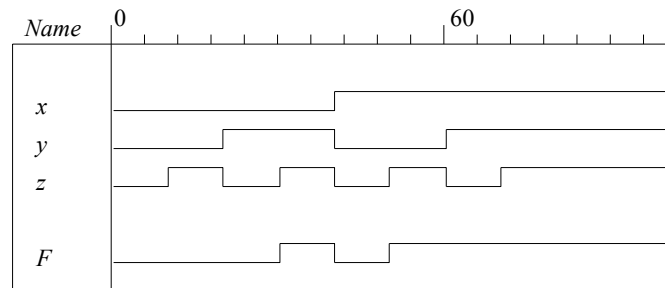
```

Simulation results:

```

0 xyz = 000 F = 0
10 xyz = 001 F = 0
20 xyz = 010 F = 0
30 xyz = 011 F = 1
40 xyz = 100 F = 0
50 xyz = 101 F = 1
60 xyz = 110 F = 1
70 xyz = 111 F = 1

```



Supplement to Experiment 5: See the solution to Prob. 4.42.

Supplement to Experiment 7:

(a)

```

//BEHAVIORAL DESCRIPTION OF 7483 4-BIT ADDER,
module Adder_7483 (
    output S4, S3, S2, S1, C4,
    input A4, A3, A2, A1, B4, B3, B2, B1, C0, VCC, GND
);
    // Note: connect VCC and GND to supply1 and supply0 in the test bench
    wire [4: 1] sum;
    wire [4: 1] A = {A4, A3, A2, A1};
    wire [4: 1] B = {B4, B3, B2, B1};
    assign S4 = sum[4];
    assign S3 = sum[3];
    assign S2 = sum[2];
    assign S1 = sum[1];

    assign {C4, sum} = A + B + C0;
endmodule

```

```

module t_Adder_7483 ();
  wire S4, S3, S2, S1, C4;
  wire A4, A3, A2, A1, B4, B3, B2, B1;
  reg C0;
  supply1 VCC;
  supply0 GND;
  reg [4:1] A, B;
  assign A4 = A[4];
  assign A3 = A[3];
  assign A2 = A[2];
  assign A1 = A[1];
  assign B4 = B[4];
  assign B3 = B[3];
  assign B2 = B[2];
  assign B1 = B[1];

  Adder_7483 M0 (S4, S3, S2, S1, C4, A4, A3, A2, A1, B4, B3, B2, B1, C0, VCC, GND);

  initial #2600 $finish;
  initial begin
    A = 0; B = 0; C0 = 0;
    repeat (256) #5 {A, B} = {A, B} + 1;
    A = 0; B = 0; C0 = 1;
    repeat (256) #5 {A, B} = {A, B} + 1;
  end
endmodule

```

(b)

```

module Supp_9_17b (output [4: 1] S, output carry, input [4: 1] A, B, input M, VCC, GND);
  wire B4, B3, B2, B1;
  xor (B4, M, B[4]);
  xor (B3, M, B[3]);
  xor (B2, M, B[2]);
  xor (B1, M, B[1]);
  Adder_7483 M0 (S[4], S[3], S[2], S[1], carry, A[4], A[3], A[2], A[1], B4, B3, B2, B1, M, VCC, GND);
endmodule

module Adder_7483 (
  output S4, S3, S2, S1, C4,
  input A4, A3, A2, A1, B4, B3, B2, B1, C0, VCC, GND
);
// Note: connect VCC and GND to supply1 and supply0 in the test bench
wire [4: 1] sum;
wire [4: 1] A = {A4, A3, A2, A1};
wire [4: 1] B = {B4, B3, B2, B1};
assign S4 = sum[4];
assign S3 = sum[3];
assign S2 = sum[2];
assign S1 = sum[1];
assign {C4, sum} = A + B + C0;
endmodule

```

```

module t_Supp_9_17b ();
  wire [4: 1] S;
  wire carry;
  reg C0;
  reg [4: 1] A, B;
  reg M;
  supply1 VCC;
  supply0 GND;

  Supp_9_17b M0 (S, carry, A, B, M, VCC, GND);
  initial #2600 $finish;
  initial begin
    A = 0; B = 0; M = 0;
    repeat (256) #5 {A, B} = {A, B} + 1;
    A = 0; B = 0; M = 1;
    repeat (256) #5 {A, B} = {A, B} + 1;
  end
endmodule

```

(c), (d)

```

module supp_9_7c (output S3, S2, S1, S0, C, V, input A3, A2, A1, A0, B3, B2, B1, B0, M);
  wire [3: 0] Sum, B;
  assign S3 = Sum[3];
  assign S2 = Sum[2];
  assign S1 = Sum[1];
  assign S0 = Sum[0];
  wire [3:0] A = {A3, A2, A1, A0};
  xor(B[3], B3, M);
  xor(B[2], B2, M);
  xor(B[1], B1, M);
  xor(B[0], B0, M);
  xor (V, C, C3);
  ripple_carry_4_bit_adder M0 (Sum, C, C3, A, B, M);
endmodule

module t_supp_9_7c ();
  wire S3, S2, S1, S0, C, V;
  reg A3, A2, A1, A0, B3, B2, B1, B0, M;
  wire [3: 0] sum = {S3, S2, S1, S0};
  wire [3: 0] A = {A3, A2, A1, A0};
  wire [3: 0] B = {B3, B2, B1, B0};

  supp_9_7c M0 (S3, S2, S1, S0, C, V, A3, A2, A1, A0, B3, B2, B1, B0, M);

  initial #2600 $finish;
  initial begin
    {A3, A2, A1, A0, B3, B2, B1, B0} = 0; M = 0;
    repeat (256) #5 {A3, A2, A1, A0, B3, B2, B1, B0} = {A3, A2, A1, A0, B3, B2, B1, B0} + 1;
    {A3, A2, A1, A0, B3, B2, B1, B0} = 0; M = 1;
    repeat (256) #5 {A3, A2, A1, A0, B3, B2, B1, B0} = {A3, A2, A1, A0, B3, B2, B1, B0} + 1;
  end
endmodule

```

```

module half_adder (output S, C, input x, y);    // Verilog 2001, 2005 syntax
// Instantiate primitive gates
  xor (S, x, y);
  and (C, x, y);
endmodule

module full_adder (output S, C, input x, y, z);
  wire S1, C1, C2;

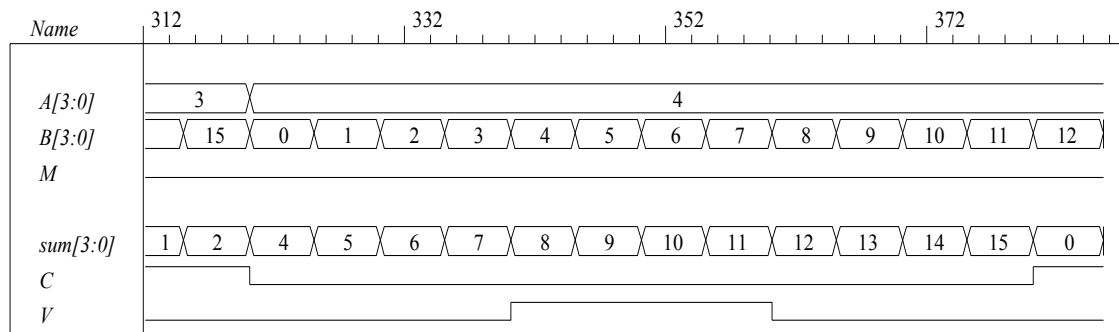
// Instantiate half adders
  half_adder HA1 (S1, C1, x, y);
  half_adder HA2 (S, C2, S1, z);
  or G1 (C, C2, C1);
endmodule

// Modify for C3 output
module ripple_carry_4_bit_adder ( output [3: 0] Sum, output C4, C3, input [3:0] A, B, input C0);
  wire C1, C2; // Intermediate carries
// Instantiate chain of full adders
  full_adder FA0 (Sum[0], C1, A[0], B[0], C0),
              FA1 (Sum[1], C2, A[1], B[1], C1),
              FA2 (Sum[2], C3, A[2], B[2], C2),
              FA3 (Sum[3], C4, A[3], B[3], C3);

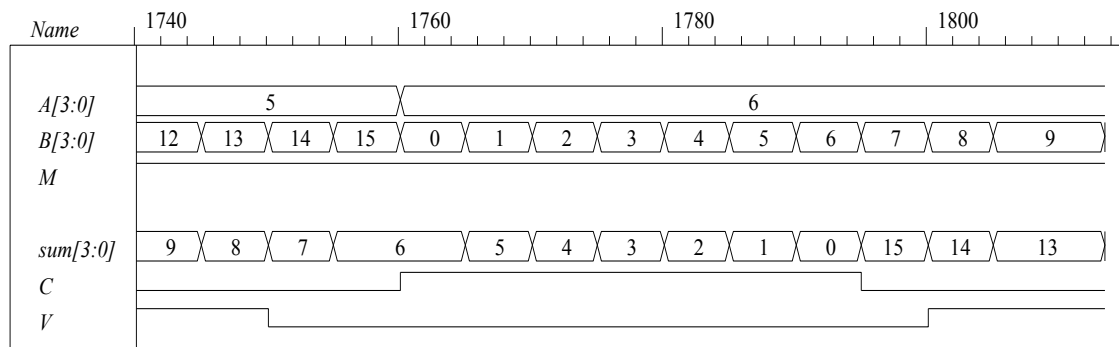
endmodule

```

Addition:



Subtraction:



Supplement to Experiment 8:

(a)

```

module Flip_flop_7474 (output reg Q, input D, CLK, preset, clear);
  always @ (posedge CLK, negedge preset , negedge clear)
    if (!preset)      Q <= 1'b1;
    else if (!clear)   Q <= 1'b0;
    else              Q <= D;
endmodule

```

```

module t_Flip_flop_7474 ();
  wire Q;
  reg D, CLK, preset, clear;

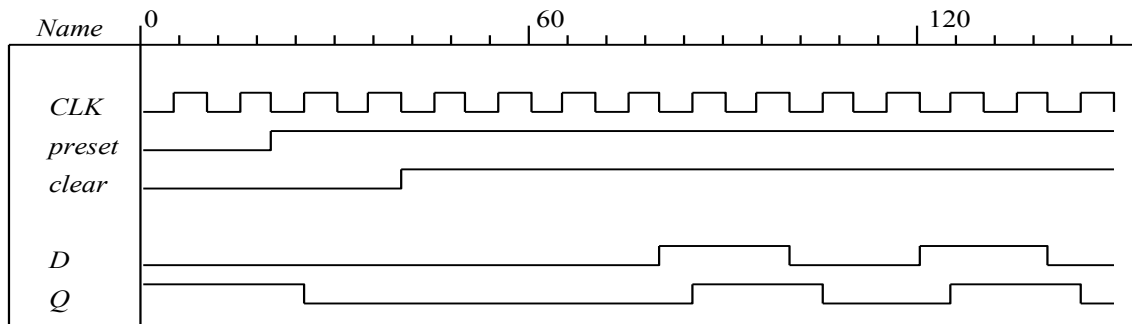
  Flip_flop_7474 M0 (Q, D, CLK, preset, clear);

  initial #150 $finish;
  initial begin CLK = 0; forever #5 CLK = ~CLK; end

  initial fork
    preset = 0; clear = 0;
    #20 preset = 1;
    #40 clear = 1;
  join

  initial begin D = 0; #60 forever #20 D = ~D; end
endmodule

```



(b)

```

//Solution to supplement Experiment 8(b)
//Behavioral description of a 7474 D flip-flop with Q_not
module Flip_Flop_7474_with_Q_not (output reg Q, Q_not, input D, CLK, Preset, Clear);

  always @ (posedge CLK, negedge Preset, negedge Clear)
    /* case ({Preset, Clear})
      2'b00: begin Q <= 1; Q_not <= 1; end
      2'b01: begin Q <= 1; Q_not <= 0; end
      2'b10: begin Q <= 0; Q_not <= 1; end
      2'b11: begin Q <= D; Q_not <= ~D; end

      // NOTE: Q_not <= ~Q will produce a pipeline effect and delay Q_not by one clock
    endcase*/
    if (Preset == 0) begin Q <= 1; if (Clear == 0) Q_not <= 1; else Q_not <= 0; end
    else if (Clear == 0) begin Q <= 0; Q_not <= 1; end

```

```

else begin Q <= D; Q_not <= ~D; end
endmodule

```

// Note: this model will not work if Preset and Clear are // both brought low and then high again.
 // A case statement for both Q and Q_not is also OK.

```

module t_Flip_Flop_7474_with_Q_not ();

```

```

  wire Q, Q_not;

```

```

  reg D, CLK, Preset, Clear;

```

```

  Flip_Flop_7474_with_Q_not M0 (Q, Q_not, D, CLK, Preset, Clear);

```

```

  initial #250 $finish;

```

```

  initial begin CLK = 0; forever #5 CLK = ~CLK; end

```

```

  initial fork

```

```

    Preset = 1; Clear = 1;

```

```

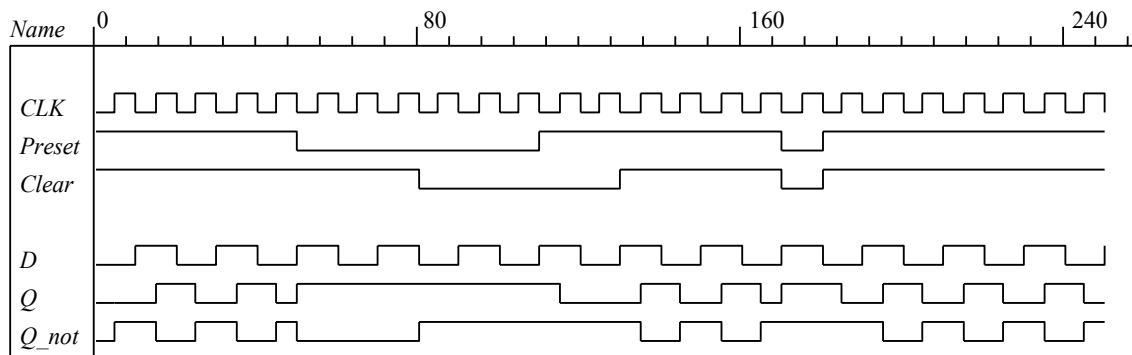
    #50 Preset = 0;

```

```

    #80 Clear =

```



Supplement to Experiment #9:

(a)

```

module Figure_9_9a (output reg y, input x, clock, reset_b);

```

```

  reg [1: 0] state, next_state;

```

```

  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

```

```

  always @ (posedge clock, negedge reset_b) if (reset_b == 0) state <= S0; else state <= next_state;

```

```

  always @ (state, x) begin

```

```

    y = 0;

```

```

    case (state)

```

```

      S0:if (x) begin next_state = S0; y = 1; end else begin next_state = S1; y = 0; end

```

```

      S1:if (x) begin next_state = S3; y = 0; end else begin next_state = S2; y = 1; end

```

```

      S2:if (x) begin next_state = S1; y = 0; end else begin next_state = S0; y = 1; end

```

```

      S3:if (x) begin next_state = S2; y = 1; end else begin next_state = S3; y = 0; end

```

```

    endcase

```

```

  end

```

```

endmodule

```

```

module t_Figure_9_9a ();

```

```

  wire y;

```

```

  reg x, clock, reset_b;

```

```

  Figure_9_9a M0 (y, x, clock, reset_b);

```

```

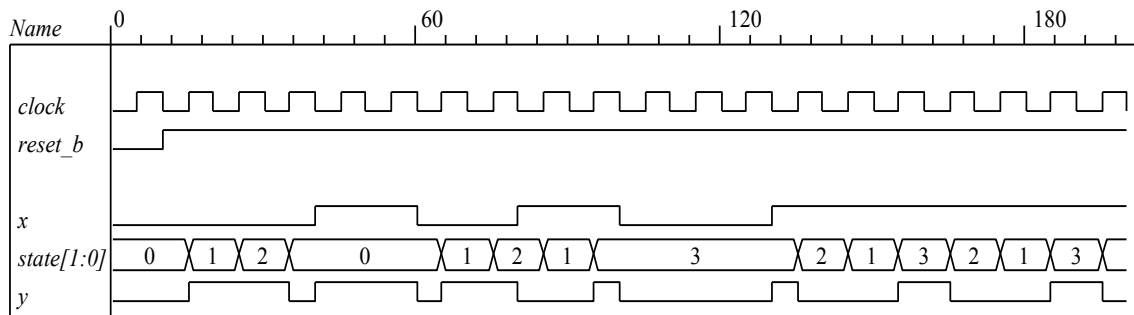
  initial #200 $finish;

```

```

initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
  reset_b = 0;
  x = 0;      // S0, S1, S2 after release of reset_b
  #10 reset_b = 1;
  #40 x = 1;  // Stay in S0
  #60 x = 0;  // S1, S2
  #80 x = 1;  // s1, S3,
  #100 x = 0; // S3
  #130 x = 1; // S2, S1, S3 cycle
join
endmodule

```



(b) The solution depends on the particular design.

(c, d)

Note: The HDL description of the state diagram produces outputs *T0*, *T1*, and *T2*. Additional logic must form the signals that control the datapath unit (*Load_regs*, *Incr_P*, *Add_regs*, and *Shift_regs*). An alternative controller that generates the control signals, rather than the states, as the outputs is given below too. It produces identical simulation results.

```

module Supp_9_9cd # (parameter dp_width = 5)
(
  output [2*dp_width - 1: 0] Product,
  output Ready,
  input [dp_width - 1: 0] Multiplicand, Multiplier,
  input Start, clock, reset_b
);
  wire Load_regs, Incr_P, Add_regs, Shift_regs, Done, Q0;

  Controller M0 (
    Ready, Load_regs, Incr_P, Add_regs, Shift_regs, Start, Done, Q0,
    clock, reset_b
  );

  Datapath M1(Product, Q0, Done, Multiplicand, Multiplier,
    Start, Load_regs, Incr_P, Add_regs, Shift_regs, clock, reset_b);
endmodule

/* // This alternative controller directly produces the signals needed to control the datapath.
module Controller (
  output Ready,
  output reg Load_regs, Incr_P, Add_regs, Shift_regs,
  input Start, Done, Q0, clock, reset_b
);

```



```

parameter S_idle = 3'b001, // one-hot code
            S_add = 3'b010,
            S_shift = 3'b100;
reg [2: 0] state, next_state; // sized for one-hot
assign Ready = (state == S_idle);

always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;

always @ (state, Start, Q0, Done) begin
    next_state = S_idle;
    Load_regs = 0;
    Incr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
        S_idle: if (Start) begin next_state = S_add; Load_regs = 1; end
        S_add: begin next_state = S_shift; Incr_P = 1; if (Q0) Add_regs = 1; end
        S_shift: begin Shift_regs = 1;
                if (Done) next_state = S_idle;
                else next_state = S_add;
            end
        default: next_state = S_idle;
    endcase
end
endmodule
*/

// This controller has an embedded unit to generate T0, T1, and T2 and additional logic to form // the
// signals needed to control the datapath.

module Controller (
    output Ready, Load_regs, Incr_P, Add_regs, Shift_regs,
    input Start, Done, Q0, clock, reset_b
);

    State_Generator M0 (T0, T1, T2, Start, Done, Q0, clock, reset_b);
    assign Ready = T0;
    assign Load_regs = T0 && Start;
    assign Incr_P = T1;
    assign Add_regs = T1 && Q0;
    assign Shift_regs = T2;
endmodule

module State_Generator (output T0,T1, T2, input Start, Done, Q0, clock, reset_b);
    parameter S_idle = 3'b001, // one-hot code
            S_add = 3'b010,
            S_shift = 3'b100;
    reg [2: 0] state, next_state; // sized for one-hot
    assign T0 = (state == S_idle);
    assign T1 = (state == S_add);
    assign T2 = (state == S_shift);

    always @ (posedge clock, negedge reset_b)
        if (~reset_b) state <= S_idle; else state <= next_state;

```

```

always @ (state, Start, Q0, Done) begin
    next_state = S_idle;
    case (state)
        S_idle:    if (Start) next_state = S_add;
        S_add:     next_state = S_shift;
        S_shift:   if (Done) next_state = S_idle; else next_state = S_add;
        default:   next_state = S_idle;
    endcase
end
endmodule

module Datapath #(parameter dp_width = 5, BC_size = 3) (
    output [2*dp_width - 1: 0] Product, output Q0, output Done,
    input [dp_width - 1: 0] Multiplicand, Multiplier,
    input Start, Load_regs, Incr_P, Add_regs, Shift_regs, clock, reset_b
);
// Default configuration: 5-bit datapath
reg   [dp_width - 1: 0]    A, B, Q;           // Sized for datapath
reg                                     C;
reg   [BC_size - 1: 0]    P;                 // Bit counter

assign Q0 = Q[0];
assign Done = (P == dp_width);               // Multiplier is exhausted
assign Product = {C, A, Q};
always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) begin                   // Added to this solution, but
        P <= 0;                               // not really necessary since Load_regs
        B <= 0;                               // initializes the datapath
        C <= 0;
        A <= 0;
        Q <= 0;
    end
    else begin
        if (Load_regs) begin
            P <= 0;
            A <= 0;
            C <= 0;
            B <= Multiplicand;
            Q <= Multiplier;
        end
        if (Add_regs) {C, A} <= A + B;
        if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
        if (Incr_P) P <= P+1 ;
    end
endmodule

module t_Supp_9_9cd;
    parameter dp_width = 5;                 // Width of datapath
    wire [2 * dp_width - 1: 0] Product;
    wire Ready;
    reg [dp_width - 1: 0] Multiplicand, Multiplier;
    reg Start, clock, reset_b;
    integer Exp_Value;
    reg Error;

    Supp_9_9cd M0(Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b);

    initial #115000 $finish;
    initial begin clock = 0; #5 forever #5 clock = ~clock; end

```

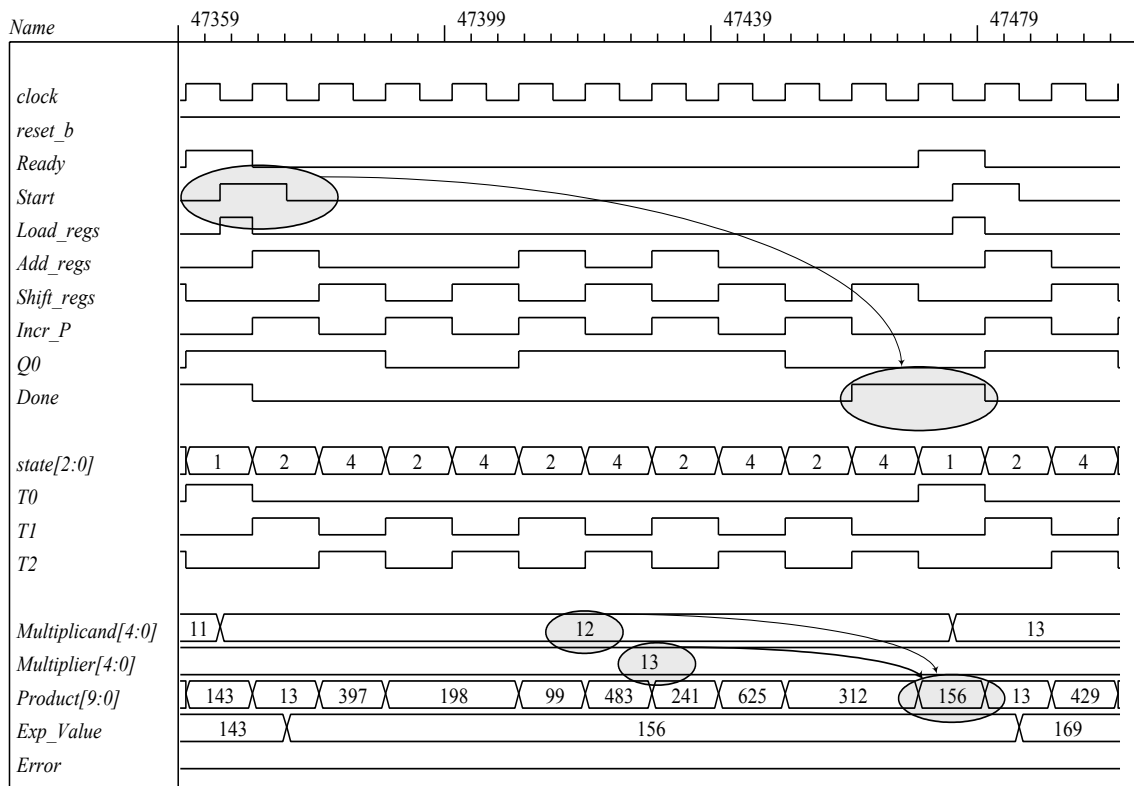
```

initial fork
  reset_b = 1;
  #2 reset_b = 0;
  #3 reset_b = 1;
join
always @ (negedge Start) begin
  Exp_Value = Multiplier * Multiplicand;
  //Exp_Value = Multiplier * Multiplicand +1; // Inject error to confirm detection
end
always @ (posedge Ready) begin
  # 1 Error <= (Exp_Value ^ Product) ;
end

initial begin
  #5 Multiplicand = 0;
  Multiplier = 0;

  repeat (32) #10 begin
    Start = 1;
    #10 Start = 0;
    repeat (32) begin
      Start = 1;
      #10 Start = 0;
      #100 Multiplicand = Multiplicand + 1;
    end
    Multiplier = Multiplier + 1;
  end
end
endmodule

```



Supplement to Experiment #10:

```

module Counter_74161 (
  output      QD, QC, QB, QA,    // Data output
  output      COUT,              // Output carry
  input       D, C, B, A,        // Data input
  input       P, T,              // Active high to count
                                // Active low to load
                                // Positive edge sensitive
                                // Active low to clear
  L,
  CK,
  CLR
);

  reg [3: 0] A_count;
  assign QD = A_count[3];
  assign QC = A_count[2];
  assign QB = A_count[1];
  assign QA = A_count[0];

  assign COUT = ((P == 1) && (T == 1) && (L == 1) && (A_count == 4'b1111));

  always @ (posedge CK, negedge CLR)
    if (CLR == 0)      A_count <= 4'b0000;
    else if (L == 0)    A_count <= {D, C, B, A};
    else if ((P == 1) && (T == 1)) A_count <= A_count + 1'b1;
    else A_count <= A_count; // redundant statement
endmodule

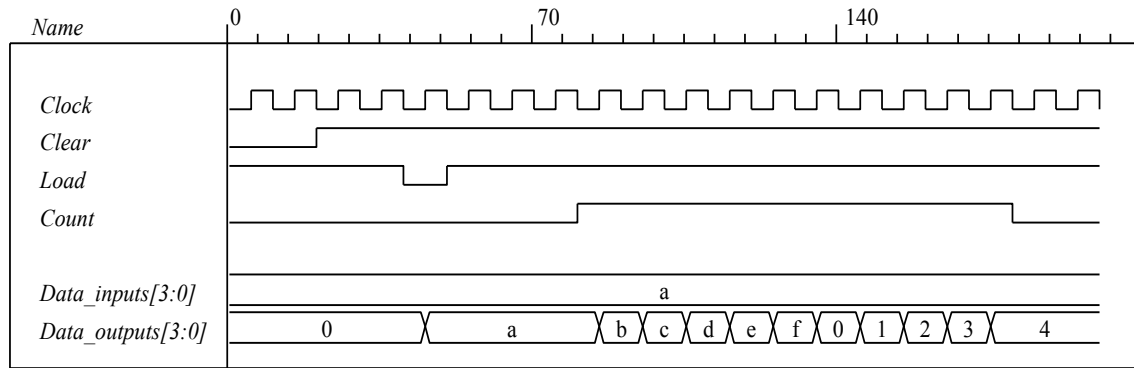
module t_Counter_74161 ();
  wire      QD, QC, QB, QA;
  wire [3: 0] Data_outputs = {QD, QC, QB, QA};
  wire      Carry_out;      // Output carry
  reg [3:0] Data_inputs;     // Data input
  reg       Count,           // Active high to count
            Load,            // Active low to load
            Clock,           // Positive edge sensitive
            Clear;           // Active low to clear

  Counter_74161 M0 (QD, QC, QB, QA, Carry_out,
    Data_inputs[3], Data_inputs[2], Data_inputs[1], Data_inputs[0], Count, Count, Load, Clock, Clear);

  initial #200 $finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end

  initial fork
    Clear = 0;
    Load = 1;
    Count = 0;
    #20 Clear = 1;
    #40 Load = 0;
    #50 Load = 1;
    #80 Count = 1;
    #180 Count = 0;
    Data_inputs = 4'ha;      // 10
  join
endmodule

```



Supplement to Experiment #11.

(a)

// Note: J and K_bar are assumed to be connected together.

```

module SReg_74195 (
  output reg QA, QB, QC, QD,
  output QD_bar,
  input A, B, C, D, SH_LD, J, K_bar, CLR_bar, CK
);
  assign QD_bar = ~QD;

  always @ (posedge CK, negedge CLR_bar)
    if (!CLR_bar) {QA, QB, QC, QD} <= 4'b0;
    else if (!SH_LD) {QA, QB, QC, QD} <= {A, B, C, D};
    else case ({J, K_bar})
      2'b00: {QA, QB, QC, QD} <= {1'b0, QA, QB, QC};
      2'b11: {QA, QB, QC, QD} <= {1'b1, QA, QB, QC};
      2'b01: {QA, QB, QC, QD} <= {QA, QA, QB, QC}; // unused
      2'b10: {QA, QB, QC, QD} <= {~QA, QA, QB, QC}; // unused
    endcase
endmodule

```

```

module t_SReg_74195 ();
  wire QA, QB, QC, QD;
  wire QD_bar;
  reg A, B, C, D, SH_LD, CLR_bar, CK;
  reg Serial_Input;
  wire J = Serial_Input;
  wire K_bar = Serial_Input;
  wire [3: 0] Data_inputs = {A, B, C, D};
  wire [3: 0] Data_outputs = {QA, QB, QC, QD};

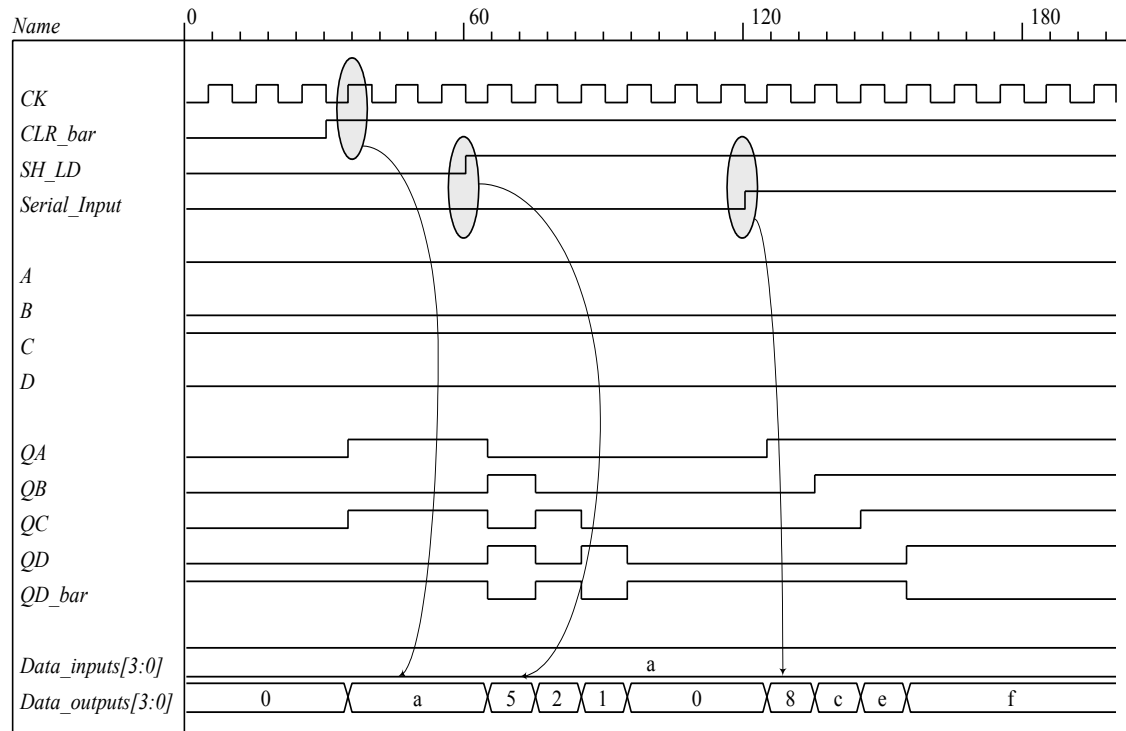
  SReg_74195 M0 (QA, QB, QC, QD, QD_bar, A, B, C, D, SH_LD, J, K_bar, CLR_bar, CK);

```

```

initial #200 $finish;
initial begin CK = 0; forever #5 CK = ~CK; end
initial fork
  {A, B, C, D} = 4'ha;
  CLR_bar = 0;
  Serial_Input = 0;
  SH_LD = 0;
  #30 CLR_bar = 1;
  #60 SH_LD = 1;
  #120 Serial_Input = 1;
join
endmodule

```



(b)

```

module Mux_74157 (
  output reg Y1, Y2, Y3, Y4,
  input A1, A2, A3, A4, B1, B2, B3, B4, SEL, STB
);

```

```

  wire [4: 1] In_A = {A1, A2, A3, A4};
  wire [4: 1] In_B = {B1, B2, B3, B4};

```

```

  always @ (In_A, In_B, SEL, STB)
    if (STB) {Y1, Y2, Y3, Y4} = 4'b0;
    else if (SEL) {Y1, Y2, Y3, Y4} = In_B;
    else {Y1, Y2, Y3, Y4} = In_A;
endmodule

```

```

module t_Mux_74157 ();
  wire Y1, Y2, Y3, Y4;
  reg A1, A2, A3, A4, B1, B2, B3, B4, SEL, STB;
  wire [4: 1] In_A = {A1, A2, A3, A4};
  wire [4: 1] In_B = {B1, B2, B3, B4};
  wire [4: 1] Y = {Y1, Y2, Y3, Y4};

```

```

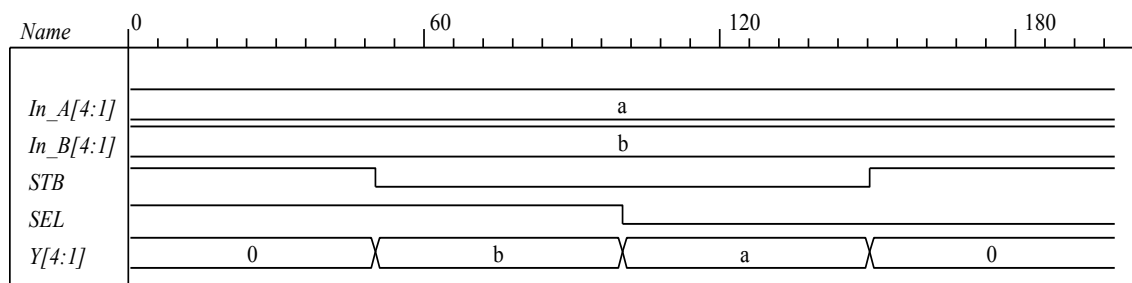
  Mux_74157 M0 (Y1, Y2, Y3, Y4, A1, A2, A3, A4, B1, B2, B3, B4, SEL, STB);

```

```

  initial #200 $finish;
  initial fork
    {A1, A2, A3, A4} = 4'ha;
    {B1, B2, B3, B4} = 4'hb;
    STB = 1;
    SEL = 1;
    #50 STB = 0;
    #100 SEL = 0;
    #150 STB = 1;
  join
endmodule

```



(c)

```

module Bi_Dir_Shift_Reg (output [1: 4] D_out, input [1: 4] D_in, input SEL, STB, SH_LD, clock,
  CLR_bar);
  wire QD_bar;
  wire [1: 4] Y;
  SReg_74195 M0 (D_out[1], D_out[2], D_out[3], D_out[4], QD_bar, Y[1], Y[2], Y[3], Y[4],
    SH_LD, D_out[4], D_out[4], CLR_bar, clock

```

```

);
Mux_74157 M1 (Y[1], Y[2], Y[3], Y[4], D_in[1], D_in[2], D_in[3], D_in[4],
D_out[2], D_out[3], D_out[4], D_out[1], SEL, STB
);
endmodule

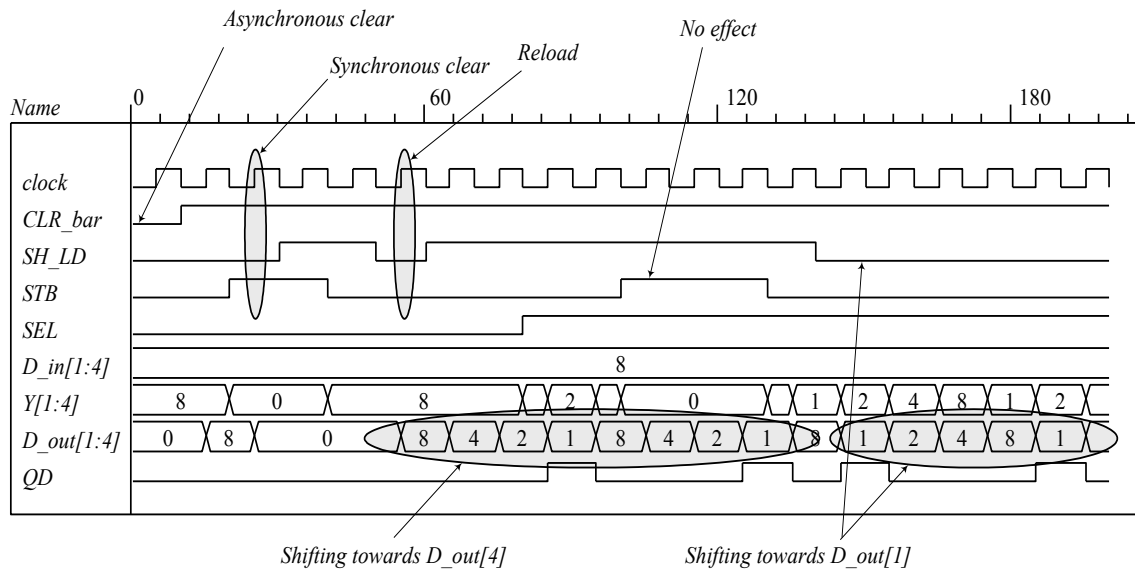
module SReg_74195 (
output reg QA, QB, QC, QD,
output QD_bar,
input A, B, C, D, SH_LD, J, K_bar, CLR_bar, CK
);
assign QD_bar = ~QD;
always @ (posedge CK, negedge CLR_bar)
if (!CLR_bar) {QA, QB, QC, QD} <= 4'b0;
else if (!SH_LD) {QA, QB, QC, QD} <= {A, B, C, D};
else case ({J, K_bar})
2'b00: {QA, QB, QC, QD} <= {1'b0, QA, QB, QC};
2'b11: {QA, QB, QC, QD} <= {1'b1, QA, QB, QC};
2'b01: {QA, QB, QC, QD} <= {QA, QA, QB, QC}; // unused
2'b10: {QA, QB, QC, QD} <= {~QA, QA, QB, QC}; // unused
endcase
endmodule

module Mux_74157 (
output reg Y1, Y2, Y3, Y4,
input A1, A2, A3, A4, B1, B2, B3, B4, SEL, STB
);
wire [4: 1] In_A = {A1, A2, A3, A4};
wire [4: 1] In_B = {B1, B2, B3, B4};

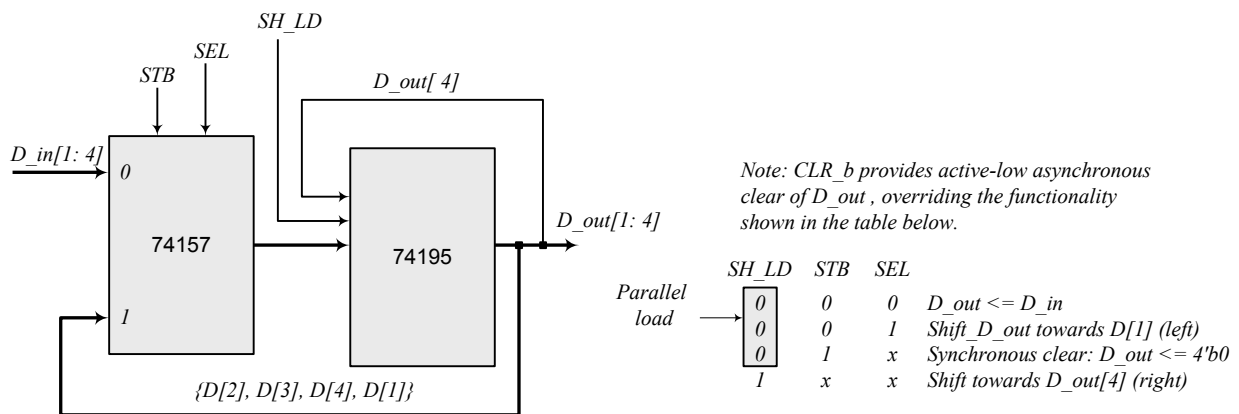
always @ (In_A, In_B, SEL, STB)
if (STB) {Y1, Y2, Y3, Y4} = 4'b0;
else if (SEL) {Y1, Y2, Y3, Y4} = In_B; // SEL = 1
else {Y1, Y2, Y3, Y4} = In_A; // SEL = 0
endmodule

module t_Bi_Dir_Shift_Reg ();
wire [1: 4] D_out;
reg [1: 4] D_in;
reg SEL, STB, SH_LD, clock, CLR_bar;
Bi_Dir_Shift_Reg M0 (D_out, D_in, SEL, STB, SH_LD, clock, CLR_bar);
initial #200 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
D_in = 4'h8; // Data for walking 1 to right
CLR_bar = 0;
STB = 0;
SEL = 0; // Selects D_in
SH_LD = 0; // load D_in
#10 CLR_bar = 1;
#20 STB = 1;
#40 STB = 0;
#30 SH_LD = 1;
#50 SH_LD = 0; // Interrupt count to load
#60 SH_LD = 1;
#80 SEL = 1;
#100 STB = 1;
#130 STB = 0;
#140 SH_LD = 0;
// #150 SH_LD = 1;
join
endmodule

```

The behavioral model is listed below. The two models have matching simulation results.



```

module Bi_Dir_Shift_Reg_beh (output reg [1: 4] D_out, input [1: 4] D_in, input SEL, STB, SH_LD, clock,
CLR_bar);
always @ (posedge clock, negedge CLR_bar)
if (!CLR_bar) D_out <= 4'b0;
else if (SH_LD) D_out <= {D_out[4], D_out[1], D_out[2], D_out[3]};
else if (!STB) D_out <= SEL ? {D_out[2: 4], D_out[1]}: D_in;
else D_out <= 4'b0;
endmodule

module t_Bi_Dir_Shift_Reg_beh ();
wire [1: 4] D_out;
reg [1: 4] D_in;
reg SEL, STB, SH_LD, clock, CLR_bar;

```

Bi_Dir_Shift_Reg_beh M0 (D_out, D_in, SEL, STB, SH_LD, clock, CLR_bar);

```

initial #200 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
    D_in = 4'h8;           // Data for walking 1 to right
    CLR_bar = 0;
    STB = 0;
    SEL = 0;           // Selects D_in
    SH_LD = 0;        // load D_in
    #10 CLR_bar = 1;
    #20 STB = 1;
    #40 STB = 0;
    #30 SH_LD = 1;
    #50 SH_LD = 0;    // Interrupt count to load
    #60 SH_LD = 1;
    #80 SEL = 1;
    #100 STB = 1;
    #130 STB = 0;
    #140 SH_LD = 0;
    // #150 SH_LD = 1;
join
endmodule

```

Supplement to Experiment #13.

module RAM_74189 (**output** S4, S3, S2, S1, **input** D4, D3, D2, D1, A3, A2, A1, A0, CS, WE);
 // Note: active-low CS and WE

```

wire [3: 0]      address = {A3, A2, A1, A0};
reg [3: 0]        RAM [0: 15];           // 16 x 4 memory
wire [4: 1]      Data_in = { D4, D3, D2, D1}; // Input word
tri [4: 1]        Data;                  // Output data word, three-state output
assign S1 = Data[1];                    // Output bits
assign S2 = Data[2];
assign S3 = Data[3];
assign S4 = Data[4];

```

```

always @ (Data_in, address, CS, WE) if (~CS && ~WE) RAM[address] = Data_in;
assign Data = (~CS && WE) ? ~RAM[address] : 4'bz;
endmodule

```

```

module t_RAM_74189 ();
reg [4: 1] Data_in;
reg [3: 0] address;
reg CS, WE;
wire S1, S2, S3, S4;
wire D1, D2, D3, D4;
wire A0, A1, A2, A3;
wire [4: 1] Data_out = {S4, S3, S2, S1};
assign D1 = Data_in [1];
assign D2 = Data_in [2];
assign D3 = Data_in [3];
assign D4 = Data_in [4];
assign A0 = address[0];
assign A1 = address[1];
assign A2 = address[2];
assign A3 = address[3];

```

```

wire [3: 0] RAM_0 = M0.RAM[0];
wire [3: 0] RAM_1 = M0.RAM[1];
wire [3: 0] RAM_2 = M0.RAM[2];

```

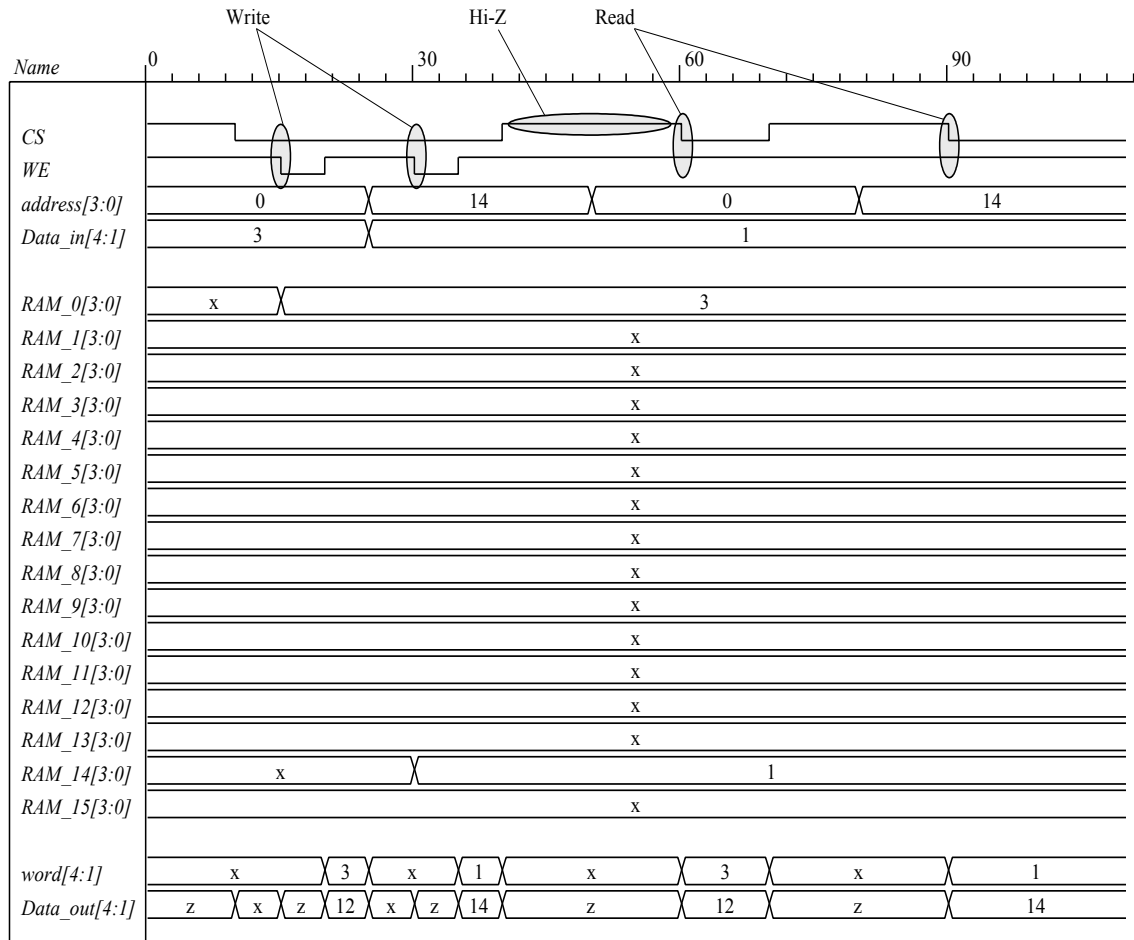
```

wire [3: 0] RAM_3 = M0.RAM[3];
wire [3: 0] RAM_4 = M0.RAM[4];
wire [3: 0] RAM_5 = M0.RAM[5];
wire [3: 0] RAM_6 = M0.RAM[6];
wire [3: 0] RAM_7 = M0.RAM[7];
wire [3: 0] RAM_8 = M0.RAM[8];
wire [3: 0] RAM_9 = M0.RAM[9];
wire [3: 0] RAM_10 = M0.RAM[10];
wire [3: 0] RAM_11 = M0.RAM[11];
wire [3: 0] RAM_12= M0.RAM[12];
wire [3: 0] RAM_13 = M0.RAM[13];
wire [3: 0] RAM_14 = M0.RAM[14];
wire [3: 0] RAM_15 = M0.RAM[15];
wire [4: 1] word = ~Data_out;

RAM_74189 M0 (S4, S3, S2, S1, D4, D3, D2, D1, A3, A2, A1, A0, CS, WE);

initial #110 $finish;
initial fork
  WE = 1;
  CS = 1;
  address = 0;
  Data_in = 3;
  #10 CS = 0;
  #15 WE = 0;
  #20 WE = 1;
  #25 address = 14;
  #25 Data_in = 1;
  #30 WE = 0;
  #35 WE = 1;
  #40 CS = 1;
  #50 address = 0;
  #60 CS = 0;
  #70 CS = 1;
  #80 address = 14;
  #90 CS = 0;
join
endmodule

```



Note: Data_out is the complement of the stored value

Supplement to Experiment #14.

```

module Bi_Dir_Shift_Reg_74194 (
  output reg  QA, QB, QC, QD,
  input      A, B, C, D, SIR, SIL, s1, s0, CK, CLR
);
  always @ (posedge CK, negedge CLR)
    if (!CLR) {QA, QB, QC, QD} <= 4'b0;
    else case ({s1, s0})
      2'b00: {QA, QB, QC, QD} <= {QA, QB, QC, QD};
      2'b01: {QA, QB, QC, QD} <= {SIR, QA, QB, QC};
      2'b10: {QA, QB, QC, QD} <= {QB, QC, QD, SIL};
      2'b11: {QA, QB, QC, QD} <= {A, B, C, D};
    endcase
endmodule

module t_Bi_Dir_Shift_Reg_74194 ();
  wire QA, QB, QC, QD;
  reg A, B, C, D, SIR, SIL, s1, s0, clock, CLR;

  Bi_Dir_Shift_Reg_74194 M0 (QA, QB, QC, QD, A, B, C, D, SIR, SIL, s1, s0, clock, CLR);

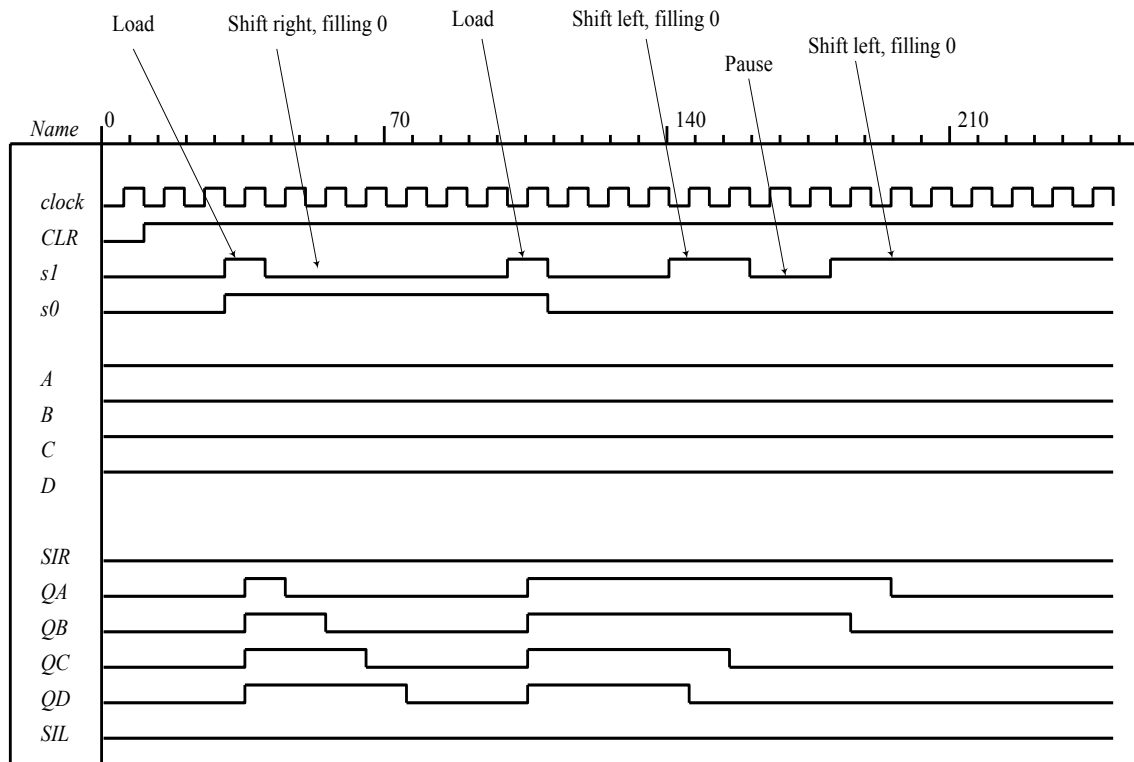
  initial #250 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end

```

```

initial fork
  CLR = 0;
  {A, B, C, D} = 4'hf;
  s1 = 0;
  s0 = 0;
  SIL = 0;
  SIR = 0;
  #10 CLR = 1;
  #30 begin s1 = 1; s0 = 1; end // load
  #40 s1 = 0; // shift right
  #100 s1 = 1; // load
  #110 begin s1 = 0; s0 = 0; end
  #140 s1 = 1; // shift left
  #160 s1 = 0; // pause
  #180 s1 = 1; // resume
join
endmodule

```



Supplement to Experiment #16.

The HDL behavioral descriptions of the components in the block diagram of Fig. 9.23 are described in the solutions of previous experiments, along with their test benches and simulations results: 74189 is described in Experiment 13(a); 74157 in Experiment 11(b); 74161 in Experiment 10; 7483 in Experiment 7(a); 74194 in Experiment 14; and 7474 in Experiment 8(a). The structural description of the parallel adder instantiates these components to show how they are interconnected (see the solution to the supplement for Experiment 17 for a similar procedure). A test bench and simulation results for the integrated unit are given below.

```
// LOAD condition for 74194: s1 = 1, s0 = 1
// SHIFT condition: s1 = 0, s0 = 1
// NO CHANGE condition: s1 = 0, s0 = 0

module Supp_9_16 (
    output [3: 0] accum_sum,
    output carry,
    input [3: 0] Data_in, Addr_in,
    input SIR, SIL, CS, WE, s1, s0, count, Load, select, STB, clock, preset, clear, VCC, GND
);
    wire B4 = Data_in[3]; // Data word to memory
    wire B3 = Data_in[2];
    wire B2 = Data_in[1];
    wire B1 = Data_in[0];
    wire S4, S3, S2, S1;
    wire D4, D3, D2, D1;
    wire S4b = ~S4; // Inverters
    wire S3b = ~S3;
    wire S2b = ~S2;
    wire S1b = ~S1;
    wire D = Addr_in[3]; // For parallel load of address counter
    wire C = Addr_in[2];
    wire B = Addr_in[1];
    wire A = Addr_in[0];
    wire Ocar, Y1, Y2, Y3, Y4, QA, QB, QC, QD, A3, A2, A1, A0;
    assign accum_sum = {D4, D3, D2, D1};

    Flip_flop_7474 M0 (Ocar, carry, clock, preset, clear);
    Adder_7483 M1 (D4, D3, D2, D1, carry, S4b, S3b, S2b, S1b, QD, QC, QB, QA, Ocar, VCC, GND);
    Mux_74157 M2 (Y4, Y3, Y2, Y1, QD, QC, QB, QA, B4, B3, B2, B1, select, STB);
    Counter_74161 M3 (A3, A2, A1, A0, COUT, D, C, B, A, count, count, Load, clock, clear);
    RAM_74189 M4 (S4, S3, S2, S1, Y4, Y3, Y2, Y1, A3, A2, A1, A0, CS, WE);
    Reg_74194 M5 (QD, QC, QB, QA, D4, D3, D2, D1, Ocar, SIL, s1, s0, clock, clear);
endmodule

module t_Supp_9_16 ();
    wire [3: 0] sum;
    wire carry;
    reg [3: 0] Data_in, Addr_in;
    reg SIR, SIL, CS, WE, s1, s0, count, Load, select, STB, clock, preset, clear;
    supply1 VCC;
    supply0 GND;
    wire [3: 0] RAM_0 = M0.M4.RAM[0];
    wire [3: 0] RAM_1 = M0.M4.RAM[1];
    wire [3: 0] RAM_2 = M0.M4.RAM[2];
    wire [3: 0] RAM_3 = M0.M4.RAM[3];
endmodule
```

```

wire [3: 0] RAM_4 = M0.M4.RAM[4];
wire [3: 0] RAM_5 = M0.M4.RAM[5];
wire [3: 0] RAM_6 = M0.M4.RAM[6];
wire [3: 0] RAM_7 = M0.M4.RAM[7];
wire [3: 0] RAM_8 = M0.M4.RAM[8];
wire [3: 0] RAM_9 = M0.M4.RAM[9];
wire [3: 0] RAM_10 = M0.M4.RAM[10];
wire [3: 0] RAM_11 = M0.M4.RAM[11];
wire [3: 0] RAM_12 = M0.M4.RAM[12];
wire [3: 0] RAM_13 = M0.M4.RAM[13];
wire [3: 0] RAM_14 = M0.M4.RAM[14];
wire [3: 0] RAM_15 = M0.M4.RAM[15];

```

```

wire [4: 1] word = {M0.S4b, M0.S3b, M0.S2b, M0.S1b};
wire [4: 1] mux_out = { M0.Y4, M0.Y3, M0.Y2, M0.Y1};
wire [4: 1] Reg_Output = {M0.QD, M0.QC, M0.QB, M0.QA};

```

Supp_9_16 M0 (sum, carry, Data_in, Addr_in, SIR, SIL, CS, WE, s1, s0, count, Load, select, STB, clock, preset, clear, VCC, GND);

```

integer k;
initial #600 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end
initial fork
  #10 begin preset = 1; clear = 0; s1 = 0; s0 = 0; Load = 1; count = 0; CS = 1; WE = 1; STB = 0; end
  // initialize memory
  #10 begin k = 0; repeat (16) begin M0.M4.RAM[k] = 4'hf; k = k + 1; end end
  #20 begin Data_in = 4'hf; Addr_in = 0; select = 1; end
  #30 begin clear = 1; WE = 0; end
  // load memory
  #40 begin
    count = 1;
    CS = 0;
    begin
      repeat (16) @ (negedge clock) Data_in = Data_in + 1;
      count = 0;
      @ (negedge clock) CS = 1;
    end
  end
  #200 count = 1;           // Establish address
  #240 count = 0;
  #250 WE = 1;
  #260 CS = 0;           // Read from memory
  #280 clear = 0;
  #290 clear = 1;
  #300 count = 1;           // Establish address
  #340 begin s1 = 1; s0 = 1; count = 0; end
  #390 CS = 0;
  #400 clear = 0;           // Clear the registers
  #410 clear = 1;
  #420 begin count = 1; CS = 0; end           // Accumulate values
  #490 begin count = 0; CS = 1; end
join
endmodule

```

```

module Flip_flop_7474 (output reg Q, input D, CLK, preset, clear);
  always @ (posedge CLK, negedge preset, negedge clear)
    if (!preset)           Q <= 1'b1;
    else if (!clear)       Q <= 1'b0;
    else                   Q <= D;

```

endmodule

```
module Adder_7483 (
    output S4, S3, S2, S1, C4,
    input A4, A3, A2, A1, B4, B3, B2, B1, C0, VCC, GND
);
// Note: connect VCC and GND to supply1 and supply0 in the test bench
    wire [4: 1] sum;
    wire [4: 1] A = {A4, A3, A2, A1};
    wire [4: 1] B = {B4, B3, B2, B1};
    assign S4 = sum[4];
    assign S3 = sum[3];
    assign S2 = sum[2];
    assign S1 = sum[1];
    assign {C4, sum} = A + B + C0;
endmodule
```

```
module Mux_74157 (
    output reg Y1, Y2, Y3, Y4,
    input A1, A2, A3, A4, B1, B2, B3, B4, SEL, STB
);
    wire [4: 1] In_A = {A1, A2, A3, A4};
    wire [4: 1] In_B = {B1, B2, B3, B4};

    always @ (In_A, In_B, SEL, STB)
        if (STB) {Y1, Y2, Y3, Y4} = 4'b0;
        else if (SEL) {Y1, Y2, Y3, Y4} = In_B;
        else {Y1, Y2, Y3, Y4} = In_A;
endmodule
```

```
module Counter_74161 (
    output QD, QC, QB, QA, // Data output
    output COUT, // Output carry
    input D, C, B, A, // Data input
    input P, T, // Active high to count
    L, // Active low to load
    CK, // Positive edge sensitive
    CLR // Active low to clear
);
    reg [3: 0] A_count;
    assign QD = A_count[3];
    assign QC = A_count[2];
    assign QB = A_count[1];
    assign QA = A_count[0];
    assign COUT = ((P == 1) && (T == 1) && (L == 1) && (A_count == 4'b1111));
```



```

always @ (posedge CK, negedge CLR)
if (CLR == 0)          A_count <= 4'b0000;
else if (L == 0)        A_count <= {D, C, B, A};
else if ((P == 1) && (T == 1)) A_count <= A_count + 1'b1;
else                   A_count <= A_count; // redundant statement
endmodule

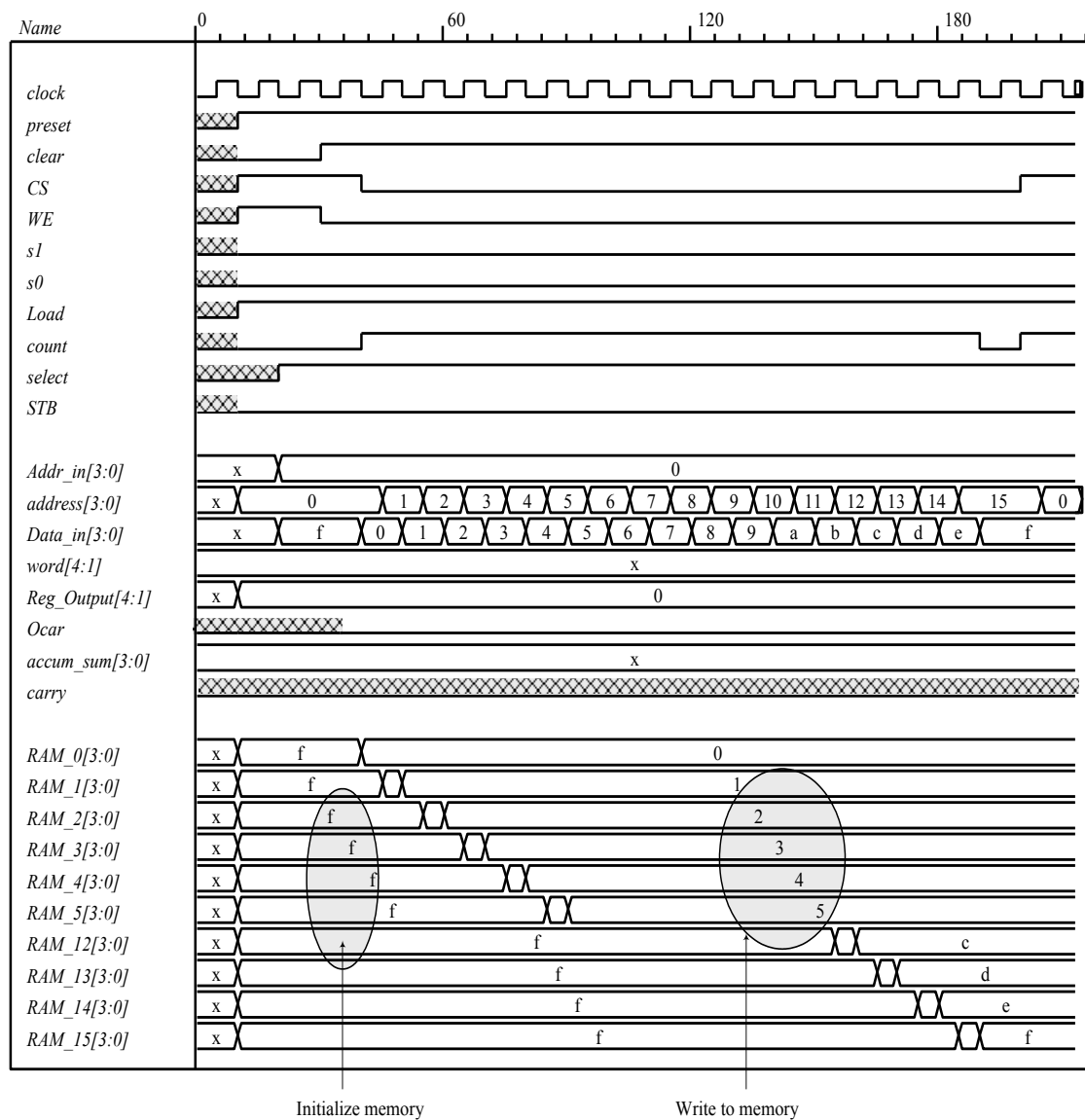
module RAM_74189 (output S4, S3, S2, S1, input D4, D3, D2, D1, A3, A2, A1, A0, CS, WE);
// Note: active-low CS and WE
  wire [3: 0] address = {A3, A2, A1, A0};
  reg [3: 0] RAM [0: 15]; // 16 x 4 memory
  wire [4: 1] Data_in = { D4, D3, D2, D1}; // Input word
  tri [4: 1] Data; // Output data word, three-state output
  assign S1 = Data[1]; // Output bits
  assign S2 = Data[2];
  assign S3 = Data[3];
  assign S4 = Data[4];

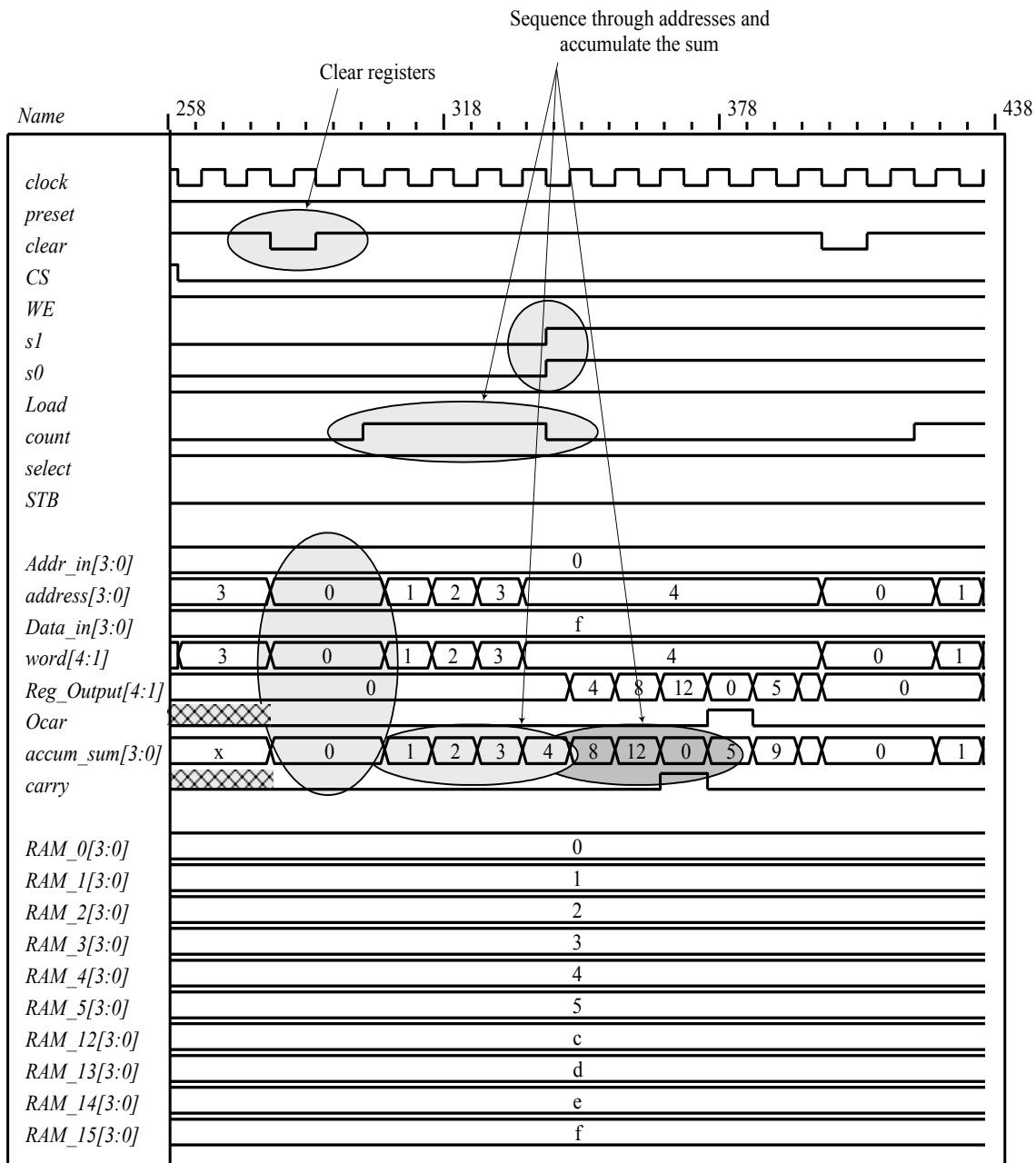
  always @ (Data_in, address, CS, WE) if (~CS && ~WE) RAM[address] = Data_in;
  assign Data = (~CS && WE) ? ~RAM[address] : 4'bz; // Note complement of data word
endmodule

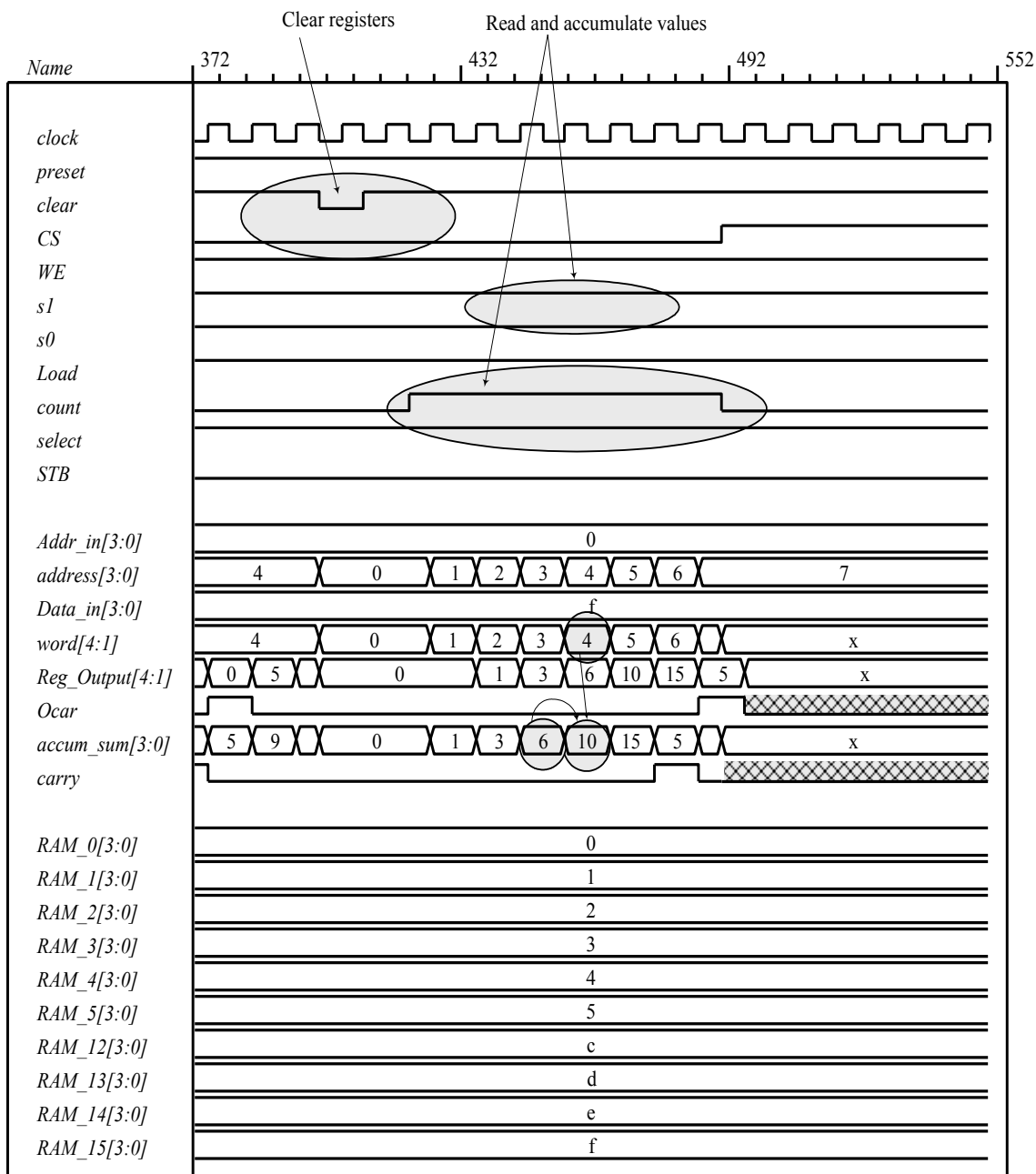
module Reg_74194 (
  output reg QA, QB, QC, QD,
  input A, B, C, D, SIR, SIL, s1, s0, CK, CLR
);
always @ (posedge CK, negedge CLR)
  if (!CLR) {QA, QB, QC, QD} <= 4'b0;
  else case ({s1, s0})
    2'b00: {QA, QB, QC, QD} <= {QA, QB, QC, QD};
    2'b01: {QA, QB, QC, QD} <= {SIR, QA, QB, QC};
    2'b10: {QA, QB, QC, QD} <= {QB, QC, QD, SIL};
    2'b11: {QA, QB, QC, QD} <= {A, B, C, D};
  endcase
endmodule

```

Simulation results: initializing memory to 4'hf, then writing to memory. Note: the values of the inputs are ambiguous until the clear signal is asserted. Signals Ocar and carry are ambiguous because the output of memory is high-z until memory is read is read.







Supplement to Experiment #17.

The HDL behavioral descriptions of the components in the block diagram of Fig. 9.23 are described in the solutions of previous experiments, along with their test benches and simulations results: 74161 in Experiment 10; 7483 in Experiment 7(a); 74194 in Experiment 14; and 7474 in Experiment 8(a). The structural description of the parallel adder instantiates these components to show how they are interconnected (see the

solution to the supplement for Experiment 17 for a similar procedure). A test bench and simulation results for the integrated unit are given below.

```
// Control unit is obtained by modifying the solution to Prob. 8.24.
// Datapath is implemented with a structural HDL model and IC components.
// LOAD condition for 74194: s1 = 1, s0 = 1
// SHIFT condition: s1 = 0, s0 = 1
// NO CHANGE condition: s1 = 0, s0 = 0

module Supp_9_17_Par_Mult # (parameter dp_width = 4)
(
  output [2*dp_width - 1: 0] Product,
  output Ready,
  input [dp_width - 1: 0] Multiplicand, Multiplier,
  input Start, clock, reset_b, VCC, GND
);
  wire Load_regs, Incr_P, Add_regs, Shift_regs, Done, Q0;

  Controller M0 (
    Ready, Load_regs, Incr_P, Add_regs, Shift_regs, Start, Done, Q0,
    clock, reset_b);

  Datapath M1(Product, Q0, Done, Multiplicand, Multiplier,
    Start, Load_regs, Incr_P, Add_regs, Shift_regs, clock, reset_b, VCC, GND);
endmodule

module Controller (
  output Ready,
  output reg Load_regs, Incr_P, Add_regs, Shift_regs,
  input Start, Done, Q0, clock, reset_b
);

  parameter S_idle = 3'b001, // one-hot code
             S_add = 3'b010,
             S_shift = 3'b100;
  reg [2: 0] state, next_state; // sized for one-hot
  assign Ready = (state == S_idle);

  always @ (posedge clock, negedge reset_b)
    if (~reset_b) state <= S_idle; else state <= next_state;
  always @ (state, Start, Q0, Done) begin
    next_state = S_idle;
    Load_regs = 0;
    Incr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    case (state)
      S_idle: if (Start) begin next_state = S_add; Load_regs = 1; end
      S_add: begin next_state = S_shift; Incr_P = 1; if (Q0) Add_regs = 1; end
      S_shift: begin
        Shift_regs = 1;
        if (Done) next_state = S_idle;
        else next_state = S_add;
      end
      default: next_state = S_idle;
    endcase
  end
endmodule

module Datapath #(parameter dp_width = 4, BC_size = 3) (
```

```

output [2*dp_width - 1: 0] Product, output Q0, output Done,
input [dp_width - 1: 0] Multiplicand, Multiplier,
input Start, Load_regs, Incr_P, Add_regs, Shift_regs, clock, clear, VCC, GND
);
wire C;
wire Cout, Sum3, Sum2, Sum1, Sum0, P3, P2, P1, P0, A3, A2, A1, A0;
wire Q3, Q2, Q1;
wire [dp_width - 1: 0] A = {A3, A2, A1, A0};
wire [dp_width - 1: 0] Q = {Q3, Q2, Q1, Q0};
assign Product = {C, A, Q};
wire [BC_size - 1: 0] P = {P3, P2, P1, P0};

// Registers must be controlled separately to execute add and shift operations correctly.
// LOAD condition for 74194: s1 = 1, s0 = 1
// SHIFT condition: s1 = 0, s0 = 1
// NO CHANGE condition: s1 = 0, s0 = 0

wire B3 = Multiplicand[3];      // Data word to adder
wire B2 = Multiplicand[2];
wire B1 = Multiplicand[1];
wire B0 = Multiplicand[0];
wire Q3_in = Multiplier[3];
wire Q2_in = Multiplier[2];
wire Q1_in = Multiplier[1];
wire Q0_in = Multiplier[0];
assign Done = ({P3, P2, P1, P0} == dp_width);      // Counts bits of multiplier
wire s1A = Load_regs || Add_regs;      // Controls for A register
wire s0A = Load_regs || Add_regs || Shift_regs;
wire s0Q = Load_regs || Shift_regs;      // Controls for Q register
wire s1Q = Load_regs;
wire Pout;      // Unused
wire clr_P = clear && ~Load_regs;
Flip_flop_7474 M0_C (C, Cout, clock, VCC, clr_P);
Adder_7483 M1 (Sum3, Sum2, Sum1, Sum0, Cout, A3, A2, A1, A0, B3, B2, B1, B0, GND, VCC, GND);
Counter_74161 M3_P (P3, P2, P1, P0, Pout, GND, GND, GND, GND, Incr_P, Incr_P, VCC, clock,
clr_P);
Reg_74194 M4_A (A3, A2, A1, A0, Sum3, Sum2, Sum1, Sum0, C, GND, s1A, s0A, clock, clr_P);
Reg_74194 M5_Q (Q3, Q2, Q1, Q0, Q3_in, Q2_in, Q1_in, Q0_in, A0, GND, s1Q, s0Q, clock, clear);

endmodule

module t_Supp_9_17_Par_Mult;
parameter dp_width = 4;      // Width of datapath
wire [2 * dp_width - 1: 0] Product;
wire Ready;
reg [dp_width - 1: 0] Multiplicand, Multiplier;
reg Start, clock, reset_b;
integer Exp_Value;
reg Error;
supply0 GND;
supply1 VCC;
Supp_11_17_Par_Mult M0 (Product, Ready, Multiplicand, Multiplier, Start, clock, reset_b, VCC, GND);
wire [dp_width - 1: 0] sum = {M0.M1.Sum3, M0.M1.Sum2, M0.M1.Sum1, M0.M1.Sum0};
initial #115000 $finish;
initial begin clock = 0; #5 forever #5 clock = ~clock; end
initial fork
    reset_b = 1;
    #2 reset_b = 0;
    #3 reset_b = 1;
join
always @ (negedge Start) begin
    Exp_Value = Multiplier * Multiplicand;

```

```

    //Exp_Value = Multiplier * Multiplicand +1; // Inject error to confirm detection
end

```

```

always @ (posedge Ready) begin
    # 1 Error <= (Exp_Value ^ Product) ;
end
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (32) #10 begin
        Start = 1;
        #10 Start = 0;
        repeat (32) begin
            Start = 1;
            #10 Start = 0;
            #100 Multiplicand = Multiplicand + 1;
        end
        Multiplier = Multiplier + 1;
    end
end
endmodule

```

```

module Flip_flop_7474 (output reg Q, input D, CLK, preset, clear);
    always @ (posedge CLK, negedge preset , negedge clear)
        if (!preset)          Q <= 1'b1;
        else if (!clear)      Q <= 1'b0;
        else                  Q <= D;
endmodule

```

```

module Adder_7483 (
    output S4, S3, S2, S1, C4,
    input A4, A3, A2, A1, B4, B3, B2, B1, C0, VCC, GND
);
// Note: connect VCC and GND to supply1 and supply0 in the test bench
    wire [4: 1] sum;
    wire [4: 1] A = {A4, A3, A2, A1};
    wire [4: 1] B = {B4, B3, B2, B1};
    assign S4 = sum[4];
    assign S3 = sum[3];
    assign S2 = sum[2];
    assign S1 = sum[1];
    assign {C4, sum} = A + B + C0;
endmodule

```

```

module Counter_74161 (
    output QD, QC, QB, QA, // Data output
    output COUT, // Output carry
    input D, C, B, A, // Data input
    input P, T, // Active high to count
            L, // Active low to load
            CK, // Positive edge sensitive
            CLR // Active low to clear
);
    reg [3: 0] A_count;
    assign QD = A_count[3];
    assign QC = A_count[2];
    assign QB = A_count[1];

```

```

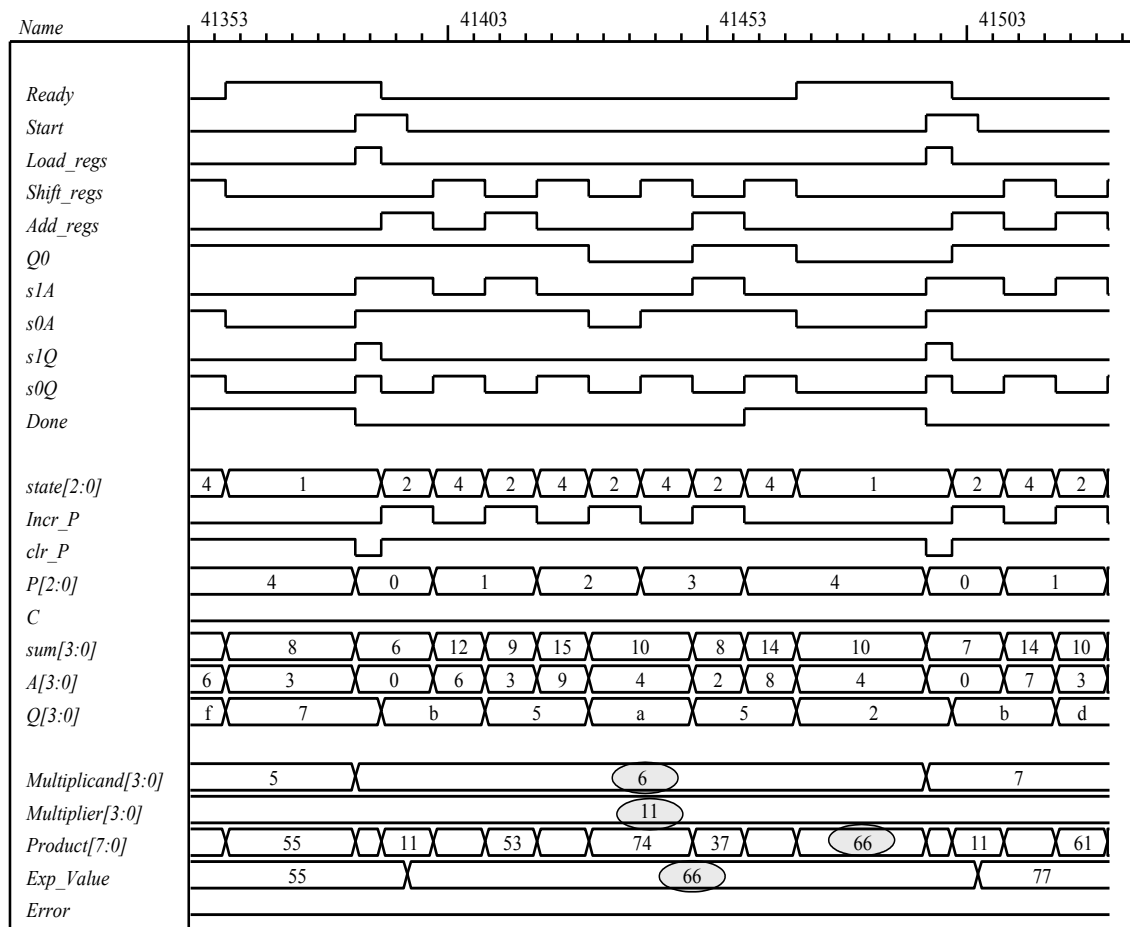
assign QA = A_count[0];
assign COUT = ((P == 1) && (T == 1) && (L == 1) && (A_count == 4'b1111));
always @ (posedge CK, negedge CLR)
if (CLR == 0)      A_count <= 4'b0000;
else if (L == 0)    A_count <= {D, C, B, A};
else if ((P == 1) && (T == 1))  A_count <= A_count + 1'b1;
else               A_count <= A_count; // redundant statement
endmodule

```

```

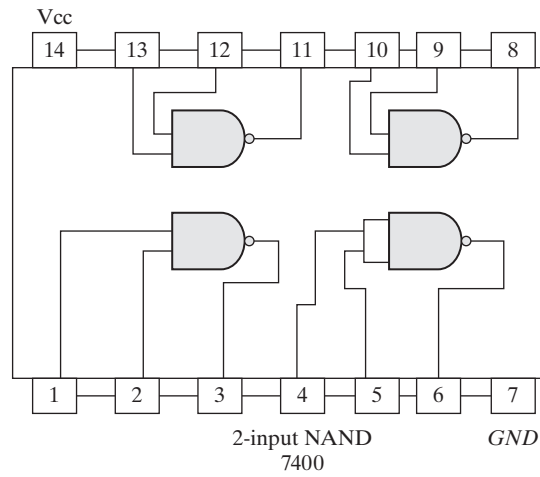
module Reg_74194 (
output reg QA, QB, QC, QD,
input A, B, C, D, SIR, SIL, s1, s0, CK, CLR
);
always @ (posedge CK, negedge CLR)
if (!CLR) {QA, QB, QC, QD} <= 4'b0;
else case ({s1, s0})
2'b00: {QA, QB, QC, QD} <= {QA, QB, QC, QD};
2'b01: {QA, QB, QC, QD} <= {SIR, QA, QB, QC};
2'b10: {QA, QB, QC, QD} <= {QB, QC, QD, SIL};
2'b11: {QA, QB, QC, QD} <= {A, B, C, D};
endcase
endmodule

```

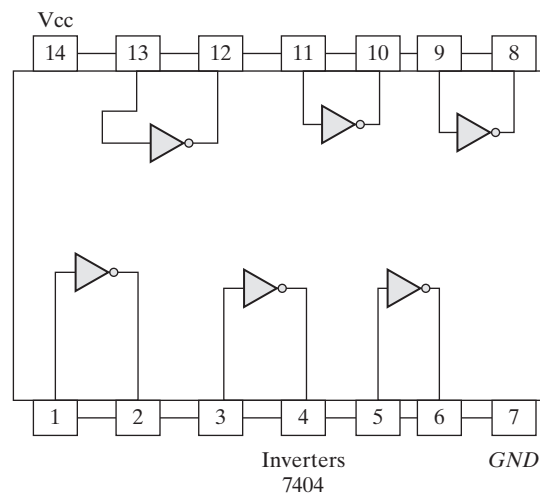


CHAPTER 10

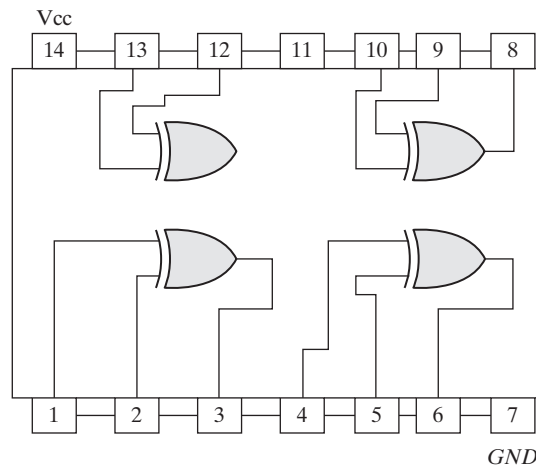
10.1 The rectangular shaped graphic symbol for 7400 is shown below.



The rectangular shaped graphic symbol for 7404 is shown below.

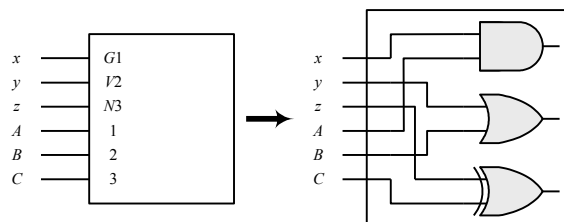


The rectangular shaped graphic symbol for 7486 is shown below.

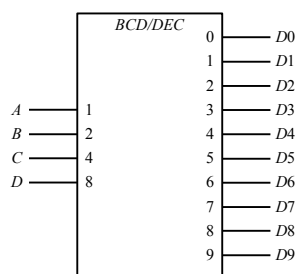


10.2 See textbook.

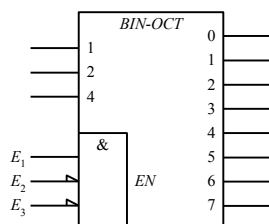
10.3



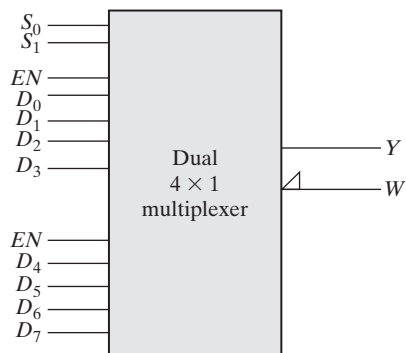
10.4 BCD-to-decimal decoder (similar to IC 7442)



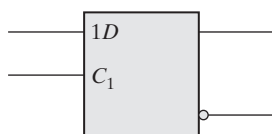
10.5 Similar to 7438:

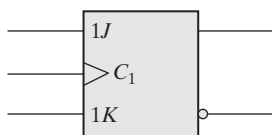
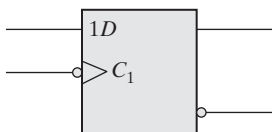


10.6 IC type 74153.

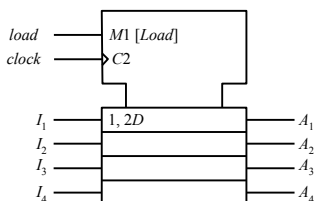
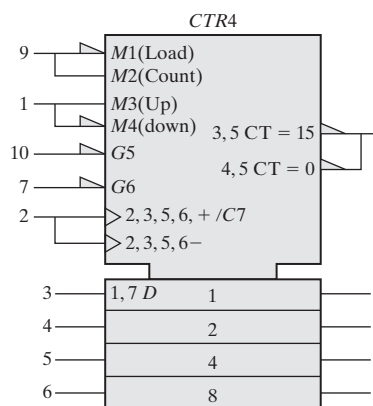


10.7 (a) D latch



(b) Positive-edge-triggered J - K flip-flop**(c) Positive-edge-triggered J - K flip-flop**

10.8 Magnitude comparator: COMP
 Multiplexer: MUX
 Code converter: X/Y
 Counter: CTR

10.9**10.10** See textbook.**10.11****10.12**